

**UNIVERSITÀ
CAMPUS BIO-MEDICO DI ROMA**

**FACOLTÀ DIPARTIMENTALE DI INGEGNERIA
CORSO DI LAUREA MAGISTRALE IN
INGEGNERIA DEI SISTEMI INTELLIGENTI**

**IMPLEMENTAZIONE E OTTIMIZZAZIONE
DELL'ALGORITMO DI SHOR IN AMBIENTI
QUANTISTICI RUMOROSI MEDIANTE MODELLI
DI INTELLIGENZA ARTIFICIALE**

Relatore

Prof. Paolo Soda

Correlatore

Ing. Floriano Caprio

Laureando

Claudio Dragotta

*Alla mia famiglia,
per il sostegno incondizionato e l'amore che non è mai venuto meno.
A me stesso, per la determinazione e la perseveranza con cui ho affrontato questo
percorso.*

Abstract

Introduzione

L’algoritmo di Shor [1] fattorizza un intero composto in tempo polinomiale nella taglia dell’input. La sua complessità è $O((\log N)^3)$; il *general number field sieve* [2], il miglior algoritmo classico noto, richiede invece tempo sub-esponenziale. È questo divario a minacciare RSA [3], ma si tratta di un vantaggio asintotico che i dispositivi *Noisy Intermediate-Scale Quantum* [4] non realizzano. Il rumore distrugge la struttura periodica su cui l’algoritmo si fonda e le dimostrazioni restano confinate a moduli didattici [5]. Anche la stima delle risorse per RSA-2048, pur scendendo di circa un ordine di grandezza — da venti milioni di qubit fisici nel 2019 [6] a meno di un milione nel 2025 [7] —, resta lontana dai dispositivi disponibili. I due rimedi principali sono la mitigazione a valle e la correzione d’errore durante il calcolo; a entrambi un numero crescente di lavori applica modelli appresi, spesso senza un termine di confronto che separi il contributo del modello da quello della regola decisionale.

Il lavoro riprende un progetto preesistente — Shor in Qiskit e una prima campagna rumorosa su $N = 15$ — e lo rigenera dopo che una verifica per truth table ne ha invalidato il moltiplicatore modulare controllato, il blocco che realizza $|y\rangle \mapsto |ay \bmod N\rangle$. Chiede quindi *in quale punto della pipeline un modello appreso migliori davvero la resa di Shor in ambiente rumoroso*, e con la stessa tecnologia e lo stesso impianto statistico ne confronta due: un classificatore dell’istogramma di output e un decoder addestrato sulle sindromi di un codice correttore. La risposta ha due facce, negativa a valle e positiva ma solo residuale sulla sindrome. Il contributo è metodologico: disegno appaiato con *ablazione* — un braccio di controllo che esegue la sola regola decisionale, senza il modello —, separazione fra metriche di correzione d’errore e successo di Shor, artefatti riproducibili (seed, manifest e hash del circuito).

Materiali e Metodi

Formalizzazione matematica del problema

Per N composto, dispari e non potenza di un primo, e a coprimo con N , la fattorizzazione si riduce alla ricerca dell’ordine $r = \text{ord}_N(a) = \min\{k \in \mathbb{N}^+ : a^k \equiv 1 \pmod{N}\}$: se r è pari e $a^{r/2} \not\equiv -1 \pmod{N}$, allora $\gcd(a^{r/2} \pm 1, N)$ è un fattore non banale, con probabilità almeno $1/2$ su a casuale [8]. La parte quantistica stima r per *phase estimation* dell’unitario $U_a|y\rangle = |ay \bmod N\rangle$: con n qubit di conteggio e $M = 2^n$, dopo la trasformata di Fourier quantistica inversa la misura si concentra

sui multipli di M/r , con dispersione descritta dal nucleo di Fejér. Il post-processing Φ è deterministico: frazioni continue di y/M , esatte quando $M \geq N^2$, poi parità e massimo comun divisore. Detto $\mathcal{U}$ l'insieme degli y da cui Φ ricava un fattore non banale, ogni esecuzione è delimitata da due costanti in forma chiusa,

$$P_{\text{id}} = \sum_{y \in \mathcal{U}} P(y) = \frac{3}{4}, \quad P_{\text{unif}} = \frac{|\mathcal{U}|}{M} = \frac{63}{256} \simeq 0,2461, \quad (1)$$

per l'istanza canonica $N = 15$, $a = 7$, $n = 8$, dove $r = 4$ divide M e P è supportata sui quattro picchi $\{0, 64, 128, 192\}$: di questi solo $y = 0$ non porta informazione sull'ordine, da cui i tre quarti. Le due costanti fissano in anticipo la scala di lettura: P_{id} è la resa per shot della distribuzione ideale; P_{unif} è il *pavimento uniforme*, la resa attesa quando il rumore rende equiprobabili tutte le M celle, non nulla perché il post-processing accetta 63 esiti su 256. Non sono limiti universali: sono i riferimenti ideale e uniforme, e toccare il pavimento segnala la perdita del segnale utile, non l'impossibilità di fattorizzare.

Modello di rumore, scenari e mitigazione a valle

Il circuito è compilato deterministicamente (ottimizzazione 2 e seed registrato) nella base hardware $\{r_z, \sqrt{X}, X, CX\}$, in cui r_z è una rotazione virtuale attorno a z , e il rumore è modellato su quelle porte. Dopo ogni gate g si compone un canale depolarizzante di parametro λ con il rilassamento termico, descritto dai tempi T_1 (rilassamento energetico) e T_2 (perdita di coerenza di fase) e applicato in proporzione alla durata di g : r_z ha durata nulla nel modello, $\sqrt{X}/X$ dura 50 ns e CX 300 ns. La lettura è una matrice di confusione simmetrica di parametro p_{ro} ; su S shot l'istogramma normalizzato $\hat{h}$ è la frequenza campionaria multinomiale della distribuzione così letta. Due preset definiscono gli scenari confrontati: **UC1** (riferimento) con $\lambda_{1q} = 10^{-3}$, $\lambda_{2q} = 10^{-2}$, $T_1 = 100 \mu\text{s}$, $T_2 = 80 \mu\text{s}$, $p_{\text{ro}} = 0,02$; **UC2** (stress) con $\lambda_{1q} = 5 \times 10^{-3}$, $\lambda_{2q} = 5 \times 10^{-2}$, $T_1 = 50 \mu\text{s}$, $T_2 = 30 \mu\text{s}$, $p_{\text{ro}} = 0,05$: errori di gate cinque volte più intensi e tempi di coerenza dimezzati. Sono scenari uniformi illustrativi, non calibrazioni di una QPU. Come termine di paragone a valle si valuta la *zero-noise extrapolation* [9]: il circuito gira con il rumore scalato di $\lambda = 1, 2, 3$ e l'istogramma è estrapolato a $\lambda \rightarrow 0$ (lineare su due livelli, Richardson su tre), al prezzo di altrettante repliche.

Politiche decisionali, modello appreso e stimando statistico

Su $\hat{h}$ si definisce l'ordinamento totale $\prec$ per conteggio decrescente, con pareggi risolti da una priorità deterministica seeded (SHA-256); la politica π_K prova i primi K candidati secondo $\prec$. Il Metodo 1 è π_1 , l'ablazione è π_4 , il Metodo 2 applica un classificatore $g_\theta(\hat{h}) \in \{0, 1\}$ ed esegue π_4 solo se il filtro è positivo. Il classificatore riceve $\hat{h}$ e l'etichetta binaria $Y_1 = \mathbf{1}[\pi_1 \text{ ha successo}]$; è addestrato su 2000 istogrammi per scenario, campionati entro il $\pm 50\%$ attorno al preset, con partizione stratificata 60/20/20. Fra random forest, *support vector machine* a kernel RBF calibrata e perceptrone multistrato, la selezione per F1 sulla validation sceglie in entrambi gli scenari la *support vector machine*, valutata una sola volta sul test. Per ogni replica si osserva il numero di iterazioni $\mathcal{M}^{(\pi)} = \min\{m \geq 1 : \pi \text{ riesce all'iterazione } m\}$, posto uguale a $\mathcal{M}_{\max} + 1$ in caso di fallimento, con $\mathcal{M}_{\max} = 50$: una sentinella che codifica esplicitamente la censura. Il carattere calligrafico tiene il conteggio delle iterazioni distinto dalla dimensione $M = 2^n$ del registro. Lo stimando dichiarato a priori, $\rho = \bar{\mathcal{M}}_1/\bar{\mathcal{M}}_2$, resta la misura di effetto; l'inferenza è però affidata alle differenze appaiate $D_i = \mathcal{M}_i^{(A)} - \mathcal{M}_i^{(B)}$, perché un rapporto di medie su una variabile discreta e censurata non ammette test esatto: test dei ranghi con segno di Wilcoxon (zeri trattati con il metodo di Pratt), H_0 di simmetria attorno a zero, $\alpha = 0,05$ e correzione di Holm entro famiglia. Poiché ogni esecuzione Monte Carlo dà un istogramma diverso anche a parametri identici, le tre strategie ne ricevono uno identico a ogni coppia (replica, iterazione): D_i confronta le regole sulla stessa realizzazione, senza variabilità spuria. Un audit separato scarta la variante Y_{16} dell'etichetta — successo entro i primi sedici candidati — perché degenerata a classe unica.

Correzione d'errore: formalismo, metriche e decodifica appresa

Un codice stabilizzatore protegge l'informazione misurando ripetutamente osservabili che lasciano invariati gli stati codificati (il sottogruppo abeliano $\mathcal{S} \subset \mathcal{P}_n$ del gruppo di Pauli): l'esito di quelle misure è la *sindrome* $\sigma(E)$ dell'errore E , e un decoder $\mathcal{D}$ è la regola che dalla sindrome inferisce la correzione. Il decoder fallisce quando la correzione riporta lo stato nel codice applicandovi però un operatore logico non banale, cioè un errore che nessuna sindrome poteva rivelare; la probabilità di quell'evento è il *logical error rate* $p_L(p, d)$, dove d è la distanza del codice, il peso minimo di un errore non rivelabile. Si simulano il codice a ripetizione, con la forma chiusa $p_L = 3p^2 - 2p^3$ come test di correttezza; il codice di Steane [10], con verifica della corrispondenza sindrome→errore sui 21 casi e stima di γ in $p_L \propto p^\gamma$; il surface code ruotato [11]

sotto rumore *circuit-level*, decodificato per matching di peso minimo (MWPM) e descritto dalla legge di scala

$$p_L(p, d) = A (p/p_{\text{th}})^{\lfloor (d+1)/2 \rfloor}, \quad (2)$$

con A e p_{th} adattati ai punti sotto soglia per $d = 3, 5, 7, 9$, così da stimare la soglia indipendentemente dall'incrocio delle curve. Il decoder appreso è un percettore multistrato f_θ sugli stessi *detection events*, le variazioni di sindrome fra cicli consecutivi, confrontato con il matching sia come sostituto sia nella forma ibrida $\mathcal{D}_{\text{hyb}}(\sigma) = \mathcal{D}_{\text{MWPM}}(\sigma) \oplus \mathbf{1}[f_\theta(\sigma) > t]$, in cui la rete, quando segnala che il matching sta sbagliando, ne ribalta la decisione applicando l'operatore logico; la soglia t è scelta in validazione e la significatività dal test di McNemar.

Proxy fenomenologico per la sensibilità di Shor

Dopo ogni gate compilato si applica un canale di Pauli isotropo, che con probabilità p_g sostituisce l'identità con un errore di Pauli non banale scelto uniformemente, ossia $\lambda_1 = \frac{4}{3}p_g$ e $\lambda_2 = \frac{16}{15}p_g$ in forma depolarizzante. È indicato p_g , non p_L : è la probabilità di un Pauli non identità *per gate compilato*, non il tasso logico per ciclo di sindrome della sezione *Correzione d'errore*, e le due unità non sono convertibili senza un modello architetturale esplicito. Su una griglia preregistrata di p_g si misura il successo per singola misura P_s , da cui la cumulata $P(k) = 1 - (1 - P_s)^k$ su k tentativi. Non è una compilazione fault-tolerant: ne risulta un'analisi di sensibilità, non una previsione di prestazione.

Strumenti, verifica e validazione

Qiskit/Aer [12] gestisce i circuiti generali, Stim [13] e PyMatching [14] quelli stabilizzatori e scikit-learn [15] i modelli appresi; ambiente e versioni sono registrati in un formato JSON versionato (input, seed, manifest e hash della netlist). Una demo web separa esecuzione ideale e rumorosa; codice e artefatti sono resi disponibili con la tesi (demo: <https://shor-demo-6knp.onrender.com>). L'aritmetica di Beauregard [16], realizzata anche per $N = 21$ e $N = 35$, è validata nel solo caso ideale: $N = 15$ resta l'unica istanza rumorosa. La verifica è *funzionale* (truth table, azzeramento delle ancille e confronto con la phase estimation teorica: distanza di variazione totale $1,38 \times 10^{-9}$ per $N = 21$), *di identità* (profondità, conteggi e hash) e *statistica* (ipotesi e griglie preregistrate, seed separati, intervalli di Wilson, Bonferroni sulla monotonia,

Holm entro famiglia e censura esplicita). L’ablazione sugli stessi dati resta il controllo causale.

Risultati

Fase 1. Sul circuito verificato ($N = 15$, $a = 7$, profondità 412, 224 CX, hash registrato) i classificatori raggiungono le prestazioni ipotizzate a priori ($F1 = 0,919$ e $0,923$; $AUC = 0,965$ e $0,958$), ma l’ablazione appaiata mostra che il guadagno non è loro. Su 30 repliche per scenario, con successo 30/30 in ogni braccio, la sola ricerca π_4 porta le iterazioni al minimo ($\bar{\mathcal{M}} = 1,00$ contro 1,37 in UC1 e 1,50 in UC2; $p_{\text{Holm}} = 0,0024$ e $0,0015$), mentre il Metodo 2 non migliora la baseline (1,50 e 1,37; $p_{\text{Holm}} = 0,7314$ e $0,0874$) e peggiora la propria ablazione ($p_{\text{Holm}} = 0,0033$ e $0,0096$). Nemmeno la zero-noise extrapolation migliora la baseline, e triplica il costo di campionamento.

Fase 2. I codici riproducono le previsioni a priori, con la stima di p_{th} dall’Eq. (2) concorde con l’incrocio delle curve: la ripetizione a $d = 3$ accorda con $p_L = 3p^2 - 2p^3$ entro l’errore statistico; Steane $[[7, 1, 3]]$ dà $\gamma = 1,94$ e pseudo-soglia (il rumore oltre il quale, a distanza fissata, codificare non conviene) $p \simeq 0,08$; il surface code per $d = 3-9$ dà $p_{\text{th}} = 0,86\%$ (Z) e $0,81\%$ (X), prefattore $A = 3,5 \times 10^{-2}$ e residui di RMS $0,08$ in $\ln p_L$; la decodifica appresa, nelle quattordici configurazioni testate ($d = 3, 5$), non supera mai il matching come sostituto e, come correttore ibrido, riduce p_L del 18–21% a $d = 3$ *soltanto in presenza di errori correlati* — crosstalk fra qubit dati adiacenti, non rappresentabile nel grafo di matching —, dell’1–3% a $d = 5$ e per nulla a $d = 7$.

Chiusura quantitativa. Nel proxy fenomenologico descritto sopra — in unità di p_g , non del p_L appena riportato — gli estremi della curva di sensibilità coincidono con le costanti dell’Eq. (1): il punto senza rumore ($p_g = 0$) vale $0,7489$ $[0,7459; 0,7518]$ e stima $P_{\text{id}} = 3/4$; il plateau è già raggiunto a $p_g \simeq 0,1$ e vale $0,2459$ a $p_g = 0,5$, cioè il pavimento uniforme $P_{\text{unif}} = 63/256 = 0,2461$. La curva non descrive un algoritmo che fallisce, ma uno che degrada verso l’estrazione casuale, con entrambi gli asintoti fissati in anticipo dalla teoria. **Trasformata approssimata.** Una verifica esplorativa condotta a chiusura del lavoro mostra che troncare le rotazioni della Trasformata di Fourier Quantistica (*Quantum Fourier Transform*) (QFT) sotto una soglia d’angolo dimezza le porte a due qubit del circuito per $N = 21$ (da 49 661 a 23 178) e, sulla stima di fase, supera la trasformata piena in tutti e nove i confronti

eseguiti su tre dimensioni di registro, fino a +0,126 su dieci qubit di conteggio. Il grado ottimo di troncamento non cresce con il registro, mentre il risparmio sì.

Discussione e Conclusioni

La domanda di localizzazione ha una risposta a due facce. A valle dell'esecuzione l'ipotesi principale, $\rho \gg 1$, è *smentita* dai suoi stessi dati: il classificatore raggiunge le soglie di F1 e AUC fissate a priori e resta inutile, perché scarta istogrammi che π_4 avrebbe risolto. Sulla sindrome, dove il dato conserva una struttura locale più ricca del solo istogramma finale, la stessa tecnologia guadagna, ma solo nella forma ibrida — la rete affianca il matching invece di sostituirlo — e solo finché il rumore contiene correlazioni che il decoder analitico non rappresenta. Il criterio unificante è verificabile entro il perimetro misurato: un modello appreso aggiunge valore quando sfrutta informazione che il metodo di riferimento lascia inutilizzata, non quando ne replica la regola decisionale. Il braccio negativo dice che cosa non si ottiene con le risorse di questo lavoro, e non si estende ai decoder appresi in generale.

L'autovalutazione riguarda il metodo: senza il braccio π_4 la riduzione da 1,37 a 1,00 sarebbe stata attribuita al classificatore, un falso positivo del tutto presentabile; l'audit dell'etichetta e la verifica per truth table del moltiplicatore, che ha imposto la rigenerazione dell'intera campagna, sono stati più informativi delle metriche di accuratezza.

I limiti delimitano ogni conclusione: il rumore della campagna su Shor è uniforme, indipendente e stazionario, senza topologia, crosstalk, leakage o deriva; $N = 15$ è un'istanza didattica con post-processing permissivo; le stime QEC valgono entro il modello Clifford–Pauli, indicato come potenzialmente ottimistico [17]; l'analisi di sensibilità è un proxy per gate, non una simulazione fault-tolerant. Rispetto al piano, $N = 21$ e $N = 35$ restano validati solo idealmente ed esclusi per costo dalle campagne rumorose, l'accelerazione GPU prevista non è stata impiegata e nessun esperimento è stato eseguito su hardware reale: l'accesso a una macchina è stato valutato e scartato d'accordo con il relatore, sicché tutta la validazione poggia su modelli di rumore e snapshot di calibrazione. Sul fronte delle risorse la letteratura recente percorre l'asse opposto: approssimando l'aritmetica modulare si scende per RSA-2048 a 1730 qubit logici contro 6144, ma pagando $2^{40,9}$ porte Toffoli contro $2^{31,3}$ [18]. Qui si ottimizzano invece le porte, perché nel regime *Noisy Intermediate-Scale Quantum* (NISQ) il vincolo dominante è la profondità rispetto al tempo di coerenza e non il numero di qubit.

Gli sviluppi futuri discendono dai limiti — mappatura per operazione logica, rumore *channel-first*, decoder ricorrenti o a grafo [19] — ma la ricaduta più immediata è metodologica: ablazione appaiata, contratto interpretativo fra unità diverse e artefatti versionati formano un protocollo riutilizzabile per valutare qualunque mitigazione appresa del rumore quantistico.

Indice

Abstract	i
Elenco delle figure	xv
Elenco delle tabelle	xvii
Elenco degli Acronimi	xxi
1 Introduzione	1
1.1 Il limite del calcolo classico	1
1.2 Motivazione e contributo del presente lavoro	2
1.3 Struttura della tesi	3
2 Problema di ricerca, obiettivi e criteri di valutazione	4
2.1 Obiettivi generali	4
2.2 Obiettivi specifici	5
2.3 Ipotesi di lavoro	6
2.4 Contributo originale	7
2.5 Perimetro sperimentale	7
2.6 Metriche e criteri di valutazione	8
2.6.1 Metriche della parte QEC	8
2.7 Articolazione del lavoro di ricerca	9
3 Fondamenti teorici e metodologia sperimentale	10
3.1 Fondamenti del calcolo quantistico	10
3.1.1 Richiami di Calcolo Quantistico	10
3.2 L'algoritmo di Shor	13
3.2.1 Il Problema della Fattorizzazione Intera	13
3.2.2 Riduzione alla Ricerca del Periodo	14

3.2.3	La Quantum Fourier Transform	15
3.2.4	L'Algoritmo di Shor: Schema Completo	19
3.2.5	Analisi della Complessità	22
3.2.6	Stato dell'Arte e Confronto con Altri Algoritmi	22
3.3	Rumore quantistico nei sistemi NISQ	23
3.3.1	Introduzione al Rumore Quantistico	23
3.3.2	La Macchina Fisica: i Tre Fatti che Contano	23
3.3.3	Le Quattro Fonti Fisiche di Rumore	23
3.3.4	Dal Fenomeno Fisico al Modello Matematico	24
3.3.5	I Quattro Canali Matematici del Rumore	25
3.3.6	Impatto del Rumore sull'Algoritmo di Shor	26
3.3.7	Sfide Pratiche su Hardware NISQ	26
3.4	Strategie per la riduzione del rumore quantistico	27
3.4.1	Le Quattro Strategie: Panoramica e Scelta	27
3.4.2	Machine Learning Classico per la Robustezza al Rumore	27
3.4.3	Correzione degli Errori Quantistici e Ruolo del Machine Learning	28
3.4.4	Motivazione Pratica: il Contributo di questa Tesi	28
3.4.5	Una Strategia sul Circuito: la Trasformata di Fourier Approssimata	28
3.4.6	Dall'esito della prima fase alla struttura del lavoro	29
3.5	Disegno e metodo sperimentale	29
3.5.1	Disegno e perimetro sperimentale	29
3.5.2	Pipeline appaiata	30
3.5.3	Modello di rumore e controlli	31
3.5.4	Regole decisionali e analisi statistica	31
3.5.5	Matrice di verifica dell'intero disegno	31
4	Architettura e implementazione del sistema sperimentale	33
4.1	Strumenti e configurazione dell'ambiente	33
4.1.1	Strumenti e ambiente di simulazione	33
4.1.2	Ambiente e contratto operativo	34
4.2	L'algoritmo di Shor: implementazione di riferimento	34
4.3	Configurazione del modello di rumore	35
4.4	Implementazione dei metodi proposti	35
4.4.1	Metodo 1: selezione TOP-1	35
4.4.2	Metodo 2: classificatore ML e ricerca TOP-4	35

4.5	Infrastruttura di test e raccolta dati	36
4.6	Implementazione della correzione d'errore	36
5	Risultati e discussione sperimentale	38
5.1	Fase NISQ: classificatore ML e strategia TOP-K	38
5.1.1	Setup Sperimentale	38
5.1.2	Perimetro Ideale e Rumoroso	38
5.1.3	La Baseline: Metodo 1 (TOP-1)	39
5.1.4	L'Ipotesi ML e il Verdetto dell'Ablazione	39
5.1.5	Robustezza Esplorativa della Strategia TOP-4	40
5.1.6	Bilancio della Verifica e Ponte verso la Correzione d'Errore . .	43
5.2	Correzione d'errore: codici a ripetizione e di Steane	43
5.2.1	Dalla Ridondanza Ingenua alla Codifica	44
5.2.2	Il Ciclo della Correzione: Stabilizzatori, Sindrome, Decoder . .	44
5.2.3	Il Panorama dei Codici e la Scelta Metodologica	44
5.2.4	Laboratorio I: il Codice a Ripetizione	44
5.2.5	Laboratorio II: il Codice di Steane $[[7, 1, 3]]$	46
5.3	Surface code e decodifica	48
5.3.1	Il Codice su Griglia: Concetto Operativo	48
5.3.2	La Decodifica: Minimum Weight Perfect Matching	48
5.3.3	Disegno Sperimentale	48
5.3.4	Risultati	48
5.3.5	Oltre il Modello di Pauli: Errori Coerenti e Leakage	51
5.3.6	La Decodifica Appresa: Sintesi della Campagna	51
5.4	Analisi di sensibilità di Shor	61
5.4.1	L'architettura a quattro livelli	61
5.4.2	Definizione operativa del proxy	61
5.4.3	Il contesto delle stime fault-tolerant	62
5.4.4	Risultati M8 e M13	62
5.5	Ottimizzazione topologica della QFT	65
5.6	Discussione complessiva dei risultati	68
6	Conclusioni e sviluppi futuri	71
6.1	Conclusioni	71
6.1.1	Le Domande di Ricerca	71
6.1.2	Verifica delle Ipotesi	73

6.1.3	Contributi del Lavoro	73
6.1.4	Limiti	74
6.1.5	Il Criterio che Unifica il Lavoro	74
6.2	Sviluppi futuri	75
6.2.1	Scalabilità a Istanze di Maggiore Dimensione	75
6.2.2	Validazione su Hardware Quantistico Reale	76
6.2.3	Generalizzazione ad Altri Algoritmi Quantistici	76
6.2.4	Integrazione con la Quantum Error Correction	76
6.2.5	Evoluzione del Classificatore ML	77
A	Trattazione teorica estesa	78
A.1	Fondamenti del calcolo quantistico	78
A.1.1	Dalla Meccanica Classica alla Meccanica Quantistica	78
A.1.2	I Postulati della Meccanica Quantistica	79
A.1.3	Il Qubit	81
A.1.4	Sistemi Multi-Qubit e Prodotto Tensoriale	82
A.1.5	Porte Quantistiche	83
A.1.6	Circuiti Quantistici	87
A.1.7	Misura Quantistica	87
A.1.8	Interferenza Quantistica	87
A.1.9	Modello di Complessità Quantistica	88
A.2	L'anatomia fisica di una QPU	89
A.2.1	Qubit, Controllo e Vincoli Fisici della QPU	89
A.2.2	Le quattro fonti fisiche: parametri e modelli di misura	94
A.3	L'algoritmo di Shor — approfondimenti e stato dell'arte	96
A.3.1	Sviluppi Recenti e Stato dell'Arte	96
A.3.2	Contesto: Confronto con l'Algoritmo di Grover	99
A.3.3	Sensibilità delle componenti di Shor al rumore	100
A.3.4	Scelta dell'aritmetica e perimetro delle istanze	101
A.4	I canali di rumore: derivazioni complete	102
A.4.1	Canale Bit Flip	102
A.4.2	Canale Phase Flip	102
A.4.3	Canale di Amplitude Damping	103
A.4.4	Confronto geometrico dei quattro canali	103
A.5	Strategie anti-rumore e richiami estesi di QEC	105
A.5.1	I quattro livelli di intervento	105

A.5.2	Usi possibili dei metodi appresi	106
A.5.3	Principio, codici e decoder QEC	106
A.5.4	Verifica del filtro appreso nella prima fase	107
A.5.5	AQFT e scambio fra qubit e porte	107
A.5.6	Transizione fra le due fasi sperimentali	108
A.5.7	Dettagli tecnici del contratto sperimentale	108
B	Strumenti, ambiente e codice	109
B.1	Strumenti e ambiente di simulazione — trattazione estesa	109
B.1.1	Qiskit: il Framework Quantistico di Riferimento	109
B.1.2	Qiskit Aer: il Simulatore	111
B.1.3	Accelerazione GPU con Qiskit Aer	113
B.1.4	scikit-learn: la Libreria per il Classificatore	114
B.1.5	Ambiente di Esecuzione: WSL con Ubuntu	114
B.2	Codice sorgente dei componenti implementati	115
B.2.1	Contratto implementativo e riproducibilità	115
B.2.2	Organizzazione completa del codice sperimentale	116
B.2.3	Dettagli implementativi dei componenti QEC	117
B.2.4	Estratti dei sorgenti	118
C	Contesto e motivazione	126
C.1	Contesto esteso dell'introduzione	126
C.1.1	Approfondimento della Sezione 1.1: dal Limite Classico al Paradigma Quantistico	126
C.1.2	Dal Bit al Qubit: i Principi Fisici del Calcolo Quantistico . . .	127
C.1.3	La Rivoluzione Algoritmica degli Anni Novanta	131
C.1.4	RSA e le Implicazioni Crittografiche dell'Algoritmo di Shor . .	132
C.1.5	Crittografia Post-Quantistica: la Risposta dell'Industria	135
C.1.6	Lo Stato dell'Arte: dall'Era NISQ ai Computer Fault-Tolerant	137
C.1.7	Approfondimento della Sezione 1.2: Filo delle Due Fasi e Risultati Guida	138
C.1.8	Approfondimento della Sezione 1.3: Struttura Estesa della Tesi	139
C.2	Dettaglio degli obiettivi e del piano sperimentale	142
C.2.1	Motivazione: la Vulnerabilità della Crittografia Attuale	142
C.2.2	Approfondimento della Sezione 2.2: Obiettivi Specifici	143

C.2.3	Approfondimento della Sezione 2.4: Contributi e Confini Interpretativi	144
C.2.4	Requisiti Funzionali	145
C.2.5	Approfondimento della Sezione 2.5: Campagne e Validazioni .	146
C.2.6	Parametri di Input e Output	148
C.2.7	Approfondimento della Sezione 2.6: Definizioni Complete delle Metriche	149
C.2.8	Approfondimento della Sezione 2.3: Protocollo Statistico e Test	151
D	Fase 1: la campagna classica nel dettaglio	152
D.1	Campagna parametrica e confronto con la Zero-Noise Extrapolation .	152
D.1.1	Contratto sperimentale e perimetro delle validazioni	152
D.1.2	Variazione di K	154
D.1.3	Variazione di λ_{2q}	154
D.1.4	Variazione degli Shot	155
D.1.5	Analisi Congiunta $K \times \lambda_{2q}$	155
D.1.6	Parametri Secondari	156
D.1.7	Livello di Ottimizzazione	157
D.1.8	Diagnostiche della struttura QPE	157
D.1.9	Confronto con Zero-Noise Extrapolation	157
D.2	Il classificatore ML: architettura, addestramento e metriche	158
D.2.1	L’Ipotesi alla Prova: il Classificatore ML	158
D.2.2	Audit delle etichette TOP-1 e TOP-16	159
E	Fase 2: la campagna QEC nel dettaglio	163
E.1	Laboratori preliminari QEC: codici a ripetizione e di Steane	163
E.1.1	Dalla ridondanza alla codifica per stabilizzatori	163
E.1.2	Protocollo completo del codice a ripetizione	164
E.1.3	Struttura e protocollo completo del codice di Steane	165
E.2	Surface code: protocollo e controlli sul modello	166
E.2.1	Surface code e decodifica MWPM	166
E.2.2	Griglia sperimentale e riproducibilità	167
E.2.3	Errori coerenti, leakage e controllo full-state	168
E.3	La decodifica appresa: la campagna completa	169
E.3.1	E6 — modello del decoder errato?	172
E.3.2	E7 — confronto con un decoder analitico migliore	174

E.3.3	E8/E10 — rumori combinati e pavimento comune	177
E.3.4	E5 — il decoder è trasferibile fra dispositivi?	181
E.3.5	Bilancio	182
E.4	Analisi di sensibilità di Shor: dettagli e verifiche	183
E.4.1	Separazione dei quattro livelli	184
E.4.2	Contesto delle stime fault-tolerant	184
E.4.3	Griglia M8 completa e probabilità cumulativa	185
E.4.4	Perché le metriche QEC non entrano direttamente in M8 . . .	185
E.5	Compilazione consapevole della struttura: un'esplorazione	186
E.5.1	Previsione e selezione non sono la stessa domanda	186
E.5.2	M11: pilota sui layout	186
E.5.3	M11b: readout effettivo dopo il routing	191
E.5.4	M13: TOP-4 sotto il proxy per gate	192
E.5.5	Conclusione e limiti	193

Bibliografia

195

Elenco delle figure

3.1	La sfera di Bloch	11
3.2	Circuito per lo stato di Bell $ \Phi^+\rangle$	12
3.3	Periodicità di $f(x) = 7^x \bmod 15$	15
3.4	Circuito della QFT su 3 qubit	16
3.5	Circuito QPE per $N = 15$, $a = 2$	17
3.6	Blocco di moltiplicazione modulare U_2 per $N = 15$	18
3.7	Blocco di moltiplicazione modulare U_4 per $N = 15$	18
3.8	Istogramma QPE ideale per $N = 15$	19
3.9	Schema a due registri dell'algoritmo di Shor	20
3.10	Dove colpisce il rumore nella pipeline di Shor	26
3.11	Ogni istogramma è riutilizzato dalle tre strategie: le differenze non dipendono da realizzazioni Monte Carlo diverse.	30
5.1	Analisi di ablazione M1, TOP-4 e M2	40
5.2	Repliche UC1 con massa massima e minima sui quattro picchi QPE teorici.	41
5.3	Il proxy di nessun evento decade più rapidamente della massa strutturata osservata sui picchi.	42
5.4	Errore logico del codice a ripetizione a 3 qubit	45
5.5	Errore logico del codice di Steane $[[7, 1, 3]]$	47
5.6	Comportamento a soglia del surface code nelle due basi	49
5.7	Decodifica appresa a parità di modello di rumore	55
5.8	Decodifica appresa in presenza di rumore correlato	57
5.9	Precisione e copertura del flag di validazione	58
5.10	Il decoder ibrido a confronto con matching e rete	60
5.11	Shor sotto un proxy fenomenologico per gate	63
5.12	Probabilità cumulativa dai punti M8	64
A.1	Porte quantistiche a singolo e due qubit	86

A.2	Gerarchia delle classi di complessità quantistica	89
A.3	Anatomia schematica di una QPU superconduttiva a 5 qubit	90
A.4	La QPU reale: sistema integrato e criostati con cablaggio	91
A.5	Chip quantistico superconduttivo a 6 transmon con giunzioni di test	92
A.6	Il bordo di coerenza: budget di operazioni e rischio di misura	94
A.7	Proxy di nessun evento al crescere del circuito	101
A.8	Effetto dei canali di rumore sulla sfera di Bloch	104
D.1	Curve ROC dei classificatori TOP-1 selezionati	159
D.2	Istogramma TOP-1 negativo ma recuperabile con TOP-4	160
D.3	Tetto combinatorio alla classe negativa	162
E.1	Errore coerente vs errore di Pauli	168
E.2	Ortogonalità delle due sorgenti di guadagno	176
E.3	Layout M11: selezione contro holdout	189
E.4	Collocazione dei layout selezionati sulla coupling map	190

Elenco delle tabelle

2.1	Sintesi del perimetro, delle metriche e dei criteri di valutazione.	8
3.1	Le porte quantistiche essenziali	12
3.2	Fonti fisiche di rumore nei processori superconduttori; gli intervalli sono ordini di grandezza indicativi, non parametri dei preset sperimentali.	23
3.3	Corrispondenza operativa tra fonti fisiche e canali simulabili.	24
3.4	Confronto delle strategie anti-rumore nel perimetro della tesi.	27
3.5	Contratto essenziale delle campagne rumorose su $N = 15$	30
3.6	Corrispondenza fra componenti, controlli e criteri di evidenza.	32
5.1	Confronto appaiato M1, TOP-4 e M2 (30 repliche per scenario)	39
5.2	Sintesi degli otto sweep schema 2 su $N = 15$	41
5.3	Massa efficace sui picchi e proxy di nessun evento 2Q.	42
5.4	Distanza e costo in qubit richiesti dalla legge di scala	50
5.5	Sintesi della campagna sulla decodifica appresa	52
5.6	Disegno sperimentale della decodifica appresa	54
5.7	Decodifica in presenza di rumore correlato	56
5.8	La rete come validatore della correzione del matching	58
5.9	Prestazioni del decoder ibrido	59
5.10	M8: probabilità di ricavare i fattori per singola misura, 20 repliche da 4096 shot per punto.	62
6.1	Esito delle ipotesi	73
A.1	Confronto tra tecnologie di QPU: superconduttori vs ioni intrappolati	93
A.2	Confronto tra l'algoritmo di Shor e l'algoritmo di Grover	99
A.3	Confronto fra i quattro canali fondamentali [8].	104
B.1	Confronto tra i principali framework per la simulazione quantistica con rumore.	111

B.2	Organizzazione del codice sperimentale	117
C.1	Preset uniformi illustrativi per le campagne rumorose N15. Non rappresentano la calibrazione di una QPU reale.	147
C.2	Parametri principali del sistema sperimentale. λ segue la convenzione Aer e non coincide con la probabilità aggregata di Pauli non identità.	149
D.1	Contratto riproducibile della campagna N=15	153
D.2	Perimetro delle validazioni	154
D.3	Sweep del numero di candidati	154
D.4	Sweep del parametro Aer λ_{2q}	155
D.5	Sweep degli shot	155
D.6	Griglia $K \times \lambda_{2q}$	155
D.7	Sweep di T_1/T_2	156
D.8	Sweep di λ_{1q}	156
D.9	Sweep del readout simmetrico	156
D.10	Sweep di <code>optimization_level</code>	157
D.11	Confronto ZNE appaiato	157
D.12	Metriche TOP-1 sul test set indipendente (400 istogrammi per scenario)	158
D.13	Audit delle etichette TOP-1 e TOP-16	159
D.14	Distribuzione della moda su UC1	160
D.15	Tetto combinatorio alla classe negativa in funzione di K	161
E.1	Confronto tra codici correttori quantistici	164
E.2	Verifica della sindrome del codice a ripetizione	165
E.3	Parametri del codice di Steane	165
E.4	Syndrome table del codice di Steane	166
E.5	Disegno sperimentale surface code	167
E.6	Larghezza dell'ingresso contro distanza del codice	170
E.7	Discriminazione e guadagno al crescere della larghezza dell'ingresso	171
E.8	Effetto di un modello di rumore mis-calibrato sul MWPM	173
E.9	BP+OSD contro MWPM e contro il decoder ibrido	175
E.10	Il residuo appreso costruito sopra BP+OSD	178
E.11	Sei decoder, un solo pavimento	179
E.12	Il pavimento alla prova di modelli più potenti	180
E.13	Trasferibilità del decoder appreso fra profili di rumore	181
E.14	Architettura a quattro livelli della parte sperimentale	184

E.15 Traiettoria delle stime di risorsa per Shor su scala crittografica	184
E.16 Contratto tra risultati QEC e curva M8	185
E.17 Disegno del pilota sui layout	187
E.18 Stimandi del pilota M11	187
E.19 Disegno osservazionale M11b	191
E.20 Strategie descrittive di M11b	191
E.21 Endpoint preregistrati di M13	193

Elenco degli Acronimi

AQFT Trasformata di Fourier Quantistica Approssimata (*Approximate Quantum Fourier Transform*)

AUC Area Sotto la Curva ROC (*Area Under the Curve*)

BQP *Bounded-error Quantum Polynomial time*

CCNOT Controlled-Controlled NOT (porta di Toffoli)

CNOT Controlled-NOT

CPTP *Completely Positive Trace-Preserving*

CPU Unità di Elaborazione Centrale (*Central Processing Unit*)

CSS Calderbank-Shor-Steane

CX Controlled-X (porta CNOT)

DEM *Detector Error Model*

DFT Trasformata Discreta di Fourier (*Discrete Fourier Transform*)

ECDSA *Elliptic Curve Digital Signature Algorithm*

GCD Massimo Comune Divisore (*Greatest Common Divisor*)

GNFS *General Number Field Sieve*

GPU Unità di Elaborazione Grafica (*Graphics Processing Unit*)

HTTPS *HyperText Transfer Protocol Secure*

IBM International Business Machines

ML Apprendimento Automatico (*Machine Learning*)

MLP *Percetttrone Multistrato (Multi-Layer Perceptron)*

MWPM *Minimum Weight Perfect Matching*

NISQ *Noisy Intermediate-Scale Quantum*

NIST *National Institute of Standards and Technology*

NMR *Risonanza Magnetica Nucleare (Nuclear Magnetic Resonance)*

PEC *Probabilistic Error Cancellation*

PQC *Post-Quantum Cryptography*

QEC *Correzione degli Errori Quantistici (Quantum Error Correction)*

QFT *Trasformata di Fourier Quantistica (Quantum Fourier Transform)*

QMA *Quantum Merlin-Arthur*

QML *Quantum Machine Learning*

QPE *Quantum Phase Estimation*

QPU *Unità di Elaborazione Quantistica (Quantum Processing Unit)*

RSA *Rivest-Shamir-Adleman*

SDK *Software Development Kit*

SSH *Secure Shell*

SVM *Support Vector Machine*

WSL *Windows Subsystem for Linux*

ZNE *Zero-Noise Extrapolation*

Capitolo 1

Introduzione

1.1 Il limite del calcolo classico

La crescita dell'hardware classico, descritta per decenni dalla Legge di Moore, ha moltiplicato la potenza disponibile senza eliminare i limiti di scala: i transistor si avvicinano alle dimensioni atomiche e, soprattutto, alcuni problemi restano difficili per ragioni di complessità, non per insufficienza dei processori. La fattorizzazione di grandi interi, la ricerca non strutturata e la simulazione di sistemi quantistici sono esempi in cui aumentare soltanto le risorse classiche non modifica la natura della crescita computazionale.

Il calcolo quantistico nasce dall'idea di usare sovrapposizione, entanglement e interferenza come risorse. Feynman osservò che simulare sistemi quantistici generici può richiedere risorse classiche esponenziali [20]; Deutsch formalizzò poi il modello di calcolatore quantistico universale [21]. Negli anni Novanta, Shor mostrò che fattorizzazione e logaritmo discreto possono essere affrontati in tempo polinomiale su una macchina quantistica ideale [1]. Il risultato espone una vulnerabilità strutturale di Rivest-Shamir-Adleman (RSA) e di altre primitive a chiave pubblica, ma tra l'algoritmo teorico e un attacco crittograficamente rilevante restano profondità circuitale, qubit logici, overhead fault-tolerant e rumore.

La rilevanza non dipende soltanto dall'arrivo futuro di una macchina capace di fattorizzare RSA-2048. Dati cifrati oggi possono essere conservati per una decifratura successiva, mentre la sostituzione delle primitive vulnerabili richiede anni di inventario, aggiornamento e interoperabilità. Gli standard post-quantistici rispondono a questo rischio sul piano crittografico; sul piano del calcolo quantistico resta necessario misurare quanto separi i circuiti dimostrativi da un'esecuzione affidabile. L'era NISQ

rende visibile questa separazione: il numero nominale di qubit non compensa fedeltà limitate, connettività, accumulo degli errori e assenza di protezione logica scalabile.

Questa distanza fra potenziale algoritmico e realizzabilità costituisce il punto di partenza della tesi. Il contesto storico e fisico, dai qubit superconduttori alla rivoluzione algoritmica, è approfondito nelle Sezioni C.1.1–C.1.3; RSA, la transizione post-quantistica e il passaggio dall’era NISQ ai sistemi fault-tolerant sono trattati nelle Sezioni C.1.4–C.1.6. I fondamenti formali necessari all’analisi sono invece raccolti nella Sezione 3.1.1.

1.2 Motivazione e contributo del presente lavoro

L’algoritmo di Shor è quindi un banco di prova particolarmente esigente: la sua complessità asintotica minaccia la crittografia moderna, mentre i circuiti profondi che ne realizzano il *period finding* sono sensibili a decoerenza, errori di gate e misura. Il problema affrontato è comprendere quali interventi conservino informazione utile in presenza di rumore e, in particolare, *dove* un modello appreso possa aggiungere valore rispetto a una regola analitica trasparente.

Il divario metodologico studiato è fra due livelli d’intervento. Il post-processing può sfruttare meglio l’informazione sopravvissuta nella distribuzione di misura, ma non modifica lo stato durante il circuito. La correzione quantistica usa invece ridondanza ed estrazione di sindrome per rilevare errori senza leggere direttamente l’informazione logica, pagando un costo in qubit, gate e decodifica. Valutare entrambi i livelli sotto baseline esplicite evita di attribuire a una tecnica un miglioramento dovuto al diverso insieme di candidati, ai seed o a metriche non confrontabili.

Il lavoro procede in **due fasi**. La prima studia il post-processing dell’output di Shor su simulatore, con due preset di rumore uniformi e illustrativi che non rappresentano la calibrazione di una specifica Unità di Elaborazione Quantistica (*Quantum Processing Unit*) (QPU). Sugli stessi istogrammi vengono confrontati la baseline TOP-1 (Metodo 1), la ricerca TOP-4 e un Metodo 2 che antepone a TOP-4 un classificatore Apprendimento Automatico (*Machine Learning*) (ML). L’appaiamento conserva per le tre regole lo stesso rumore campionato, mentre l’ablazione impedisce di confondere il contributo del classificatore con quello della ricerca multi-candidato. I dati mostrano che il filtro appreso non migliora significativamente la baseline e risulta peggiore di TOP-4 senza classificatore. Il risultato non nega che l’istogramma contenga candidati utili; delimita invece ciò che può essere recuperato *dopo* che il rumore ha agito. Il protocollo e i valori numerici anticipati da questa motivazione

sono raccolti, con l'indicazione della loro origine, nella Sezione C.1.7; la verifica completa è nel Capitolo 5.

La **seconda fase**, nucleo della tesi, sposta perciò l'intervento durante il calcolo. Costruisce e simula una progressione di codici di Correzione degli Errori Quantistici (*Quantum Error Correction*) (QEC) — ripetizione, Steane $[[7, 1, 3]]$ e surface code con decodifica *Minimum Weight Perfect Matching* (MWPM) — e misura il *logical error rate* p_L in funzione dell'errore fisico p e della distanza d . Una successiva analisi applica a Shor un proxy Pauli fenomenologico per gate compilato, mantenendolo distinto dal tasso di fallimento per ciclo o per memoria misurato nei benchmark QEC.

L'apprendimento automatico viene infine valutato nel punto in cui riceve informazione diagnostica diretta: la **sindrome**. Un decoder appreso che sostituisce MWPM non offre un vantaggio stabile a parità di modello; un decoder **ibrido**, che apprende soltanto quando ribaltare la decisione del matching, riduce invece l'errore logico fino al 21% nel modello simulato con correlazioni non rappresentate dal grafo analitico e ricade sul riferimento quando il rumore coincide con quello previsto. Ne emerge il contributo unificante: l'ML è utile non per il solo fatto di essere applicato, ma quando accede a struttura informativa pertinente che il metodo di riferimento non rappresenta.

Il problema di ricerca, le sei domande, le ipotesi e il contratto sperimentale sono definiti nel Capitolo 2. Codice, sorgenti L^AT_EX, seed e artefatti sono disponibili nel repository pubblico <https://github.com/claudio-dragotta/shor-nisq-qec-thesis>; la demo interattiva di Shor è all'indirizzo <https://shor-demo-6knp.onrender.com> (sorgenti: <https://github.com/claudio-dragotta/shor-demo>).

1.3 Struttura della tesi

L'elaborato comprende sei capitoli e cinque appendici: dopo questa introduzione, il Capitolo 2 formula problema e criteri; il Capitolo 3 presenta teoria e metodologia; il Capitolo 4 descrive l'implementazione; il Capitolo 5 espone e discute le due campagne, l'analisi di sensibilità e la QFT approssimata; il Capitolo 6 risponde alle domande di ricerca. Le Appendici A–E raccolgono, rispettivamente, teoria estesa, strumenti e codice, contesto e obiettivi, dettagli della fase classica e della fase QEC. La roadmap completa, con il contenuto referenziabile di ciascun capitolo e appendice, è trasferita nella Sezione C.1.8.

Capitolo 2

Problema di ricerca, obiettivi e criteri di valutazione

Il Capitolo 1 ha individuato il divario fra la rilevanza teorica di Shor e la sua realizzabilità su dispositivi rumorosi. Questo capitolo formula il problema di ricerca e fissa il contratto essenziale con cui vengono valutati gli esperimenti; obiettivi analitici, requisiti, configurazioni e protocolli completi sono trasferiti nella Sezione C.2. La motivazione crittografica estesa — confronto fra *General Number Field Sieve* (GNFS) e Shor, vulnerabilità di RSA e rischio *harvest now, decrypt later* — è nella Sezione C.2.1.

2.1 Obiettivi generali

L'obiettivo generale è analizzare l'algoritmo di Shor e stabilire come il rumore limiti la sua esecuzione, distinguendo il potenziale computazionale asintotico dalle condizioni necessarie per ottenerlo. Il lavoro non assume che una tecnica sia utile per definizione: ne misura il contributo rispetto a baseline esplicite e sotto unità coerenti.

Il disegno segue due fasi. La prima verifica se un classificatore applicato a valle dell'esecuzione riduca il costo rispetto alla baseline TOP-1 e alla sola ricerca TOP-4. La seconda studia la protezione durante l'esecuzione mediante codici a ripetizione, Steane e surface code, quindi confronta decoder analitici e appresi sulla sindrome. Un'analisi separata misura la sensibilità di Shor a un proxy Pauli per gate compilato senza identificarlo con il tasso logico per ciclo o per memoria dei benchmark QEC.

2.2 Obiettivi specifici

Gli obiettivi operativi sono:

1. fissare i fondamenti matematici e fisici indispensabili a leggere circuiti e rumore quantistici;
2. descrivere Shor attraverso *period finding*, QFT, phase estimation, complessità e implicazioni per RSA;
3. implementare e validare in Qiskit la QFT, l'esponenziazione modulare e il circuito di fattorizzazione sulle istanze comprese nel perimetro;
4. caratterizzare decoerenza, errori di gate e di misura, distinguendo mitigazione e correzione d'errore;
5. determinare *dove* un modello appreso aggiunga valore, confrontandone l'uso sull'istogramma finale e sulla sindrome;
6. misurare soppressione, break-even e soglia di ripetizione, Steane $[[7, 1, 3]]$ e surface code, quindi collegare i risultati a Shor soltanto tramite un'analisi di sensibilità dichiaratamente fenomenologica.

La formulazione estesa, con attività, output attesi e limiti di ciascun obiettivo, è riportata nella Sezione C.2.2 come approfondimento di questa sezione.

La parte sperimentale risponde alle seguenti **domande di ricerca**:

- DR1.** Come degrada la probabilità di successo di Shor al crescere dell'errore fisico sui gate, e quale fonte di rumore è la più dannosa?
- DR2.** In che modo un codice a ripetizione permette di introdurre il concetto di sindrome senza collassare lo stato logico?
- DR3.** Quali vantaggi e limiti offre il codice di Steane $[[7, 1, 3]]$ come primo codice capace di correggere un errore arbitrario su un qubit?
- DR4.** Perché il surface code è più rilevante per scenari scalabili, e come si stima il suo logical error rate tramite decodifica? Esiste un comportamento di soglia osservabile?

DR5. Come varia il successo di Shor al crescere di un proxy Pauli uniforme per gate, e quante ripetizioni indipendenti servono per superare una soglia operativa prefissata?

DR6. Esiste un regime in cui un decoder *appreso dai dati* supera il decoder analitico standard, e da che cosa dipende? In particolare: il vantaggio dipende dalla qualità della calibrazione del modello di rumore, oppure dalla sua rappresentabilità nella struttura del decoder?

Le prime cinque domande seguono una progressione: Shor, rumore, codifica, decodifica e integrazione. DR6 chiude il confronto fra i due impieghi dell'apprendimento automatico.

2.3 Ipotesi di lavoro

Ipotesi principale. Il classificatore del Metodo 2 riduce significativamente il numero medio di iterazioni rispetto al Metodo 1 sugli stessi istogrammi. Il fattore descrittivo è

$$\rho = \frac{\bar{M}_1}{\bar{M}_2} \gg 1, \quad (2.1)$$

mentre l'inferenza riguarda le differenze appaiate fra repliche. **Ipotesi secondaria 1:** il classificatore distingue esecuzioni corrette ed errate con $F1 > 0.85$ e $AUC > 0.90$ sul test indipendente. **Ipotesi secondaria 2:** l'eventuale vantaggio si mantiene dal preset di riferimento a quello di stress sul medesimo circuito $N = 15$; $N = 21$ resta una validazione ideale dell'aritmetica e delle risorse e non entra nella verifica inferenziale.

La fase QEC verifica tre ulteriori ipotesi. **QEC 1, soppressione:** per i codici di distanza $d = 3$, $p_L \propto p^2$ a piccolo p ed esiste un regime $p_L < p$. **QEC 2, soglia:** per il surface code le curve $p_L(p)$ a distanze crescenti si incrociano in prossimità di p_{th} ; sotto tale valore aumentare d riduce p_L . **QEC 3, sensibilità:** nel proxy M8 il successo di Shor è compatibile con una funzione non crescente del parametro Pauli per gate e le ripetizioni indipendenti possono superare una soglia cumulativa non raggiunta dal singolo run ideale. Quest'ultima ipotesi non converte direttamente i benchmark Steane o surface code.

Test, trattamento dei fallimenti, pareggi, correzioni per confronti multipli e criteri di accettazione sono specificati nella Sezione C.2.8.

2.4 Contributo originale

Il contributo originale è articolato in quattro risultati metodologici:

- **TOP-K e ablazione:** M1, TOP-4 e M2 ricevono lo stesso istogramma, così il confronto separa la ricerca multi-candidato dal filtro ML senza introdurre una diversa realizzazione Monte Carlo;
- **quantificazione riproducibile:** il fattore ρ e i confronti appaiati sono calcolati sui due preset $N = 15$ usando gli artefatti schema 2 successivi alla correzione del moltiplicatore $\times 7 \bmod 15$; $N = 21$ fornisce un controllo ideale separato;
- **caratterizzazione a livelli:** ripetizione, Steane e surface code sono valutati nelle rispettive unità QEC, mentre M8 misura separatamente la sensibilità di Shor a un proxy per gate; senza compilazione fault-tolerant tali grandezze non vengono sovrapposte;
- **criterio di utilità dell'apprendimento:** l'impiego sull'output finale e quello sulla sindrome sono entrambi confrontati con una baseline analitica a parità di informazione, delimitando il regime in cui il modello appreso produce un guadagno misurabile.

I requisiti verificabili e gli input/output che rendono operativi questi contributi sono nelle Sezioni C.2.4 e C.2.6.

2.5 Perimetro sperimentale

Il corpo si concentra su due istanze con ruoli distinti. $N = 15$, $a = 7$ alimenta le due campagne rumorose uniformi, *referimento* e *stress*, sulle quali vengono confrontati M1, TOP-4 e M2. $N = 21$, $a = 2$ è invece usata soltanto per validare idealmente l'aritmetica reversibile di Beauregard, il ripristino degli ancilla, la QPE e il costo compilato; non le vengono attribuiti risultati rumorosi. I preset numerici, il razionale completo delle istanze e gli audit ulteriori sono trasferiti nelle Sezioni C.2.5 e C.2.5.

2.6 Metriche e criteri di valutazione

Esperimento	Confronto	Metrica primaria	Criterio
Shor rumoroso, $N = 15$	M1–TOP-4–M2 appaiati	iterazioni, successo entro $M_{\max}, \rho$	vantaggio significativo e non solo descrittivo
Validazione $N = 21$	aritmetica e QPE ideali	correttezza, ancilla, costo compilato	trasformazione esatta; nessuna inferenza rumorosa
Codici QEC	fisico–logico e distanze d	p_L , pendenza, break-even, p_{th}	$p_L < p$ e soppressione al crescere di d sotto soglia
Decoder di sindrome	appreso/ibrido–MWPM	differenza di p_L sugli stessi campioni	guadagno riproducibile nel regime dichiarato
Shor M8	livelli del proxy Pauli	$P_{\text{success}}(p_L)$ e $P(k)$	sensibilità non crescente, senza conversione QEC diretta

Tabella 2.1: Sintesi del perimetro, delle metriche e dei criteri di valutazione.

Per M1 e M2, $\bar{M}$ è accompagnata dal tasso di successo e dalla dispersione; il test usa vettori appaiati completi e conserva i fallimenti entro il budget. Accuratezza, F1 e AUC valutano il classificatore ma non sostituiscono la metrica operativa. Il readout error è incluso nel noise model, senza correzione post-hoc; la ricerca TOP-4 costituisce l’ablazione che isola il contributo del classificatore. Definizioni, preset, seed, gestione dei pareggi e protocollo statistico completo sono nella Sezione C.2.8.

2.6.1 Metriche della parte QEC

Il *logical error rate* p_L è stimato con numero di campioni, seed e incertezza binomiale dichiarati. Pendenza di soppressione, break-even e soglia descrivono i codici; in M8 lo stesso simbolo indica esplicitamente la probabilità totale di un Pauli non identità dopo ciascun gate incluso nel proxy. La distinzione delle unità impedisce di trasformare un memory experiment in una previsione hardware non giustificata. Le definizioni operative complete sono nella Sezione C.2.7.

2.7 Articolazione del lavoro di ricerca

Il Capitolo 3 sviluppa teoria e metodologia, il Capitolo 4 traduce il contratto nell'architettura sperimentale e il Capitolo 5 valuta ipotesi e domande sui dati; la roadmap analitica dell'intero elaborato è nella Sezione C.1.8.

Capitolo 3

Fondamenti teorici e metodologia sperimentale

3.1 Fondamenti del calcolo quantistico

Questa sezione richiama soltanto gli elementi di calcolo quantistico necessari a seguire Shor, il modello di rumore e i codici correttori: stato del qubit, rappresentazione di Bloch, porte essenziali, algebra di Pauli, misura e interferenza. La costruzione assiomatica completa, il prodotto tensoriale, l'insieme esteso delle porte e il modello di complessità sono raccolti nella Sezione A.1 dell'Appendice A.

3.1.1 Richiami di Calcolo Quantistico

L'algoritmo di Shor [1] fattorizza un intero N in tempo polinomiale nel numero di bit,

$$T_{\text{Shor}}(N) = O((\log N)^3), \quad (3.1)$$

combinando riduzione classica al *period finding*, nucleo quantistico basato su *Quantum Phase Estimation* (QPE) e QFT, e post-elaborazione classica. La sezione successiva sviluppa questi passaggi; qui si fissano le primitive che essi utilizzano.

Qubit, Sovrapposizione e Sfera di Bloch

Un qubit è un vettore unitario di $\mathbb{C}^2$,

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1, \quad (3.2)$$

dove la regola di Born associa $|\alpha|^2$ e $|\beta|^2$ alle probabilità dei due esiti. Eliminata la fase globale, ogni stato puro ammette la parametrizzazione

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right) |0\rangle + e^{i\phi} \sin\left(\frac{\theta}{2}\right) |1\rangle, \quad \theta \in [0, \pi], \quad \phi \in [0, 2\pi), \quad (3.3)$$

visualizzata dalla sfera di Bloch in Figura 3.1. I poli sono $|0\rangle$ e $|1\rangle$; l'equatore raccoglie le sovrapposizioni equiprobabili con fasi diverse.

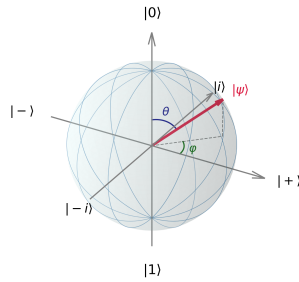


Figura 3.1: Rappresentazione di uno stato puro mediante gli angoli della (3.3). Nel Capitolo 3 i canali sono descritti come contrazioni o traslazioni di questa sfera.

Per n qubit lo spazio diventa $\mathbb{C}^{2^n}$. Uno stato è *entangled* quando non può essere fattorizzato negli stati dei singoli qubit; in Shor questa correlazione lega registro di controllo e registro di lavoro durante la stima di fase. La dimensione esponenziale consente tali correlazioni ma, da sola, non prova un vantaggio computazionale. Definizioni, stati di Bell e fragilità dell'entanglement sono approfonditi nella Sezione A.1.4.

Uno stato generale del registro ha la forma

$$|\psi\rangle = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle, \quad \sum_x |\alpha_x|^2 = 1. \quad (3.4)$$

L'esempio minimo è lo stato di Bell $|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$, che non ammette una fattorizzazione $|\psi_A\rangle \otimes |\psi_B\rangle$. Il circuito della Figura 3.2 mostra che una Hadamard crea le due ampiezze e una CNOT le correla: misurare il primo qubit determina così anche il secondo.

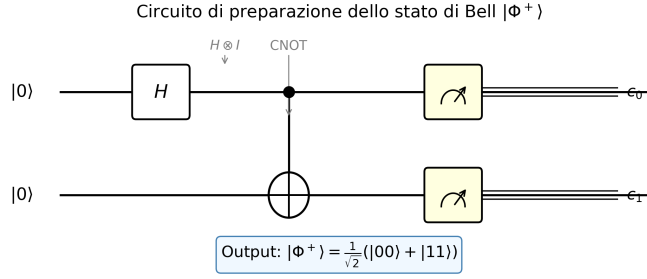


Figura 3.2: Preparazione di $|\Phi^+\rangle$. La porta H crea la sovrapposizione e la CNOT introduce la correlazione non separabile fra i due qubit.

Per Shor l'entanglement non è soltanto un esempio formale: trasferisce la periodicità dal registro di lavoro a quello di controllo durante la QPE. L'accoppiamento con l'ambiente trasferisce invece informazione di fase fuori dal sistema e riduce l'interferenza osservabile. Questa fragilità collega direttamente il richiamo astratto ai tempi T_1 e T_2 e ai canali del rumore discussi nelle sezioni successive.

Porte, Algebra di Pauli e Interferenza

Le porte sono trasformazioni unitarie ($U^\dagger U = I$). La Tabella 3.1 contiene le sei primitive richiamate nel seguito; matrici, circuiti e universalità sono sviluppati nell'Appendice A.

Porta	Matrice	Qubit	Effetto
X	$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$	1	<i>bit flip</i>
Y	$\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$	1	bit e phase flip
Z	$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$	1	<i>phase flip</i>
H	$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$	1	sovrapposizione uniforme
R_ϕ	$\begin{pmatrix} 1 & 0 \\ 0 & e^{i\phi} \end{pmatrix}$	1	rotazione di fase
CNOT	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$	2	nega il target se il control è 1

Tabella 3.1: Le porte impiegate nella trattazione: X, Y, Z descrivono gli errori di Pauli; H e R_ϕ costruiscono la QFT; la CNOT domina il costo a due qubit.

Insieme a I , le matrici di Pauli formano una base degli operatori su un qubit. Poiché $Y = iXZ$, correggere le componenti X e Z permette, per linearità, di trattare anche le loro combinazioni. Le relazioni

$$\begin{aligned} X^2 &= Y^2 = Z^2 = I, \\ XY &= iZ, \quad YZ = iX, \quad ZX = iY, \\ \{X, Y\} &= \{Y, Z\} = \{X, Z\} = 0 \end{aligned} \tag{3.5}$$

spiegano il criterio di sindrome: un errore è rilevato quando anticommuta con almeno uno stabilizzatore misurato. Nei codici Calderbank-Shor-Steane (CSS), stabilizzatori di tipo X e Z diagnosticano separatamente le due componenti.

La misura di $|\psi\rangle = \sum_x \alpha_x |x\rangle$ restituisce x con probabilità $|\alpha_x|^2$ ed è irreversibile. Prima della misura, un algoritmo utile orchestra l'**interferenza**: cancella le ampiezze degli esiti errati e rinforza quelle corrette. In Shor è la QFT a rendere osservabile, mediante questo meccanismo, la periodicità cercata.

3.2 L'algoritmo di Shor

3.2.1 Il Problema della Fattorizzazione Intera

La **fattorizzazione intera** consiste nel trovare i fattori primi di un numero intero N . Per numeri piccoli il problema è banale, ma la sua difficoltà cresce in modo impressionante all'aumentare di N : i migliori algoritmi classici noti, come il GNFS, richiedono un tempo sub-esponenziale ma comunque super-polinomiale:¹

$$T_{\text{classico}}(N) = O\left(\exp\left((c + o(1))(\log N)^{1/3}(\log \log N)^{2/3}\right)\right) \tag{3.6}$$

Nella sua formulazione decisionale il problema appartiene a NP, e più precisamente a $\text{NP} \cap \text{co-NP}$: sia l'esistenza sia l'assenza di un fattore in un dato intervallo ammettono un certificato verificabile in tempo polinomiale. È questa collocazione a

¹Il GNFS detiene il record mondiale per la fattorizzazione classica: nel 2020, un consorzio internazionale ha fattorizzato RSA-250 (829 bit, 250 cifre decimali) [22], impiegando l'equivalente di circa 2700 anni-core. In precedenza, nel 2009, era stato fattorizzato RSA-768 (768 bit). I numeri RSA usati in produzione sono di 2048 o 4096 bit, rendendo la fattorizzazione classica del tutto impraticabile con qualunque tecnologia convenzionale prevedibile.

rendere improbabile che la fattorizzazione sia NP-completa — lo fosse, ne seguirebbe $NP = co-NP$, contro l'opinione prevalente. Non è però noto se appartenga a P: nessun algoritmo classico polinomiale è *conosciuto*, e si congettura che non esista, ma una dimostrazione di questa impossibilità manca.

Perché la Fattorizzazione Conta

La difficoltà di fattorizzare interi non è una curiosità matematica: è il fondamento su cui poggia RSA, e con esso buona parte della sicurezza delle comunicazioni odierne. Un algoritmo polinomiale per la fattorizzazione compromette RSA in modo irrecuperabile, e non per una debolezza implementativa ma perché ne viola l'assunzione di sicurezza. La trattazione estesa — il confronto fra la complessità sub-esponenziale del GNFS e quella polinomiale di Shor, la risposta della crittografia post-quantistica con gli standard *National Institute of Standards and Technology* (NIST) del 2024, e lo scenario *harvest now, decrypt later* — è nella Sezione C.2 dell'Appendice C. Fra l'algoritmo teorico e un'applicazione crittografica restano scala, compilazione fault-tolerant, overhead QEC e qualità dell'hardware; questa tesi isola soprattutto il ruolo del rumore.

3.2.2 Riduzione alla Ricerca del Periodo

L'intuizione geniale di Shor consiste nel ricondurre la fattorizzazione a un problema di **period finding**, trattabile quantisticamente in modo efficiente.

Osservazione chiave: Dato N e un intero a con $\gcd(a, N) = 1$ (coprimo con N), la funzione

$$f(x) = a^x \bmod N \quad (3.7)$$

è **periodica** con periodo r (detto *ordine* di a modulo N): $f(x + r) = f(x)$ per ogni x .

Una volta trovato il periodo r , se r è pari si può calcolare:

$$\gcd(a^{r/2} - 1, N) \quad \text{e} \quad \gcd(a^{r/2} + 1, N) \quad (3.8)$$

Con alta probabilità, almeno uno di questi due valori è un fattore non banale di N . La prova utilizza proprietà della teoria dei numeri: in particolare, vale $(a^{r/2} - 1)(a^{r/2} + 1) = a^r - 1 \equiv 0 \pmod{N}$.

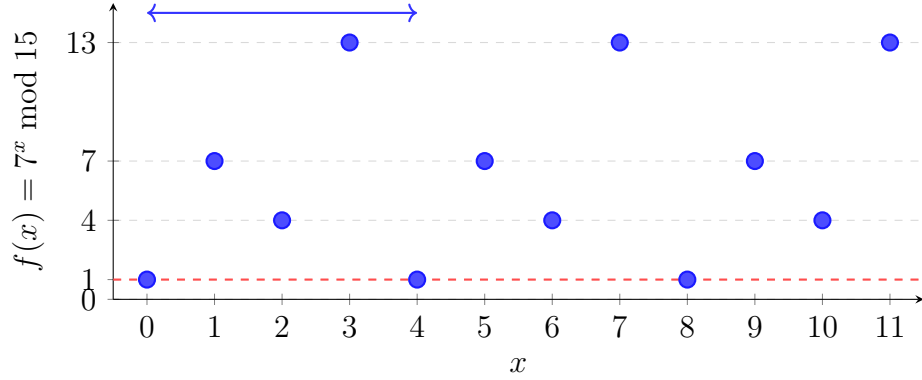


Figura 3.3: La funzione $f(x) = 7^x \bmod 15$ è periodica con periodo $r = 4$. La periodicità è evidente dai valori che si ripetono: $1, 7, 4, 13, 1, 7, 4, 13, \dots$. Trovare r permette di calcolare $\gcd(7^2 - 1, 15) = \gcd(48, 15) = 3$ e $\gcd(7^2 + 1, 15) = \gcd(50, 15) = 5$, ottenendo la fattorizzazione $15 = 3 \times 5$.

3.2.3 La Quantum Fourier Transform

Il componente quantistico fondamentale dell'algoritmo di Shor è la **Quantum Fourier Transform** (QFT), l'analogo quantistico della Trasformata Discreta di Fourier (Trasformata Discreta di Fourier (*Discrete Fourier Transform*) (DFT)).

Definizione

La QFT su $\mathbb{Z}_{2^n}$ agisce su uno stato base come:

$$\text{QFT} |j\rangle = \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} e^{2\pi i j k / 2^n} |k\rangle \quad (3.9)$$

L'azione su uno stato generico $|\psi\rangle = \sum_j \alpha_j |j\rangle$ produce:

$$\text{QFT} |\psi\rangle = \sum_k \hat{\alpha}_k |k\rangle \quad (3.10)$$

dove $\hat{\alpha}_k = \frac{1}{\sqrt{2^n}} \sum_j \alpha_j e^{2\pi i j k / 2^n}$ sono i coefficienti della DFT.

Implementazione Efficiente

La DFT classica su $N = 2^n$ elementi richiede $O(N \log N)$ operazioni (con il Fast Fourier Transform), mentre la QFT si realizza con $O(n^2) = O((\log N)^2)$ porte quantistiche: un conteggio esponenzialmente inferiore.

Questo confronto va però interpretato correttamente, perché la formulazione ingenua — “la QFT calcola la trasformata di Fourier esponenzialmente più in fretta” — è fuorviante. La QFT non restituisce i coefficienti $\hat{\alpha}_k$: li lascia *codificati nelle ampiezze* di uno stato quantistico, e la misura ne estrae un solo valore per esecuzione, con probabilità $|\hat{\alpha}_k|^2$. Non esiste quindi alcun modo di leggere lo spettro completo, e la QFT non sostituisce la FFT come strumento di calcolo generale. Il vantaggio è utilizzabile soltanto quando l’informazione cercata è una *proprietà globale* dello spettro che una singola misura è sufficiente a rivelare — ed è esattamente il caso della periodicità nell’algoritmo di Shor, dove i picchi della distribuzione trasformata cadono sui multipli di $2^n/r$ e una manciata di misure basta a ricostruire r .

Il circuito della QFT è costruito ricorsivamente tramite porte di Hadamard e porte di fase controllate CR_k dove:

$$CR_k = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{2\pi i/2^k} \end{pmatrix} \quad (3.11)$$

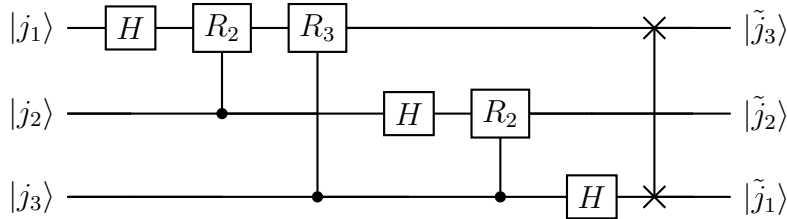


Figura 3.4: Circuito della QFT su $n = 3$ qubit. Ogni qubit riceve una porta di Hadamard H seguita da porte di fase controllate $R_k = \text{diag}(1, e^{2\pi i/2^k})$: la rotazione applicata al qubit j dipende dai qubit successivi. Lo SWAP finale riordina i qubit nell’ordine corretto. La struttura si generalizza a n qubit richiedendo $O(n^2)$ porte.

Phase Estimation

La QFT è impiegata, all’interno dell’algoritmo di Shor, attraverso la subroutine di **Phase Estimation** (QPE). Data una porta U con autovettore $|u\rangle$ e autovalore $e^{2\pi i\theta}$,

la QPE stima θ con precisione arbitraria in tempo $O(n^2)$. Nell'algoritmo di Shor, U è l'operatore di moltiplicazione modulare $U|x\rangle = |ax \bmod N\rangle$, e il suo spettro di fase contiene l'informazione sul periodo r .

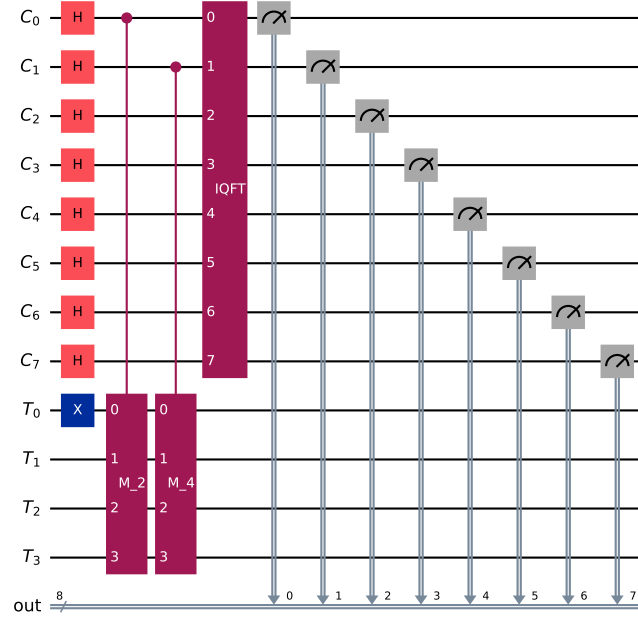


Figura 3.5: Circuito di Quantum Phase Estimation per $N = 15$, $a = 2$, implementato con Qiskit. Il registro di controllo (C , 8 qubit) viene posto in sovrapposizione uniforme, entangolato tramite gli operatori M_2 e M_4 controllati, e trasformato con la QFT^{-1} . La misura del registro C produce una stima della fase $\theta = k/r$, da cui si ricava il periodo r tramite frazioni continue.

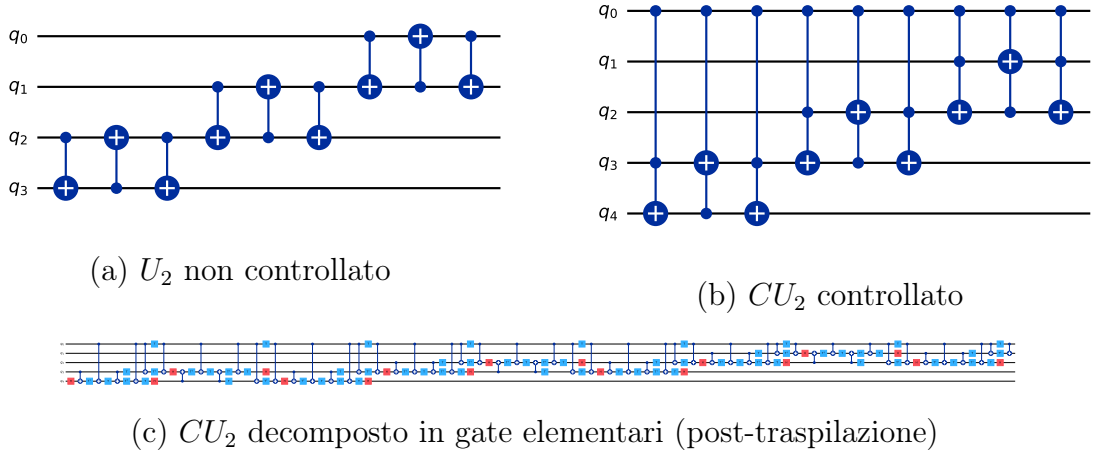


Figura 3.6: Gate di moltiplicazione modulare $U_2 |x\rangle = |2x \bmod 15\rangle$ per $N = 15$, $a = 2$. (a) Versione non controllata su 4 qubit: implementata come sequenza di SWAP. (b) Versione controllata CU_2 su 5 qubit utilizzata nel circuito QPE. (c) Decomposizione in gate elementari dopo la traspilazione: le porte SWAP si espandono in triple di CNOT, rivelando il costo effettivo in gate a due qubit del blocco.

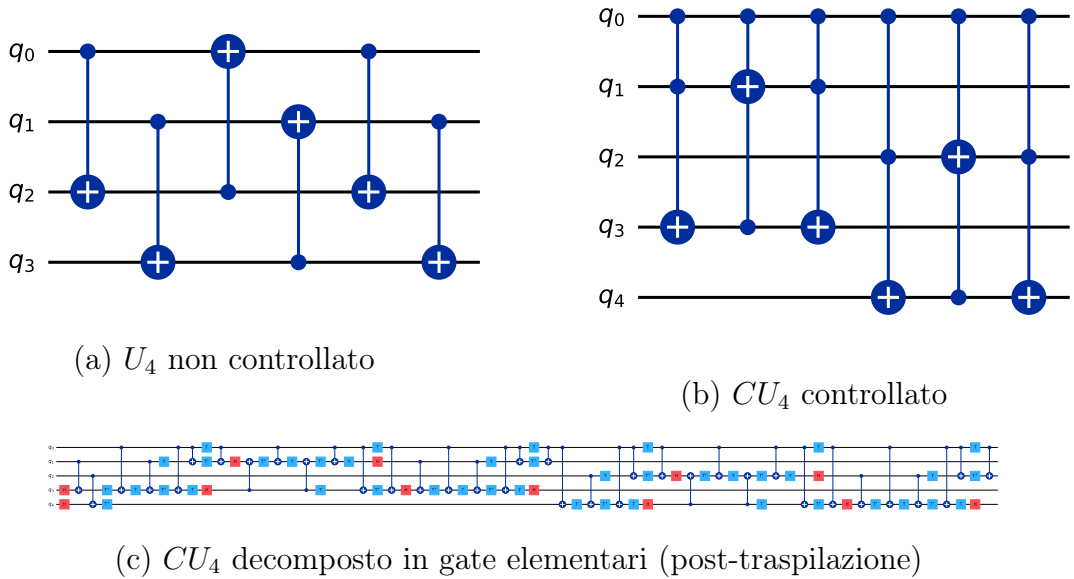


Figura 3.7: Gate di moltiplicazione modulare $U_4 |x\rangle = |4x \bmod 15\rangle$ per $N = 15$, $a = 4 = 2^2$. (a) Versione non controllata: il pattern di SWAP differisce da U_2 poiché la permutazione modulare per $a = 4$ è diversa. (b) Versione controllata CU_4 su 5 qubit, secondo blocco del circuito QPE. (c) Decomposizione in gate elementari: il costo aggiuntivo rispetto a CU_2 riflette la maggiore complessità della permutazione per $a = 4$.

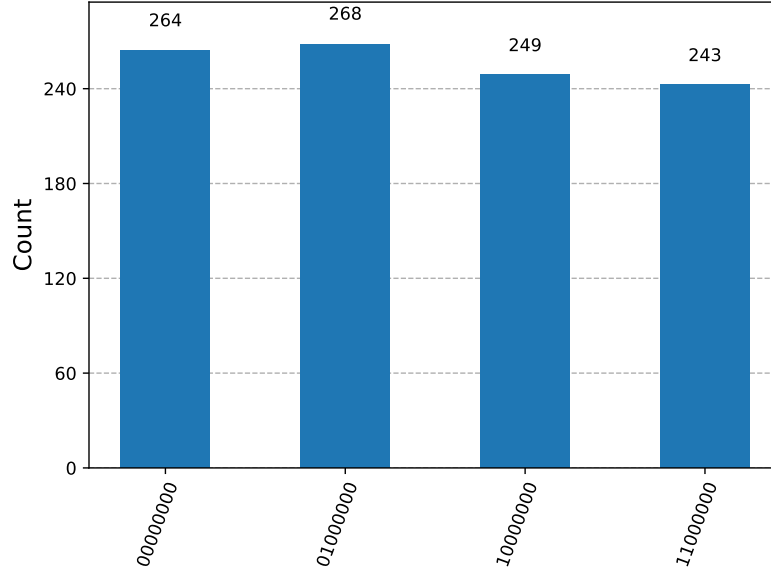


Figura 3.8: Distribuzione ideale campionata del registro di controllo per $N = 15$, $a = 7$ (1024 shot). I quattro stati attesi — $\{0, 64, 128, 192\}$ — sono i multipli di $2^8/r = 64$ per $r = 4$; le oscillazioni dei conteggi fra i picchi sono fluttuazioni di campionamento.

3.2.4 L'Algoritmo di Shor: Schema Completo

L'algoritmo si articola in tre fasi: un preprocessing classico, l'esecuzione del circuito quantistico per il *period finding*, e una post-elaborazione classica per estrarre i fattori.

Fase 1 — Preprocessing classico

1. Scegliere $a \in \{2, \dots, N-1\}$ uniformemente a caso.
2. Calcolare $g = \gcd(a, N)$: se $g > 1$, restituire g (fattore trovato direttamente).

Fase 2 — Circuito quantistico (period finding)

3. Inizializzare $n = 2\lceil \log_2 N \rceil$ qubit nel registro di controllo: $|0\rangle^{\otimes n} |1\rangle$.
4. Applicare $H^{\otimes n}$ al registro di controllo, creando la sovrapposizione uniforme $\frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle$.
5. Applicare la moltiplicazione modulare controllata: $|x\rangle |1\rangle \mapsto |x\rangle |a^x \bmod N\rangle$.
6. Misurare il registro di lavoro: il registro di controllo collassa in una sovrapposizione periodica di passo r . Il passo è *concettuale* — serve a rendere evidente

l'origine della periodicità — e nell'implementazione viene omesso, come chiarito nella Sezione 3.2.4.

7. Applicare la QFT^{-1} al registro di controllo.
8. Misurare il registro di controllo: ottenere un intero proporzionale a s/r , con $s \in \{0, \dots, r-1\}$.

Fase 3 — Post-elaborazione classica

9. Applicare l'algoritmo delle frazioni continue al risultato della misura per stimare il periodo r .
10. Se r è dispari o $a^{r/2} \equiv -1 \pmod{N}$: tornare al passo 1 con un nuovo valore di a .
11. Calcolare $p = \gcd(a^{r/2} - 1, N)$ e $q = \gcd(a^{r/2} + 1, N)$.
12. Se p o q è un fattore non banale di N : restituirlo.
Altrimenti: tornare al passo 1.

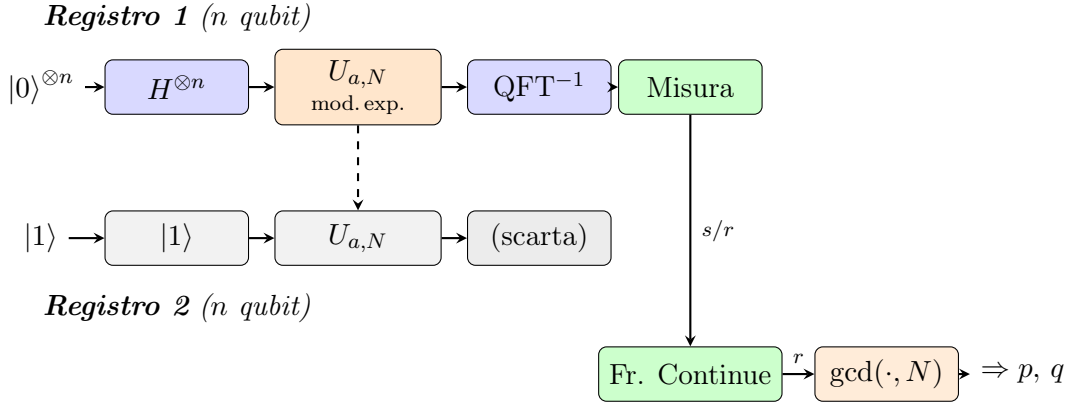


Figura 3.9: Schema a due registri dell'algoritmo di Shor. Il **Registro 1** (di controllo) viene posto in sovrapposizione uniforme con $H^{\otimes n}$, entangled con il Registro 2 tramite la moltiplicazione modulare $U_{a,N}$, quindi trasformato con la QFT^{-1} . La misura produce un valore s/r elaborato classicamente: le frazioni continue stimano il periodo r , e il calcolo del MCD restituisce i fattori p e q di N .

Il Ruolo dell'Entanglement nel Period Finding

Il collasso descritto al passo 6 dello schema non è un effetto arbitrario: è la conseguenza diretta dell'**entanglement** creato al passo 5. Vale la pena rendere esplicito questo meccanismo, che è il cuore quantistico dell'intero algoritmo.

Dopo l'applicazione della moltiplicazione modulare controllata, i due registri si trovano nello stato entangled:

$$|\Psi\rangle = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle \otimes |a^x \bmod N\rangle \quad (3.12)$$

Questo stato **non è separabile**: il valore del registro di lavoro è correlato in modo univoco al valore del registro di controllo. I due registri formano un unico sistema entangled, esattamente come accade in uno stato di Bell — dove misurare un qubit determina istantaneamente l'esito dell'altro.

Se a questo punto si misurasse il registro di lavoro e si ottenesse un valore $v = a^{x_0} \bmod N$, il registro di controllo collasserebbe *istantaneamente* nell'unica sovrapposizione compatibile con v : quella di tutti i valori x per cui $a^x \equiv v \pmod{N}$, ovvero

$$|x_0\rangle + |x_0 + r\rangle + |x_0 + 2r\rangle + \dots \quad (3.13)$$

La correlazione tra i due registri è **bidirezionale**: conoscere il valore del registro di lavoro vincola il registro di controllo a una sovrapposizione periodica di passo esattamente r . Questa è la “versione inversa” propria dell'entanglement: misurare uno dei due sottosistemi determina (o in questo caso, restringe) lo stato dell'altro.

In pratica il circuito di Shor non misura esplicitamente il registro di lavoro: la sola creazione dell'entanglement è sufficiente. Applicando la QFT^{-1} al registro di controllo prima della misura, si trasforma la periodicità nascosta dell'equazione (3.13) in picchi netti alle frequenze k/r , con $k \in \{0, \dots, r-1\}$. La misura finale del registro di controllo restituisce allora, con alta probabilità, un valore da cui le frazioni continue estraggono r .

In sintesi: l'entanglement tra i due registri è il meccanismo che *trasferisce* la periodicità della funzione $f(x) = a^x \bmod N$ — che esiste nel registro di lavoro — verso il registro di controllo, dove la QFT la rende misurabile. Senza entanglement, questa trasmissione di informazione tra i due registri sarebbe impossibile, e l'intero speedup esponenziale di Shor verrebbe meno.

3.2.5 Analisi della Complessità

La complessità dell’algoritmo di Shor è **polinomiale** nel numero di bit di N :

- Il circuito quantistico richiede $O(n^3)$ porte elementari con aritmetica modulare elementare, dove $n = \log N$; ricorrendo alla moltiplicazione veloce il conteggio scende a $O(n^2 \log n \log \log n)$;
- Il numero di qubit necessari è $O(\log N)$; l’implementazione ottimizzata di Beauregard [16] lo riduce a $2n + 3$ tramite addizione modulare in-place;
- Il numero atteso di iterazioni (con basi a diverse) prima del successo è $O(1)$.

Complessivamente, nella versione con aritmetica elementare,

$$T_{\text{Shor}}(N) = O((\log N)^3),$$

a fronte della complessità sub-esponenziale dell’Equazione (3.6) del miglior algoritmo classico generale noto. In funzione della lunghezza dell’input $n = \log N$, il divario è **superpolinomiale**; chiamarlo semplicemente “esponenziale” nasconderebbe il fatto che GNFS è già sub-esponenziale in n .

3.2.6 Stato dell’Arte e Confronto con Altri Algoritmi

Le realizzazioni sperimentali di Shor su hardware reale, i miglioramenti algoritmici più recenti — fra cui la variante di Regev del 2023 — e la roadmap dei costruttori verso il calcolo fault-tolerant sono discussi nella Sezione A.3 dell’Appendice A, insieme al confronto con l’algoritmo di Grover. Due elementi di quel quadro vanno però anticipati qui, perché delimitano il perimetro della parte sperimentale.

Il primo è che **nessuna implementazione ha finora fattorizzato un numero non banale senza semplificazioni**: le dimostrazioni note sfruttano la conoscenza preventiva del risultato per ridurre il circuito, e questo colloca il presente lavoro — che esegue il circuito completo su simulatore rumoroso — in un regime diverso da quello delle dimostrazioni su hardware. Il secondo è che il divario fra Shor dimostrativo e Shor crittograficamente rilevante non è di scala ma di natura: le stime di risorse per moduli a 2048 bit derivano da analisi architetturali complete, non da estrapolazioni di istanze piccole, e il Capitolo 5 riprende esplicitamente questo punto.

3.3 Rumore quantistico nei sistemi NISQ

3.3.1 Introduzione al Rumore Quantistico

Shor è polinomiale nel modello ideale, ma sui dispositivi NISQ la profondità utile è limitata da errori, connettività e assenza di correzione scalabile [4]. Il rumore quantistico non è riducibile a un semplice errore binario: rotazioni arbitrarie, perdita di fase, rilassamento e lettura imperfetta agiscono su ampiezze e correlazioni che non possono essere protette copiando il qubit, per il teorema del no-cloning [23]. Nel seguito si collegano tre livelli: origine fisica, canale matematico e impatto sulla pipeline di Shor.

3.3.2 La Macchina Fisica: i Tre Fatti che Contano

Per l’analisi bastano tre vincoli. Le porte sono impulsi analogici generati dall’elettronica esterna e quindi accumulano imprecisioni; il rapporto tra durata dei gate e tempi T_1, T_2 impone un budget finito di profondità; la topologia richiede routing per qubit non adiacenti e condiziona crosstalk e costo delle porte a due qubit. Architettura criogenica, tecnologie, lettura fisica e “bordo di coerenza” sono sviluppati nella Sezione A.2 dell’Appendice A.

3.3.3 Le Quattro Fonti Fisiche di Rumore

Le quattro sorgenti rilevanti sono riassunte nella Tabella 3.2. T_1 descrive il decadimento $|1\rangle \rightarrow |0\rangle$, mentre T_2 include sia tale rilassamento sia la perdita pura di fase; gli errori di gate derivano da controllo imperfetto, accoppiamenti residui e leakage; il readout scambia gli esiti con probabilità calibrabili; il crosstalk introduce correlazioni spaziali difficili da rappresentare con un modello indipendente. Formule, valori tecnologici e matrici di calibrazione sono conservati nella Sezione A.2.2.

Fonte	Causa	Parametro	Ordine tipico
Decoerenza	ambiente	T_1, T_2	50–500 μs
Gate	pulse/leakage	infedeltà	0.03–1%
Readout	rivelatore	e_0, e_1	1–3%
Crosstalk	accoppiamento	correlazioni	~ 0.1 –1%

Tabella 3.2: Fonti fisiche di rumore nei processori superconduttori; gli intervalli sono ordini di grandezza indicativi, non parametri dei preset sperimentali.

3.3.4 Dal Fenomeno Fisico al Modello Matematico

La Tabella 3.3 esplicita la traduzione usata per passare dal fenomeno alla simulazione. Il mapping non è biunivoco: un meccanismo può richiedere più canali e un canale efficace può riassumere più cause.

Fonte fisica	Canale matematico	Parametro
Decadimento energetico (T_1)	Amplitude damping	$\gamma = 1 - e^{-t/T_1}$
Perdita di fase (T_2)	Phase flip	$p = \frac{1}{2}(1 - e^{-t/T_2})$
Gate isotropici	Depolarizzante	p / infedeltà
Gate asimmetrici	Bit/phase flip	profilo d'errore
Readout	Matrice di confusione	e_0, e_1
Crosstalk	Canale multi-qubit	correlazioni

Tabella 3.3: Corrispondenza operativa tra fonti fisiche e canali simulabili.

In Qiskit Aer, `thermal_relaxation_error` combina rilassamento e dephasing a partire da T_1, T_2 e dalla durata del gate. I preset della prima campagna compongono questo contributo con canali depolarizzanti 1Q/2Q e una matrice di readout. È un modello Markoviano controllato: non riproduce automaticamente leakage, deriva, errori coerenti o crosstalk.

Parametri di coerenza, lettura e correlazione

I due tempi che collegano più direttamente il dispositivo al modello sono T_1 e T_2 . Il primo descrive il rilassamento energetico $|1\rangle \rightarrow |0\rangle$; il secondo la perdita della fase relativa. Il dephasing osservato combina rilassamento e perdita pura di fase secondo

$$\frac{1}{T_2} = \frac{1}{2T_1} + \frac{1}{T_\phi}, \quad T_2 \leq 2T_1, \quad (3.14)$$

dove T_ϕ è il tempo di puro dephasing. Il confronto pertinente non è fra T_2 e il numero totale di porte, ma fra il cammino temporale critico del circuito e il budget di coerenza:

$$\tau_{\text{circuito}} \ll \min(T_1, T_2). \quad (3.15)$$

Profondità, parallelismo, tempi di idle e routing determinano τ_{circuito} ; per questo nelle campagne si registrano sia i conteggi sia la profondità della netlist compilata.

Il readout viene descritto da una matrice di confusione $A_{ij} = P(\text{lettura } i \mid \text{stato } j)$. Nel modello indipendente a n qubit vale $A = \bigotimes_k A_k$, con

$$A_k = \begin{pmatrix} 1 - e_0^{(k)} & e_1^{(k)} \\ e_0^{(k)} & 1 - e_1^{(k)} \end{pmatrix}. \quad (3.16)$$

Questa fattorizzazione è utile come baseline ma non rappresenta errori di lettura correlati. Analogamente, il crosstalk fra qubit vicini produce meccanismi spaziali che possono sfuggire a un decoder costruito su archi indipendenti. Tale scarto strutturale è sottoposto a prova nella Sezione 5.3.6; ordini di grandezza e limiti fisici dei parametri sono conservati nella Sezione A.2.2.

3.3.5 I Quattro Canali Matematici del Rumore

Un canale quantistico è una mappa *Completely Positive Trace-Preserving* (CPTP) sulla matrice densità e ammette rappresentazione di Kraus [8]:

$$\mathcal{E}(\rho) = \sum_k K_k \rho K_k^\dagger, \quad \sum_k K_k^\dagger K_k = I. \quad (3.17)$$

Il Canale Depolarizzante

È il canale centrale nella campagna perché modella l'errore stocastico dei gate. Con probabilità p applica una Pauli non identità scelta uniformemente:

$$\mathcal{E}_{dep}(\rho) = (1 - p)\rho + \frac{p}{3}(X\rho X + Y\rho Y + Z\rho Z), \quad (3.18)$$

ossia

$$\mathcal{E}_{dep}(\rho) = \left(1 - \frac{4p}{3}\right)\rho + \frac{4p}{3}\frac{I}{2}. \quad (3.19)$$

Il vettore di Bloch è contratto isotropicamente. Nel parametro Aer λ , la probabilità aggregata di una componente Pauli non-identità è $3\lambda/4$ per un qubit e $15\lambda/16$ per due: questa distinzione è necessaria per interpretare il proxy sperimentale senza chiamarlo probabilità di successo di Shor.

Gli Altri Tre Canali

Il **bit flip** applica X e lascia invariata la componente r_x ; il **phase flip** applica Z , conserva le popolazioni e sopprime le coerenze; l'**amplitude damping** modella T_1 e porta asimmetricamente lo stato verso $|0\rangle$. Kraus, azione sulla matrice densità,

confronto geometrico e Tabella A.3 sono nella Sezione A.4; la Figura A.8 ne visualizza gli effetti.

3.3.6 Impatto del Rumore sull'Algoritmo di Shor

La vulnerabilità di Shor nasce dall'elevata profondità e dalla precisione di fase. La Figura 3.10 localizza le sorgenti nella pipeline.

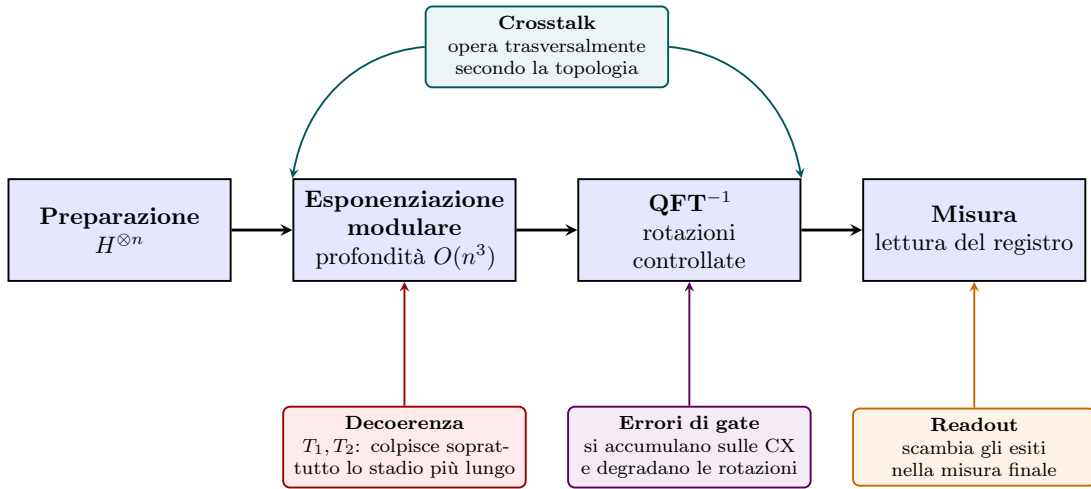


Figura 3.10: Decoerenza, errori di gate, readout e crosstalk colpiscono stadi differenti della pipeline.

Nella QFT, rotazioni sempre più piccole rendono critica la precisione di fase; nella QPE, il registro di controllo deve restare coerente lungo il cammino causale dell'esponenziazione; nell'aritmetica modulare, l'elevato numero di gate a due qubit accumula eventi. Sotto la sola ipotesi illustrativa di L eventi indipendenti, il proxy di nessun evento è $(1 - \epsilon)^L$: non è una probabilità di successo algoritmico. Derivazione, esempio numerico e Figura A.7 sono trasferiti nella Sezione A.3.3.

3.3.7 Sfide Pratiche su Hardware NISQ

La difficoltà operativa dipende insieme da qubit logici, routing e profondità. Il circuito canonico $N = 15$ usato qui, compilato nella base astratta **rz/sx/x/cx**, contiene 224 cx e profondità 412; una mappatura hardware aggiungerebbe routing dipendente dalla topologia. Le dimostrazioni di $N = 15$ e $N = 21$ su dispositivi reali usano circuiti fortemente semplificati e non generalizzabili [5]. Il seguito confronta quindi strategie controllate: post-processing nella prima fase, poi QEC e decodifica delle sindromi, mantenendo distinti tassi logici e successo di Shor.

3.4 Strategie per la riduzione del rumore quantistico

La risposta al rumore può agire sull’hardware, sulla stima dell’output, sulla codifica dell’informazione o sul circuito. Questa sezione confronta tali leve e isola quelle effettivamente valutate; la rassegna estesa è nella Sezione A.5 dell’Appendice A.

3.4.1 Le Quattro Strategie: Panoramica e Scelta

La **soppressione** riduce fisicamente l’accoppiamento con l’ambiente; la **mitigazione** stima a posteriori l’output ideale mediante campionamento aggiuntivo; la **fault tolerance** codifica qubit logici in molti qubit fisici e, sotto soglia, sopprime l’errore aumentando la distanza [24]; i **metodi appresi** svolgono infine compiti classici di classificazione o decodifica. Queste categorie non coincidono: in particolare, il machine learning può assistere una fase della QEC, ma non la sostituisce.

Strategia	Quando agisce	Costo dominante	Uso qui
Soppressione	prima/durante il calcolo	controllo hardware	no
Mitigazione	dopo il calcolo	campioni aggiuntivi	confronto ZNE
Fault tolerance/QEC	durante il calcolo	qubit e cicli	sì, simulata
Metodi appresi	post-processing/decoder	dati e calcolo classico	valutati

Tabella 3.4: Confronto delle strategie anti-rumore nel perimetro della tesi.

Nel regime studiato non si assume un vantaggio del modello appreso: la prima fase lo confronta con TOP-1 e con l’ablazione TOP-4 sullo stesso istogramma; la seconda lo colloca soltanto nella decodifica delle sindromi. La acZNE resta un controllo secondario, non la strategia principale.

3.4.2 Machine Learning Classico per la Robustezza al Rumore

Random Forest, *Support Vector Machine* (SVM) e Percettrone Multistrato (*Multi-Layer Perceptron*) (MLP) sono qui modelli **classici applicati a dati quantistici**, non *Quantum Machine Learning* (QML): addestramento e inferenza non impiegano un circuito quantistico. Possibili usi comprendono caratterizzazione del rumore, ottimizzazione circuitale e decodifica delle sindromi; il lavoro verifica il post-processing dell’istogramma e, separatamente, un correttore residuo del decoder. Le altre possibilità sono documentate nell’Appendice A.

3.4.3 Correzione degli Errori Quantistici e Ruolo del Machine Learning

La QEC distribuisce un qubit logico su più qubit fisici e ricava informazione sull'errore senza misurare lo stato logico. Il ciclo essenziale è:

1. **codifica** dello stato $|\psi_L\rangle$;
2. **misura degli stabilizzatori**, che produce una sindrome;
3. **decodifica e recupero**, analitici o assistiti da un modello.

Il codice di Shor $[[9, 1, 3]]$ dimostra la correzione di un errore arbitrario; Steane $[[7, 1, 3]]$ riduce l'overhead con una costruzione CSS; il surface code usa una griglia locale e sopprime l'errore logico sotto una soglia che dipende da circuito, rumore e decoder. Stati logici e proprietà complete sono riportati nella Sezione A.5.3.

Machine Learning come Possibile Componente del Decoder QEC

La sindrome conserva informazione strutturata sull'errore e rende sensato un confronto fra decoder analitico e appreso sugli stessi campioni [25, 19]. Nella Sezione 5.3.6 il modello non sostituisce con vantaggio il matching; come correttore residuo, invece, recupera parte delle correlazioni che il grafo analitico non rappresenta. Il risultato resta circoscritto al modello simulato.

3.4.4 Motivazione Pratica: il Contributo di questa Tesi

Nella prima fase M1 prova la moda, TOP-4 i primi quattro candidati e M2 usa un classificatore per decidere se attivare la stessa ricerca TOP-4. La riduzione rispetto a M1 esiste, ma l'ablazione la attribuisce alla ricerca multi-candidato: il filtro appreso non aggiunge informazione alla regola semplice. Questo esito negativo motiva lo spostamento del focus dalla sola lettura dell'output alla correzione durante il calcolo.

3.4.5 Una Strategia sul Circuito: la Trasformata di Fourier Approssimata

Una leva complementare riduce il circuito prima che agisca il rumore. La Trasformata di Fourier Quantistica Approssimata (*Approximate Quantum Fourier Transform*) (AQFT) elimina le rotazioni controllate più piccole: introduce errore di approssimazione, ma riduce porte e profondità. Per $N = 21$, il troncamento nelle trasformate

interne agli addizionatori porta le porte 2Q da 49 661 a 23 178; nella stima di fase la variante troncata supera quella completa in tutti i nove confronti eseguiti. È un risultato esplorativo, ripreso nella Sezione 5.5; confronto con le proposte che riducono i qubit logici e relativi costi sono nella Sezione A.5.5.

3.4.6 Dall’esito della prima fase alla struttura del lavoro

Il filo sperimentale procede quindi da un controllo NISQ appaiato, che separa effetto TOP-K ed effetto ML, alla protezione durante il calcolo: codice a ripetizione, Steane, surface code e decodifica. La sensibilità di Shor al proxy per gate resta distinta dai tassi logici per ciclo; l’AQFT chiude il percorso agendo direttamente sul costo circuitale.

3.5 Disegno e metodo sperimentale

Il disegno distingue correttezza ideale dell’aritmetica, comportamento sotto rumore controllato e capacità del post-processing di utilizzare l’istogramma. Qui sono fissati perimetro, appaiamento e regole decisionali; i risultati sono nel Capitolo 5.

3.5.1 Disegno e perimetro sperimentale

La prima fase applica tre strategie allo *stesso* istogramma: M1 verifica la sola moda (TOP-1), TOP-4 i primi quattro candidati, M2 usa un classificatore per decidere se attivare la stessa ricerca TOP-4. Quest’ultima è l’ablazione che separa il contributo del modello da quello della ricerca multi-candidato.

Le campagne rumorose usano $N = 15$, $a = 7$, con i due preset uniformi della Tabella 3.5, che non simulano la calibrazione di una QPU specifica. Le istanze ulteriori sono validate soltanto in condizioni ideali; scelte aritmetiche, costi e perimetro completo sono nella Sezione A.3.4.

Scelta dell’aritmetica e limite di scala

La sintesi dell’esponenziazione modulare come operatore unitario denso cresce come $O(4^k)$ su k qubit [26]. Per le istanze oltre il circuito didattico viene quindi usata l’aritmetica reversibile di Beauregard [16], che costruisce nel dominio della QFT gli addizionatori modulari con costo $O(n^3)$ e $2n + 3$ qubit. La riduzione asintotica

Tabella 3.5: Contratto essenziale delle campagne rumorose su $N = 15$.

Parametro	Riferimento	Stress
a / periodo atteso	7/4	7/4
qubit di controllo / shot	8/1024	8/1024
$\lambda_{1q} / \lambda_{2q}$	$10^{-3}/10^{-2}$	$5 \times 10^{-3}/5 \times 10^{-2}$
T_1/T_2 (ns)	100 000/80 000	50 000/30 000
p_{ro}	0.02	0.05

non elimina il costo concreto: per $N = 21$, $a = 2$ e otto qubit di conteggio, la compilazione congiunta in **rz/sx/x/cx** produce 21 036 porte Controlled-X (porta CNOT) (CX) e profondità 23 081 su 22 qubit.

Per questo la correttezza di $N = 21$ viene verificata idealmente mediante truth table, azzeramento delle ancille e distribuzione della QPE, mentre la campagna rumorosa rimane su $N = 15$. Il costo dominante non è la sola trasformata finale della phase estimation, ma anche le trasformate interne agli addizionatori: l'esperimento AQFT del Capitolo 5 agisce proprio in quel punto. Questa separazione evita di presentare un limite della simulazione classica come limite dell'algoritmo quantistico; la derivazione estesa è nella Sezione A.3.4.

3.5.2 Pipeline appaiata

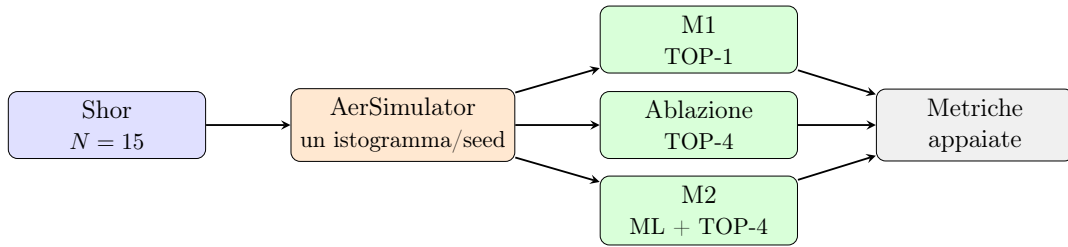


Figura 3.11: Ogni istogramma è riutilizzato dalle tre strategie: le differenze non dipendono da realizzazioni Monte Carlo diverse.

Il circuito è compilato una volta nella base **rz/sx/x/cx**, con livello di ottimizzazione 2 e seed fissato; la netlist di riferimento ha 224 **cx** e profondità 412. Per ogni coppia replica-iterazione Aer produce un istogramma poi condiviso da M1, TOP-4 e M2. Manifest, hash e regola deterministica per i pareggi sono riportati nella Sezione A.5.7.

3.5.3 Modello di rumore e controlli

Le rotazioni $\mathbf{rz}$ sono cambi di frame virtuali. Su $\mathbf{sx}/\mathbf{x}$ e $\mathbf{cx}$ si compongono depolarizzazione e rilassamento termico, con durate di 50 e 300 ns; il readout è una matrice simmetrica. Aer definisce $\mathcal{E}(\rho) = (1 - \lambda)\rho + \lambda I/2^n$; pertanto, sulle 224 CX,

$$P_{\text{no-Pauli-2Q}} = \left(1 - \frac{15\lambda}{16}\right)^{224}. \quad (3.20)$$

È un proxy di nessun evento Pauli 2Q, non la probabilità di successo di Shor, e non comprende errori 1Q, rilassamento o readout.

Il contratto base non inverte la matrice di readout. T_1/T_2 , errori 1Q/2Q, readout, shot, K e ottimizzazione sono variati nelle sensibilità; griglia e confronto *Zero-Noise Extrapolation* (ZNE) sono nella Sezione D.1.

3.5.4 Regole decisionali e analisi statistica

M1 prova un candidato, TOP-4 fino a quattro e M2 gli stessi quattro soltanto se il classificatore accetta l'istogramma. Dataset, modello TOP-1 e audit TOP-16 sono documentati nella Sezione D.2. Ogni scenario usa 30 repliche, al massimo 50 iterazioni e sentinella 51 per i fallimenti. I confronti M1>TOP-4 e M1>M2 impiegano Wilcoxon appaiato unilaterale con `zero_method='pratt'`; TOP-4–M2 è bilaterale. Le correzioni multiple sono applicate entro la famiglia dichiarata; medie e tassi sono descrittivi, l'inferenza usa i vettori appaiati completi.

3.5.5 Matrice di verifica dell'intero disegno

La Tabella 3.6 collega ogni componente alla prova che la rende controllabile. In questo modo la prima fase, i laboratori QEC, il surface code, la decodifica, la sensibilità di Shor e l'AQFT appartengono a un solo contratto sperimentale pur mantenendo metriche non intercambiabili.

Oggetto	Test o controllo	Evidenza richiesta
Circuito $N = 15$	moltiplicatore $\times 7 \bmod 15$, periodo e fattori	truth table, $r = 4$, fattori 3 e 5, hash della netlist
Aritmetica $N = 21$	azione sui vettori validi e uncomputation	output modulare corretto, ancille a zero e costo compilato
M1/TOP-4/M2	stessi istogrammi, budget e seed	vettori appaiati, Wilcoxon–Pratt e correzione Holm
Classificatore	separazione train/validation/test	F1, AUC e audit delle etichette sul test indipendente
Ripetizione e Steane	iniezioni correggibili e sweep a piccolo p	sindrome, $\gamma \simeq 2$ e regione di break-even
Surface code	sweep con distanze crescenti	intervalli su p_L e incrocio compatibile con p_{th}
Decoder appreso/ibrido	stessi detection events del MW-PM	differenza appaiata nei modelli matched e mismatched
Shor M8	sweep del proxy per gate e ripetizioni	curva e $P(k)$, senza conversione dai tassi QEC
AQFT	sweep della soglia nelle QFT interne	risparmio di porte contro perdita algoritmica

Tabella 3.6: Corrispondenza fra componenti, controlli e criteri di evidenza.

Capitolo 4

Architettura e implementazione del sistema sperimentale

Il Capitolo 2 ha fissato l'ipotesi da verificare: ridurre il numero medio di esecuzioni necessarie per ottenere fattori validi di Shor mediante un filtro classico. Il Capitolo 3 ha poi definito rumore, metriche e metodologia. Qui si presenta soltanto l'architettura che collega tali elementi alle due campagne sperimentali: la prima confronta TOP-1, TOP-4 e il filtro ML; la seconda studia codici correttori e decodifica della sindrome. Le motivazioni estese degli strumenti e gli estratti del codice sono raccolti, rispettivamente, nelle Sezioni B.1 e B.2; i risultati sono nel Capitolo 5.

L'architettura operativa è articolata in quattro livelli. Il livello circuitale costruisce Shor e produce una netlist compilata una sola volta per ogni configurazione; il livello di simulazione associa a quella netlist il preset di rumore e genera i campioni; il livello decisionale applica post-processing, classificatore o decoder; il livello di analisi salva metriche e metadati prima di generare tabelle e figure. La separazione impedisce che una modifica al classificatore alteri il circuito o che una differenza di compilazione venga scambiata per un effetto del metodo. Nella prima fase l'oggetto condiviso è l'istogramma delle misure; nella seconda è la sequenza dei detection event. Cambia quindi il dato su cui si decide, non la regola generale di confronto su campioni comuni.

4.1 Strumenti e configurazione dell'ambiente

4.1.1 Strumenti e ambiente di simulazione

La toolchain segue la natura dei circuiti. **Qiskit** [12] e **Qiskit Aer** costruiscono, compilano e simulano lo Shor generale di piccola taglia; il classificatore della prima

fase usa **scikit-learn** [15]. Per i milioni di campioni stabilizer richiesti dal surface code si impiegano invece **Stim** [13] e **PyMatching** [14]. Questa separazione evita di attribuire a un simulatore statevector un regime che cresce come 2^n , ma non rende Shor simulabile con Stim perché le sue rotazioni di fase non sono Clifford. Confronto dei framework, metodi di Aer e ragioni della scelta sono sviluppati nella Sezione B.1.

La campagna rumorosa principale usa `AerSimulator(method='statevector')` per il circuito didattico a 12 qubit di $N = 15$; $N = 21$ e $N = 35$ sono riservati ai controlli ideali di correttezza e costo. Il noise model uniforme e illustrativo combina depolarizzazione, rilassamento termico e readout e viene applicato, dopo compilazione, alla base `rz/sx/x/cx`. Non è una calibrazione di una QPU nominata.

4.1.2 Ambiente e contratto operativo

Le esecuzioni canoniche avvengono su Unità di Elaborazione Centrale (*Central Processing Unit*) (CPU) in *Windows Subsystem for Linux* (WSL)2 con dipendenze fissate; interprete, librerie e backend sono registrati nel manifest di ciascun artefatto. L'accelerazione Unità di Elaborazione Grafica (*Graphics Processing Unit*) (GPU), prevista inizialmente, non è entrata nel contratto riproducibile perché il backend Aer non era utilizzabile nell'ambiente di sviluppo. Versioni, configurazione WSL, limite di memoria e diagnosi del fallimento sono documentati nelle Sezioni B.1.5 e B.1.3; il comando di installazione e l'errore osservato compaiono nei Listati B.1 e B.2.

L'implementazione segue un unico contratto: preset rumorosi controllati per $N = 15$, validazioni ideali separate per $N = 21/35$, confronto appaiato delle strategie e artefatti schema 2. I dettagli di netlist, manifest e organizzazione dei sorgenti sono trasferiti nella Sezione B.2.1.

4.2 L'algoritmo di Shor: implementazione di riferimento

Il circuito costruisce la QPE con un registro di conteggio e uno di lavoro; applica la moltiplicazione modulare controllata, la QFT inversa e la misura. Il post-processing ricava il periodo mediante frazioni continue e verifica i fattori non banali con il Massimo Comune Divisore (*Greatest Common Divisor*) (GCD). La QFT inversa è decomposta in Hadamard, rotazioni di fase controllate e scambi; i relativi estratti sono nei Listati B.5 e B.3.

Per $N = 15$, $a = 7$, la rete textbook realizza realmente l'operazione elementare $\times 7 \bmod 15$ per il numero di potenze richiesto. Ciò corregge la precedente selezione mediante il solo valore `power % 4`, che non rappresentava in generale U^{power} (Listato B.4). Per $N = 21$ e $N = 35$ si impiega invece l'aritmetica reversibile di Beauregard, verificata idealmente mediante truth table, pulizia degli ausiliari e prove QPE.

La compilazione, comune a training, baseline e sweep, fissa il livello di ottimizzazione 2, il seed del transpiler e la base `rz/sx/x/cx`. Le istanze maggiori sono escluse dalla campagna rumorosa completa per il costo della netlist, ma non dai controlli ideali. Profondità, conteggi delle porte e hash SHA-256 delle compilazioni di riferimento sono riportati nella Sezione B.2.1.

4.3 Configurazione del modello di rumore

Il modello `qiskit_aer.noise` è **uniforme e illustrativo**, non una calibrazione hardware. Le `rz` sono cambi di frame virtuali, quindi senza durata né errore; `sx/x` durano 50 ns e `cx` 300 ns. Depolarizzazione e rilassamento termico agiscono su `sx/x` e `cx`, mentre il readout è descritto da una matrice di confusione simmetrica separata (Listato B.6).

I simboli ϵ_{1q} e ϵ_{2q} indicano il parametro Aer λ di `depolarizing_error`; per due qubit la probabilità Pauli non-identità è $15\lambda/16$. Per i 224 gate `cx` della compilazione N15, il proxy che nessuno di tali eventi si verifichi è $(1 - 15\lambda/16)^{224}$. Non è una probabilità di successo: esclude rumore 1Q, rilassamento e readout.

4.4 Implementazione dei metodi proposti

4.4.1 Metodo 1: selezione TOP-1

M1 seleziona il primo esito per frequenza, tenta il post-processing e, in caso di fallimento, passa all'iterazione successiva. Un tie-break SHA-256 derivato da seed e bitstring rende l'ordinamento indipendente dal dizionario e dal valore numerico del candidato (Listato B.7).

4.4.2 Metodo 2: classificatore ML e ricerca TOP-4

M2 combina addestramento offline e inferenza online. La stessa netlist genera, da ogni istogramma, le etichette TOP-1 e TOP-16; Random Forest, SVM RBF e MLP

sono confrontati con split stratificato. Il modello schema 2 registra definizione dell’etichetta, revisione, preset, seed e hash del circuito, e l’inferenza rifiuta manifest incompatibili. Se il filtro accetta l’istogramma, vengono provati fino a quattro candidati: M1, TOP-4 e M2 ricevono pertanto gli stessi conteggi, tie-break e post-processing. I Listati B.8 e B.9 rendono verificabile il flusso; l’audit completo è nella Sezione D.2.

4.5 Infrastruttura di test e raccolta dati

Le campagne N15 usano $K = 30$ repliche appaiate e una sola simulazione per produrre l’istogramma comune alle tre strategie. Seed sufficientemente distanziati evitano la quasi sovrapposizione degli stream Aer; il primo successo è registrato separatamente e un fallimento entro `max_iter` riceve la sentinella `max_iter+1`. I confronti impiegano il Wilcoxon appaiato con `zero_method='pratt'`. Artefatti JSON, parametri, vettori appaiati, versioni, hash e regole esatte per seed e test sono descritti nella Sezione B.2.1 e sintetizzati dal Listato B.10.

4.6 Implementazione della correzione d’errore

La seconda fase sostituisce l’istogramma finale di un circuito profondo con la sindrome di circuiti di memoria. Qiskit Aer gestisce il codice a ripetizione, Steane e il proxy di Shor; Stim e PyMatching rendono praticabile il surface code, mentre scikit-learn serve il decoder appreso. La mappa completa fra moduli, strumenti e costo dominante è nella Tabella B.2 dell’Appendice B.

Il flusso comune prepara lo stato logico, inserisce il canale nel punto dichiarato, estrae la sindrome su ancille e consegna gli eventi al decoder; l’esito è un successo o fallimento logico, non una misura diretta della fedeltà di Shor. Repetition, Steane e surface code realizzano questo schema con livelli crescenti di protezione. La curva di sensibilità di Shor costituisce invece un secondo livello: riceve un proxy per gate e misura la resa in fattori. Tenere separati i due livelli evita di convertire senza giustificazione il tasso per memoria dei codici nel parametro per gate del circuito.

Nel codice a ripetizione la parità è copiata su ancille senza misurare i dati; nel codice di Steane $|0_L\rangle$ è preparato e decodificato con una lookup table a peso minimo, dichiarando ideale l’estrazione di sindrome. Per il surface code il crosstalk viene iniettato anche dentro i blocchi REPEAT e MWPM opera sui detection event. Nel

proxy di Shor, il p_L per gate viene convertito nella convenzione depolarizzante di Aer; non è identificato con il logical error rate per round senza un mapping aggiuntivo. Il decoder appreso e MWPM ricevono gli stessi eventi. Algoritmi, assunzioni e listati per ciascun componente sono raccolti nella Sezione B.2.3.

Ogni script propaga il seed e registra nel JSON campioni, parametri e versioni; le figure sono generate dagli artefatti senza trascrizione manuale. Questa disciplina costituisce il contratto di riproducibilità comune alle due fasi.

Capitolo 5

Risultati e discussione sperimentale

5.1 Fase NISQ: classificatore ML e strategia TOP-K

Il Capitolo 4 ha descritto M1 (TOP-1), M2 (classificatore TOP-1 seguito da TOP-4) e l'ablazione M_{TOP4} . Questa prima parte del capitolo presenta la campagna rigenerata dopo la correzione dell'operatore $\times 7 \bmod 15$ e dei contratti statistici. Il risultato principale è che la ricerca multi-candidato riduce le iterazioni, mentre il classificatore a valle non aggiunge beneficio rispetto alla stessa ricerca priva di filtro.

5.1.1 Setup Sperimentale

Il confronto rumoroso usa un'unica istanza controllata, $N = 15$, $a = 7$, e due preset uniformi illustrativi. Le tre strategie ricevono, replica per replica, gli stessi istogrammi: il disegno isola così la regola di post-processing senza confonderla con una diversa realizzazione Monte Carlo. Il modello combina depolarizzazione, rilassamento/dephasing e readout, ma non riproduce la calibrazione o la topologia di una QPU specifica. Ogni confronto principale comprende 30 repliche da 1 024 shot per iterazione e usa Wilcoxon–Pratt con correzione di Holm.

Il contratto completo — circuito e versioni, seed, manifest, gestione dei pareggi e della censura, insieme al resoconto della campagna storica non riutilizzata — è conservato nella Sezione D.1.1 dell'Appendice D.

5.1.2 Perimetro Ideale e Rumoroso

La verifica di scala resta distinta dalla campagna rumorosa. Per $N = 21$, $a = 2$, la truth table, la pulizia delle ancille e la legge QPE ideale confermano l'aritmetica;

la compilazione globale raggiunge però 21 036 CX e profondità 23 081, rendendo la simulazione rumorosa completa estranea al confronto principale. L'ulteriore controllo ideale su $N = 35$, le distanze di variazione totale, il proxy di nessun evento e la tabella del perimetro sono riportati nella Sezione D.1.1. Nel seguito, i risultati rumorosi riguardano esclusivamente $N = 15$.

5.1.3 La Baseline: Metodo 1 (TOP-1)

La diagnostica indipendente di 60 repliche trova la moda $y = 0$ in 30/60 casi e una moda utile negli altri 30/60, mentre TOP-4 trova sempre almeno un candidato utile. L'instabilità di TOP-1 deriva quindi da quale dei quattro picchi teorici risulta dominante, non dalla completa scomparsa del segnale. Gli istogrammi estremi, selezionati con criterio prefissato e seed registrati, sono mostrati nella Figura 5.2 dell'Appendice D.

5.1.4 L'Ipotesi ML e il Verdetto dell'Ablazione

I classificatori TOP-1 sono stati rigenerati su 2 000 istogrammi per scenario; TOP-16 ha prodotto 2 000/2 000 positivi in entrambi gli scenari e viene registrato come audit a classe unica. Le metriche e il protocollo train/validation/test sono riportati nella Sezione D.2.1. La Tabella 5.1 riporta invece la misura operativa, calcolata sui medesimi istogrammi per tutte le strategie.

Tabella 5.1: Confronto appaiato M1, TOP-4 e M2 (30 repliche per scenario)

Scenario	Strategia	Successo	$\bar{M}$	ρ vs M1	p_{Holm} vs M1
UC1	M1	30/30	1.37	—	—
UC1	TOP-4	30/30	1.00	1.367	0.0024
UC1	M2	30/30	1.50	0.911	0.7314
UC2	M1	30/30	1.50	—	—
UC2	TOP-4	30/30	1.00	1.500	0.0015
UC2	M2	30/30	1.37	1.098	0.0874

TOP-4 converge al primo tentativo in tutte le repliche. M2 non migliora significativamente M1 e, rispetto alla propria ablazione, richiede più iterazioni: Wilcoxon bilaterale con correzione di Holm $p = 0.0033$ (UC1) e $p = 0.0096$ (UC2). Il filtro scarica dunque istogrammi che la ricerca TOP-4 avrebbe risolto; il guadagno appartiene alla ricerca multi-candidato, non al classificatore.

Il confronto con la letteratura recente chiarisce perché l’ablazione sia il risultato portante e non un dettaglio di metodo. Yang e Markidis addestrano alberi e foreste casuali su 680 distribuzioni raccolte da sistemi International Business Machines (IBM) reali per prevedere se il post-processing a frazioni continue recupererà l’ordine vero, usando quattro caratteristiche della distribuzione, fra cui la frazione di massa verificata dominante, che risulta la più informativa [27]. Il loro impianto ha un vantaggio che questo lavoro non possiede — i dati provengono da hardware, non da un modello di rumore — ma pone una domanda diversa: quanto valga il classificatore *rispetto a una regola semplice* disponibile sugli stessi istogrammi resta fuori dal confronto. È precisamente la quantità che l’ablazione qui misura, e il suo esito è negativo. Un risultato negativo di questo tipo non contraddice quei lavori: ne delimita la portata, indicando che il valore incrementale del filtro appreso va misurato contro la ricerca multi-candidato prima di essere attribuito all’apprendimento.

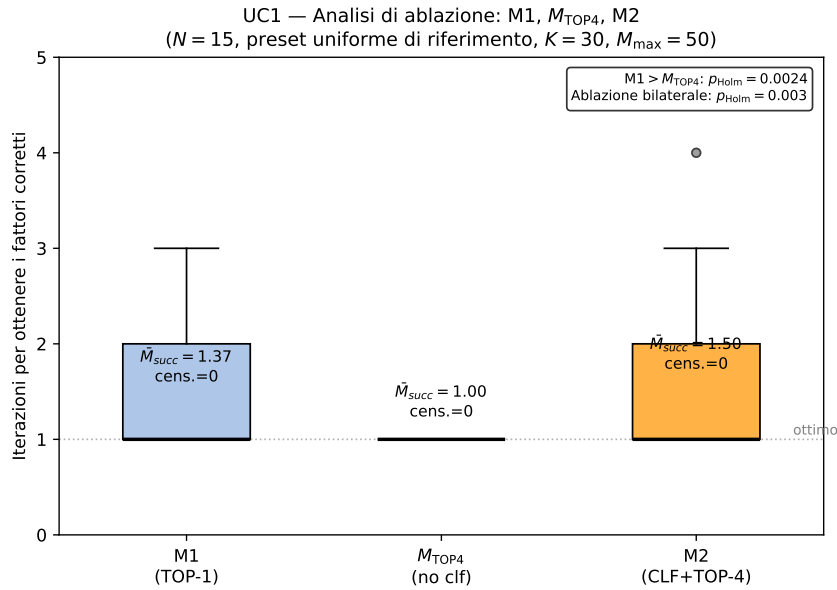


Figura 5.1: Distribuzione delle iterazioni nelle 30 repliche appaiate. I fallimenti, se presenti, sarebbero mostrati separatamente con sentinella 51; in questa baseline tutte le repliche hanno successo.

5.1.5 Robustezza Esplorativa della Strategia TOP-4

Otto famiglie di sweep variano un parametro per volta. I test sono appaiati all’interno del singolo punto, ma i p non ricevono una correzione globale fra le otto famiglie: i risultati vanno quindi letti come esplorativi. La Tabella 5.2 evita di trasformare l’assenza di variazione osservata in una pretesa universale di insensibilità al rumore.

Tabella 5.2: Sintesi degli otto sweep schema 2 su $N = 15$

Famiglia	Intervallo	Esito TOP-4
K	1, 2, 3, 4, 6, 8	soglia osservata $K = 2$
λ_{2q}	10^{-3} – 10^{-1}	$\bar{M} = 1.00$ – 1.07 , successo 100%
shot	128–2 048	$\bar{M} = 1.00$
$K \times \lambda_{2q}$	$K = 2$ – 8 , 5×10^{-3} – 5×10^{-2}	$\bar{M} = 1.00$
T_1/T_2	20/16–500/400 μs	$\bar{M} = 1.00$
λ_{1q}	10^{-4} – 2×10^{-2}	$\bar{M} = 1.00$
readout simmetrico	0–20%	$\bar{M} = 1.00$
ottimizzazione	livelli 0–3	$\bar{M} = 1.00$

Il livello di ottimizzazione modifica il circuito: livello 0 produce 242 CX e profondità 427, livello 1 produce 236/421, mentre i livelli 2 e 3 coincidono a 224/412. Nonostante ciò TOP-4 resta a una iterazione nei quattro punti. Le oscillazioni di M1 negli sweep non sono monotone e, con 30 repliche, non sono interpretate come leggi causali.

Diagnostiche e confronto con ZNE. La sola iterazione di successo non descrive quanta struttura periodica resti nell’istogramma. La Figura 5.2 mostra, fra le 30 repliche nominali UC1, i casi con massa massima e minima sui quattro picchi teorici $\{0, 64, 128, 192\}$.

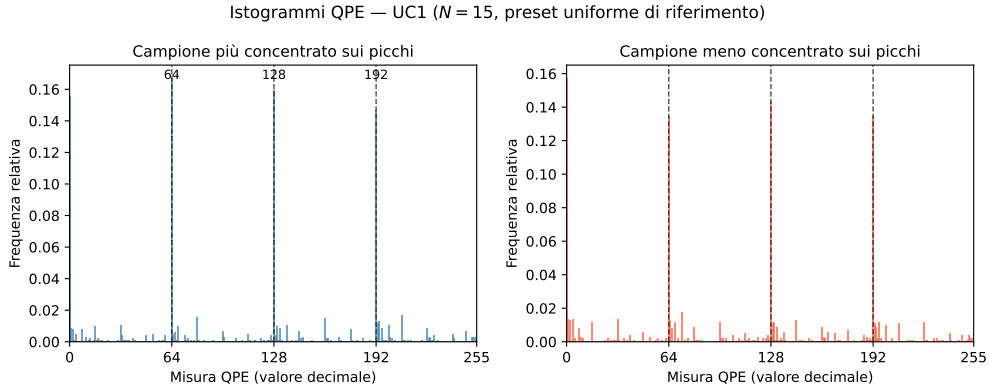


Figura 5.2: Repliche UC1 con massa massima e minima sui quattro picchi QPE teorici.

La massa media m sui picchi viene convertita nell’indice descrittivo

$$f = \frac{m - 4/256}{1 - 4/256}, \quad (5.1)$$

che vale zero per il fondo uniforme e uno per una distribuzione concentrata sui picchi. Non è una fedeltà. La Tabella 5.3 lo confronta con il proxy $P_0 = (1 - 15\lambda_{2q}/16)^{224}$.

Tabella 5.3: Massa efficace sui picchi e proxy di nessun evento 2Q.

λ_{2q}	f efficace	P_0 proxy
10^{-3}	0.8036	8.11×10^{-1}
5×10^{-3}	0.7039	3.49×10^{-1}
10^{-2}	0.5938	1.21×10^{-1}
2×10^{-2}	0.4286	1.44×10^{-2}
5×10^{-2}	0.1688	2.14×10^{-5}
10^{-1}	0.0421	2.65×10^{-10}
2×10^{-1}	0.0047	6.32×10^{-21}
3×10^{-1}	0.0015	7.47×10^{-33}

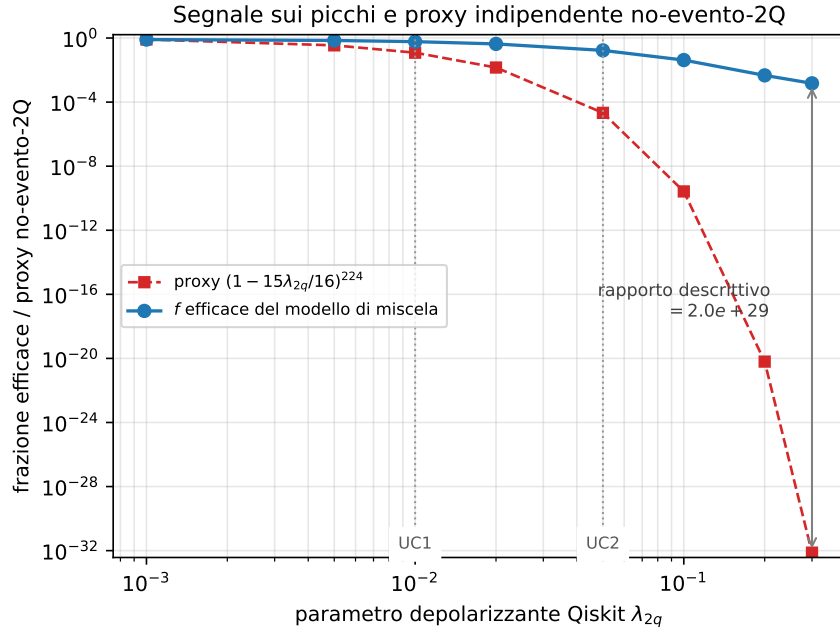


Figura 5.3: Il proxy di nessun evento decade più rapidamente della massa strutturata osservata sui picchi.

La struttura QPE sopravvive dunque anche quando il proxy moltiplicativo di nessun evento Pauli è già prossimo a zero: i due indicatori non sono né fedeltà né probabilità di fattorizzazione e non vanno identificati. Il protocollo completo e gli altri sweep restano nell'Appendice D.

Nello stesso perimetro, la ZNE digitale non migliora M1 e porta il costo medio da 1 399 a 3 072–4 813 shot, mentre TOP-4 raggiunge $\bar{M} = 1.00$ con 1 024 shot. Il

protocollo, che scala solo i parametri depolarizzanti e non equivale al gate folding su hardware, e la tabella appaiata completa sono conservati nella Sezione D.1.9 e nella Tabella D.11. Il verdetto è locale al modello uniforme e a questa istanza, non una valutazione generale della ZNE.

5.1.6 Bilancio della Verifica e Ponte verso la Correzione d’Errore

La campagna consente quattro conclusioni delimitate dai dati:

1. l’operatore $\times 7 \bmod 15$ e il circuito compilato sono fissati da truth table, manifest e hash;
2. TOP-4 riduce $\bar{M}$ da 1.37/1.50 a 1.00 nei due scenari;
3. il classificatore TOP-1 non migliora M1 e peggiora la propria ablazione TOP-4;
4. la robustezza osservata negli sweep è esplorativa e non rende il modello uniforme equivalente a una QPU reale.

Il post-processing recupera informazione ancora presente nell’istogramma, ma non può correggere uno stato già corrotto durante il calcolo. Questo limite motiva il passaggio ai codici di correzione e alla decodifica della sindrome nelle sezioni successive.

5.2 Correzione d’errore: codici a ripetizione e di Steane

La prima fase ha mostrato che TOP-4 recupera candidati ancora presenti nell’istogramma, ma nessun post-processing può ricostruire la coerenza persa durante l’evoluzione. La seconda fase interviene quindi durante il calcolo: introduce sindrome e decoder con il codice a ripetizione, passa allo Steane $[[7, 1, 3]]$ e raggiunge infine il surface code. La trattazione formale e i protocolli completi dei primi due laboratori sono raccolti nella Sezione E.1 dell’Appendice E; qui restano gli esperimenti, le curve e le conclusioni quantitative.

5.2.1 Dalla Ridondanza Ingenua alla Codifica

Il no-cloning impedisce di copiare uno stato ignoto e una misura diretta ne distruggerebbe la sovrapposizione [23]. La QEC codifica invece un qubit logico nelle correlazioni di più qubit fisici e misura operatori che rivelano l'errore senza rivelare l'informazione logica [28, 10, 29]. Questa distinzione fra copia e codifica è sviluppata nella Sezione E.1.1.

5.2.2 Il Ciclo della Correzione: Stabilizzatori, Sindrome, Decoder

Ogni codice segue il ciclo codifica–rumore–estrazione della sindrome–decodifica–correzione. Gli stabilizzatori commutano con l'informazione logica e segnalano l'uscita dal sottospazio valido; il decoder traduce la sindrome in una correzione fisica o in un aggiornamento del *Pauli frame*. La distanza d determina quanti errori il codice può correggere secondo l'Eq. E.1; definizioni e dimostrazione operativa sono nell'Appendice E.

5.2.3 Il Panorama dei Codici e la Scelta Metodologica

La progressione è deliberata: il repetition rende leggibile la sindrome, lo Steane corregge un errore arbitrario e il surface code introduce soglia e scalabilità. Il confronto con Shor e five-qubit code è conservato nella Tabella E.1 dell'Appendice E.

5.2.4 Laboratorio I: il Codice a Ripetizione

Il codice bit-flip a 3 qubit

Il codice codifica $\alpha|0\rangle + \beta|1\rangle$ nello stato entangled $\alpha|000\rangle + \beta|111\rangle$. Le parità Z_1Z_2 e Z_2Z_3 identificano un singolo errore X senza misurare i dati; il duale nella base di Hadamard protegge invece da Z. Struttura, circuito di estrazione e limite X/Z sono descritti nella Sezione E.1.2.

Risultati del laboratorio repetition

L'iniezione deterministica verifica tutti i casi della lookup table (Tabella E.2). Il test quantitativo usa 20 000 campioni per punto: il codice fallisce quando almeno due dei

tre qubit subiscono un errore, quindi

$$p_L = 3p^2(1 - p) + p^3 = 3p^2 - 2p^3. \quad (5.2)$$

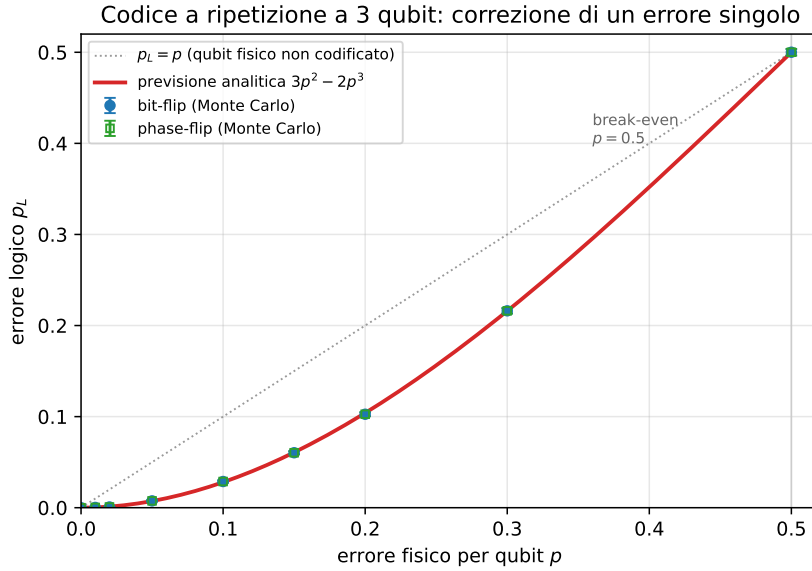


Figura 5.4: Errore logico p_L in funzione dell'errore fisico p per il codice a ripetizione a 3 qubit. I punti Monte Carlo del codice bit-flip e del duale phase-flip (20 000 ripetizioni per punto) coincidono con la previsione analitica $3p^2 - 2p^3$ (Eq. 5.2) entro l'errore statistico. Nella regione $p < 0.5$ la curva giace sotto la bisettrice $p_L = p$: la codifica rende il qubit logico più affidabile del qubit fisico non protetto. Il punto di *break-even* è a $p = 0.5$, dove correzione e assenza di correzione si equivalgono; oltre, la ridondanza amplifica gli errori anziché sopprimerli.

L'accordo con l'Eq. 5.2 è quantitativo su tutto l'intervallo: ad esempio, per $p = 0.10$ si osserva $p_L = 0.0288 \pm 0.0012$ contro il valore teorico 0.0280, e per $p = 0.20$ si ha $p_L = 0.1026 \pm 0.0021$ contro 0.1040. Il messaggio operativo è quello che l'intera parte QEC svilupperà su codici più potenti: sotto una soglia di errore fisico, la codifica *sopprime* l'errore — qui da p a $\mathcal{O}(p^2)$ — mentre sopra la soglia lo *amplifica*. Il codice a ripetizione paga però un limite strutturale già discusso: protegge da un solo tipo di errore per volta. Un errore Z sul codice bit-flip commuta con entrambi gli stabilizzatori, non produce sindrome e resta invisibile — ragione per cui il passo successivo richiede un codice, come quello di Steane, capace di correggere *qualunque* errore su un qubit.

5.2.5 Laboratorio II: il Codice di Steane $[[7, 1, 3]]$

Struttura del codice

Lo Steane [10] è un codice CSS $[[7, 1, 3]]$: la struttura di Hamming separa le sindromi X e Z e consente di correggere un errore arbitrario su un qubit. Parametri, generatori e lookup table sono riportati nella Sezione E.1.3; nel corpo interessa verificare che la codifica appartenga al code space e che la soppressione sia quadratica.

Protocollo di laboratorio

Si preparano $|0_L\rangle$ e $|+_L\rangle$, si iniettano X , Z e Y su tutti i sette qubit e si stima p_L al crescere di p . L'estrazione implementata non è fault-tolerant: il modello Monte Carlo agisce sui dati e assume la sindrome ideale. Questa assunzione, insieme al protocollo completo, è esplicitata nell'Appendice E.

Risultati del laboratorio Steane

La codifica restituisce sindrome nulla 000000 su tutti i 2000 campioni, quindi lo stato preparato appartiene al code space. Le sette sindromi sono distinte e non nulle; la verifica copre tutti i 21 casi X , Z e Y su sette qubit e conferma la biiettività della Tabella E.4.

Il qubit logico batte il fisico: soppressione quadratica. Il terzo controllo misura il beneficio della correzione. Sotto un canale depolarizzante di intensità p per qubit, si stima la probabilità di errore logico p_L con una simulazione Monte Carlo nel formalismo di Pauli (200 000 campioni per punto, seed = 42): per ogni campione si estrae la sindrome, si applica la correzione a peso minimo dalla Tabella E.4 e si verifica se il residuo contiene un operatore logico non banale. La Figura 5.5 mostra il risultato in scala doppio-logaritmica: i punti seguono un andamento $p_L \propto p^2$, con pendenza log-log misurata 1.94 — la firma di un codice di distanza $d = 3$, che corregge ogni errore singolo e fallisce solo a partire da due errori. Poiché gli eventi di peso due, il cui numero di coppie di qubit scala come $\binom{7}{2}$, forniscono il contributo dominante al fallimento, l'errore logico è soppresso da $\mathcal{O}(p)$ a $\mathcal{O}(p^2)$.

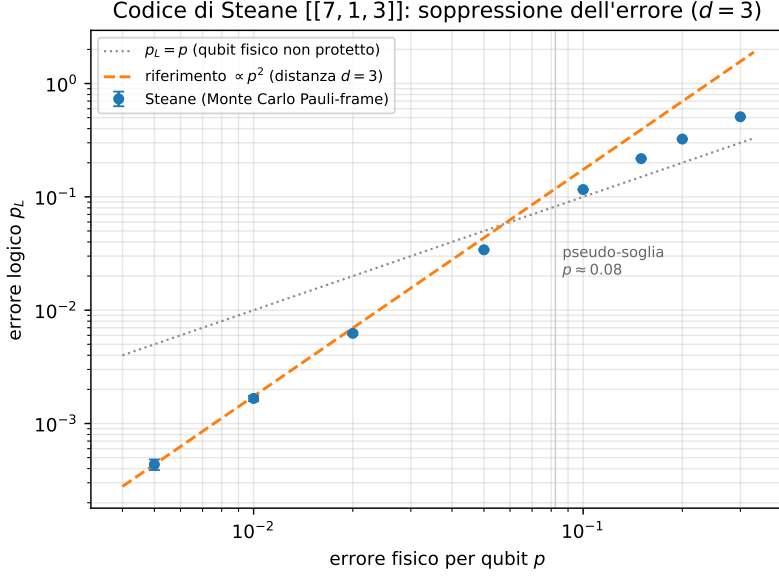


Figura 5.5: Errore logico p_L in funzione dell'errore fisico p (canale depolarizzante), scala doppio-logaritmica. I punti Monte Carlo seguono il riferimento $\propto p^2$ (pendenza $d = 3$); la bisettrice $p_L = p$ rappresenta il qubit fisico non protetto. L'intersezione individua la **pseudo-soglia** $p \approx 0.08$: sotto di essa il qubit logico è più affidabile del fisico, sopra la ridondanza amplifica l'errore. Curva rigenerabile da `figure_src/gen_qec_steane.py`.

Il regime di convenienza è netto: per $p = 0.01$ si ha $p_L = 1.67 \times 10^{-3}$ (già un ordine di grandezza sotto p), per $p = 0.05$ si ha $p_L = 3.4 \times 10^{-2} < p$, mentre a $p = 0.10$ si ha $p_L = 0.116 > p$. La pseudo-soglia — l'intersezione tra la curva e la bisettrice $p_L = p$ — cade attorno a $p \approx 0.08$: al di sotto, codificare conviene.

Confronto con il repetition e assunzioni. Rispetto al repetition code (Sezione 5.2.4), che pure esibisce una soppressione $\propto p^2$, lo Steane paga sette qubit fisici invece di tre ma ottiene un vantaggio qualitativo decisivo: protegge simultaneamente da errori X , Z e Y , mentre il repetition è cieco a un intero tipo di errore. Va infine ribadita l'assunzione dichiarata nella Sezione 5.2.5: l'estrazione di sindrome qui implementata *non è fault-tolerant*. Il modello Monte Carlo inietta errori solo sui qubit dati e assume una lettura di sindrome ideale; un errore sulle ancilla durante l'estrazione può corrompere la sindrome e indurre una correzione errata. La quantificazione di questo effetto — e le tecniche fault-tolerant che lo mitigano — appartiene al terreno del surface code, oggetto del Capitolo 5, dove il decoder opera su cicli ripetuti di misura proprio per distinguere gli errori di dato da quelli di misura.

5.3 Surface code e decodifica

Lo Steane mostra la soppressione a distanza fissa; il surface code [30, 11] introduce invece un parametro scalabile, la distanza d . Questa sezione misura $p_L(p, d)$ con Stim e PyMatching, verifica il comportamento di soglia e mette alla prova separatamente il modello con cui il codice è simulato e quello con cui è decodificato. La teoria operativa, il protocollo completo e i controlli full-state sono conservati nella Sezione E.2 dell'Appendice E.

5.3.1 Il Codice su Griglia: Concetto Operativo

Qubit dati e ancille di misura occupano una griglia locale. Un patch ruotato di distanza d usa d^2 dati e un numero comparabile di ancille; le variazioni di sindrome fra round consecutivi costituiscono i *detection events*. La costruzione completa è nella Sezione E.2.1.

5.3.2 La Decodifica: Minimum Weight Perfect Matching

Il MWPM accoppia gli estremi delle catene osservate minimizzandone il peso [14]. Se errore e correzione differiscono per un ciclo logico, il fallimento è silenzioso e contribuisce a p_L . Stim campiona efficientemente i circuiti stabilizer e PyMatching decodifica il *Detector Error Model* (DEM); Shor non appartiene a questa classe, quindi le metriche non sono trasferite direttamente. Dettagli e listato sono nelle Sezioni E.2.1 e B.2.

5.3.3 Disegno Sperimentale

La campagna copre $d = 3, 5, 7, 9$, le basi memory X/Z, rounds = d e $p \in [2 \times 10^{-3}, 10^{-2}]$. Il criterio è l'inversione dell'ordine delle curve: sotto soglia aumentare d deve ridurre p_L , sopra soglia deve aumentarlo. Griglia prescritta, parte eseguita e numerosità sono nella Tabella E.5 dell'Appendice E.

5.3.4 Risultati

Stim e PyMatching realizzano memory experiment a rounds = d sotto rumore circuit-level. Il campione adattivo, da 2×10^5 a 4×10^7 shot, raccoglie circa duecento eventi logici per punto. La Figura 5.6 riporta $p_L(p)$ per $d = 3, 5, 7, 9$ nelle due basi; protocollo e audit dei seed sono nella Sezione E.2.2.

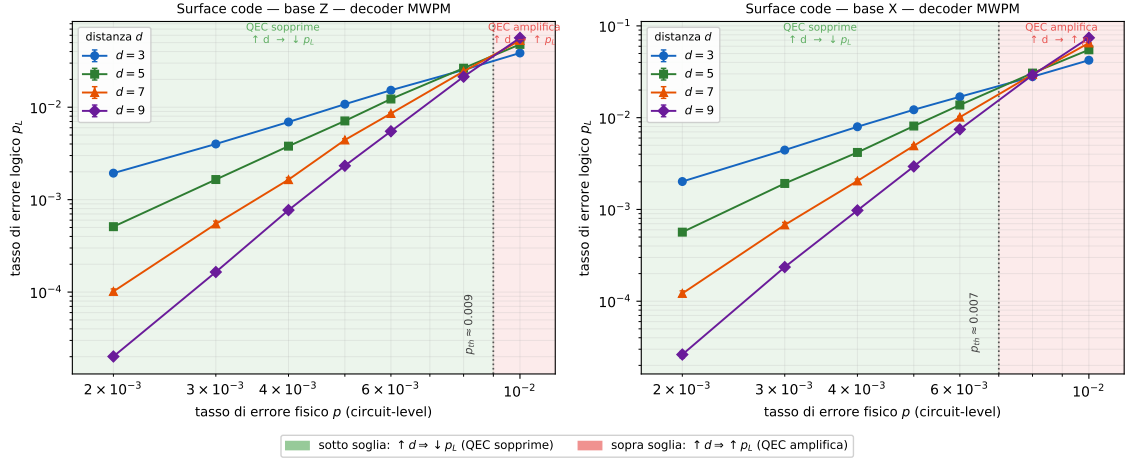


Figura 5.6: Errore logico p_L in funzione dell'errore fisico p (*circuit-level*) per il surface code ruotato a distanza $d = 3, 5, 7, 9$, decoder MWPM, scala doppio-logaritmica; pannello sinistro memory Z, pannello destro memory X. A p basso le curve *divergono* — distanza maggiore dà errore logico molto minore — e si *incrociano* in prossimità della soglia. È la firma del comportamento *threshold-like*: sotto soglia la ridondanza sopprime l'errore, sopra soglia lo amplifica. Il confine fra zona verde e zona rossa è posto alla soglia *letta dall'incrocio* (0.9% in base Z, 0.7% in base X), non a quella stimata dal fit della legge di scala discusso più avanti (0.86% e 0.81%): le due stime usano regimi diversi e vanno confrontate come tali. La campitura è un ausilio di lettura relativo a *questo* circuito, a *questo* decoder e a *questo* modello di rumore, non una partizione universale del piano. Barre d'errore statistiche (spesso più piccole del marcatore); figura rigenerabile da `figure_src/gen_qec_surface_zone.py`.

La soglia è osservata. Il risultato centrale è la presenza di un incrocio netto. Nel regime sotto-soglia la soppressione è marcata e cresce con la distanza: a $p = 2 \times 10^{-3}$ si passa da $p_L = 1.94 \times 10^{-3}$ ($d = 3$) a 5.10×10^{-4} ($d = 5$), a 1.02×10^{-4} ($d = 7$), a 2.01×10^{-5} ($d = 9$) — con fattori di soppressione 3.8, 5.0 e 5.1 per incremento di distanza. La vicinanza degli ultimi due fattori è compatibile con una soppressione approssimativamente geometrica nell'intervallo $d = 5-9$; quattro sole distanze non bastano per affermare che sia stato raggiunto il regime asintotico. L'ordine si mantiene fino a circa $p = 6 \times 10^{-3}$; oltre, le curve si accavallano e a $p = 10^{-2}$ è *invertito* ($p_L = 3.9\%$ per $d = 3$ contro 5.6% per $d = 9$): sopra soglia aumentare la distanza peggiora l'errore. La soglia osservata, $p_{th} \approx 0.9\%$ in base Z e $\approx 0.7\%$ in base X, è una stima specifica del circuito, del decoder e del modello di rumore adottati. Il comportamento qualitativo comune alle due basi costituisce un controllo di coerenza, non una prova di universalità della soglia.

Dalla constatazione alla legge di scala. Con quattro distanze anziché tre la soglia non va più soltanto letta dall'incrocio: può essere *stimata*. La teoria prevede che, sotto soglia, l'errore logico segua

$$p_L(d, p) = A \left(\frac{p}{p_{th}} \right)^{\lfloor (d+1)/2 \rfloor}, \quad (5.3)$$

dove l'esponente è il numero di errori fisici necessari a produrre un errore logico non rilevabile. Adattando (5.3) ai punti con $p \leq 6 \times 10^{-3}$ per tutte e quattro le distanze si ottiene $A = 3.5 \times 10^{-2}$ e $p_{th} = 0.86\%$ in base Z ($A = 3.4 \times 10^{-2}$, $p_{th} = 0.81\%$ in base X), con residuo quadratico medio di 0.08 in $\ln p_L$ — ossia un accordo entro l'8% su cinque ordini di grandezza di errore logico. La stima della soglia ricavata dal fit e quella letta dall'incrocio concordano, il che è un controllo non banale: la prima usa il comportamento sotto soglia, la seconda quello al confine.

Il valore di questa formulazione è che rende il risultato *predittivo* anziché descrittivo. Invertendo la (5.3) si ottiene la distanza minima necessaria a raggiungere un dato errore logico, e con essa il costo in qubit fisici, che per il surface code ruotato è $2d^2 - 1$ per qubit logico.

Tabella 5.4: Distanza minima e costo in qubit fisici per qubit logico, ricavati invertendo la (5.3) con i parametri stimati in base Z. Il confronto fra le due colonne quantifica quanto la fedeltà dei gate valga in risorse: dimezzare l'errore fisico da 2×10^{-3} a 10^{-3} riduce di oltre tre volte il numero di qubit necessari a raggiungere $p_L = 10^{-6}$.

p_L richiesto	$p = 2 \times 10^{-3}$		$p = 10^{-3}$	
	d	qubit fisici	d	qubit fisici
10^{-4}	9	161	5	49
10^{-6}	15	449	9	161
10^{-9}	23	1 057	17	577
10^{-12}	33	2 177	23	1 057

Distanza dall'hardware attuale. Il confronto fra la soglia e i preset uniformi del Capitolo 3 è soltanto qualitativo. Il parametro p del memory experiment circuit-level non coincide con λ_{2q} del circuito di Shor, e il logical error rate dell'osservabile di memoria non coincide automaticamente con una probabilità per gate logico. Nel modello simulato, ridurre p sotto la regione di incrocio rende vantaggioso aumentare la distanza; sopra tale regione la ridondanza amplifica il rumore. Il Capitolo 5 esplora invece una griglia esogena e dichiarata di errore Pauli per gate compilato:

non trasferisce questi valori uno-a-uno. Per una previsione end-to-end servirebbe una mappatura per operazione logica e relativo circuito fault-tolerant.

La legge di scala permette però di andare oltre il singolo punto e di rispondere in modo diretto alla domanda di risorse. L'istanza $N = 15$ richiede dodici qubit logici (otto di controllo e quattro di lavoro); combinando questo dato con la Tabella 5.4 si ottiene che una versione codificata al livello $p_L = 10^{-4}$ — quello che il Capitolo 5 individua come sufficiente — costerebbe **circa 1 900 qubit fisici** a $p = 2 \times 10^{-3}$, e ne basterebbero **poco meno di 600** se la fedeltà dei gate migliorasse di un fattore due. È il modo più concreto di enunciare il costo della correzione: non un fattore moltiplicativo astratto, ma due ordini di grandezza di qubit per eseguire in modo protetto la fattorizzazione di un numero a quattro bit.

Riproducibilità. Seed, numerosità adattiva e intervalli binomiali sono registrati punto per punto; una seconda esecuzione indipendente riproduce le curve e le soglie entro l'errore statistico. Il protocollo a due passate e i limiti dello scaling a $2d$ sono documentati nella Sezione E.2.2.

5.3.5 Oltre il Modello di Pauli: Errori Coerenti e Leakage

Stim fornisce una baseline Clifford–Pauli, non una previsione calibrata dell'hardware: sovrarotazioni coerenti e leakage possono violarne le assunzioni [17]. Il controllo full-state a distanza 3 trova, a $p = 3 \times 10^{-2}$, meno flip grezzi sotto sovrarotazione (0.237 contro 0.262) ma un errore MWPM leggermente maggiore (0.0789 contro 0.0767, +2.9%). Le misure ripetute interrompono parte dell'accumulo coerente; il canale Pauli resta un'approssimazione, non un'identità.

Sul decoder ibrido, i guadagni sotto Pauli sono 1.019–1.041 e sotto errore coerente 1.029–1.052: lo scarto è circa una deviazione standard e non significativo. Il vantaggio appreso non dipende quindi dal generico carattere non-Pauli, ma dalla non decomponibilità del meccanismo in archi del grafo. Teoria, esempio a singolo qubit, Figura E.1, dati completi e implicazioni channel-first sono conservati nella Sezione E.2.3 dell'Appendice E.

5.3.6 La Decodifica Appresa: Sintesi della Campagna

Il MWPM richiede che ogni meccanismo accenda al più due detection events e sia rappresentabile come un arco. Il crosstalk fra due dati adiacenti può accenderne quattro: resta Pauli, ma non è decomponibile. Questa è la seconda idealizzazione

sottoposta a prova. Lo stesso tipo di modello appreso giudicato superfluo sull'istogramma viene applicato alla sindrome, che conserva relazioni spaziali e temporali assenti dall'output.

La Tabella 5.5 anticipa l'esito dei sette esperimenti principali. Le evidenze decisive E1–E4 sono presentate subito dopo la sintesi; i controlli E5–E12 sono riportati per esteso nella Sezione E.3.

	domanda	esito
E1	sostituire il matching con una rete?	No : nessuna vittoria su 14 configurazioni; pareggio a $d = 3$, sconfitta a $d = 5$
E1b	quanti dati servirebbero?	10^5 – 10^6 campioni solo per pareggiare a $d = 3$
E2	e se il rumore esce dal modello del decoder?	la rete vince a $d = 3$ (1.14 – $1.19\times$)
E3	sa prevedere <i>quando</i> il matching sbaglia?	sì, $AUC = 0.941$
E4	affiancarlo invece di sostituirlo?	$-18/ - 21\%$ a $d = 3$, $-1/ - 3\%$ a $d = 5$, nullo a $d = 7$
E5	è trasferibile fra profili di rumore?	sì, 12 trasferimenti su 12 battono il matching
E6	un modello <i>mal calibrato</i> apre un margine?	No : 0 su 12; il matching perde solo l'1–3%
E7	un decoder analitico migliore lo cattura già?	a $d = 5$ sì, e meglio della rete

Tabella 5.5: Sintesi dei sette esperimenti principali. E1–E4 sono sviluppati nel corpo; E5–E12, con tabelle, figure e test di significatività, sono nella Sezione E.3.

Che cosa se ne ricava. Il perimetro è una rete densa e dati alla portata di questa tesi, non il limite dei decoder neurali industriali [31]. In tale perimetro **sostituire** il decoder analitico non conviene; **affiancarlo** può ridurre p_L fino al 21% quando il rumore correlato esce dalla struttura del grafo, senza peggiorarlo entro la risoluzione quando il modello è corretto. E6 esclude che basti una cattiva calibrazione; E7 mostra che BP+OSD recupera il 17–27% a $d = 5$, contro l'1–3% dell'ibrido.

E8–E11 mostrano inoltre che i guadagni non si sommano: matching con residuo, BP+OSD con residuo e rete autonoma raggiungono lo stesso pavimento entro l'1–2%, anche aumentando di quaranta volte i parametri. Ne segue il criterio definitivo: un modello appreso produce valore quando la classe di errore non è rappresentabile nella struttura del metodo analitico, non quando lo è con parametri imprecisi. A $d = 7$ il vantaggio si annulla; prima di introdurre apprendimento va quindi adottato il miglior decoder analitico disponibile.

Evidenze principali E1–E4. La sintesi precedente è ora motivata sui dati e sui confronti appaiati che sostengono il risultato centrale.

Le evidenze principali seguono la catena logica controllo negativo–violazione strutturale–costruzione residuale: E1 verifica la sostituzione a parità di modello, E2 introduce il crosstalk non decomponibile, E3 misura la capacità di riconoscere i fallimenti del matching ed E4 costruisce il decoder ibrido. I controlli di trasferibilità, calibrazione, confronto con BP+OSD, composizione e scalabilità sono raccolti nella Sezione E.3 dell’Appendice E.

La Sezione 5.3.5 ha mostrato che il modello di rumore usato per *simulare* il codice è un’idealizzazione. Esiste però una seconda idealizzazione, indipendente dalla prima e altrettanto conseguente, che riguarda il modello di rumore usato per *decodificare*. Il MWPM non è un algoritmo generico: presuppone che gli errori si organizzino in un grafo, ossia che ogni meccanismo d’errore accenda al più due detection events e sia quindi rappresentabile come un arco. È questa proprietà — la **decomponibilità in archi** — a rendere il problema della decodifica un accoppiamento di peso minimo, ed è ciò che la libreria richiede esplicitamente quando si costruisce il modello d’errore con `decompose_errors=True`. Un errore correlato che colpisca simultaneamente due qubit dati adiacenti accende invece fino a quattro detector e non ammette alcun arco che lo rappresenti: non viola l’ipotesi di Pauli, che resta soddisfatta, ma viola quella di decomponibilità. Le due idealizzazioni sono distinte e vanno tenute separate.

Questa distinzione individua con precisione la domanda a cui la presente sezione risponde: *esiste un regime in cui un decoder **appreso dai dati** — che non riceve un grafo esplicito degli errori, ma stima la regola decisionale dai campioni — fa meglio del decoder analitico?* La domanda chiude anche il cerchio metodologico dell’intera tesi. Il Capitolo 5 ha stabilito che l’apprendimento automatico applicato *a valle* dell’esecuzione non aggiunge nulla; qui lo stesso strumento — un percettrone multistrato di `scikit-learn`, la medesima tecnologia del classificatore giudicato superfluo — viene applicato alla **sindrome**, cioè a un dato che conserva relazioni spaziali e temporali non rappresentate dal solo istogramma finale. Cambia il punto di applicazione, non la tecnologia: è esattamente il confronto che permette di attribuire l’esito al *dove*, e non al *come*.

Disegno sperimentale

Il dato di addestramento non richiede alcuna infrastruttura aggiuntiva: è già ciò che Stim produce per stimare p_L . Il campionatore restituisce la matrice dei detection

events e il corrispondente esito dell'osservabile logico, che sono rispettivamente le variabili indipendenti e l'etichetta del problema di apprendimento supervisionato:

```
det, obs = sampler.sample(shots, separate_observables=True)
# X = det (detection events) -> sindrome misurata
# y = obs (logical observable) -> flip logico da predire
```

Listing 5.1: Il dataset del decoder appreso coincide con l'output del campionatore di Stim: `det` è la sindrome osservata, `obs` l'esito logico da predire. Il decoder analitico e quello appreso ricevono quindi esattamente la stessa informazione.

Il modello è un MLP con due strati nascosti (128, 64), attivazione ReLU, ottimizzatore Adam e arresto anticipato su una frazione di validazione interna; il decoder di riferimento è PyMatching sullo stesso insieme di test. Tutte le stime adottano il codice ruotato in base Z con $\text{rounds} = d$ e rumore *circuit-level*, come nella campagna della Sezione 5.3.4. La Tabella 5.6 riassume i quattro esperimenti.

	Domanda	Condizione	Campioni
E1	la rete sostituisce il matching?	rumore nominale, modello esatto	4×10^5 per punto
E1b	quanto costa in dati il pareggio?	$d = 3, p = 3 \times 10^{-3}$	fino a 8×10^5
E2	e se il rumore esce dal modello?	crosstalk correlato	4×10^5 per punto
E4	e se la si affianca invece di sostituirla?	crosstalk correlato	8×10^5 per punto

Tabella 5.6: I quattro esperimenti sulla decodifica appresa. E3, discusso nella Sezione 5.3.6, riusa la configurazione di E2 senza nuovi campionamenti.

E1 — la sostituzione a parità di modello

Il primo esperimento pone il confronto nelle condizioni più favorevoli al decoder analitico: rumore uniforme e modello d'errore esatto, cioè esattamente ciò per cui il MWPM è costruito. Su quattordici configurazioni ($d = 3, 5$; sette valori di p) la rete **non supera mai** il matching con significatività statistica. A distanza 3 lo affianca, con un rapporto $p_L^{\text{MWPM}}/p_L^{\text{rete}}$ che sale da 0.83 a $p = 2 \times 10^{-3}$ fino a 1.01 a $p = 10^{-2}$; a distanza 5 resta indietro di un ordine di grandezza (da 0.05 a 0.29).

L'andamento monotono suggerisce la spiegazione, che il pannello (b) della Figura 5.7 verifica direttamente: la rete è limitata dal **volume di dati**, non dalla capacità del modello. Fissando $d = 3$ e $p = 3 \times 10^{-3}$ e facendo variare il solo

insieme di addestramento, il rapporto cresce in modo regolare da 0.50 (2.5×10^4 campioni) a 1.02 (8×10^5 campioni). L'ultimo punto misura $p_L = 3.98 \times 10^{-3}$ contro 4.04×10^{-3} del matching: un sorpasso nominale che, su un insieme di test di 10^5 campioni, vale 0.2 deviazioni standard e **non è statisticamente significativo**. Il risultato di E1 è dunque un pareggio, e il suo contenuto informativo è il prezzo di quel pareggio: dell'ordine di 10^5 – 10^6 campioni a distanza 3, con un fabbisogno che cresce rapidamente con d — coerente con il fatto che i decoder neurali allo stato dell'arte ricorrono ad architetture ricorrenti e a volumi di dati di diversi ordini di grandezza superiori [19, 32]. La stessa diagnosi è raggiunta per via indipendente da Yan, Li e Du, che studiano decoder neurali su surface code fino a distanza 9 (161 qubit) sotto vincoli congiunti di accuratezza e latenza, e concludono che le prestazioni a breve termine sono determinate più dalla *scala dei dati* che dalla complessità architetturale [33]. La convergenza è utile perché i due percorsi differiscono: qui il volume di dati è la variabile manipolata a modello fisso, là è il confronto fra architetture a essere condotto sistematicamente.

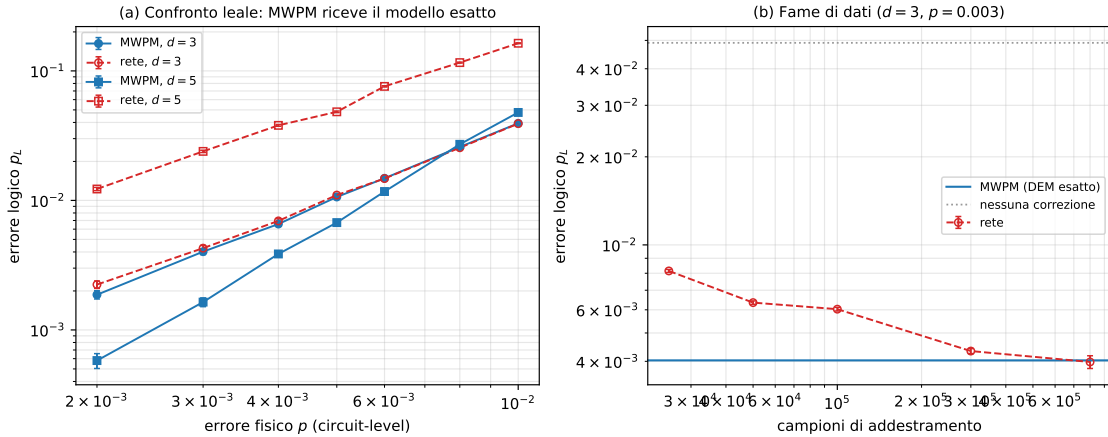


Figura 5.7: (a) Errore logico del decoder appreso e del MWPM in funzione dell'errore fisico, per $d = 3$ e $d = 5$, con modello d'errore esatto fornito al matching. La rete affianca il matching a distanza 3 e resta al di sopra a distanza 5. (b) Errore logico della rete al crescere dell'insieme di addestramento ($d = 3$, $p = 3 \times 10^{-3}$): l'avvicinamento al matching è monotono e il pareggio si raggiunge fra 3×10^5 e 8×10^5 campioni. Rigenerabile da `figure_src/gen_qec_neural.py`.

E2 — quando il rumore esce dal modello del decoder

Il secondo esperimento rimuove l'ipotesi di decomponibilità. Al circuito viene aggiunto un canale correlato a due qubit fra i **qubit dati fisicamente adiacenti** sulla griglia, una volta per ciclo di sindrome e con intensità p_{ct} : è il modello elementare

del **crosstalk**, la quarta fonte fisica di rumore discussa nel Capitolo 3, che l'accoppiamento capacitivo residuo produce fra qubit vicini. L'errore fisico resta fissato a $p = 3 \times 10^{-3}$, sotto la soglia misurata nella Sezione 5.3.4. Si confrontano tre bracci: il matching con il modello *nominale* (rumore uniforme — ciò che la calibrazione del dispositivo dichiara), il matching con il modello del circuito *reale* (che include il crosstalk), e la rete addestrata sui campioni effettivamente osservati.

Il crosstalk degrada la decodifica in modo severo: a $p_{ct} = 5 \times 10^{-3}$ l'errore logico a $d = 3$ passa da 3.8×10^{-3} a 3.1×10^{-2} , quasi un ordine di grandezza, pur restando l'errore fisico invariato. In questo regime la rete supera il matching su tutte e quattro le intensità testate a distanza 3, con guadagni fra 1.14 e 1.19 (Tabella 5.7). A distanza 5 il vantaggio non si manifesta: la rete perde, per la stessa ragione di E1 — lo spazio delle sindromi cresce più rapidamente della sua capacità di generalizzazione.

d	p_{ct}	nessuna corr.	MWPM nominale	MWPM reale	rete
3	5×10^{-3}	0.0845	0.03097	0.05656	0.02707
	1×10^{-2}	0.1163	0.05966	0.08975	0.05012
	2×10^{-2}	0.1746	0.11184	0.15107	0.09547
	4×10^{-2}	0.2652	0.20806	0.25481	0.17568
5	5×10^{-3}	0.2007	0.03521	0.02796	0.08266
	1×10^{-2}	0.2648	0.08655	0.06526	0.13957
	2×10^{-2}	0.3565	0.19933	0.15662	0.28906
	4×10^{-2}	0.4459	0.37458	0.32515	0.44634

Tabella 5.7: Errore logico p_L in presenza di crosstalk fra qubit dati adiacenti, a errore fisico costante $p = 3 \times 10^{-3}$. “Nessuna correzione” è la frequenza con cui l’osservabile logico risulta alterato in assenza di decodifica. A distanza 3 la rete domina entrambe le varianti del matching; a distanza 5 il quadro si inverte.

Un’osservazione inattesa: conoscere il rumore non basta. Il braccio con il modello d’errore *reale* — quello che conosce il crosstalk — si comporta in modo opposto alle due distanze, ed è il risultato più istruttivo dell’esperimento. A $d = 3$ è il **peggiore** dei tre (5.7×10^{-2} contro 3.1×10^{-2} del modello nominale a $p_{ct} = 5 \times 10^{-3}$): fornire al matching l’informazione corretta lo peggiora. La ragione è strutturale e non contraddittoria. Poiché l’errore correlato non è rappresentabile come arco, la libreria è costretta a scomporlo in archi equivalenti, e su un grafo piccolo come quello di distanza 3 gli archi spurî introdotti dalla scomposizione sviano l’accoppiamento più di quanto l’informazione aggiuntiva lo guidi. A $d = 5$, dove il grafo è più ricco e

la scomposizione pesa relativamente meno, il bilancio si inverte e il modello reale torna a essere il migliore.

Ne discende un enunciato che vale la pena isolare, perché delimita esattamente ciò che un decoder analitico può e non può fare: *quando la classe di errori non è esprimibile nella struttura del decoder, il limite non è la calibrazione ma la struttura stessa* — e migliorare la stima del rumore può non produrre alcun beneficio, o risultare controproducente.

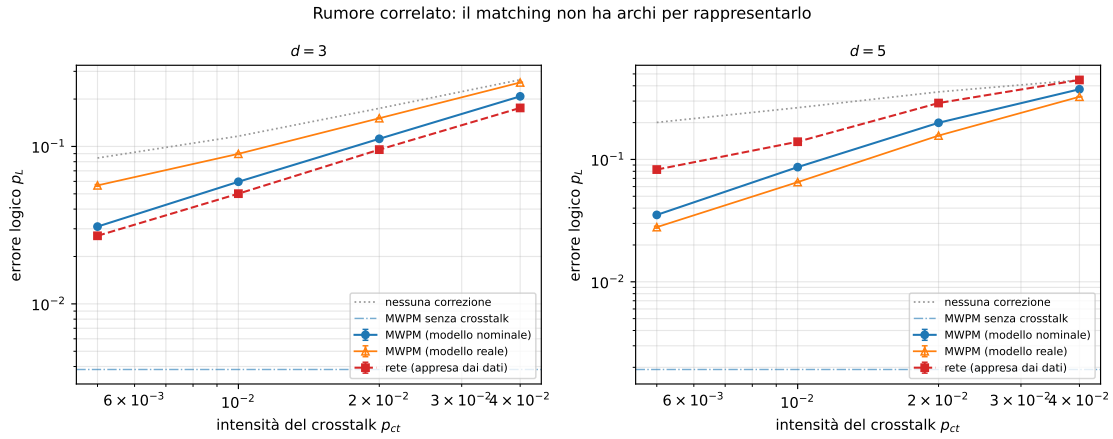


Figura 5.8: Errore logico in funzione dell'intensità del crosstalk, a errore fisico costante $p = 3 \times 10^{-3}$. A distanza 3 la rete (rosso) domina il matching sia con modello nominale (blu) sia con modello reale (arancio), che risulta il peggiore; a distanza 5 l'ordine si inverte. La linea tratteggiata inferiore indica l'errore logico del matching in assenza di crosstalk, cioè il livello che la decodifica raggiungerebbe se il modello fosse corretto.

E3 — la rete come validatore della correzione

I due esperimenti precedenti valutano la rete come *sostituto* del decoder. Una domanda diversa, e operativamente più modesta, è se la rete sappia riconoscere gli shot in cui il matching ha sbagliato — cioè se possa **confermare o negare** la correzione proposta, senza doverla rimpiazzare. Si definisce allora la confidenza che la rete assegna alla risposta effettivamente data dal matching, e si verifica quanto una confidenza bassa predica un fallimento reale.

Nella configurazione $d = 3$, $p_{ct} = 10^{-2}$, in cui il matching sbaglia 5966 shot su 10^5 , il flag così costruito discrimina i fallimenti con $AUC = 0.941$. La Tabella 5.8 ne riporta il compromesso: abbassando la soglia si segnalano meno shot con precisione crescente, fino a individuare l'85.9% di errori veri segnalando appena lo 0.59% delle esecuzioni.

Soglia di confidenza	Shot segnalati	Precision	Recall
0.5	2.92%	0.664	0.324
0.3	1.54%	0.783	0.202
0.2	1.16%	0.825	0.160
0.1	0.59%	0.859	0.085

Tabella 5.8: Capacità della rete di segnalare gli shot in cui il MWPM ha sbagliato ($d = 3$, $p_{ct} = 10^{-2}$, $AUC = 0.941$). La precisione cresce al restringersi della segnalazione: segnalando lo 0.59% degli shot, l'85.9% delle segnalazioni corrisponde a un fallimento reale.

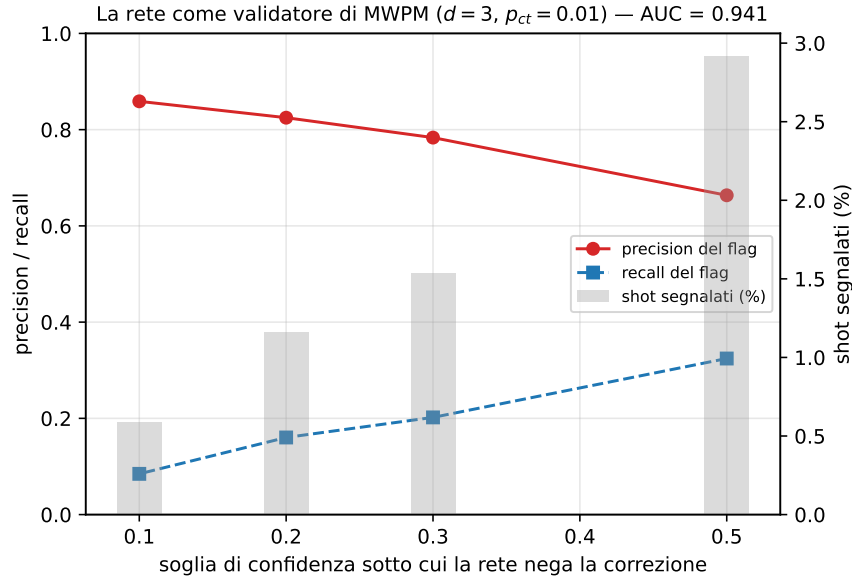


Figura 5.9: Precision e recall del flag in funzione della soglia di confidenza, con la frazione di shot segnalati (barre, asse destro). Il flag non sostituisce la decodifica: la annota.

E4 — il decoder ibrido

L'esito di E3 suggerisce la costruzione che chiude la sezione. Se la rete riconosce i fallimenti del matching senza saper decodificare meglio di esso, allora il suo impiego corretto non è sostituirlo ma **correggerlo**. Si definisce quindi un decoder **ibrido** in forma residuale:

$$\hat{y}_{\text{ibrido}} = \hat{y}_{\text{MWPM}} \oplus [\text{Pr}(\text{MWPM sbaglia} \mid \text{sindrome}) \geq \tau], \quad (5.4)$$

dove $\oplus$ è la somma modulo 2 e la rete riceve in ingresso la sindrome e la decisione del matching. La soglia di ribaltamento τ è scelta su un insieme di **validazione separato** e poi applicata invariata all'insieme di test: sceglierla sul test costituirebbe una fuga di informazione, e su guadagni dell'ordine di pochi punti percentuali sarebbe un vizio sufficiente a invalidare il risultato. Fra le soglie candidate figura il valore che non ribalta mai, cosicché il criterio di validazione può sempre ricadere sul matching puro quando nessun ribaltamento conviene.

La formulazione ha una motivazione precisa, oltre alla convenienza pratica: alla rete non si chiede più di riprodurre da zero il calcolo di parità che il matching già esegue correttamente nella grande maggioranza dei casi, ma soltanto di modellarne il *residuo*, che è un evento raro e strutturato. È un compito più facile, ed è per questo che riesce dove la sostituzione fallisce.

d	p_{ct}	MWPM	rete sola	ibrido	ribalta	McNemar
3	0	0.00415	0.00409	0.00417	0.10%	$p = 0.67$
	5×10^{-3}	0.03089	0.02680	0.02607	1.19%	3×10^{-87}
	1×10^{-2}	0.05829	0.04917	0.04851	2.69%	1×10^{-156}
	2×10^{-2}	0.11144	0.09248	0.09183	5.38%	2×10^{-312}
	4×10^{-2}	0.20750	0.17615	0.17455	11.04%	$< 10^{-320}$
5	0	0.00165	0.01900	0.00165	0.00%	—
	5×10^{-3}	0.03561	0.06311	0.03521	0.32%	1×10^{-3}
	1×10^{-2}	0.08704	0.12874	0.08428	1.17%	3×10^{-30}
	2×10^{-2}	0.20231	0.25198	0.19662	2.88%	6×10^{-51}
	4×10^{-2}	0.37581	0.44182	0.37385	2.96%	4×10^{-7}
7	0	0.00055	0.06888	0.00055	0.00%	—
	5×10^{-3}	0.03707	0.24671	0.03707	0.00%	—
	1×10^{-2}	0.11319	0.36749	0.11318	0.00%	$p = 0.32$
	2×10^{-2}	0.29048	0.46219	0.29048	0.00%	—
	4×10^{-2}	0.46741	0.49980	0.46741	0.00%	—

Tabella 5.9: Errore logico del decoder ibrido (5.4) confrontato con il MWPM e con la rete usata come sostituto. “Ribalta” è la frazione di shot in cui la rete inverte la decisione del matching. Il valore di significatività è quello del test di McNemar, appaiato sugli stessi campioni: poiché i due decoder differiscono *soltanto* sugli shot ribaltati, le coppie discordanti coincidono con essi e il test è esatto.

L’ibrido riduce l’errore logico in tutte e otto le configurazioni con crosstalk a distanza 3 e 5, con guadagni del 18–21% nel primo caso e dell’1–3% nel secondo, tutti significativi al test di McNemar. Il dettaglio delle coppie discordanti rende l’effetto leggibile: a $d = 3$ e $p_{ct} = 4 \times 10^{-2}$ la rete rompe 7745 shot che il matching

aveva risolto correttamente e ne aggiusta 14 335, un rapporto di quasi due a uno; a $d = 5$ il bilancio si stringe (2 769 contro 3 160), da cui il guadagno più contenuto ma comunque netto.

Due proprietà meritano attenzione perché riguardano il comportamento del sistema fuori dal regime per cui è stato addestrato. La prima è che la frazione di ribaltamenti **scala con l'intensità del crosstalk** (dallo 0.10% all'11.04% a distanza 3): la rete non interviene a caso, ma in misura proporzionale a quanto il modello del matching è sbagliato. La seconda è il comportamento in assenza di crosstalk, dove il matching è già adeguato e non c'è nulla da correggere: a $d = 5$ la rete effettua **zero ribaltamenti** e l'ibrido coincide cifra per cifra con il MWPM, mentre a $d = 3$ ribalta lo 0.10% degli shot con effetto statisticamente nullo ($p = 0.67$). Il decoder ibrido *degrada con grazia*: riconosce di non avere nulla da aggiungere e si fa da parte. Va detto con esattezza che questa garanzia riguarda il criterio di validazione e non ogni singolo punto dell'insieme di test — il valore leggermente superiore a $d = 3$, $p_{ct} = 0$ ne è un esempio, entro il rumore statistico — sicché la formulazione corretta è che l'ibrido non risulta *mai peggiore del matching in modo statisticamente rilevabile*.

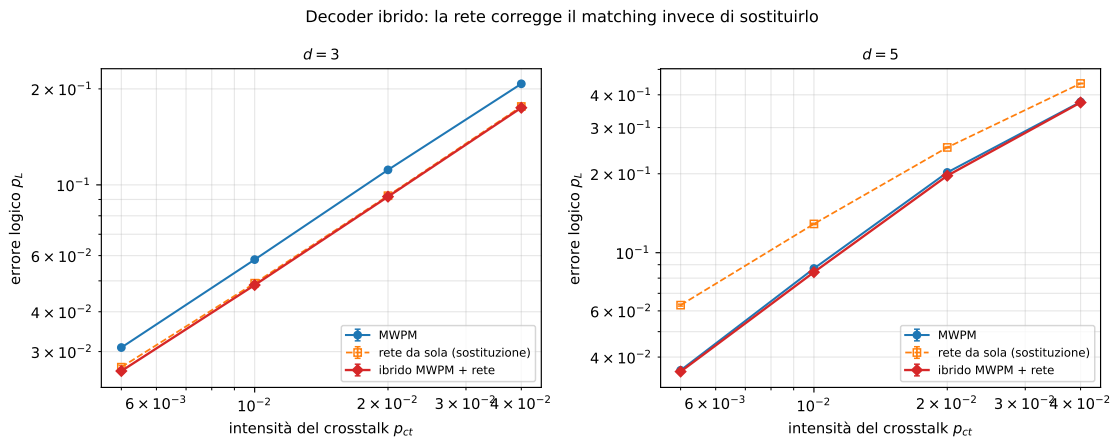


Figura 5.10: Errore logico dei tre decoder in funzione dell'intensità del crosstalk. L'ibrido (rosso) è il migliore a entrambe le distanze; la rete usata come sostituto (arancio) domina il matching a $d = 3$ ma lo perde a $d = 5$, dove l'ibrido continua invece a guadagnare.

5.4 Analisi di sensibilità di Shor

Le sezioni precedenti misurano la correzione d'errore su circuiti dedicati, mentre le prime analisi del capitolo descrivono la degradazione di Shor sotto rumore fisico. Collegare direttamente i due insiemi richiederebbe una compilazione fault-tolerant completa, con registri codificati, cicli di sindrome, decoder e risorse non-Clifford. Poiché questa architettura non viene simulata, qui si adotta un esperimento di **sensibilità fenomenologica**: si applica al circuito compilato un errore Pauli indipendente per gate, indicato con p_L , e si osserva la probabilità di fattorizzazione. Il pedice “L” ne segnala il ruolo concettuale, ma non rende il parametro intercambiabile con le metriche logiche aggregate ottenute dai codici correttori.

5.4.1 L'architettura a quattro livelli

L'analisi separa Shor ideale, Shor con rumore fisico, QEC studiata in circuiti isolati e proxy uniforme per gate. Questa distinzione evita di interpretare il quarto livello come una simulazione fault-tolerant. Esistono infatti sia una barriera dimensionale — codificare i dodici qubit di $N = 15$ conduce rapidamente a centinaia di qubit — sia una barriera strutturale: Stim tratta efficientemente circuiti stabilizer, mentre l'aritmetica modulare e la QFT rendono Shor non-Clifford. Schema completo, strumenti e manifest sono riportati nella Sezione E.4.1.

5.4.2 Definizione operativa del proxy

In M8, p_L è la **probabilità totale di applicare un Pauli non identità dopo ciascun gate compilato incluso nel modello**. Per un gate su q qubit il canale distribuisce p_L uniformemente sui $4^q - 1$ Pauli non identità. Poiché Qiskit Aer usa il parametro depolarizzante λ_q , lo script imposta

$$\lambda_1 = \frac{4}{3}p_L, \quad \lambda_2 = \frac{16}{15}p_L. \quad (5.5)$$

Il rumore agisce su **sx/x** e **cx**; le rotazioni **rz** sono virtuali. p_L è dunque un proxy *per gate nella base compilata*, non una probabilità per ciclo QEC, round di sindrome o blocco logico.

Il circuito canonico usa $N = 15$, $a = 7$, otto qubit di conteggio e quattro di lavoro. Ogni punto aggrega 20 repliche indipendenti da 4096 shot; una misura ha successo quando frazioni continue e controlli con il massimo comun divisore restituiscono

fattori non banali. Seed, conteggi dei gate e hash del circuito sono conservati nella Sezione E.4.1. La domanda supportata è quindi come vari il successo di questa specifica implementazione al crescere di un residuo Pauli uniforme, non quale distanza di surface code corrisponda a ciascun punto.

5.4.3 Il contesto delle stime fault-tolerant

Le stime per istanze crittografiche comprendono routing, cicli di sindrome, fabbriche di stati magici e schedulazione e non derivano dall'estrapolazione di $N = 15$. La letteratura passa dai circa 20 milioni di qubit fisici stimati da Gidney ed Ekerå per RSA-2048 [6] a proposte molto meno costose, ma fondate su codici e assunzioni architettureali differenti. La traiettoria completa, incluse le più recenti stime non ancora sottoposte a revisione paritaria, è discussa nella Sezione E.4.2 e nella Tabella E.15. M8 resta pertanto una sensibilità locale, non una previsione di risorse hardware.

5.4.4 Risultati M8 e M13

M8: successo per singola misura

La Tabella 5.10 riporta la griglia completa e gli intervalli Wilson al 95%. Il controllo di monotonia combina incertezza binomiale e dispersione fra repliche con copertura simultanea sui confronti adiacenti.

p_L	P_{success}	IC 95%	p_L	P_{success}	IC 95%
0	0,7489	[0,7459;0,7518]	0,005	0,6450	[0,6417;0,6483]
0,0001	0,7469	[0,7439;0,7498]	0,01	0,5651	[0,5617;0,5685]
0,0005	0,7373	[0,7343;0,7403]	0,02	0,4518	[0,4484;0,4552]
0,001	0,7250	[0,7219;0,7280]	0,05	0,2953	[0,2921;0,2984]
0,0017	0,7083	[0,7051;0,7114]	0,1	0,2503	[0,2474;0,2533]
0,002	0,6992	[0,6961;0,7024]	0,2	0,2441	[0,2412;0,2471]
			0,5	0,2459	[0,2430;0,2489]

Tabella 5.10: M8: probabilità di ricavare i fattori per singola misura, 20 repliche da 4096 shot per punto.

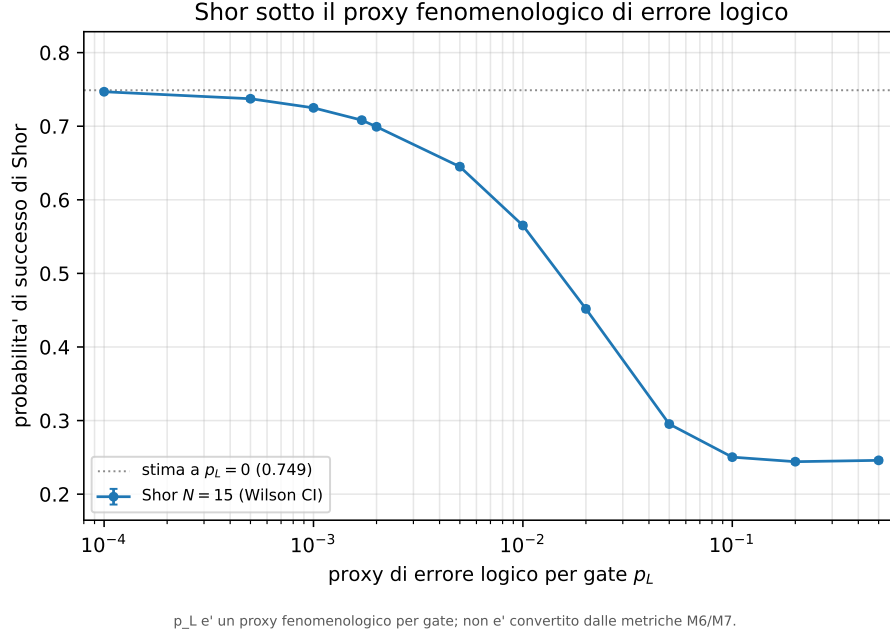


Figura 5.11: Probabilità di successo di Shor ($N = 15$, $a = 7$) in funzione di p_L , con IC Wilson al 95%. Non sono sovrapposti valori Steane o surface code, perché le rispettive metriche aggregate non sono direttamente convertibili nel proxy M8. Figura rigenerata dal JSON schema 2 con `figure_src/gen_shor_logico.py`.

La curva è compatibile con una funzione non crescente al livello simultaneo prefissato, ma non ogni diminuzione adiacente è significativa: l'affermazione supportata è “nessun aumento significativo”. Il successo passa da 0,7489 a 0,4518 tra $p_L = 0$ e 0,02, poi raggiunge un plateau empirico vicino a 0,245 oltre $p_L = 0,1$. Il plateau dipende dagli esiti che il post-processing di $N = 15$ riconosce come utili e non è universale.

Ripetizioni indipendenti

Il valore ideale 0,7489 riguarda una singola misura. Per tentativi indipendenti la probabilità cumulativa è

$$P(k) = 1 - (1 - P_s)^k. \quad (5.6)$$

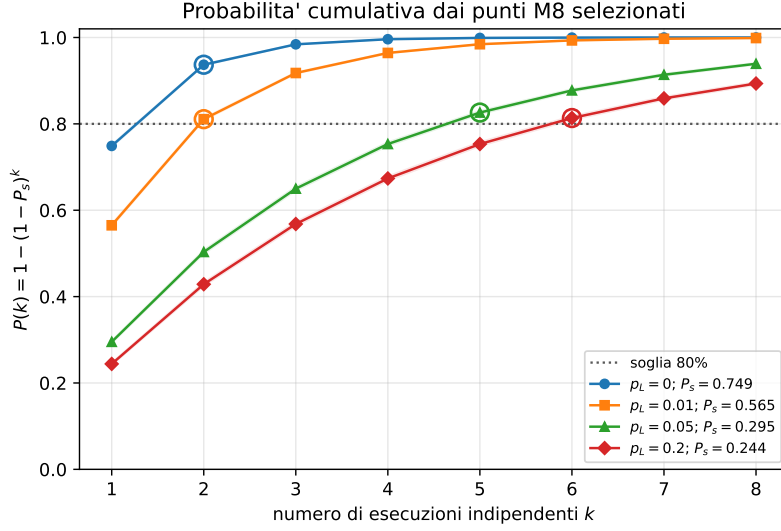


Figura 5.12: Probabilità cumulativa per $p_L \in \{0; 0,01; 0,05; 0,2\}$. La soglia dell'80% richiede rispettivamente 2, 2, 5 e 6 tentativi; le bande propagano gli estremi degli intervalli Wilson.

Due tentativi danno circa 0,937 nel caso ideale e 0,811 per $p_L = 0,01$. L'80% va quindi letto come soglia cumulativa, non come requisito del singolo run; ipotesi e controllo statistico sono richiamati nella Sezione E.4.3.

M13: TOP-4 con pareggi espliciti

M13 usa lo stesso proxy in 200 repliche da 1024 shot per punto. Il TOP-4 resta 1,000 fino a $p_L = 0,05$, benché la massa utile per misura scenda da 0,7512 a 0,2979; a $p_L = 0,2$ vale invece 0,670, con intervallo identificato $[0,535; 0,860]$ dovuto ai pareggi. Il contrasto preregistrato rispetto a $p_L = 0$ mostra una diminuzione di 0,330, IC Newcombe 95% $[0,266; 0,398]$, superiore al margine 0,10. Protocollo, griglia e analisi di sensibilità ai pareggi sono conservati nella Sezione E.5.4 dell'Appendice E.5.

Barriera di scalabilità e compatibilità delle metriche

M8 e M13 restano limitati a $N = 15$. La variante Beauregard validata per $N = 21$ raggiunge 21 036 gate cx e profondità 23 081 su 22 qubit: il limite osservato riguarda la simulazione classica rumorosa adottata, non l'algoritmo quantistico. Inoltre errore Steane, errore logico del surface code e guadagno del decoder hanno unità statistiche diverse da p_L e non possono essere proiettati direttamente sulla curva. Il contratto completo è esplicitato nella Sezione E.4.4 e nella Tabella E.16.

Discussione

M8 stabilisce una relazione quantitativa entro il proprio modello: ridurre il residuo Pauli per gate riporta il successo verso il limite ideale. Non dice quale codice realizzi un determinato p_L , né confronta Steane e surface code a parità di hardware. Servirebbero un gate set logico, il conteggio dei cicli di correzione, latenze e correlazioni del decoder e operazioni non-Clifford fault-tolerant.

Il confronto tra successo per misura e TOP-K chiarisce inoltre che quattro candidati possono coprire rapidamente un esito utile nella piccola istanza $N = 15$, anche quando la massa utile è degradata. Un TOP-4 elevato non prova quindi che l'informazione quantistica sia intatta e va letto insieme alla metrica per misura e all'incertezza dei pareggi.

Esperimenti recenti su qubit logici confermano il verso qualitativo del beneficio, ma non forniscono la conversione fra risultati QEC e M8. La loro portata, la natura pre-compilata della dimostrazione di Shor e la distinzione da altri esperimenti su atomi riconfigurabili sono documentate nella Sezione E.4.4. Perciò $N = 15$ resta un'istanza didattica: la fattibilità crittografica è una domanda architetturale separata, dominata dall'overhead fault-tolerant.

5.5 Ottimizzazione della trasformata di Fourier consapevole della topologia

La QFT approssimata è una delle varianti naturali da esplorare. Una campagna esplorativa condotta a chiusura del lavoro ne ha misurato il margine, e il risultato merita di essere riportato perché delimita l'intera famiglia di ottimizzazioni di questo tipo.

Il punto di partenza è che nell'aritmetica di Beauregard il costo non si concentra nella trasformata finale del period finding, ma nelle QFT interne agli addizionatori modulari: per $N = 21$ la compilazione conta 11 436 porte `cp` contro 3 406 `cx`, e la trasformata conclusiva vale 28 rotazioni sulle circa 47 000 porte complessive. Troncare le rotazioni sotto una soglia d'angolo negli addizionatori porta le porte a due qubit da 49 661 a 23 178, cioè le dimezza; troncare quella finale non sposta l'ordine di grandezza. Il primo dato utile è dunque *dove* agire.

Il risparmio non è però gratuito. Sulla stessa istanza la resa ideale scende da 0,4667 a 0,4013: circa sei punti e mezzo pagati in precisione algoritmica. Ne segue

in linea di principio un incrocio, perché le due quantità scalano diversamente con la qualità delle porte: sotto una certa soglia di errore conviene approssimare, sopra conviene la trasformata intera.

La misura colloca quell'incrocio e ne quantifica l'ampiezza. Su questa istanza il vantaggio massimo ottenibile dalla variante troncata è di circa **tre punti percentuali**, e si realizza soltanto con porte a due qubit circa **duecento volte più accurate** di quelle oggi disponibili. Nel regime accessibile alla simulazione con calibrazione realistica nessuna delle configurazioni esaminate supera il pavimento uniforme dell'istanza, e nel solo punto in cui un segnale emerge è la trasformata *piena* a prevalere. Su $N = 21$ il circuito è del resto centocinquanta volte più lungo del tempo di coerenza tipico, quindi il confronto non discrimina né in simulazione né su hardware.

Questo limite appartiene però all'istanza, non al metodo. Ripetendo la misura sulla **stima di fase** — che della trasformata è l'uso canonico, e che con un unitario di fase costa decine di porte a due qubit invece di decine di migliaia — il quadro si rovescia. Su nove confronti, distribuiti su tre dimensioni di registro e su fasi bersaglio sia rappresentabili sia non rappresentabili in n bit, la trasformata troncata batte quella piena in **tutti e nove**, con intervallo di confidenza interamente positivo in sei. Su registro da dieci qubit il vantaggio raggiunge $+0,126$, con intervallo di Newcombe al 95% $[+0,098; +0,153]$.

Tre regolarità rendono il risultato utilizzabile in fase di progetto. Il grado ottimo di troncamento **non cresce con la dimensione del registro**: resta due o tre a tutte le taglie esaminate, mentre la trasformata piena ne impiega $n - 1$; il risparmio cresce quindi con n , da venticinque porte a due qubit su sei a settantotto su dieci. Di conseguenza **il vantaggio aumenta con la dimensione**, da circa due punti percentuali a sei qubit fino a tredici a dieci. E le fasi esattamente rappresentabili guadagnano più di quelle che non lo sono: è il meccanismo che **spiega la differenza fra le istanze di Shor**, perché per $N = 15$ l'ordine $r = 4$ produce fasi $0, 1/4, 1/2, 3/4$, esatte in due bit, mentre per $N = 21$ l'ordine $r = 6$ dà fasi $j/6$, dove le rotazioni fini trasportano ancora informazione.

La prima di queste regolarità — quella che a prima vista sorprende di più, perché un registro più lungo sembrerebbe richiedere più rotazioni fini — ha una spiegazione teorica indipendente, pubblicata mentre questo lavoro era in corso. Akoramurthy e Surendiran derivano per la stima di fase con trasformata troncata un limite in forma chiusa sulla distanza in variazione totale fra la distribuzione esatta e quella

troncata [34]:

$$\text{TV}(P_\phi, P_\phi^d) \leq \frac{\pi(m-d)}{2^d}, \quad (5.7)$$

dove m è la dimensione del registro e d il grado di troncamento. Il numeratore cresce linearmente in m , il denominatore esponenzialmente in d : un d piccolo e *costante* basta quindi a schiacciare il termine d'errore anche al crescere del registro, ed è esattamente il comportamento osservato qui per via empirica. Gli autori riportano inoltre una riduzione del 31,3–43,7% nel conteggio di porte e un passaggio di complessità da $O(m^2)$ a $O(m \log m)$ per $d = O(\log m)$. Il loro impianto sperimentale si ferma però alla stima di fase e non affronta la fattorizzazione: la delimitazione riportata sopra — che su Shor la stessa leva vale al massimo tre punti percentuali, e solo con porte a due qubit molto più accurate delle attuali — resta propria di questo lavoro, così come la selezione del grado su dati disgiunti da quelli di verifica.

Ne segue che il grado di troncamento conveniente è **determinabile in anticipo**, dai soli conteggi di porte del circuito compilato e dai dati di calibrazione del dispositivo, senza eseguire l'algoritmo. È questa la forma in cui il contributo si generalizza: non una proprietà della fattorizzazione, ma una regola di progetto per la trasformata di Fourier quantistica e quindi per la famiglia di algoritmi che la contiene.

Tre direzioni indipendenti convergono qui, e il passo successivo più promettente è **comporle**. La prima è quella misurata in questo lavoro: approssimare la trasformata per ridurre le *porte*, e con esse la profondità rispetto al tempo di coerenza. La seconda approssima l'aritmetica modulare per ridurre i *qubit* logici [18], ed è alla base della discesa nelle stime di risorsa degli ultimi anni [7], di cui la Tabella E.15 riporta la traiettoria e i relativi caveat. La terza riorganizza la stima di fase in blocchi indipendenti, riducendo il *registro* di conteggio a pochi qubit [35].

Agiscono su tre grandezze distinte — porte, qubit di lavoro, qubit di conteggio — ma sarebbe un errore darle per semplicemente additive. La riduzione dei qubit di lavoro si ottiene infatti *spendendo* porte: per RSA-2048 il conteggio Toffoli complessivo passa da $2^{31,3}$ a $2^{40,9}$ a fronte del calo da 6144 a 1730 qubit logici [18], e un lavoro successivo ne recupera buona parte con ottimizzazioni aritmetiche [7]. Comporre quella leva con un troncamento della trasformata significa quindi agire due volte, in versi opposti, sulla stessa grandezza.

La terza direzione è invece dichiarata complementare dagli autori stessi, che la presentano come rivolta allo strato di stima di fase anziché a quello aritmetico [35]: il registro di conteggio scende da $2n + 1$ a tre o quattro qubit lasciando invariato il registro di lavoro. Il meccanismo è descritto per esteso nella loro proposta per

la stima di fase, dove blocchi indipendenti di più qubit di controllo stimano più bit di fase entro una finestra, con una risoluzione esplicita dell’ambiguità dal bit meno significativo verso il più significativo [36]. Determinare se i tre guadagni si sommino davvero, e a quale prezzo in profondità, richiede un disegno che li vari congiuntamente ed è la domanda che questo lavoro lascia aperta con maggiore fiducia, perché le tre riduzioni sono ciascuna già documentata e quantificata.

Va infine osservato che la trasformata di Fourier quantistica non è una particolarità della fattorizzazione. Compare nella stima di fase, nell’amplitude estimation e nel conteggio quantistico, nella risoluzione di sistemi lineari, nel problema del sottogruppo nascosto di cui Shor è un caso particolare, e nelle simulazioni che alternano dominio delle posizioni e dei momenti. In tutti questi casi il costo si concentra nelle stesse rotazioni controllate di piccolo angolo, e la regola di progetto ricavata qui — scegliere il grado di troncamento dai conteggi di porte e dai dati di calibrazione, prima di eseguire — si applicherebbe senza modifiche sostanziali. È il motivo per cui il contributo va letto come metodologico e non come risultato sulla fattorizzazione.

I limiti della verifica vanno però dichiarati. Le misure usano un layout per ciascuna dimensione, una sola snapshot di calibrazione e tre fasi bersaglio; il confronto è appaiato, sugli stessi batch e con gli stessi semi, quindi il contrasto interno è pulito, ma la generalizzazione ad altri dispositivi e ad altre topologie non è dimostrata.

Il precedente più vicino è la compilazione *noise-adaptive*, che ordina o vincola il collocamento del circuito usando i dati di calibrazione [37, 38]. La differenza rispetto a quella linea, e il motivo per cui resta spazio, è che tali metodi ottimizzano la fedeltà del circuito, mentre per un algoritmo come Shor conta soltanto che il post-processing restituisca i fattori. I due obiettivi non coincidono, e la campagna sulla compilazione riportata in Appendice E misura quanto non coincidano.

5.6 Discussione complessiva dei risultati

Letti congiuntamente, i risultati descrivono tre livelli distinti di intervento sul calcolo quantistico: il recupero dell’informazione dall’istogramma finale, la protezione dello stato mediante correzione d’errore e la riduzione preventiva del costo circuitale. Il confronto è significativo soltanto mantenendo separate le rispettive metriche: il numero di iterazioni della campagna NISQ, l’errore logico dei benchmark QEC e il proxy Pauli per gate dell’analisi di Shor non sono quantità direttamente sostituibili

l'una all'altra. Proprio questa separazione permette però di ricavare un criterio comune su dove ciascun metodo produca valore.

Nel regime NISQ il risultato più netto non appartiene al classificatore, ma alla regola di ricerca. Nei due scenari appaiati, TOP-4 porta il numero medio di iterazioni da 1.37 e 1.50 a 1.00, con successo in tutte le repliche, mentre M2 non migliora significativamente la baseline e risulta peggiore della propria ablazione priva di filtro (Tabella 5.1). Anche la ZNE digitale non riduce le iterazioni e porta gli shot medi totali a 3072 e 4813, contro i 1024 di TOP-4 (Tabella D.11). In questo perimetro, dunque, conviene prima sfruttare in modo esplicito i pochi picchi ancora informativi; un modello appreso a valle non ricostruisce informazione che il rumore ha già cancellato e deve essere valutato contro una regola semplice disponibile sul medesimo istogramma.

I laboratori QEC mostrano invece che intervenire durante la conservazione dello stato può cambiare l'ordine dell'errore, purché si operi nel regime appropriato. Il codice a ripetizione segue $p_L = 3p^2 - 2p^3$ entro l'incertezza Monte Carlo (Figura 5.4); lo Steane $[[7, 1, 3]]$ estende la protezione a errori arbitrari, con pendenza log-log 1.94 e pseudo-soglia attorno a $p \simeq 0.08$ (Figura 5.5). Il surface code rende scalabile la stessa idea: i fit forniscono $p_{th} = 0.86\%$ in base Z e 0.81% in base X e, a $p = 2 \times 10^{-3}$, l'errore logico scende da 1.94×10^{-3} a distanza 3 a 2.01×10^{-5} a distanza 9 (Figura 5.6). Questi valori costituiscono una baseline entro il modello Pauli circuit-level; non sono ancora una previsione calibrata di un'esecuzione fault-tolerant di Shor.

La campagna sulla decodifica precisa anche quando l'apprendimento sia giustificato. La sostituzione diretta di MWPM non vince in nessuna delle quattordici configurazioni considerate e richiede 10^5 – 10^6 campioni per arrivare al pareggio a distanza 3. Quando però il rumore correlato viola la scomponibilità in archi assunta dal matching, il decoder ibrido riduce l'errore logico fino al 21%; il vantaggio si restringe a distanza 5 e si annulla a distanza 7 (Tabella 5.5). Il confronto con BP+OSD delimita ulteriormente il risultato: un decoder analitico più espressivo ottiene a distanza 5 guadagni del 17–27%, contro l'1–3% dell'ibrido. Il valore del modello appreso emerge quindi da una cecità strutturale del metodo analitico, non dalla sola imprecisione dei suoi parametri.

L'analisi di sensibilità di Shor collega questi risultati senza forzare una conversione tra metriche. Nel proxy uniforme per gate, il successo per singola misura passa da 0.7489 in assenza di rumore a 0.4518 per $p_L = 0.02$, quindi raggiunge un plateau empirico vicino a 0.245 oltre $p_L = 0.1$ (Figura 5.11). TOP-4 resta pari a 1.000 fino a $p_L = 0.05$, ma scende a 0.670 a $p_L = 0.2$: il post-processing conserva quindi efficacia

anche quando la massa utile è degradata, senza rendere il circuito immune al rumore. Come formalizzato nella Tabella E.16, trasformare le curve QEC in probabilità di fattorizzazione richiederebbe ancora un gate set logico, cicli di sindrome, operazioni non-Clifford e un modello delle correlazioni residue.

Infine, l'ottimizzazione della QFT mostra che ridurre le porte è utile solo se il rumore evitato compensa l'approssimazione introdotta. Su Shor per $N = 21$, il troncamento porta le operazioni a due qubit da 49 661 a 23 178, ma riduce la resa ideale da 0.4667 a 0.4013 e offre al massimo circa tre punti percentuali di vantaggio, in un regime che richiede porte a due qubit circa duecento volte più accurate di quelle considerate realistiche. Sulla stima di fase, invece, la variante troncata vince in tutti e nove i confronti e raggiunge +0.126 su dieci qubit; il grado ottimo resta due o tre (Sezione 5.5). La stessa trasformazione può dunque essere sfavorevole dentro un circuito troppo profondo e vantaggiosa nel sottoproblema per cui il suo costo è dominante.

Nel complesso emerge una regola unitaria: la complessità è giustificata solo quando agisce sul collo di bottiglia osservato. TOP-K sfrutta informazione residua; la QEC sopprime l'errore sotto soglia; l'apprendimento recupera struttura assente dal decoder analitico; il troncamento della QFT conviene se il risparmio circuitale supera la perdita algoritmica. Restano tre limiti: l'istanza rumorosa principale è $N = 15$, le soglie dipendono dal modello e manca una simulazione end-to-end fault-tolerant. Sono quindi criteri sperimentali, non previsioni per QPU reali.

Capitolo 6

Conclusioni e sviluppi futuri

6.1 Conclusioni

Questo lavoro ha affrontato una domanda operativa: quanta informazione dell'output rumoroso di Shor può essere recuperata a valle e quando diventa necessario correggere l'errore durante il calcolo? La risposta delimita i due regimi. Su $N = 15$ una ricerca multi-candidato sfrutta meglio l'informazione già misurata, mentre un classificatore applicato allo stesso output non aggiunge valore. Oltre questo margine occorre intervenire sulle sindromi e sulla correzione d'errore.

6.1.1 Le Domande di Ricerca

DR1 — Come cambia Shor al variare dei canali di rumore? Nel modello uniforme per $N = 15$, aumentando λ_{2q} la massa sui quattro picchi QPE scende da 0.804 a 0.042, mentre il proxy di nessun evento Pauli sulle 224 CX decade da 0.811 a 2.65×10^{-10} : quest'ultimo non predice quindi il successo algoritmico. Gli sweep di λ_{1q} , T_1/T_2 , readout e shot non ordinano causalmente i canali con 30 repliche; TOP-4 resta invece a una iterazione, salvo il valore 1.07 a $\lambda_{2q} = 0.1$. Risultati e diagnostiche complete sono nelle Sezioni 5.1.5 e D.1.

DR2 — Come si estrae una sindrome senza misurare lo stato logico? Nel codice a ripetizione si misurano le parità Z_1Z_2 e Z_2Z_3 , non i qubit logici: la sindrome localizza il bit alterato senza rivelarne l'ampiezza. La curva Monte Carlo coincide con $p_L = 3p^2 - 2p^3$ entro l'incertezza, ma protegge un solo tipo di errore per volta (Sezione 5.2.4).

DR3 — Quali vantaggi e limiti offre Steane $[[7, 1, 3]]$? La struttura CSS corregge un errore Pauli arbitrario su un qubit e la corrispondenza sindrome–errore è verificata nei 21 casi. La pendenza $\log\text{--}\log$ 1.94 conferma la soppressione quadratica attesa; distanza fissa ed estrazione non fault-tolerant ne impediscono però una lettura scalabile (Sezione 5.2.5).

DR4 — Il surface code mostra un comportamento di soglia? Sì, entro il memory experiment e il modello circuit-level adottati: le curve per $d = 3, 5, 7, 9$ si incrociano intorno allo 0.9% in base Z e allo 0.7% in base X, con soppressione sotto soglia e amplificazione sopra soglia. Il relativo p non coincide con λ_{2q} della campagna Shor, perciò il confronto è solo qualitativo (Sezione 5.3.4).

DR5 — Quale errore logico è compatibile con Shor? Nel modello fenomenologico, a $p_L = 0$ il successo per esecuzione resta circa 0.75 per la quota intrinsecamente non utile degli esiti QPE; le ripetizioni ne aumentano la probabilità cumulativa. La curva è però una sensibilità, non una previsione end-to-end: il logical error rate di una memoria non può essere assegnato a ogni gate di Shor senza operazioni fault-tolerant e canali residui espliciti (Sezione 5.4.4).

DR6 — Quando un decoder appreso può superare quello analitico? Il vantaggio compare quando il rumore contiene correlazioni assenti dal grafo del matching. Il decoder ibrido riduce l'errore rispetto a MWPM a distanza 3, ma il margine si restringe e non è significativo a distanza 7 per la rete densa adottata. Il risultato individua quindi una nicchia residuale, non una sostituzione generale dei decoder analitici (Sezione 5.3.6 e Appendice E.3).

6.1.2 Verifica delle Ipotesi

Tabella 6.1: Esito dopo la rigenerazione schema 2.

Ipotesi	Esito	Evidenza
Il classificatore riduce le iterazioni	smentita	M2 non migliora M1; TOP-4 supera M2 in UC1 e UC2 ($p_{\text{Holm}} = 0.0033/0.0096$).
$F1 > 0.85$ e $AUC > 0.90$	confermata, ma insufficiente	SVM: $F1$ 0.919/0.923 e AUC 0.965/0.958; le metriche descrivono TOP-1, non l'utilità operativa.
Il vantaggio si mantiene crescendo N	non verificata	$N = 21/35$ sono validati idealmente ma esclusi dal rumore per costo; va controllato anche l'ordine r .
Soppressione quadratica a distanza 3	confermata nel modello	Ripetizione coerente con $3p^2 - 2p^3$; Steane con pendenza 1.94.
Soglia del surface code	confermata nel modello	Incrocio delle curve $d = 3, 5, 7, 9$ e inversione del beneficio sopra soglia.
Trasferimento diretto di p_L a Shor	non dimostrato	La curva è fenomenologica; manca una mappatura per operazione fault-tolerant.

L'audit chiarisce perché buone metriche non bastano: TOP-1 genera classi addestrabili, mentre TOP-16 rende positivi tutti i 2000 campioni di entrambi gli scenari. Con 63 esiti utili su 256, la probabilità combinatoria di non includerne alcuno fra sedici celle è appena $\binom{193}{16}/\binom{256}{16} = 0.9275\%$. La costruzione delle etichette e i relativi controlli sono documentati nell'Appendice D.2.2.

6.1.3 Contributi del Lavoro

Demo ideale e rumorosa separate. Il percorso ideale espone circuito, stato e QPE; quello rumoroso rende selezionabili depolarizzazione 1Q/2Q, T_1/T_2 , readout ed errore coerente. $N = 21/35$ restano nel perimetro ideale e $N = 15$ in quello rumoroso.

Ablazione appaiata e riproducibile. M1, TOP-4 e M2 ricevono lo stesso istogramma. TOP-4 porta $\bar{M}$ da 1.37 a 1.00 nello scenario moderato e da 1.50 a 1.00 nello stress, mentre il classificatore peggiora la propria ablazione; seed, tie-break, sentinelle, revisioni e manifest sono conservati negli artefatti.

Validazione funzionale dell'aritmetica. Il moltiplicatore $\times 7 \bmod 15$ è coperto da truth table; il percorso Beauregard per $N = 21/35$ da proprietà esaustive e confronti QPE. Il controllo mostra perché picchi o ordine, isolatamente, non validano l'operatore.

Progressione QEC con limiti espliciti. Ripetizione, Steane, surface code e decodifica appresa sono collegati da metriche dichiarate senza rendere commentabili modelli di rumore diversi; Shor logico chiude la progressione come analisi fenomenologica.

6.1.4 Limiti

Nessun risultato deriva da hardware quantistico. È una scelta concordata: la disponibilità di un dispositivo è sopraggiunta dopo la chiusura delle domande sui modelli di rumore e sulle snapshot di calibrazione. La validazione resta quindi simulativa; il solo riscontro su macchina reale è quello discusso per citazione nella Sezione 5.4.4.

La campagna Shor usa rumore uniforme, indipendente e stazionario, senza topologia, routing, crosstalk, leakage o deriva. $N = 15$ è didattico, TOP-K ha una verifica classica permissiva, gli sweep impiegano 30 repliche e i test non sono corretti globalmente fra otto famiglie; $N = 21/35$ non sono simulati sotto rumore. Nella QEC, Steane non ha estrazione fault-tolerant, memoria e proxy per gate non condividono lo stesso p_L , e il decoder denso non mantiene il vantaggio alle maggiori distanze. Le stime crittografiche richiedono dunque architetture fault-tolerant complete, non estrapolazioni dalle curve di $N = 15$.

6.1.5 Il Criterio che Unifica il Lavoro

Un complemento semplice è utile quando sfrutta informazione trascurata dal metodo base senza alterarne la generazione: TOP-4 verifica candidati già nell'istogramma e il decoder residuale interviene dove il matching ha una debolezza rappresentazionale. La complessità non crea valore se replica una regola semplice, usa etichette degeneri o costa più del beneficio. Ne segue il confine fra mitigazione e correzione: TOP-K recupera un picco già misurato, ma non ricostruisce informazione distrutta; la scalabilità richiede operazioni logiche fault-tolerant e l'apprendimento è utile solo se dispone di struttura assente nel decoder analitico.

L'AQFT applica lo stesso criterio a monte. Troncare le rotazioni interne agli addizionatori dimezza le porte a due qubit ($49\,661 \rightarrow 23\,178$), ma su $N = 21$ riduce la resa ideale di circa 6.5 punti; il vantaggio massimo, circa tre punti, richiederebbe porte quasi duecento volte più accurate, mentre il circuito resta circa 150 volte più lungo della coerenza disponibile. Nella stima di fase, molto più corta, la variante troncata vince invece tutti i nove confronti e raggiunge +0.126 a dieci qubit. Il grado ottimo

resta due o tre, il risparmio cresce con il registro e le fasi esattamente rappresentabili beneficiano di più: ciò spiega la differenza fra $r = 4$ per $N = 15$ e $r = 6$ per $N = 21$. Conteggi compilati e calibrazione permettono quindi di scegliere il troncamento prima dell'esecuzione. Dati, limite teorico e perimetro di generalizzazione sono nella Sezione 5.5.

Questa riduzione delle porte è complementare, ma non automaticamente additiva, alla riduzione dei qubit di lavoro ottenuta con aritmetica approssimata [18, 7] e alla riduzione del registro di fase mediante blocchi indipendenti [35]. Le tre leve agiscono su porte, qubit di lavoro e qubit di conteggio; studiarne congiuntamente costi e profondità è la domanda aperta più naturale.

6.2 Sviluppi futuri

I risultati separano ciò che è dimostrato da ciò che resta da verificare: $N = 21/35$ sono validati nel percorso ideale, mentre il confronto controllato fra canali di rumore è sostenibile su $N = 15$. Le estensioni seguenti colmano questa distanza senza estrapolare oltre il perimetro sperimentale.

6.2.1 Scalabilità a Istanze di Maggiore Dimensione

La priorità è ridurre l'aritmetica modulare. Per $N = 21$, $a = 2$ e otto controlli, la compilazione congiunta in base `rz/sx/x/cx` produce 21 036 CX e profondità 23 081: conteggi effettivi, non somme di blocchi. AQFT con soglia esplicita, addizionatori alternativi, sintesi per il gate set del backend e riduzione delle operazioni controllate sono le leve immediate. Ogni variante dovrà però superare le verifiche già adottate: moltiplicazione corretta, ancille pulite, coerenza di fase e QPE ideale vicina alla teoria; il solo conteggio dei gate non dimostra la correttezza. La Sezione 5.5 fornisce già il primo protocollo quantitativo per scegliere il troncamento.

La scalabilità va stratificata anche per ordine moltiplicativo r . Infatti $N = 35$, $a = 6$ ha ordine 2 e non costituisce una progressione monotona rispetto a $N = 21$, $a = 2$, che ha ordine 6. Larghezza e ordine devono quindi essere variati e dichiarati separatamente.

6.2.2 Validazione su Hardware Quantistico Reale

La validazione hardware, esclusa deliberatamente dal perimetro corrente, richiede di ripetere il protocollo $N = 15$ registrando backend, timestamp, calibrazione dei qubit, layout iniziale e finale, conteggi post-routing e job identifier, a parità di shot e con seed dichiarati. Ai quattro scenari della demo andrebbe aggiunta una modalità “calibrazione acquisita”, datata e costruita dallo snapshot reale: diventerebbe così misurabile il *simulation gap* rispetto al rumore uniforme, che non include coupling map, routing, crosstalk, leakage e deriva.

Per $N = 21/35$ la QPU evita il costo dello statevector classico, ma non la profondità né i vincoli di connettività. Prima dell’esecuzione servono pertanto una sintesi più corta e un budget d’errore riferito al circuito realmente instradato.

6.2.3 Generalizzazione ad Altri Algoritmi Quantistici

TOP-K è trasferibile solo quando più candidati sono verificabili classicamente a basso costo, come in Shor o in alcuni problemi di ricerca; non segue che migliori VQE [39] o QAOA [40], il cui output è un valore atteso. Ogni estensione dovrà fissare una funzione di validazione, il costo delle K prove, una baseline TOP-1 sugli stessi campioni e un controllo contro il successo puramente combinatorio. Grover con verifica economica è un primo banco di prova; per gli algoritmi variazionali sono più naturali l’allocazione adattiva degli shot e la mitigazione del bias dell’osservabile.

6.2.4 Integrazione con la Quantum Error Correction

Il p_L dello Shor logico è una probabilità fenomenologica di Pauli non-identità per gate del proxy, non il logical error rate di una memoria surface-code. Una previsione end-to-end richiede prima la mappatura delle operazioni fault-tolerant sui rispettivi canali residui e poi, per ciascun tipo, conteggi logici, costo della magic-state distillation, decoder e latenza di feed-forward, correlazioni e propagazione degli intervalli di confidenza. Fino ad allora il modello resta una sensibilità, non una stima hardware.

Per la decodifica appresa sono promettenti modelli a grafo o convoluzionali coerenti con la località del reticolo, storia temporale dei cicli e soft information analogica. Queste direzioni introducono struttura o informazione, anziché ingrandire una rete densa sul medesimo vettore di sindrome.

6.2.5 Evoluzione del Classificatore ML

TOP-1 è addestrabile ma predice soltanto il candidato dominante; TOP-16 rende positivi tutti i 2 000 campioni per scenario e M2 perde contro TOP-4. Non è quindi utile cambiare solo l'architettura mantenendo la stessa etichetta. Un nuovo obiettivo dovrebbe incorporare il costo, per esempio gli shot attesi risparmiati al netto di uno scarto errato; training e test andrebbero separati anche per backend o giorno di calibrazione, con soglia scelta su validation. Bilanciamento e confronto con massa sui picchi, entropia e TOP-K devono precedere l'addestramento (Appendice D.2.2).

Il ruolo più credibile dell'apprendimento resta comunque sulla sindrome, dove può rappresentare correlazioni assenti da un decoder analitico, non sull'output finale già degradato.

Appendice A

Trattazione teorica estesa

A.1 Fondamenti del calcolo quantistico

Questa sezione sviluppa in forma estesa i richiami di meccanica quantistica che la Sezione 3.1.1 del Capitolo 3 usa nella misura minima necessaria a seguire l'algoritmo.

A.1.1 Dalla Meccanica Classica alla Meccanica Quantistica

La meccanica quantistica descrive il comportamento della materia e dell'energia alle scale atomiche e subatomiche, dove i principi della fisica classica cessano di essere validi. I suoi postulati, formulati nella prima metà del Novecento da Heisenberg, Schrödinger, Dirac e Bohr, introducono concetti radicalmente diversi da quelli classici: lo stato di un sistema fisico non è determinato in modo assoluto, ma è descritto da una **funzione d'onda** la cui evoluzione è governata dall'equazione di Schrödinger.

Il calcolo quantistico traduce questi principi fisici in risorse computazionali. La **sovrapposizione** consente a un bit quantistico di trovarsi in una combinazione lineare di 0 e 1 simultaneamente; l'**entanglement** crea correlazioni non classiche tra qubit distinti; l'**interferenza** permette di amplificare le soluzioni corrette e cancellare quelle errate.

A.1.2 I Postulati della Meccanica Quantistica

Il formalismo matematico della meccanica quantistica poggia su quattro postulati fondamentali, che costituiscono l'assiomatica della teoria e il punto di partenza rigoroso per la descrizione di qualsiasi sistema quantistico [8].

1. **Postulato dello spazio degli stati.** Ad ogni sistema fisico isolato è associato uno **spazio di Hilbert** $\mathcal{H}$, detto *spazio degli stati* del sistema. Lo stato del sistema è completamente descritto da un vettore unitario $|\psi\rangle \in \mathcal{H}$, detto *vettore di stato* o *ket*. Per un sistema a due livelli (qubit), $\mathcal{H} = \mathbb{C}^2$ e lo stato generale è $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ con $|\alpha|^2 + |\beta|^2 = 1$.
2. **Postulato dell'evoluzione.** L'evoluzione di un sistema quantistico chiuso è descritta da una **trasformazione unitaria**: se lo stato del sistema all'istante t_1 è $|\psi(t_1)\rangle$, allora all'istante t_2 esso si trova nello stato

$$|\psi(t_2)\rangle = U(t_1, t_2) |\psi(t_1)\rangle \quad (\text{A.1})$$

dove $U(t_1, t_2)$ è un operatore unitario ($U^\dagger U = I$) che dipende dagli istanti t_1 e t_2 . In forma differenziale, l'evoluzione è governata dall'**equazione di Schrödinger**:

$$i\hbar \frac{d}{dt} |\psi(t)\rangle = \hat{H} |\psi(t)\rangle \quad (\text{A.2})$$

dove $\hat{H}$ è l'**Hamiltoniano** del sistema — un operatore hermitiano ($\hat{H} = \hat{H}^\dagger$) che rappresenta l'energia totale — e $\hbar = h/(2\pi)$ è la costante di Planck ridotta.¹

3. **Postulato della misura.** Le misure quantistiche sono descritte da un insieme $\{M_m\}$ di **operatori di misura**, indicizzati dagli esiti m , soggetti alla relazione di completezza:

$$\sum_m M_m^\dagger M_m = I \quad (\text{A.3})$$

Se il sistema si trova nello stato $|\psi\rangle$ prima della misura, la probabilità di ottenere l'esito m è:

$$p(m) = \langle \psi | M_m^\dagger M_m | \psi \rangle \quad (\text{A.4})$$

¹Nel calcolo quantistico a porte, ogni porta è la realizzazione discreta di questo postulato: la matrice unitaria U descrive l'evoluzione del registro di qubit in un passo computazionale. L'Hamiltoniano fisico che implementa una porta specifica dipende dall'architettura hardware (impulsi a microonde per qubit superconduttori, impulsi laser per ioni intrappolati, ecc.).

e lo stato immediatamente dopo la misura con esito m è il vettore normalizzato:

$$|\psi'\rangle = \frac{M_m |\psi\rangle}{\sqrt{p(m)}} \quad (\text{A.5})$$

Nel caso della **misura proiettiva in base computazionale** — il caso standard nel calcolo quantistico — gli operatori sono i proiettori $M_m = |m\rangle \langle m|$, e la probabilità di ottenere m si riduce alla **regola di Born**: $p(m) = |\langle m|\psi\rangle|^2$. La misura è un'operazione *non unitaria* e *irreversibile*: il collasso dello stato distrugge irrimediabilmente le ampiezze che non corrispondono all'esito osservato.

4. **Postulato dei sistemi composti.** Lo spazio degli stati di un sistema fisico composto da n sottosistemi con spazi $\mathcal{H}_1, \dots, \mathcal{H}_n$ è il **prodotto tensoriale**:

$$\mathcal{H} = \mathcal{H}_1 \otimes \mathcal{H}_2 \otimes \dots \otimes \mathcal{H}_n \quad (\text{A.6})$$

Se ciascun sottosistema i si trova nello stato $|\psi_i\rangle$, lo stato del sistema composto è $|\psi_1\rangle \otimes |\psi_2\rangle \otimes \dots \otimes |\psi_n\rangle$. Esistono tuttavia stati del sistema composto — gli stati **entangled** — che non possono essere scritti in questa forma fattorizzata per nessuna scelta degli stati individuali: la misura di un sottosistema modifica istantaneamente le probabilità degli esiti di misura sull'altro, indipendentemente dalla distanza fisica tra i due.

Questi quattro postulati forniscono il quadro assiomatico completo entro cui si collocano tutti i concetti che seguono: il qubit (Postulato 1), le porte quantistiche come trasformazioni unitarie (Postulato 2), la misura e il collasso dello stato (Postulato 3), i registri multi-qubit e l'entanglement (Postulato 4).

A.1.3 Il Qubit

Definizione Matematica

L'unità fondamentale del calcolo quantistico è il **qubit** (*quantum bit*). Mentre un bit classico assume uno dei due valori $\{0, 1\}$, un qubit è descritto da un vettore unitario nello spazio di Hilbert bidimensionale $\mathcal{H} = \mathbb{C}^2$.²

Introducendo la **notazione di Dirac**³, i due stati base computazionali sono:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad (\text{A.7})$$

Lo stato generico di un qubit è la **sovrapposizione**:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1 \quad (\text{A.8})$$

dove $|\alpha|^2$ e $|\beta|^2$ rappresentano le probabilità di ottenere rispettivamente 0 e 1 al momento della misura. Il vincolo di normalizzazione $|\alpha|^2 + |\beta|^2 = 1$ garantisce che le probabilità sommino a 1.

La Sfera di Bloch

Ogni stato puro di un qubit può essere visualizzato come un punto sulla superficie della **sfera di Bloch**, una sfera unitaria in $\mathbb{R}^3$.⁴ Parametrizzando le ampiezze complesse tramite angoli reali $\theta \in [0, \pi]$ e $\phi \in [0, 2\pi]$:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right) |0\rangle + e^{i\phi} \sin\left(\frac{\theta}{2}\right) |1\rangle \quad (\text{A.9})$$

²Lo **spazio di Hilbert** prende il nome dal matematico tedesco David Hilbert (1862–1943), uno dei più influenti matematici del XX secolo. In meccanica quantistica indica uno spazio vettoriale complesso dotato di prodotto interno, che fornisce il formalismo matematico per descrivere gli stati di un sistema fisico.

³La **notazione bra-ket** fu introdotta dal fisico britannico Paul Adrien Maurice Dirac (1902–1984), Premio Nobel per la Fisica nel 1933. Il simbolo $|\psi\rangle$ è detto *ket*, mentre $\langle\psi|$ è detto *bra*; il prodotto interno $\langle\phi|\psi\rangle$ forma il *bracket*, da cui il nome.

⁴La **sfera di Bloch** deve il suo nome al fisico svizzero-americano Felix Bloch (1905–1983), Premio Nobel per la Fisica nel 1952 (insieme a Edward Purcell) per lo sviluppo della spettroscopia a risonanza magnetica nucleare (Risonanza Magnetica Nucleare (*Nuclear Magnetic Resonance*) (NMR)). La stessa notazione T_1 e T_2 per i tempi di decoerenza proviene dal formalismo NMR di Bloch.

I poli nord e sud corrispondono agli stati $|0\rangle$ e $|1\rangle$, mentre i punti sull'equatore rappresentano sovrapposizioni equiprobabili con fasi diverse.

La figura corrispondente è riportata nel corpo (Figura 3.1).

A.1.4 Sistemi Multi-Qubit e Prodotto Tensoriale

Un sistema di n qubit è descritto da un vettore nello spazio di Hilbert $\mathcal{H}^{\otimes n} = \mathbb{C}^{2^n}$, ottenuto come prodotto tensoriale degli spazi individuali. Uno stato generico di n qubit è:

$$|\psi\rangle = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle, \quad \sum_x |\alpha_x|^2 = 1 \quad (\text{A.10})$$

La dimensione esponenziale 2^n dello spazio di Hilbert rende disponibili ampiezze e correlazioni che non compaiono in un registro classico di n bit: con $n = 300$ qubit, lo spazio degli stati ha dimensione 2^{300} , superiore al numero di atomi nell'universo osservabile. Questa dimensione, tuttavia, non implica da sola un vantaggio computazionale: esistono famiglie di stati e circuiti quantistici descrivibili classicamente in modo efficiente. Il vantaggio richiede che preparazione, interferenza e misura sfruttino la struttura del problema, come avviene nel period finding di Shor. Questo chiarimento era nel richiamo sintetico del Capitolo 3 ed è qui conservato nella trattazione estesa.

Entanglement

Due qubit sono detti **entangled** quando il loro stato congiunto non può essere scritto come prodotto tensoriale degli stati individuali. Gli stati di Bell costituiscono la base canonica degli stati entangled a due qubit:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \quad (\text{A.11})$$

$$|\Phi^-\rangle = \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle) \quad (\text{A.12})$$

$$|\Psi^+\rangle = \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle) \quad (\text{A.13})$$

$$|\Psi^-\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle) \quad (\text{A.14})$$

Lo stato $|\Phi^+\rangle$ non può essere fattorizzato come $|\psi_A\rangle \otimes |\psi_B\rangle$ per nessuna scelta di $|\psi_A\rangle$ e $|\psi_B\rangle$: misurare il primo qubit come 0 garantisce che il secondo sia 0, indipendentemente dalla distanza fisica tra i due sistemi.

Il circuito di preparazione è riportato nel corpo in Figura 3.2; qui si conserva la base completa dei quattro stati e la discussione fisica della loro fragilità.

L'Entanglement come Risorsa Fisica Fragile

L'entanglement non è soltanto una proprietà astratta degli stati quantistici: è la **risorsa fisica** che gli algoritmi quantistici consumano per ottenere il loro vantaggio computazionale. Nell'algoritmo di Shor, il registro di controllo e il registro di lavoro devono restare entangled per tutta la durata della phase estimation affinché la Quantum Fourier Transform possa estrarre il periodo corretto. La perdita anche parziale di questa correlazione è sufficiente a degradare il risultato.

La fragilità dell'entanglement ha un'origine fisica precisa. Un sistema quantistico reale non è mai perfettamente isolato dall'ambiente che lo circonda: campi elettromagnetici residui, vibrazioni termiche del reticolo cristallino e fluttuazioni di carica si accoppiano continuamente ai qubit. Quando questo accoppiamento avviene, il sistema entra in uno stato entangled *con l'ambiente*, trasferendo ad esso parte dell'informazione di fase. Dal punto di vista del solo sistema, le ampiezze di interferenza si riducono: il qubit non è più in una sovrapposizione pura $\alpha|0\rangle + \beta|1\rangle$, ma in un **miscuglio statistico** in cui la fase relativa tra $|0\rangle$ e $|1\rangle$ è persa. Questo processo prende il nome di **decoerenza** [41].

Il tempo caratteristico entro cui questa perdita di fase diventa apprezzabile è T_2 , il **tempo di coerenza di fase** (*dephasing time*). Esso misura per quanto tempo un qubit riesce a mantenere la propria sovrapposizione coerente prima che l'interazione con l'ambiente la degradi. La crescita della profondità di Shor rende questo budget critico, ma il solo ordine asintotico $O(n^3)$ non determina una soglia universale in bit: contano compilazione, parallelismo, idle e architettura fault-tolerant. La trattazione quantitativa e i limiti del confronto diretto con T_2 sono nel Capitolo 3.

A.1.5 Porte Quantistiche

Le operazioni su qubit sono rappresentate da **matrici unitarie**: $U^\dagger U = I$. La condizione di unitarietà garantisce la reversibilità del calcolo e la conservazione della norma (delle probabilità).

Porte a Singolo Qubit

Le principali porte a singolo qubit sono:

Le tre **matrici di Pauli** X , Y , Z costituiscono, insieme all'identità I , una base per lo spazio delle matrici 2×2 hermitiane a traccia nulla, e giocano un ruolo centrale sia nella descrizione degli stati del qubit sia nella caratterizzazione dei canali di rumore.

Porta di Pauli-X (NOT quantistico / Bit Flip):

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad X|0\rangle = |1\rangle, \quad X|1\rangle = |0\rangle \quad (\text{A.15})$$

È l'analogo quantistico del NOT classico: inverte i valori di $|0\rangle$ e $|1\rangle$.

Porta di Pauli-Y:

$$Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Y|0\rangle = i|1\rangle, \quad Y|1\rangle = -i|0\rangle \quad (\text{A.16})$$

Combina un bit flip e un phase flip: è equivalente (a meno di una fase globale) alla composizione iXZ . Compare naturalmente nella decomposizione di rotazioni generali su qubit e nella descrizione del canale di *depolarizzazione*.

Porta di Pauli-Z (Phase Flip):

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad Z|0\rangle = |0\rangle, \quad Z|1\rangle = -|1\rangle \quad (\text{A.17})$$

Lascia invariato $|0\rangle$ e aggiunge una fase -1 a $|1\rangle$: è un *phase flip*.

Le tre matrici di Pauli soddisfano le seguenti relazioni algebriche fondamentali:

$$X^2 = Y^2 = Z^2 = I \quad (\text{A.18})$$

$$XY = iZ, \quad YZ = iX, \quad ZX = iY \quad (\text{A.19})$$

$$\{X, Y\} = \{Y, Z\} = \{X, Z\} = 0 \quad (\text{A.20})$$

dove $\{A, B\} = AB + BA$ è l'**anticommutatore**. Le prime due relazioni mostrano che le matrici di Pauli formano l'algebra di Lie $\mathfrak{su}(2)$; la terza — l'anticommutazione — è alla base della struttura dei codici di correzione degli errori quantistici, dove X e Z rappresentano rispettivamente errori di bit flip e phase flip indipendenti.

Porta di Hadamard:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

$$H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \equiv |+\rangle, \quad H|1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}} \equiv |-\rangle \quad (\text{A.21})$$

La porta di Hadamard è fondamentale: applicata a n qubit inizializzati in $|0\rangle^{\otimes n}$, genera la **sovrapposizione uniforme** di tutti i 2^n stati base:

$$H^{\otimes n} |0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle \quad (\text{A.22})$$

Porta di fase R_ϕ :

$$R_\phi = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\phi} \end{pmatrix} \quad (\text{A.23})$$

Casi particolari: $T = R_{\pi/4}$ (T-gate) e $S = R_{\pi/2}$ (S-gate o Phase gate).

Porte a Due Qubit

Porta Controlled-NOT (CNOT):

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \quad (\text{A.24})$$

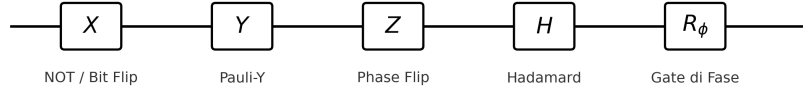
Applica una NOT sul qubit *target* se e solo se il qubit *control* è $|1\rangle$. È la porta a due qubit più comune e, insieme alle porte a singolo qubit, forma un insieme universale.

Porta Toffoli (Controlled-Controlled NOT (porta di Toffoli) (CCNOT)):

Generalizzazione a tre qubit; applica NOT sul terzo qubit solo se entrambi i qubit di controllo sono $|1\rangle$.

Principali porte quantistiche: simboli circuitali

Porte a singolo qubit:



Porta a due qubit:

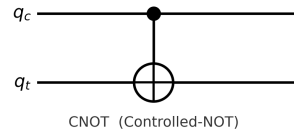


Figura A.1: Simboli circuitali delle principali porte quantistiche a singolo qubit (X , Y , Z , H , R_ϕ) e della porta CNOT a due qubit. Nella porta CNOT, il punto pieno indica il qubit di controllo q_c e il cerchio con croce indica il qubit target q_t .

A.1.6 Circuiti Quantistici

Un **circuito quantistico** è una sequenza ordinata di porte quantistiche applicata a un registro di qubit. I circuiti si leggono da sinistra a destra, con ogni filo orizzontale che rappresenta l'evoluzione temporale di un qubit. La composizione di porte corrisponde al prodotto matriciale degli operatori unitari associati.

L'equivalente quantistico del teorema di universalità classico afferma che qualunque trasformazione unitaria su n qubit può essere **approssimata con precisione arbitraria** da una sequenza finita di porte prese da un insieme universale, tipicamente $\{H, T, \text{CNOT}\}$. La distinzione fra approssimazione e decomposizione esatta non è pedanteria: le sequenze finite su un insieme di porte fissato sono numerabili, mentre le trasformazioni unitarie non lo sono, sicché una decomposizione esatta è impossibile in generale. Il teorema di Solovay–Kitaev garantisce che l'approssimazione a precisione ε richiede $O(\log^c(1/\varepsilon))$ porte, con c costante piccola: il costo dell'approssimazione cresce dunque solo polilogaritmicamente, ed è questa la ragione per cui l'universalità resta praticamente utile.

A.1.7 Misura Quantistica

La **misura** in base computazionale di uno stato $|\psi\rangle = \sum_x \alpha_x |x\rangle$ produce il risultato x con probabilità $|\alpha_x|^2$, e crolla irrevocabilmente lo stato in $|x\rangle$. La misura è una operazione *non unitaria* e *non reversibile*: una volta effettuata, l'informazione sulle ampiezze è irreversibilmente persa.

Questa caratteristica impone che gli algoritmi quantistici siano progettati per amplificare selettivamente l'ampiezza della risposta corretta prima della misura, in modo da ottenere il risultato voluto con alta probabilità.

A.1.8 Interferenza Quantistica

L'**interferenza** è il meccanismo fondamentale attraverso cui gli algoritmi quantistici realizzano il loro vantaggio computazionale. Le ampiezze quantistiche sono numeri complessi e possono combinarsi costruttivamente (quando hanno lo stesso segno/fase) o distruttivamente (quando hanno fase opposta).

Un algoritmo quantistico efficiente è costruito in modo che:

- Le ampiezze degli stati **sbagliati** si cancellino per **interferenza distruttiva**;
- Le ampiezze degli stati **corretti** si sommino per **interferenza costruttiva**.

Questo meccanismo è alla base sia dell'algoritmo di Grover sia dell'algoritmo di Shor (Capitolo 3), dove la Quantum Fourier Transform realizza l'interferenza necessaria al ritrovamento del periodo.

A.1.9 Modello di Complessità Quantistica

La complessità di un algoritmo quantistico è misurata in termini di numero di porte (equivalente delle operazioni elementari classiche) e di profondità del circuito (numero di livelli di porte parallele). Si definiscono le classi di complessità quantistiche più rilevanti:

- *Bounded-error Quantum Polynomial time* (BQP): classe dei problemi risolvibili in tempo polinomiale con probabilità di errore al più $1/3$ da un computer quantistico. È la classe centrale del calcolo quantistico.
- *Quantum Merlin-Arthur* (QMA): analogo quantistico di NP; problemi la cui soluzione può essere verificata efficientemente da un computer quantistico.

Le inclusioni $\mathbf{P} \subseteq \mathbf{BPP} \subseteq \mathbf{BQP} \subseteq \mathbf{PSPACE}$ sono *dimostrate*; ciò che resta aperto è se siano **strette**. Non è noto se BQP contenga propriamente P o BPP — cioè se esista davvero un problema che un computer quantistico risolva efficientemente e uno classico no — né se sia propriamente contenuto in PSPACE. Una dimostrazione di separazione stretta fra BQP e BPP implicherebbe $\mathbf{P} \neq \mathbf{PSPACE}$, un risultato che la teoria della complessità non ha ancora raggiunto: è questa la ragione per cui il vantaggio quantistico, per Shor come per gli altri algoritmi, resta una congettura ampiamente creduta e non un teorema.

Gerarchia delle classi di complessità nel calcolo quantistico

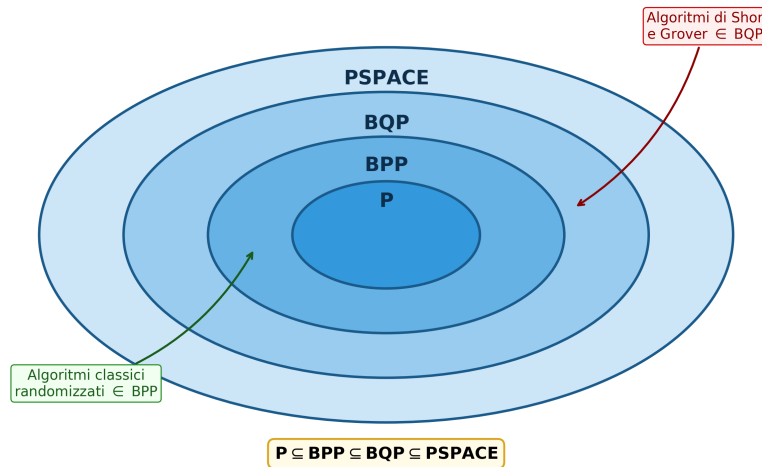


Figura A.2: Gerarchia delle classi di complessità nel calcolo quantistico. La classe BQP cattura i problemi efficientemente risolvibili da un computer quantistico; le inclusioni mostrate ($P \subseteq BPP \subseteq BQP \subseteq PSPACE$) sono dimostrate, mentre è congetturata — e non dimostrata — la loro *strettezza*. Gli algoritmi di Shor e Grover risiedono in BQP; che la fattorizzazione non appartenga a P è ritenuto probabile ma non provato.

A.2 L'anatomia fisica di una QPU

Il Capitolo 3 isola di questa materia i soli tre fatti che ricorrono nel seguito; questa sezione ne dà la trattazione completa.

A.2.1 Qubit, Controllo e Vincoli Fisici della QPU

Per capire dove nascono gli errori bisogna prima capire com'è fatta la macchina. Una QPU superconduttiva è un chip criogenico che contiene *esclusivamente* i qubit e le linee fisiche che li collegano: contrariamente all'intuizione, **le porte logiche non risiedono nel chip**. Aprendo una CPU classica si trovano registri e porte logiche incise nel silicio; aprendo una QPU si trovano solo i qubit. Le operazioni sono realizzate dall'**elettronica di controllo esterna**, che invia impulsi a microonde convogliati ai qubit tramite guide d'onda: “far passare un qubit in un gate” significa in realtà inviargli un'onda sagomata che ne ruota lo spin dell'angolo desiderato (90° ,

$30^\circ, 10^\circ, \dots$). La Figura A.3 schematizza questa architettura per una QPU a 5 qubit con topologia a catena lineare.

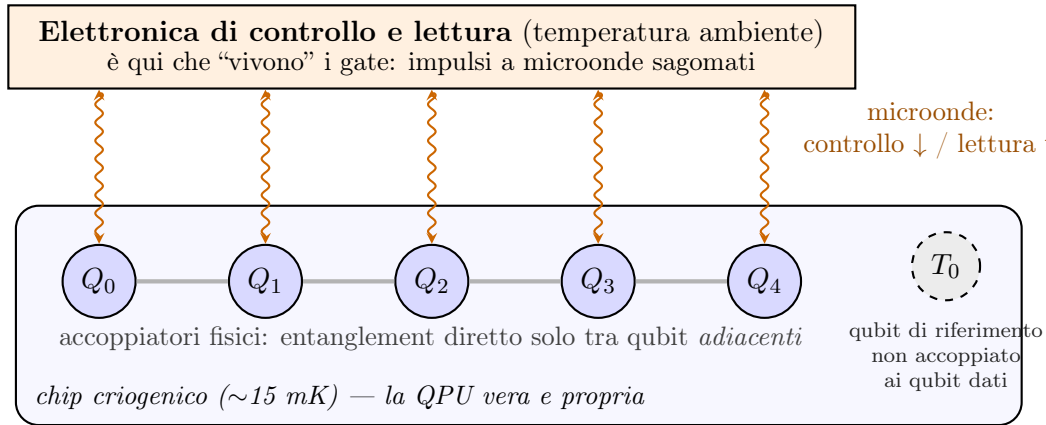


Figura A.3: Schema didattico di una QPU superconduttiva a 5 qubit con topologia a catena. Nel chip criogenico risiedono qubit, risonatori e accoppiatori; le operazioni sono pilotate dall’elettronica esterna mediante linee di controllo e lettura. L’interazione a due qubit diretta è disponibile solo lungo gli accoppiamenti disegnati (Q_1 – Q_2 sì, Q_0 – Q_2 mediante routing). T_0 rappresenta un qubit di riferimento non accoppiato alla catena, ma ancora dotato delle linee necessarie a preparazione e misura: non fornisce una misura “pura” di T_1 e non elimina perdite, Purcell effect o altre sorgenti di decoerenza.

La Figura A.4 mostra come questa architettura si presenta nella realtà: all’esterno il sistema integrato (criostato, elettronica e schermature racchiusi nella teca), all’interno della sala macchine i cilindri criogenici sospesi con il fascio di tubi e cavi coassiali che convogliano le microonde di controllo e lettura — le “guide d’onda” dello schema precedente.



(a) IBM Quantum System One (Ehningen, Germania)



(b) Criostati IBM Q con il cablaggio di controllo

Figura A.4: La QPU reale. (a) Il sistema integrato IBM Quantum System One: il cilindro nero al centro è il criostato che contiene il chip, circondato dall'elettronica di controllo. (b) Criostati IBM Q in sala macchine: dall'alto arrivano i tubi per la refrigerazione e i cavi coassiali che convogliano le microonde di controllo e lettura verso il chip, sospeso sul fondo del cilindro a ~ 15 mK. *Crediti: (a) IBM Research, licenza CC BY 2.0; (b) Connie Zhou per IBM, pubblico dominio; entrambe via Wikimedia Commons.*

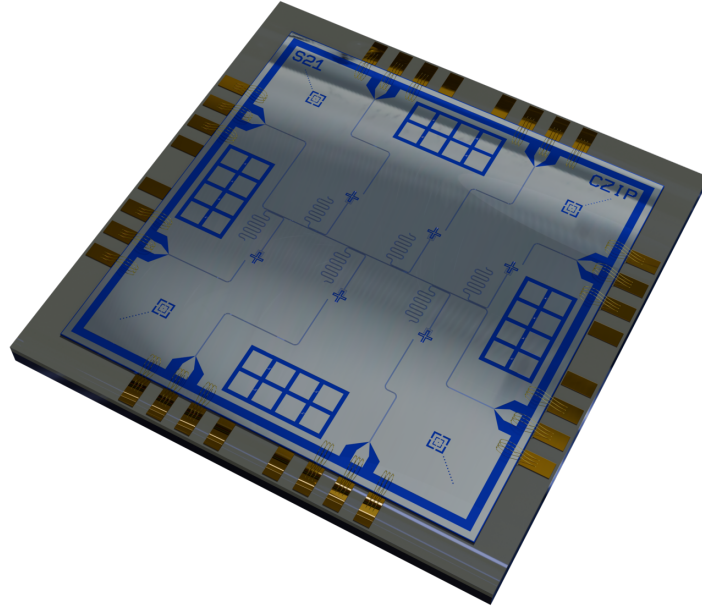


Figura A.5: Resa tridimensionale di un chip quantistico superconduttivo con 6 qubit transmon e 12 giunzioni di test. Si riconoscono gli elementi dello schema di Figura A.3: i qubit con i loro risonatori, le linee di accoppiamento, le piazzole dorate per il collegamento alle guide d'onda e — ai quattro angoli — le strutture di test isolate, l'analogo reale del qubit T_0 . *Credito: Onri Jay Benally, licenza CC BY 4.0, via Wikimedia Commons.*

Questa architettura ha tre conseguenze dirette per gli errori.

La lettura distrugge lo stato — fisicamente. Anche la misura avviene tramite microonde: per leggere un qubit lo si “bombarda” con un’onda di sonda e si osserva come essa torna indietro — un modo di risposta corrisponde a $|0\rangle$, un altro a $|1\rangle$. L’atto stesso di sondare blocca la dinamica del qubit: è per questo che la misura di uno stato quantistico lo distrugge *realmente*, non per convenzione matematica. Ed è per questo che nei circuiti le letture stanno tutte alla fine, in parallelo: misurare un solo qubit lasciando gli altri in sovrapposizione significa inviare un’onda che, per quanto focalizzata, perturba anche i vicini e può farli precipitare.

Il budget di operazioni: coerenza vs velocità dei gate. Le prestazioni di una QPU dipendono dal rapporto tra due tempi: quanto a lungo il qubit resta coerente e quanto dura una singola operazione. Con una finestra di coerenza di $100\ \mu\text{s}$ e gate da $\sim 25\ \text{ns}$, si eseguono $\sim 4\,000$ operazioni; una macchina con coerenza superiore ma gate

più lenti può eseguirne *meno*. È il rapporto che conta, non il singolo parametro — ed è questo rapporto a decidere se il circuito di Shor per una data istanza è fisicamente eseguibile. La Tabella A.1 confronta gli ordini di grandezza delle due principali tecnologie.

	Superconduttori (IBM, IQM, ...)	Ioni intrappolati (IonQ, Quantinuum, ...)
Tempo di coerenza	50–300 μ s	da secondi a minuti
Durata gate (1 qubit)	~ 15 –40 ns	~ 1 –100 μ s
Durata gate (2 qubit)	~ 60 –500 ns	~ 0.1 –1 ms
Operazioni per finestra	$\sim 10^3$ – 10^4	$\sim 10^3$
Lettura	più esposta a errori e interferenze	più stabile e affidabile
Crosstalk	presente (accoppiamento capacitivo)	ridotto
Velocità di esecuzione	molto alta	bassa (gate lenti)

Tabella A.1: Ordini di grandezza indicativi delle due principali tecnologie di QPU. La coerenza lunghissima degli ioni intrappolati non si traduce in più operazioni utili, perché i gate sono da 10^3 a 10^4 volte più lenti: il budget di operazioni per finestra di coerenza è simile, ma i superconduttori lo spendono molto più in fretta. Gli ioni sono invece più robusti su lettura e crosstalk.

Il bordo di coerenza. Gli errori non sono distribuiti uniformemente nel tempo: finché il calcolo occupa una piccola frazione della finestra di coerenza, la probabilità di decadimento è bassa; quando il circuito si spinge *verso la fine* della finestra — il “bordo di coerenza” — la probabilità che il qubit sia già decaduto al momento della misura cresce rapidamente. La Figura A.6 illustra il concetto: è la ragione per cui circuiti profondi come quello di Shor, che consumano gran parte del budget, producono letture inaffidabili anche quando “tecnicamente” rientrano nella finestra.

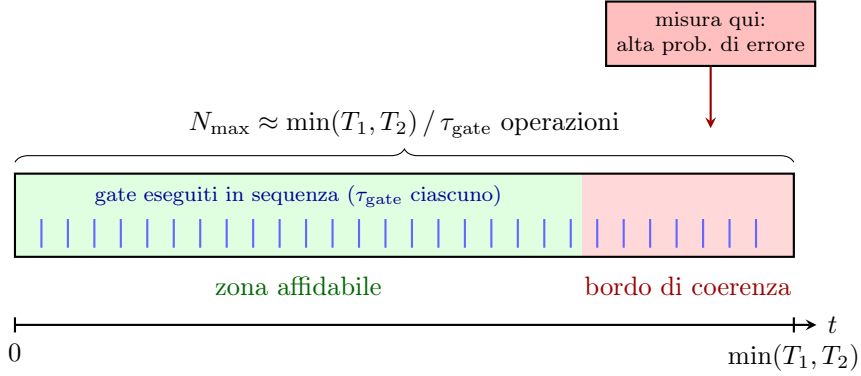


Figura A.6: Il “budget” di operazioni di un qubit: la finestra di coerenza $\min(T_1, T_2)$ divisa per la durata del singolo gate dà il numero massimo di operazioni eseguibili $N_{\max}$. Un circuito che consuma solo la prima parte della finestra opera in zona affidabile; un circuito profondo come quello di Shor spinge la misura verso il *bordo di coerenza*, dove la probabilità che il qubit sia già decaduto — e che la lettura restituisca un valore errato — è massima.

La topologia decide chi parla con chi. Le QPU differiscono anche per geometria: catene lineari, griglie (heavy-hex di IBM), connettività completa. Su una catena, l’entanglement tra qubit non adiacenti richiede un “cross-entanglement” attraverso i qubit intermedi: ogni passaggio disperde energia e accorcia la vita dello stato condiviso — più cross si fanno, prima lo stato decade. La topologia determina quindi sia *quali* circuiti si possono mappare in modo efficiente, sia *dove* il crosstalk colpirà. Questo aspetto — insieme al fatto che i parametri di coerenza variano da qubit a qubit dello stesso chip — è ripreso nel Capitolo 5, dove la località delle interazioni sulla griglia è il fondamento architetturale del surface code.

A.2.2 Le quattro fonti fisiche: parametri e modelli di misura

Questa sottosezione conserva la trattazione parametrica trasferita dalla Sezione 3.3.3: nel corpo rimangono la sintesi e il mapping operativo verso i canali, mentre qui sono raccolti formule, ordini di grandezza e limiti interpretativi.

Decoerenza: rilassamento e perdita di fase

La **decoerenza** nasce dall’interazione con campi elettromagnetici parassiti, fononi e fluttuazioni termiche. Il **tempo di rilassamento** T_1 descrive il decadimento spontaneo da $|1\rangle$ a $|0\rangle$ per dissipazione energetica; su dispositivi superconduttori è tipicamente dell’ordine di 50–500 μs . Il **tempo di dephasing** T_2 descrive invece la

randomizzazione della fase relativa e combina rilassamento e perdita pura di fase:

$$\frac{1}{T_2} = \frac{1}{2T_1} + \frac{1}{T_\phi}, \quad (\text{A.25})$$

dove T_ϕ è il tempo di puro dephasing; vale $T_2 \leq 2T_1$ e sono comuni ordini di grandezza di 50–200 μs . La notazione deriva dalla spettroscopia NMR di Felix Bloch.

Sul cammino critico, il criterio

$$\tau_{\text{circuit}} \ll \min(T_1, T_2) \quad (\text{A.26})$$

è soltanto euristico. La crescita polinomiale dell'aritmetica non determina una soglia universale in bit: compilazione, parallelismo, idle, topologia e QEC fissano la durata fisica effettiva. In particolare, durante la phase estimation la coerenza di fase deve sostenere l'entanglement fra i registri; confrontare T_2/τ_{gate} con il numero totale di porte sarebbe scorretto perché un conteggio non è una durata. Per questo il noise model usa durate esplicite e riporta separatamente conteggi e profondità.

Errori di gate

Un gate fisico realizza $\tilde{U} = U + \delta E$ anziché U . Una misura ideale della discrepanza è la gate infidelity

$$r(U, \tilde{U}) = 1 - F(U, \tilde{U}), \quad F(U, \tilde{U}) = \frac{1}{d^2} \left| \text{Tr}(U^\dagger \tilde{U}) \right|^2. \quad (\text{A.27})$$

Le cause includono errori di ampiezza o frequenza degli impulsi, accoppiamento *always-on* e leakage verso livelli non computazionali ($|2\rangle, |3\rangle, \dots$). Gli ordini indicativi 2024–2025 sono $r \simeq 0.03\text{--}0.1\%$ per gate 1Q e $0.3\text{--}1\%$ per gate 2Q; non sono assunti come calibrazione delle campagne di questa tesi.

Readout e crosstalk

Il readout è descritto da una matrice di confusione

$$A_{ij} = P(\text{misura } i \mid \text{stato } j), \quad \sum_i A_{ij} = 1. \quad (\text{A.28})$$

Sotto l'ipotesi d'indipendenza, per n qubit vale $A = \bigotimes_{k=1}^n A_k$, con

$$A_k = \begin{pmatrix} 1 - e_0^{(k)} & e_1^{(k)} \\ e_0^{(k)} & 1 - e_1^{(k)} \end{pmatrix}, \quad (\text{A.29})$$

dove $e_0^{(k)}$ è la probabilità di leggere 1 dato $|0\rangle$; valori indicativi sono 1–3%.

Il **crosstalk** è invece l'interferenza fra operazioni simultanee su qubit vicini, dovuta soprattutto all'accoppiamento capacitivo residuo. Produce rotazioni parassite, dipende dalla topologia ed è difficile da modellare analiticamente [42]. Proprio le correlazioni spaziali non rappresentate nel grafo del decoder costituiscono, nella Sezione 5.3.6, il regime in cui il correttore residuo appreso mostra un guadagno rispetto al riferimento analitico.

A.3 L'algoritmo di Shor — approfondimenti e stato dell'arte

Questa sezione raccoglie il materiale di contesto e di approfondimento dell'algoritmo di Shor: lo stato delle realizzazioni sperimentali, i miglioramenti algoritmici recenti, la roadmap verso il calcolo fault-tolerant e il confronto con l'algoritmo di Grover. Il Capitolo 3 ne riporta la sintesi.

A.3.1 Sviluppi Recenti e Stato dell'Arte

Implementazioni su Hardware Reale

Nonostante la solidità teorica, l'esecuzione di Shor su hardware reale rimane limitata a istanze molto piccole a causa dei requisiti di profondità circuitale. I principali risultati sperimentali sono:

- **2001 – NMR (Vandersypen et al.):** prima dimostrazione sperimentale di Shor su un sistema a 7 spin nucleari; fattorizzazione di $N = 15$ in 3×5 [43].
- **2016 – Ioni intrappolati (Monz et al.):** implementazione scalabile su 5 qubit ionici; fattorizzazione di $N = 15$ con un'architettura modulare [44].
- **2021 – Qubit superconduttori IBM:** dimostrazione compilata della fattorizzazione di $N = 21$ su processori IBM Quantum utilizzando 5 qubit [5].

Queste istanze evidenziano quanto la distanza dalle chiavi crittograficamente rilevanti sia ancora enorme. Una stima classica del fabbisogno, formulata sul modello di circuito privo di correzione d'errore, riporta $5K + 1$ qubit di memoria e circa $72K^3$ gate elementari per fattorizzare un numero di K bit [45]; costanti e overhead dipendono però dall'architettura aritmetica e, nelle stime moderne, dal protocollo fault-tolerant, rispetto al quale quel conteggio va inteso come limite inferiore ottimistico. In ogni caso, i valori per RSA-2048 restano fuori portata per i processori NISQ attuali.

Alla distanza teorica se ne aggiunge una operativa, documentata da una campagna recente su più piattaforme cloud: le costruzioni circuitali restano specifiche per ciascun modulo, e le fedeltà delle macchine si presentano instabili, con tassi d'errore elevati e fluttuanti [46]. È la ragione per cui, in questo lavoro, $N = 15$ è trattato come istanza illustrativa e non come misura di fattibilità.

Miglioramenti Algoritmici: Regev 2023

Un risultato teorico rilevante è stato pubblicato nel 2023 da Oded Regev [47]. La proposta fattorizza un intero di n bit eseguendo indipendentemente circa $\sqrt{n} + 4$ circuiti, ciascuno con $\tilde{O}(n^{3/2})$ gate, seguiti da post-processing classico polinomiale. Il confronto con Shor non può quindi essere ridotto al solo conteggio di gate di una singola esecuzione: aumentano il numero di run e, nella costruzione originale, anche lo spazio quantistico; inoltre l'articolo dichiara non chiaro l'effettivo vantaggio implementativo. Il risultato è un miglioramento asintotico importante, non una dimostrazione di praticabilità su dispositivi NISQ.

Roadmap IBM verso il Quantum Fault-Tolerant

IBM ha delineato una roadmap concreta verso il calcolo quantistico fault-tolerant [48]:

- **2025:** processori Loon e Nighthawk (120 qubit con 218 coupler sintonizzabili), con dimostrazione dei componenti chiave per la fault-tolerance tramite codici qLDPC;
- **2026:** target di vantaggio quantistico su problemi pratici;
- **2029:** sistema Quantum Starling con 200 qubit logici e oltre 10^8 operazioni quantistiche.

Questi traguardi sono obiettivi industriali dichiarati da IBM, soggetti a revisione, e non stabiliscono quando una macchina avrà le risorse logiche necessarie a rompere RSA-2048. Rafforzano la necessità di preparare la migrazione, ma non forniscono una previsione crittografica.

Le sfide hardware principali per l'esecuzione di Shor su istanze crittograficamente rilevanti sono analizzate in dettaglio nel Capitolo 3, mentre l'implementazione pratica con Qiskit e le tecniche di ottimizzazione basate su intelligenza artificiale sono presentate nel Capitolo 4.

A.3.2 Contesto: Confronto con l'Algoritmo di Grover

Per collocare correttamente l'algoritmo di Shor nel panorama dell'informatica quantistica, è utile confrontarlo con l'altro grande risultato algoritmico degli anni Novanta: l'algoritmo di Grover per la ricerca non strutturata [49].

Il Problema di Grover

Dato un insieme di N elementi e una funzione oracolo $f : \{0, \dots, N-1\} \rightarrow \{0, 1\}$, l'obiettivo è trovare x^* tale che $f(x^*) = 1$. Classicamente, nel caso peggiore, sono necessarie $O(N)$ interrogazioni. Grover (1996) dimostrò che un algoritmo quantistico, sfruttando il meccanismo di *amplitude amplification*, può trovare la soluzione in sole $O(\sqrt{N})$ interrogazioni – uno **speedup quadratico** che è stato dimostrato ottimale: nessun algoritmo quantistico può fare meglio [50, 51].

Differenze Fondamentali

I due algoritmi rappresentano due regimi distinti di vantaggio quantistico:

Caratteristica	Shor	Grover
Problema target	Fattorizzazione intera / period finding	Ricerca non strutturata
Speedup	Esponenziale rispetto al miglior classico	Quadratico: $O(\sqrt{N})$ vs $O(N)$
Tecnica quantistica core	QFT + Phase Estimation	Amplitude Amplification
Ottimalità	Non dimostrata in assoluto	Dimostrata ottimale [50]
Impatto crittografico	Rompe RSA, DH, ECDSA (chiave pubblica)	Dimezza sicurezza AES/SHA (simmetrica)
Requisiti hardware	$O(\log N)$ qubit, alta profondità	$O(\log N)$ qubit, bassa profondità

Tabella A.2: Confronto tra l'algoritmo di Shor e l'algoritmo di Grover

L'impatto sulla crittografia simmetrica di Grover è significativo ma gestibile: raddoppiare la lunghezza delle chiavi (da AES-128 a AES-256) è sufficiente a ripristinare il livello di sicurezza. Al contrario, non esiste un analogo rimedio per RSA: l'unica risposta è la migrazione verso gli standard post-quantum discussi nella Sezione C.1.5. È per questa ragione che l'algoritmo di Shor rappresenta la minaccia quantistica più urgente e motivante per la ricerca sul calcolo quantistico applicato.

Tutta l'analisi svolta in questa sezione ha però assunto un modello di calcolo *ideale*: porte unitarie perfette, coerenza illimitata, nessuna interazione con l'ambiente.

Su hardware reale questi presupposti non reggono, e la distanza tra il modello teorico e l'esecuzione fisica è precisamente il nodo critico che rende ancora irraggiungibile la fattorizzazione di numeri crittograficamente rilevanti. Comprendere le origini e gli effetti del rumore quantistico è quindi il passo necessario per valutare le reali prospettive dell'algoritmo di Shor.

A.3.3 Sensibilità delle componenti di Shor al rumore

La seguente analisi è stata trasferita dalla Sezione sul rumore del Capitolo 3. Nel corpo resta il nesso causale essenziale; qui si conservano derivazioni, esempio numerico e figura di scala.

Errori nella Quantum Fourier Transform

La QFT usa rotazioni $R_k = \text{diag}(1, e^{2\pi i/2^k})$ con angoli esponenzialmente piccoli. Per $k = 10$, $2\pi/1024 \simeq 6 \times 10^{-3}$ rad. Gli errori coerenti di calibrazione possono sommarsi in ampiezza, mentre il canale depolarizzante produce degradazione stocastica: non è quindi corretto ricondurli a un solo “errore di fase” lineare. Le campagne li mantengono distinti.

Errori nella Phase Estimation

I qubit di controllo devono mantenere la coerenza lungo il cammino critico dell'esponenziazione modulare. Come ordine di grandezza,

$$T_2 \gg \tau_{\text{gate}} O(n^3), \quad (\text{A.30})$$

ma questa espressione non determina una soglia universale in bit: la costante dipende da aritmetica e compilazione, mentre in regime fault-tolerant il confronto diretto è sostituito da un budget di errore logico.

Errori nell'esponenziazione modulare

Sotto l'ipotesi puramente illustrativa di L eventi indipendenti con probabilità ϵ , il proxy di nessun evento è

$$P_{\text{0-eventi}} = (1 - \epsilon)^L, \quad L \sim O(n^3). \quad (\text{A.31})$$

Ponendo soltanto per visualizzare la scala $L = n^3$, per $n = 15$ e $\epsilon = 10^{-3}$ si ottiene $(1 - 10^{-3})^{3375} \simeq 0.034$. Non è una probabilità di successo di Shor: alcuni errori possono non alterare l'esito e la formula non rappresenta rilassamento, readout o struttura circuitale.

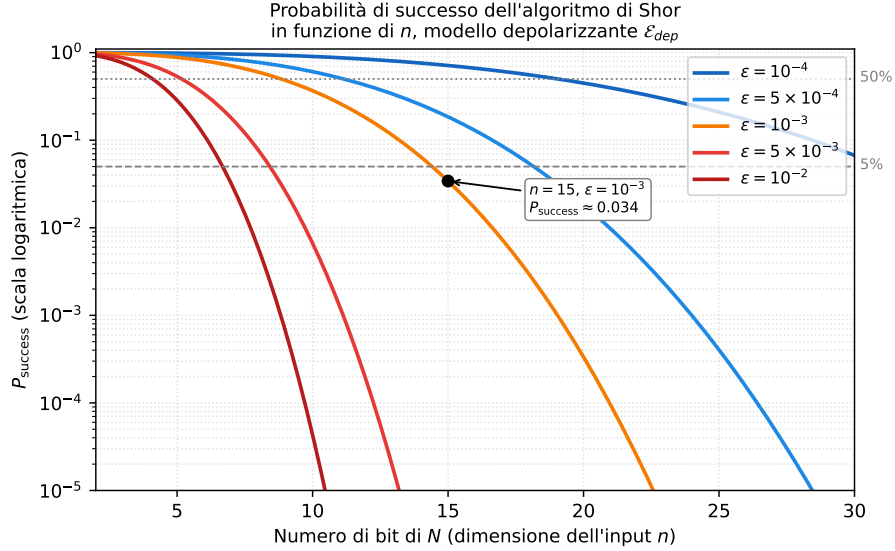


Figura A.7: Proxy illustrativo $(1 - \epsilon)^{n^3}$ per cinque valori di ϵ . La curva non deriva dai circuiti sperimentali e mostra soltanto l'accumulo moltiplicativo di eventi.

A.3.4 Scelta dell'aritmetica e perimetro delle istanze

Questo dettaglio metodologico proviene dalla sezione sul disegno sperimentale. Sintetizzare l'esponentiazione modulare come un operatore unitario denso costa $O(4^k)$ porte elementari su k qubit [26]. La decomposizione di Beauregard [16] evita tale sintesi costruendo l'aritmetica nel dominio della QFT, con addizionatori di Draper, e richiede $O(n^3)$ porte su $2n + 3$ qubit. Il costo resta elevato: per $N = 21$, $a = 2$ e otto qubit di conteggio, la compilazione congiunta in `rz/sx/x/cx` produce 21 036 porte CX e profondità 23 081. Il costo dominante non è la trasformata finale della QPE, ma le trasformate interne agli addizionatori; è lì che l'approssimazione deve agire per incidere sul conteggio di porte.

Le campagne rumorose sono perciò limitate a $N = 15$, $a = 7$. Le istanze $N = 21$, $a = 2$, e $N = 35$, $a = 6$, sono state validate idealmente mediante truth table, pulizia delle ancille e distribuzione QPE, ma escluse dal campionamento rumoroso per il costo della simulazione. Questa distinzione impedisce di confondere validazione funzionale e fattibilità rumorosa.

In termini di spazio, fattorizzare un intero di n bit richiede almeno $2n$ qubit logici per la phase estimation, oltre alle ancille necessarie all'aritmetica modulare. Il conteggio non include l'overhead fault-tolerant e non basta a stimare la fattibilità: connettività e routing possono aumentare profondità e numero di porte senza modificare il numero di qubit logici.

A.4 I canali di rumore: derivazioni complete

I quattro canali quantistici che modellano il rumore sono mappe *Completely Positive Trace-Preserving* (CPTP) in rappresentazione di Kraus, secondo l'Eq. 3.17. Il Capitolo 3 ne riporta il canale depolarizzante per esteso, essendo quello su cui poggia la campagna sperimentale, e l'enunciato compatto degli altri tre; qui se ne danno le derivazioni complete.

A.4.1 Canale Bit Flip

Il canale **bit flip** modella un errore di tipo X discreto: con probabilità p il qubit viene invertito ($|0\rangle \leftrightarrow |1\rangle$), con probabilità $1 - p$ rimane invariato. Gli operatori di Kraus sono $K_0 = \sqrt{1-p} I$ e $K_1 = \sqrt{p} X$, e il canale agisce come:

$$\mathcal{E}_{BF}(\rho) = (1-p)\rho + pX\rho X \quad (\text{A.32})$$

Sulla sfera di Bloch, le componenti r_y e r_z vengono contratte di un fattore $(1 - 2p)$ mentre la componente r_x rimane invariata. Per $p = 1/2$ il canale annulla le componenti y e z , ma conserva r_x : equivale a una completa dephasing nella base degli autostati di X , non alla distruzione completa dello stato.

A.4.2 Canale Phase Flip

Il canale **phase flip** modella la perdita di informazione di fase senza cambiamento di popolazione: la sovrapposizione decade senza che l'energia venga dissipata. Con operatori di Kraus $K_0 = \sqrt{1-p} I$ e $K_1 = \sqrt{p} Z$, il canale è:

$$\mathcal{E}_{PF}(\rho) = (1-p)\rho + pZ\rho Z \quad (\text{A.33})$$

L'effetto sulla matrice densità è immediato:

$$\mathcal{E}_{PF} \begin{pmatrix} \rho_{00} & \rho_{01} \\ \rho_{10} & \rho_{11} \end{pmatrix} = \begin{pmatrix} \rho_{00} & (1-2p) \rho_{01} \\ (1-2p) \rho_{10} & \rho_{11} \end{pmatrix} \quad (\text{A.34})$$

Le popolazioni ρ_{00} e ρ_{11} rimangono invariate, mentre le **coerenze** ρ_{01} , ρ_{10} vengono sopprese di un fattore $(1-2p)$. Questo è il canale che descrive il decadimento T_2 : parametrizzando $p(t) = \frac{1}{2}(1 - e^{-t/T_2})$, si ottiene $\rho_{01}(t) = \rho_{01}(0) e^{-t/T_2}$.

A.4.3 Canale di Amplitude Damping

Il canale di **amplitude damping** modella il rilassamento energetico T_1 : il qubit decade spontaneamente dallo stato eccitato $|1\rangle$ allo stato fondamentale $|0\rangle$ con probabilità $\gamma = 1 - e^{-t/T_1}$. Gli operatori di Kraus sono:

$$K_0 = \begin{pmatrix} 1 & 0 \\ 0 & \sqrt{1-\gamma} \end{pmatrix}, \quad K_1 = \begin{pmatrix} 0 & \sqrt{\gamma} \\ 0 & 0 \end{pmatrix} \quad (\text{A.35})$$

Il canale agisce sulla matrice densità come:

$$\mathcal{E}_{AD}(\rho) = \begin{pmatrix} \rho_{00} + \gamma \rho_{11} & \sqrt{1-\gamma} \rho_{01} \\ \sqrt{1-\gamma} \rho_{10} & (1-\gamma) \rho_{11} \end{pmatrix} \quad (\text{A.36})$$

La popolazione ρ_{11} decade verso ρ_{00} con tasso γ , mentre le coerenze vengono sopprese di $\sqrt{1-\gamma}$. A differenza dei canali precedenti, questo canale è **non simmetrico**: sulla sfera di Bloch contrae la sfera verso il polo nord $|0\rangle$, non verso il centro:

$$r_x \rightarrow \sqrt{1-\gamma} r_x, \quad r_y \rightarrow \sqrt{1-\gamma} r_y, \quad r_z \rightarrow (1-\gamma) r_z + \gamma \quad (\text{A.37})$$

Per $t \rightarrow \infty$ ($\gamma \rightarrow 1$), qualsiasi stato converge a $|0\rangle \langle 0|$, indipendentemente dallo stato iniziale.

A.4.4 Confronto geometrico dei quattro canali

La visualizzazione seguente, trasferita dal corpo, completa le derivazioni mostrando che canali con probabilità d'errore simili possono produrre geometrie molto diverse nello spazio degli stati.

Per il canale depolarizzante, l'identità $X\rho X + Y\rho Y + Z\rho Z = 2I - \rho$ (per $\text{Tr } \rho = 1$) conduce dalla forma a miscela di Pauli alla forma compatta della Eq. (3.19). Il

vettore di Bloch è moltiplicato per $1 - 4p/3$ e, per $p = 3/4$, lo stato diventa massimamente misto qualunque fosse l'ingresso. Nel parametro Aer λ , invece, la probabilità aggregata di una Pauli non identità è $3\lambda/4$ su un qubit e $15\lambda/16$ su due. I preset uniformi compongono depolarizzazione e thermal relaxation; M8 usa il solo proxy Pauli per gate, mentre M11 converte l'infedeltà media della snapshot senza aggiungere nuovamente T_1/T_2 , evitando il doppio conteggio.

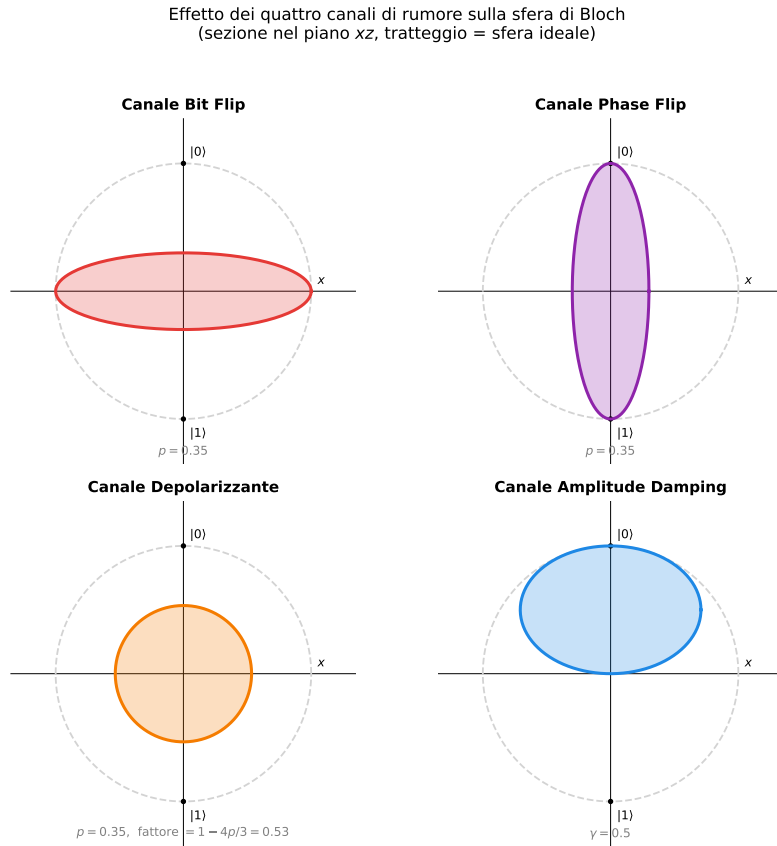


Figura A.8: Effetto dei quattro canali nel piano xz . La curva tratteggiata è la sfera ideale: bit flip e phase flip contraggono direzioni selettive, il depolarizzante contrae isotropicamente e l'amplitude damping sposta lo stato verso $|0\rangle$.

Canale	Effetto sulle coerenze	Popolazioni	Modella
Bit flip	contrae r_y, r_z	scambiate	errore X
Phase flip	sopprime ρ_{01}	invariate	dephasing T_2
Depolarizzante	contrazione isotropa	verso $I/2$	gate generico
Amplitude damping	sopprime ρ_{01}	verso $ 0\rangle$	rilassamento T_1

Tabella A.3: Confronto fra i quattro canali fondamentali [8].

A.5 Strategie anti-rumore e richiami estesi di QEC

Questa sezione raccoglie il materiale trasferito dalla parte del Capitolo 3 dedicata alle strategie. Nel corpo restano la tabella comparativa, il ciclo QEC essenziale e le scelte che guidano gli esperimenti; qui sono conservati rassegna, esempi e confronti estesi.

A.5.1 I quattro livelli di intervento

1. **Soppressione del rumore.** Agisce prima o durante il calcolo riducendo fisicamente l'accoppiamento con l'ambiente. Comprende *dynamical decoupling*, ottimizzazione della durata dei gate e isolamento criogenico ed elettromagnetico. Riduce il rumore ma è vincolata all'hardware disponibile.
2. **Mitigazione degli errori.** Opera a posteriori senza correggere i qubit. ZNE, *probabilistic error cancellation* e gate twirling combinano più esecuzioni o randomizzazioni per stimare l'output ideale. Il limite è l'overhead di campionamento, la cui legge dipende dalla tecnica e può diventare esponenziale nell'errore totale, soprattutto per la PEC.
3. **Tolleranza ai guasti.** Codifica ogni qubit logico in molti qubit fisici. Sotto le ipotesi del threshold theorem e al di sotto della soglia [24], l'errore logico può essere ridotto aumentando distanza e overhead. Le stime per algoritmi di grande scala possono richiedere centinaia o migliaia di qubit fisici per qubit logico, a seconda di codice, errore fisico e obiettivo.
4. **Metodi appresi.** Modelli classici possono classificare output, stimare qualità o assistere un decoder. Non sono automaticamente QML, né sostituiscono per definizione mitigazione o QEC.

Nel perimetro della tesi la soppressione non è controllabile, la mitigazione richiede esecuzioni aggiuntive e la fault tolerance completa eccede le risorse disponibili. Un modello a valle non modifica il circuito, ma il suo vantaggio è un'ipotesi da confrontare con regole semplici. Per questo TOP-1, TOP-4 e M2 usano gli stessi istogrammi, mentre la ZNE è un controllo secondario.

A.5.2 Usi possibili dei metodi appresi

Il termine appropriato per gli esperimenti è **machine learning classico applicato a dati quantistici**: Random Forest, SVM e MLP sono addestrati ed eseguiti classicamente. Il QML in senso stretto impiega circuiti o modelli quantistici nell'apprendimento e non è implementato. Tre usi distinti chiariscono il perimetro:

- nella **caratterizzazione del rumore**, una regressione può collegare T_1, T_2 , profondità e fedeltà per predire la qualità;
- nell'**ottimizzazione circuitale**, un agente può adattare decomposizione e routing al profilo d'errore del dispositivo;
- nella **decodifica QEC**, un modello usa le sindromi per proporre una correzione o correggere residualmente una decisione analitica.

Il terzo caso colloca il machine learning *dentro una fase* della QEC, senza ridefinire la disciplina.

A.5.3 Principio, codici e decoder QEC

Il no-cloning impedisce di proteggere un qubit duplicandolo, ma non di *distribuire* l'informazione logica su più qubit fisici. Misurando operatori collettivi si ottiene una sindrome dell'errore senza rivelare, e quindi distruggere, lo stato logico. Il paradigma comprende:

1. codifica di $|\psi_L\rangle$ su n qubit fisici;
2. misura di stabilizzatori che commutano con i codeword e anticommutano con determinate classi d'errore;
3. decodifica della sindrome e scelta dell'operazione di recupero.

Il **codice di Shor** [28] codifica un qubit in nove e corregge un errore singolo arbitrario:

$$|0_L\rangle = \frac{(|000\rangle + |111\rangle)^{\otimes 3}}{2\sqrt{2}}, \quad |1_L\rangle = \frac{(|000\rangle - |111\rangle)^{\otimes 3}}{2\sqrt{2}}. \quad (\text{A.38})$$

La struttura combina protezione dalla fase mediante interferenza e dal bit flip mediante ripetizione. Il **codice di Steane** [10] usa sette qubit e la struttura del codice classico di Hamming [7, 4, 3]; è un codice CSS [52] e corregge anch'esso

un errore singolo. Il **surface code** [11] dispone invece i qubit su una griglia 2D con interazioni locali: sotto una soglia dipendente da circuito, rumore e decoder, aumentare la distanza d sopprime l'errore logico; sopra soglia lo amplifica.

Il punto di contatto con il machine learning è la decodifica [25, 19]. Un modello può sostituire il decoder o intervenire sui suoi residui, ma deve essere confrontato a parità di sindromi e calendario. Nel protocollo studiato la sostituzione del matching non conviene; l'affiancamento recupera invece una parte degli errori correlati assenti dal grafo analitico. Questo è un risultato circoscritto al modello simulato, non una superiorità generale dei decoder appresi.

A.5.4 Verifica del filtro appreso nella prima fase

Senza ML, ogni iterazione produce un istogramma: TOP-1 prova la moda e TOP-4 i primi quattro esiti. Le iterazioni non vanno confuse con gli shot che formano ciascun istogramma; nelle campagne la media è fra 1 e 1.5 iterazioni. Con ML, il classificatore riceve le feature dell'istogramma già campionato e decide se attivare TOP-4: non classifica il singolo shot e non elimina il costo di costruzione della distribuzione.

L'ipotesi preregistrata era una riduzione rispetto a TOP-1. Essa si osserva (nello scenario moderato da $\bar{M}_1 = 1.37$ a $\bar{M} = 1.00$), ma l'ablazione mostra che deriva dalla ricerca multi-candidato: il classificatore costa più di TOP-4 in entrambi gli scenari. Il risultato negativo delimita quel filtro e quelle etichette; non è una conclusione generale su ML, mitigazione o decoder.

A.5.5 AQFT e scambio fra qubit e porte

La QFT impiega rotazioni controllate con angolo esponenzialmente più piccolo all'aumentare della distanza fra qubit. L'AQFT scarta quelle sotto una soglia: perde precisione algoritmica, ma riduce il conteggio di porte. Per $N = 21$, troncando le trasformate interne agli addizionatori porta le porte a due qubit da 49 661 a 23 178; nelle tre dimensioni del registro di phase estimation la variante troncata supera quella completa in tutti i nove confronti. Ridurre la profondità è una strategia anti-rumore perché accorcia il cammino critico rispetto a T_1 e T_2 , indipendentemente dal numero di qubit disponibili.

Chevignard, Fouque e Schrottenloher [18] perseguono uno scambio diverso: l'aritmetica modulare approssimata in un sistema di residui riduce i qubit logici da circa $2n$ a $n/2 + o(n)$; per RSA-2048 stima 1730 qubit contro i 6144 del riferimento. Gli autori riportano però $2^{35.5}$ porte Toffoli e in media quaranta esecuzioni, $2^{40.9}$ complessive,

contro $2^{31.3}$ per una singola esecuzione di riferimento. Un lavoro successivo [7] riduce il costo e arriva a 1399 qubit logici. Nel regime fault-tolerant domina il costo dei qubit; nel regime NISQ simulato qui domina la profondità rispetto alla coerenza. Le due strategie rispondono quindi a vincoli diversi.

A.5.6 Transizione fra le due fasi sperimentali

Il primo esito delimita il post-processing: l'istogramma contiene candidati utili, sfruttati da TOP-4, ma il classificatore su quelle etichette non aggiunge informazione. Il nucleo successivo si sposta durante il calcolo: codice a ripetizione, Steane, surface code e confronto fra decodifica analitica e appresa. L'analisi fenomenologica di Shor resta separata dai tassi QEC per ciclo, mentre l'AQFT valuta una leva circuitale complementare.

A.5.7 Dettagli tecnici del contratto sperimentale

Questo blocco conserva i dettagli operativi alleggeriti nella sezione metodologica. Il circuito è compilato una sola volta nella base `rz/sx/x/cx`, con `optimization_level=2` e seed del transpiler fissato. La configurazione di riferimento contiene 224 porte `cx` e ha profondità 412; hash e conteggi completi della netlist sono registrati nel manifest. Per ogni coppia (replica, iterazione), Aer genera un unico istogramma che viene consegnato senza ricampionamento a M1, TOP-4 e M2.

I pareggi di frequenza sono risolti mediante una priorità SHA-256 dipendente dal seed e dalla bitstring. In questo modo l'ordine dei candidati non dipende né dall'ordine interno del dizionario né dal valore numerico dell'esito. Il dataset conserva sia l'etichetta TOP-1 sia l'audit TOP-16, mentre la pipeline usa esclusivamente il modello TOP-1 compatibile con il manifest. Ciascuno scenario contiene 30 repliche e un budget di 50 iterazioni; il fallimento al budget assume la sentinella 51. I test inferenziali agiscono sempre sui vettori appaiati completi, non sulle sole medie aggregate.

Appendice B

Strumenti, ambiente e codice

B.1 Strumenti e ambiente di simulazione — trattazione estesa

Questa sezione accoglie la trattazione estesa originariamente inclusa nel Capitolo 4. La Sezione 4.1 ne conserva le sole conclusioni operative; qui rimangono motivazioni dei framework, metodi di simulazione, configurazione WSL e valutazione dell’accelerazione GPU.

La scelta degli strumenti di simulazione non è neutrale. In informatica quantistica, il framework adottato determina quali modelli di rumore sono disponibili, verso quale hardware è possibile traslare i circuiti e con quali risultati della letteratura è possibile confrontarsi direttamente. Questa sezione descrive gli strumenti utilizzati nella parte sperimentale di questa tesi, motivando ciascuna scelta e collocandola rispetto alle alternative disponibili.

B.1.1 Qiskit: il Framework Quantistico di Riferimento

Qiskit (*Quantum Information Science Kit*) è un *Software Development Kit* (SDK) open-source sviluppato da IBM Quantum per la programmazione, la simulazione e l’esecuzione di algoritmi quantistici [12]. Rilasciato pubblicamente nel 2017, offre integrazione diretta con l’hardware IBM ed è uno dei framework ampiamente usati nella ricerca su circuiti quantistici a corto termine [5, 53].

Architettura del Framework

Qiskit è organizzato in tre componenti principali, ciascuno con un ruolo distinto nel ciclo di vita di un esperimento quantistico:

- **Qiskit (core)** — il nucleo del framework, già noto come Qiskit Terra. Fornisce le primitive per costruire i circuiti, compilarli verso un backend target tramite `transpile` e accedere alle interfacce `Sampler` ed `Estimator`. Gestisce la pipeline dalla descrizione astratta fino all'invio al simulatore o all'hardware reale.
- **Qiskit Aer** — il modulo di simulazione classica ad alte prestazioni. Implementa diversi metodi di simulazione esatta e approssimata, con supporto nativo per modelli di rumore realistici. È il componente centrale di questa tesi e viene descritto in dettaglio nella Sezione B.1.2.
- **Qiskit IBM Runtime** — l'interfaccia per l'esecuzione su processori IBM Quantum reali tramite cloud (`qiskit-ibm-runtime`). Fornisce le primitive `Sampler` e `Estimator` ottimizzate per l'hardware reale, con supporto per tecniche di soppressione del rumore come il *Dynamical Decoupling* e il *Gate Twirling* [54].

Perché Qiskit e non un'alternativa

La scelta di Qiskit è stata formalizzata nell'incontro con il relatore del 18 marzo 2026, in seguito all'analisi del tutorial ufficiale IBM per l'algoritmo di Shor [54]. La decisione è motivata da quattro ragioni convergenti.

Prima ragione: controllo esplicito del modello. Qiskit Aer permette di comporre canali depolarizzanti, termici e di readout e di legarli a un gate set dichiarato. Questa flessibilità consente esperimenti controllati e riproducibili; non rende però i preset uniformi della tesi direttamente equivalenti alla calibrazione di una QPU reale.

Seconda ragione: copertura delle primitive necessarie. Qiskit Aer espone nello stesso ambiente i canali depolarizzanti, termici, di readout e coerenti richiesti dal disegno. Altri framework, fra cui Cirq [55] e PennyLane [56], offrono anch'essi simulazione rumorosa; la scelta non pretende di stabilire una graduatoria generale, ma riduce le conversioni rispetto al codice Qiskit adottato.

Terza ragione: disponibilità dell'implementazione di riferimento. Il circuito dell'algoritmo di Shor utilizzato in questa tesi è tratto da un'implementazione

open-source in Qiskit [57]. Questo punto di partenza, validato dalla letteratura [5], ha reso naturale mantenere Qiskit come framework unico lungo l'intero pipeline sperimentale.

Quarta ragione: allineamento con le fonti implementative. Diversi lavori usati come riferimento adottano Qiskit [5, 53, 58]. Condividere il framework facilita il confronto delle API e dei circuiti, ma non rende direttamente comparabili risultati ottenuti con gate set, versioni, transpiler o noise model differenti.

Tabella B.1: Confronto tra i principali framework per la simulazione quantistica con rumore.

Framework	Sviluppatore	Simulatore con rumore	Target hardware primario
Qiskit + Aer	IBM Quantum	Completo (T1, T2, depolarz., readout)	IBM superconduttori
Cirq	Google Quantum AI	Parziale	Google superconduttori
PennyLane	Xanadu	Limitato	Vari (plug-in)
Forest / PyQuil	Rigetti	Parziale	Rigetti superconduttori

B.1.2 Qiskit Aer: il Simulatore

Qiskit Aer è il modulo di simulazione classica di Qiskit. Il suo componente principale è `AerSimulator`, un simulatore ad alte prestazioni che supporta più metodi di simulazione selezionabili a seconda della dimensione del circuito e del tipo di analisi richiesta.

Metodi di Simulazione

`AerSimulator` espone i seguenti metodi principali tramite il parametro `method`:

- `statevector` — simulazione del vettore di stato completo. Richiede 2^n ampiezze complesse in memoria ed è il metodo usato dalla campagna rumorosa principale su $N = 15$.
- `density_matrix` — evolve la matrice densità ρ anziché il vettore di stato, permettendo di modellare canali di rumore non unitari in modo esatto. Richiede 4^n elementi e riduce il limite pratico a circa 25 qubit.

- **stabilizer** — simulazione efficiente di circuiti Clifford puri (senza porte non-Clifford come T o R_z arbitrarie). Scala a migliaia di qubit ma non è applicabile all'algoritmo di Shor, che utilizza rotazioni di fase arbitrarie.
- **matrix_product_state** — rappresenta lo stato come rete di tensori; senza troncamenti imposti resta una rappresentazione controllata, ma il costo cresce con l'entanglement. È usato in M8/M13 per contenere il costo delle molte repliche.

La scelta è quindi per campagna, non globale: **statevector** per la prima fase N15 e **matrix_product_state** per la sensibilità M8/M13. Le istanze N21/N35 sono validate idealmente ma escluse dalle campagne rumorose generali.

Modelli di Rumore

Il modulo `qiskit_aer.noise` fornisce le primitive per costruire modelli di rumore compositi. Ogni errore è rappresentato come un canale quantistico che viene applicato ai gate specificati, con la possibilità di assegnare errori diversi a qubit diversi (rumore eterogeneo) o un errore uniforme a tutti i qubit (*all-qubit error*). I tipi di errore disponibili includono:

- **depolarizing_error** — errore depolarizzante su gate a 1 o 2 qubit, parametrizzato da λ nella convenzione Aer; la probabilità Pauli non-identità è $3\lambda/4$ o $15\lambda/16$.
- **thermal_relaxation_error** — modella il rilassamento verso lo stato fondamentale (T_1) e la perdita di coerenza di fase (T_2), con un tempo di gate configurabile.
- **ReadoutError** — matrice di confusione che descrive la probabilità di misurare $|0\rangle$ come $|1\rangle$ e viceversa.
- Errori coerenti — applicazione di una rotazione indesiderata su ogni gate, modellabile tramite operatori unitari arbitrari.

Il modello di rumore usato nella prima fase combina errori depolarizzanti, rilassamento termico e readout error con parametri uniformi. È un modello illustrativo per isolare i meccanismi, non uno snapshot di `ibm_marrakesh`: non include coupling map, routing, errori per qubit, crosstalk o deriva temporale. La configurazione dettagliata è riportata nel Capitolo 2.

Transpilazione

Prima dell'esecuzione, la funzione `transpile` riscrive il circuito nella base esplicita `rz/sx/x/cx` e applica ottimizzazioni strutturali. La campagna usa il livello di ottimizzazione 2 e il seed del transpiler 20260819; questa coppia, insieme all'hash della sequenza compilata, è registrata in ogni artefatto. La base esplicita impedisce che porte composte restino opache e sfuggano al canale di rumore previsto.

B.1.3 Accelerazione GPU con Qiskit Aer

La libreria `qiskit-aer-gpu` estende `AerSimulator` con il supporto per l'accelerazione tramite GPU NVIDIA attraverso CUDA. Il backend `statevector` è interamente eseguito sulla GPU: il vettore di stato viene allocato nella memoria VRAM e le operazioni di gate sono eseguite come moltiplicazioni matrice-vettore parallelizzate su migliaia di core CUDA.

Motivazione pratica. L'accelerazione era stata valutata per la generazione dei dataset e per le campagne replicate. Non vengono però riportati fattori di accelerazione ipotetici come risultati sperimentali, poiché l'ambiente canonico finale usa la CPU.

Specifiche hardware. La GPU prevista è una NVIDIA GeForce RTX 4070 con 12 GB di memoria VRAM e 5888 core CUDA (architettura Ada Lovelace). La sola allocazione di uno statevector `complex128` avrebbe un tetto teorico di 29 qubit, inferiore nella pratica per memoria di lavoro e overhead. Questo limite dimensionale non garantisce inoltre la fattibilità: circuiti molto profondi e traiettorie rumorose moltiplicano il costo temporale, ed è proprio ciò che esclude N21/N35 dalla campagna rumorosa.¹

Esito: l'accelerazione non è stata utilizzabile. Quanto sopra descrive il piano iniziale. In sede di implementazione il driver CUDA riconosceva la scheda, ma il backend C++ di Aer sollevava a runtime l'errore riportato nel Listato B.2. Per questo motivo **tutti gli esperimenti riportati nella tesi sono stati eseguiti su CPU** e il backend effettivo è stato registrato nei metadati. Il vincolo di memoria resta quello indicato, riferito però alla memoria di sistema anziché alla VRAM. Questa diagnosi costituisce l'approfondimento della limitazione sintetizzata nella Sezione 4.1.2.

¹La memoria del solo vettore è $2^n \times 16$ byte. Con 12 GB decimali, $\log_2(12 \times 10^9/16) \simeq 29.48$, quindi il massimo intero teorico è $n = 29$.

B.1.4 scikit-learn: la Libreria per il Classificatore

Il Metodo 2 richiede l'addestramento di un classificatore binario. La libreria scelta è scikit-learn [15], una delle librerie di machine learning più consolidate nell'ecosistema Python scientifico.

La motivazione principale è pragmatica: scikit-learn espone un'interfaccia unificata (`fit` / `predict`) per un ampio catalogo di classificatori — Random Forest, SVM, MLP, e altri — permettendo di confrontarli sullo stesso dataset con minime modifiche al codice. Questo facilita la selezione sperimentale del modello migliore senza dover introdurre dipendenze da framework più pesanti (TensorFlow, PyTorch) per un classificatore che, per il problema in esame, si prevede non richieda architetture profonde.

B.1.5 Ambiente di Esecuzione: WSL con Ubuntu

Le simulazioni vengono eseguite in ambiente Linux tramite WSL (versione 2) su Windows 11. La scelta è motivata dalla necessità di mantenere la compatibilità con l'ecosistema Linux di Qiskit e delle librerie scientifiche Python, garantendo al contempo l'integrazione con i driver CUDA per WSL forniti da NVIDIA.

WSL 2 esegue un kernel Linux completo in una macchina virtuale leggera. L'ambiente Python è isolato in un virtual environment dedicato, garantendo la riproducibilità delle dipendenze e la compatibilità delle versioni tra le librerie usate; le versioni esatte sono versionate nel file `requirements.txt` della repository. È in questo stesso ambiente che l'accesso alla GPU si è rivelato non praticabile (Sezione B.1.3).

Configurazione riproducibile e versioni

La campagna canonica usa Windows 11 come sistema ospite, WSL2 con Ubuntu 24.04, Python 3.12 e Qiskit 2.5; Stim 1.16.0 e PyMatching 2.4.0 sono impiegati nella parte surface-code. L'ambiente virtuale è creato e popolato dal file `requirements.txt` come nel Listato B.1. Ogni artefatto registra inoltre le versioni effettivamente importate di Qiskit, Aer, NumPy, SciPy e scikit-learn, così che il file dei requisiti e il manifest dell'esecuzione svolgano ruoli complementari: il primo prescrive l'ambiente, il secondo certifica quello realmente usato. Tutte le campagne riportate sono state orchestrate su CPU.

Con l'ambiente così definito, il Capitolo 4 descrive come Qiskit, Aer e scikit-learn si integrano nell'architettura del pipeline sperimentale: dalla costruzione del

circuito di Shor alla generazione del dataset di addestramento, fino all’inferenza del classificatore.

B.2 Codice sorgente dei componenti implementati

Questa sezione raccoglie il materiale implementativo trasferito dal Capitolo 4: contratto di esecuzione, organizzazione dei moduli, dettagli dei codici QEC ed estratti dei sorgenti. Gli output numerici non sono codificati nei listati: ogni campagna li salva in un artefatto JSON schema 2 corredato di manifest.

B.2.1 Contratto implementativo e riproducibilità

L’ambiente Python è isolato in un virtual environment su WSL; versioni di interprete, Qiskit, Aer, NumPy, SciPy e scikit-learn vengono registrate nel manifest di ogni artefatto. La campagna canonica usa Qiskit 2.5 e simulazione CPU. Il Listato B.1 documenta la creazione dell’ambiente, mentre il Listato B.2 conserva l’esito negativo del tentativo GPU: l’accelerazione non è quindi parte del contratto riproducibile.

Netlist compilata e manifest

La compilazione è centralizzata e condivisa da training, baseline e sweep. Il contratto fissa `optimization_level=2`, seed del transpiler e base nativa `rz/sx/x/cx`. Con Qiskit 2.5, $N = 15$, $a = 7$ e otto qubit di conteggio, la netlist ha profondità 412 e contiene 224 porte `cx`. Per $N = 21$, $a = 2$, la variante Beauregard raggiunge invece profondità 23 081 e 21 036 `cx`; a $\lambda = 10^{-3}$ il proxy registrato vale 2.6987×10^{-9} . Quest’ultimo numero documenta il costo circuitale, non la probabilità di successo, ed è la ragione pratica per cui N21/N35 restano nelle validazioni ideali.

Per ogni compilazione il manifest conserva dimensione, profondità, conteggio completo delle operazioni, base, livello e seed di transpiler, versione delle librerie e SHA-256 della sequenza compilata. Modelli e risultati dichiarano inoltre schema, revisione, preset, seed e definizione dell’etichetta; l’inferenza rifiuta un classificatore con manifest incompatibile. In tal modo un artefatto non può essere associato per errore a una revisione differente del circuito.

Seed, tie-break e confronto appaiato

Per le campagne N15, una sola simulazione fornisce l'istogramma comune a M1, TOP-4 e M2. Il seed è separato fra repliche e iterazioni secondo

$$\text{seed_simulator} = \text{rep} \cdot 1\,000\,000 + \text{iteration} \cdot 10\,000, \quad (\text{B.1})$$

una distanza superiore al numero di shot che evita la quasi sovrapposizione degli stream Aer osservata con semi consecutivi. Nei pareggi, una priorità SHA-256 calcolata da seed e bitstring evita dipendenze dall'ordine del dizionario o dal valore numerico del candidato.

La prima iterazione di successo è registrata separatamente per ciascuna strategia; un fallimento entro `max_iter` è codificato, ai soli fini inferenziali, con `max_iter+1`. Il Wilcoxon appaiato usa `zero_method='pratt'`: test unilaterale per $M1 > \text{TOP-4}$ e $M1 > M2$, bilaterale per $\text{TOP-4} - M2$. Il JSON schema 2 conserva input completi, vettori appaiati, seed, test, versioni software, hash e manifest; figure e tabelle leggono questi artefatti senza trascrizione manuale.

B.2.2 Organizzazione completa del codice sperimentale

I sorgenti eseguibili sono suddivisi per campagna, così che codice, README, log e artefatti restino associati allo stesso contratto. La Tabella B.2 esplicita la mappa usata nel testo; le sigle M identificano le milestone sperimentali e non livelli di affidabilità del risultato.

Cartella	Oggetto	Strumento	Costo dominante
campagne_	Shor, TOP-K e clas-	Qiskit Aer + scikit-	traiettorie rumorose
classiche_M1-M4	sificatore	learn	
M5_repetition_	codice a ripetizione	Qiskit Aer	trascurabile
code			
M6_steane_code	Steane $[[7, 1, 3]]$	Qiskit Aer	Monte Carlo Pauli
M7_surface_code	surface code	Stim + PyMatching	decodifica
M8_shor_logico	proxy per gate e TOP-4 M13	Qiskit Aer	matrix product state
M10_neural_	decodifica appresa e ibrida	Stim + PyMatching + scikit-learn	addestramento
decoder			
M11_layout	layout su snapshot eterogenea	Qiskit Aer	compilazioni replicate
M12_coerenti	controllo su errori coerenti	Qiskit Aer	simulazione full-state
M15_qft_	QFT approssimata	Qiskit Aer	sweep di compilazione
topologia			
extra_rumore_	Pauli contro sovrarotazione	Qiskit Aer	controllo full-state
coerente			1Q

Tabella B.2: Organizzazione completa delle campagne. La toolchain cambia con la classe del circuito: Aer per circuiti generali di taglia trattabile, Stim e PyMatching per circuiti stabilizer su larga scala.

B.2.3 Dettagli implementativi dei componenti QEC

M5: codice a ripetizione

Gli stabilizzatori sono misurati indirettamente copiandone la parità su qubit ancillari, senza misurare i dati e collassare l'informazione logica. Porte identità collocate nel punto di iniezione costituiscono l'aggancio a cui Aer associa il canale senza modificare lo stato ideale. Le varianti in base Z e X condividono lo stesso schema, con i cambi di base necessari alla rilevazione, rispettivamente, di bit-flip e phase-flip (Listato B.11).

M6: codice di Steane

Lo stato $|0_L\rangle$ è preparato ponendo tre qubit seme in sovrapposizione e propagandone il supporto con CNOT. Gli stabilizzatori CSS di tipo X e Z alimentano una lookup table di correzione a peso minimo. La stima di p_L usa un modello di Pauli; l'estrazione di sindrome è ideale e non costituisce un protocollo fault-tolerant completo (Listato B.12).

M7: surface code

Stim genera il circuito di memoria e campiona i detection event; PyMatching costruisce dal detector error model il decoder MWPM. Nell'esperimento sul crosstalk, il canale fra dati adiacenti è inserito ricorsivamente anche nei blocchi REPEAT; i qubit di misura sono identificati dal primo MR, non dalla misura finale che comprende anche i dati. I Listati B.13 e B.14 mostrano i due passaggi.

M8/M13 e M10: proxy di Shor e decoder appreso

Nel proxy a livelli, p_L è la probabilità totale del Pauli non-identità applicato dopo ogni gate incluso nel modello. Prima dell'esecuzione Aer viene convertito in $\lambda_1 = 4p_L/3$ per i gate 1Q e $\lambda_2 = 16p_L/15$ per quelli 2Q; le **rz** restano virtuali e i canali agiscono su **sx/x** e **cx**. Questo parametro per gate non coincide con il logical error rate per round o memoria di M6/M7 senza una compilazione fault-tolerant e un mapping esplicito. In M10, decoder appreso e MWPM ricevono esattamente gli stessi detection event, così il confronto riguarda la regola di decodifica e non i campioni osservati.

B.2.4 Estratti dei sorgenti

```
python3 -m venv ~/quantum-env
source ~/quantum-env/bin/activate
python -m pip install -r requirements.txt
python -c "import qiskit, qiskit_aer; print(qiskit.__version__)"
```

Listing B.1: Installazione dell'ambiente Python e delle librerie necessarie

```
RuntimeError: Simulation device "GPU" is not supported on this
system
```

Listing B.2: Errore GPU osservato su WSL durante i test

```
def inverse_qft(n_qubits):
    qc = QuantumCircuit(n_qubits, name='QFT_dagger')
    for q in range(n_qubits // 2):
        qc.swap(q, n_qubits - q - 1)
    for j in range(n_qubits):
        for m in range(j):
```

```

        qc.cp(-np.pi / 2 ** (j - m), m, j)
    qc.h(j)
return qc

```

Listing B.3: Implementazione della Quantum Fourier Transform inversa

```

def c_amod15(a, power):
    if a not in [2, 7, 8, 11, 13]:
        raise ValueError("base non supportata per N=15")
    U = QuantumCircuit(4)
    for _ in range(power):
        if a in [2, 13]:
            U.swap(0, 1); U.swap(1, 2); U.swap(2, 3)
        if a in [7, 8]:
            U.swap(2, 3); U.swap(1, 2); U.swap(0, 1)
        if a == 11:
            U.swap(1, 3); U.swap(0, 2)
        if a in [7, 11, 13]:
            U.x(range(4))
    return U.control(annotated=False)

```

Listing B.4: Moltiplicazione modulare N15 corretta: l'operazione $\times a$ viene ripetuta power volte

```

def shor_circuit(N, a, n_count):
    if N != 15:
        return shor_circuit_beauregard(N, a, n_count)
    n_work = 4
    qc = QuantumCircuit(n_count + n_work, n_count)
    qc.h(range(n_count))
    qc.x(n_count + 3) #  $|1\rangle$  nella convenzione del registro
                      textbook
    for j in range(n_count):
        power = 2 ** j
        if power % 4 != 0: #  $U^4 = I$  per  $a=7 \bmod 15$ 
            qc.append(c_amod15(a, power),
                      [j] + list(range(n_count, n_count +
                                         n_work)))
    qc.barrier()
    qc.append(inverse_qft(n_count), range(n_count))

```

```

qc.measure(range(n_count), range(n_count))
return qc

def extract_factors(measured_value, n_count, N, a):
    if measured_value == 0:
        return None, None
    phase = measured_value / 2 ** n_count
    r = Fraction(phase).limit_denominator(N).denominator
    if r % 2:
        return None, None
    for candidate in (gcd(a ** (r // 2) - 1, N),
                      gcd(a ** (r // 2) + 1, N)):
        if 1 < candidate < N:
            return candidate, N // candidate
    return None, None

```

Listing B.5: Circuito N15 e post-processing classico

```

BASIS_GATES = ('rz', 'sx', 'x', 'cx')
GATE_TIME_1Q_NS = 50.0
GATE_TIME_2Q_NS = 300.0

def compose_errors(errors):
    out = None
    for error in errors:
        if error is not None:
            out = error if out is None else out.compose(error)
    return out

def build_noise_model(eps_1q, eps_2q, t1_ns, t2_ns, p_ro=0.02)
:
    # eps_1q ed eps_2q sono i lambda di depolarizing_error.
    thermal_1q = thermal_relaxation_error(
        t1_ns, t2_ns, GATE_TIME_1Q_NS)
    thermal_one_2q = thermal_relaxation_error(
        t1_ns, t2_ns, GATE_TIME_2Q_NS)
    thermal_2q = thermal_one_2q.tensor(thermal_one_2q)
    error_1q = compose_errors([
        depolarizing_error(eps_1q, 1) if eps_1q else None,

```

```

        thermal_1q,
    ])
    error_2q = compose_errors([
        depolarizing_error(eps_2q, 2) if eps_2q else None,
        thermal_2q,
    ])
    nm = NoiseModel()
    nm.add_all_qubit_quantum_error(error_1q, ['sx', 'x'])
    nm.add_all_qubit_quantum_error(error_2q, ['cx'])
    if p_ro:
        nm.add_all_qubit_readout_error(
            ReadoutError([[1-p_ro, p_ro], [p_ro, 1-p_ro]]))
    return nm

```

Listing B.6: Noise model uniforme illustrativo: RZ virtuale, durate 1Q/2Q separate

```

def compile_shor_circuit(N, a, n_count):
    return transpile(
        shor_circuit(N, a, n_count),
        basis_gates=['rz', 'sx', 'x', 'cx'],
        optimization_level=2,
        seed_transpiler=20260819,
    )

def rank_measurements(counts, tie_seed):
    def priority(bitstring):
        payload = f'{int(tie_seed)}:{bitstring}'.encode('ascii')
        return hashlib.sha256(payload).digest()
    return sorted(counts.items(),
        key=lambda item: (-item[1], priority(item[0])))

def method1_from_counts(counts, tie_seed, n_count, N, a):
    bitstring = rank_measurements(counts, tie_seed)[0][0]
    return extract_factors(int(bitstring, 2), n_count, N, a)

```

Listing B.7: Compilazione deterministica, tie-break SHA-256 e baseline TOP-1

```

LABEL_TOP_KS = (1, 16)

```

```

labels = {top_k: [] for top_k in LABEL_TOP_KS}

for sample_index in range(n_samples):
    sample_seed = seed * 1_000_000 + sample_index * 10_000
    counts = simulator.run(
        compiled, noise_model=sample_noise, shots=shots,
        seed_simulator=sample_seed,
    ).result().get_counts()
    ranked = rank_measurements(counts, tie_seed=sample_seed)
    for top_k in LABEL_TOP_KS:
        useful = any(
            extract_factors(int(bits, 2), n_count, N, a)[0] is
                not None
            for bits, _ in ranked[:top_k]
        )
        labels[top_k].append(int(useful))
    X.append(normalized_histogram(counts, n_count, shots))

# Ogni modello salva schema_version=2.0, label_top_k e
manifest.

```

Listing B.8: Dataset condiviso per l'audit delle etichette TOP-1 e TOP-16

```

def evaluate_same_histogram(counts, simulation_seed,
    classifier,
                            n_count, N, a, shots):
    ranked = rank_measurements(counts, simulation_seed)
    valid = [
        extract_factors(int(bits, 2), n_count, N, a)[0] is not
            None
        for bits, _ in ranked[:4]
    ]
    m1_success = valid[0]
    top4_success = any(valid)
    feature = normalized_histogram(counts, n_count, shots)
    m2_success = bool(classifier.predict([feature])[0]) and
        top4_success
    return m1_success, top4_success, m2_success

```

Listing B.9: Decisione M2 sullo stesso istogramma usato da M1 e TOP-4

```

PRESETS = {
    'reference': dict(N=15, a=7, n_count=8,
                      eps_1q=1e-3, eps_2q=1e-2,
                      t1_ns=100_000, t2_ns=80_000, p_ro=0.02),
    'stress':    dict(N=15, a=7, n_count=8,
                      eps_1q=5e-3, eps_2q=5e-2,
                      t1_ns=50_000, t2_ns=30_000, p_ro=0.05),
}

K_REPETITIONS, SHOTS, MAX_ITER = 30, 1024, 50

for rep in range(K_REPETITIONS):
    first = {'m1': None, 'top4': None, 'm2': None}
    for iteration in range(1, MAX_ITER + 1):
        simulation_seed = rep * 1_000_000 + iteration * 10_000
        counts = simulator.run(compiled, shots=SHOTS,
                               seed_simulator=simulation_seed).result().
                               get_counts()
        flags = evaluate_same_histogram(counts,
                                         simulation_seed, ...)
        update_first_success(first, flags, iteration)
    encoded = {name: value if value is not None else MAX_ITER
               + 1
               for name, value in first.items()}

# Inferenza: wilcoxon(paired_a, paired_b,
#                               zero_method='pratt', alternative=...)
# Output: schema_version='2.0', input espliciti e manifest/
hash.

```

Listing B.10: Orchestrazione appaiata delle sole campagne rumorose N15

```

if logical_value == 1:
    qc.x(0)
qc.cx(0, 1); qc.cx(0, 2)
if basis == 'X':
    qc.h(DATA)
for q in DATA:
    qc.id(q) # punto di iniezione del
               rumore

```

```

if basis == 'Z':
    qc.cx(0, 3); qc.cx(1, 3)
    qc.cx(1, 4); qc.cx(2, 4)
else:
    qc.h(3); qc.cx(3, 0); qc.cx(3, 1); qc.h(3)
    qc.h(4); qc.cx(4, 1); qc.cx(4, 2); qc.h(4)
qc.measure(ANC[0], 0); qc.measure(ANC[1], 1)

```

Listing B.11: Codifica ed estrazione di sindrome del codice a ripetizione.

```

def encode_zero_L(qc):
    for seed in SEED:
        qc.h(seed)
    for seed, support in SEED.items():
        for data in support:
            if data != seed:
                qc.cx(seed, data)

def measure_syndrome(qc):
    for k, support in enumerate(GZ):
        for data in support:
            qc.cx(data, ANC_Z[k])
    for k, support in enumerate(GX):
        qc.h(ANC_X[k])
        for data in support:
            qc.cx(ANC_X[k], data)
        qc.h(ANC_X[k])

```

Listing B.12: Preparazione di $|0_L\rangle$ e misura degli stabilizzatori di Steane.

```

def inject(block, pairs, p_ct):
    out = stim.Circuit()
    first = True
    for inst in block:
        if isinstance(inst, stim.CircuitRepeatBlock):
            out += inject(inst.body_copy(), pairs, p_ct) *
                inst.repeat_count
            continue
        out.append(inst)
    if inst.name == 'DEPOLARIZE1':

```

```

        if first:
            out.append('DEPOLARIZE2', pairs, p_ct)
            first = not first
    return out

```

Listing B.13: Iniezione ricorsiva del crosstalk nel circuito del surface code.

```

circuit = stim.Circuit.generated(
    'surface_code:rotated_memory_x', distance=d, rounds=rounds
    ,
    after_clifford_depolarization=p,
    after_reset_flip_probability=p,
    before_measure_flip_probability=p,
)
sampler = circuit.compile_detector_sampler()
events, observables = sampler.sample(shots,
    separate_observables=True)
model = circuit.detector_error_model(decompose_errors=True)
matching = pymatching.Matching.from_detector_error_model(model
    )
predictions = matching.decode_batch(events)
p_L = (predictions != observables).any(axis=1).mean()

```

Listing B.14: Stima di p_L con Stim e PyMatching.

Appendice C

Contesto e motivazione

C.1 Contesto esteso dell'introduzione

Questa sezione raccoglie in forma estesa il materiale sintetizzato nel Capitolo 1: l'origine del paradigma quantistico, i principi fisici e la loro implementazione hardware, la rivoluzione algoritmica degli anni Novanta, il protocollo RSA e la sua vulnerabilità all'algoritmo di Shor, la crittografia post-quantistica, lo stato dell'arte dell'hardware e la roadmap completa dell'elaborato. Le sottosezioni indicano esplicitamente la parte del capitolo da cui l'approfondimento è stato trasferito.

C.1.1 Approfondimento della Sezione 1.1: dal Limite Classico al Paradigma Quantistico

Origine nel corpo: approfondimento della Sezione “Il limite del calcolo classico” del Capitolo 1.

Per oltre cinquant'anni, lo sviluppo dell'informatica è stato guidato da una legge empirica enunciata nel 1965 da Gordon Moore¹: il numero di transistor per chip è cresciuto con andamento approssimativamente esponenziale, trasformando calcolatori grandi quanto una stanza nei dispositivi tascabili contemporanei. Questa traiettoria incontra però limiti fisici: i transistor moderni hanno dimensioni di pochi nanometri e si avvicinano alla scala atomica, alla quale gli effetti quantistici diventano inevitabili.

¹**Gordon Earle Moore** (1929–2023), cofondatore di Intel, osservò che il numero di transistor integrabili su un chip raddoppiava approssimativamente ogni due anni. La Legge di Moore non è una legge fisica, ma una previsione industriale che ha orientato decenni di investimenti tecnologici.

Al limite dell'hardware si affianca una barriera matematica. Per alcune classi di problemi, più potenza di calcolo non basta a rendere pratico un algoritmo la cui richiesta di risorse cresce troppo rapidamente con la dimensione dell'input. Fattorizzazione di grandi interi, ricerca in spazi combinatoriali e simulazione di sistemi molecolari complessi illustrano, con complessità e vantaggi quantistici differenti, questa separazione fra velocità del processore e struttura del problema.

In tale contesto nasce il calcolo quantistico: sovrapposizione, entanglement e interferenza vengono usati come risorse computazionali anziché trattati soltanto come fenomeni da controllare. L'intuizione originaria fu formulata da Richard Feynman², che nel 1982 propose una macchina governata dalle stesse leggi dei sistemi da simulare [20]. David Deutsch formalizzò nel 1985 il primo modello teorico rigoroso di computer quantistico universale [21], aprendo la strada alla successiva ricerca algoritmica.

C.1.2 Dal Bit al Qubit: i Principi Fisici del Calcolo Quantistico

Origine nel corpo: approfondimento fisico richiamato nella Sezione 1.1 del Capitolo 1.

Per comprendere appieno le potenzialità e i limiti del calcolo quantistico, è necessario esaminare le differenze fisiche fondamentali tra l'unità di informazione classica e quella quantistica.

Il Qubit e la Sovrapposizione

Un computer classico elabora l'informazione attraverso **bit** – unità fisiche che possono trovarsi in uno di due stati discreti, convenzionalmente 0 e 1, realizzati come livelli di tensione in circuiti elettronici. L'unità fondamentale del calcolo quantistico è invece il **qubit** (*quantum bit*): un sistema fisico a due livelli che obbedisce alle leggi della meccanica quantistica.

Matematicamente, lo stato di un qubit è descritto da un vettore nello spazio di Hilbert $\mathcal{H} \cong \mathbb{C}^2$:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad \alpha, \beta \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1 \quad (\text{C.1})$$

²**Richard Phillips Feynman** (1918–1988), Premio Nobel per la Fisica nel 1965, argomentò in “*Simulating Physics with Computers*” che la simulazione classica generale di sistemi quantistici comporta un overhead che può diventare esponenziale.

dove $|0\rangle$ e $|1\rangle$ sono i due stati della base computazionale, analoghi ai valori 0 e 1 del bit classico. La differenza fondamentale è che α e β possono essere simultaneamente diversi da zero: il qubit si trova in una **sovrapposizione** dei due stati base, con probabilità $|\alpha|^2$ di essere misurato in $|0\rangle$ e probabilità $|\beta|^2$ di essere misurato in $|1\rangle$.

È cruciale sottolineare che la sovrapposizione non equivale a ignoranza sullo stato del sistema: il qubit non è “segretamente” in uno stato definito che non conosciamo. Si tratta di una proprietà fisica genuina, verificata sperimentalmente con precisione straordinaria, che non ha analogo nella fisica classica.

La Sfera di Bloch

Poiché la norma è unitaria e la fase globale è fisicamente irrilevante, lo stato di un qubit puro può essere parametrizzato da soli due angoli reali:

$$|\psi\rangle = \cos\frac{\theta}{2} |0\rangle + e^{i\phi} \sin\frac{\theta}{2} |1\rangle, \quad \theta \in [0, \pi], \quad \phi \in [0, 2\pi) \quad (\text{C.2})$$

Questo definisce un punto sulla superficie di una sfera unitaria – la **sfera di Bloch** – dove il polo nord corrisponde a $|0\rangle$, il polo sud a $|1\rangle$, e i punti equatoriali a sovrapposizioni equiprobabili con fasi diverse. Questa rappresentazione geometrica è di grande utilità pratica: le operazioni unitarie su un singolo qubit corrispondono a rotazioni, mentre i canali rumorosi trasformano in generale la sfera in un ellissoide traslato. Un canale unital può contrarre verso il centro; l’amplitude damping trasla invece gli stati verso $|0\rangle$.

L’Entanglement

Quando due o più qubit interagiscono tramite porte quantistiche opportune, possono formare stati **entangled**: stati del sistema composto che non possono essere scritti come prodotto tensoriale di stati individuali. Il prototipo è lo stato di Bell:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \quad (\text{C.3})$$

In questo stato, se la misura del primo qubit produce 0, una misura del secondo nella stessa base produce anch’essa 0, e analogamente per 1, indipendentemente dalla distanza fisica; la correlazione non consente però di trasmettere informazione più velocemente della luce. Con opportune scelte delle basi di misura, gli stati entangled violano disuguaglianze di Bell e producono correlazioni incompatibili con

modelli a variabili nascoste locali [59, 60]. Un registro di n qubit descrive ampiezze in uno spazio $\mathbb{C}^{2^n}$, ma questa dimensione non basta da sola a garantire un vantaggio: alcune famiglie entangled, come gli stati stabilizer, restano simulabili classicamente in modo efficiente. Il vantaggio algoritmico nasce dalla struttura con cui preparazione, entanglement e interferenza trasformano tali ampiezze.

L'Interferenza Quantistica

Il terzo pilastro del calcolo quantistico è l'**interferenza**: la capacità di far sommare costruttivamente o annullare distruttivamente le ampiezze di probabilità di percorsi computazionali diversi. Le ampiezze α e β in (C.1) sono numeri complessi, e possono quindi avere segni opposti o sfasamenti arbitrari: quando si sommano, possono amplificarsi o cancellarsi esattamente come onde fisiche.

Un algoritmo quantistico ben progettato sfrutta questo meccanismo per modificare la distribuzione degli esiti: Grover amplifica l'ampiezza degli elementi marcati, mentre in Shor la QFT concentra la probabilità vicino alle frequenze legate al periodo. Non tutti gli esiti prodotti sono quindi “corretti”, e resta necessario un post-processing classico probabilistico.

Implementazione Fisica: i Qubit Superconduttori di IBM

Un qubit può essere realizzato con qualunque sistema fisico a due livelli: la polarizzazione di un fotone, lo spin di un elettrone, i livelli energetici di un atomo intrappolato. I **qubit superconduttori** utilizzati da IBM sono circuiti elettrici macroscopici che operano nel regime quantistico grazie al raffreddamento a temperature criogeniche estreme. In questa tesi costituiscono l'architettura di riferimento per la snapshot *FakeSherbrooke* e per la discussione hardware; gli esperimenti principali restano simulazioni classiche.

Il componente chiave è la **giunzione di Josephson** [61]: una sottile barriera isolante (spessa $\sim 1\text{--}2\text{ nm}$) interposta tra due materiali superconduttori. Questo elemento introduce una non-linearità nell'energia del circuito che rompe l'equidistanza dei livelli energetici, permettendo di indirizzare selettivamente i due stati computazionali $|0\rangle$ e $|1\rangle$. Il risultato è il **transmon** [62] – il tipo di qubit superconduttore adottato da IBM – un oscillatore anarmonico con frequenza tipica 4–6 GHz, controllato da impulsi a microonde.

Per operare nel regime quantistico, questi circuiti devono essere raffreddati a temperature dell'ordine di 15 mK (circa -273.135°C), molto inferiori ai 2.7 K della

radiazione cosmica di fondo, tramite refrigeratori a diluizione. Il raffreddamento abilita il regime superconduttivo e sopprime fortemente le eccitazioni termiche, ma non elimina la dissipazione né le altre sorgenti di decoerenza.

I parametri operativi tipici dei processori IBM (serie Eagle, Heron) sono:

- **Temperatura operativa:** ~ 15 mK
- **Tempi di coerenza:** $T_1, T_2 \sim 50\text{--}500\ \mu\text{s}$
- **Durata gate a singolo qubit:** ~ 50 ns
- **Durata gate a due qubit (ECR/CNOT):** $\sim 300\text{--}500$ ns
- **Fedeltà gate a singolo qubit:** $\sim 99.9\text{--}99.97\%$
- **Fedeltà gate a due qubit:** $\sim 99\text{--}99.7\%$

Il rapporto tra il tempo di coerenza e la durata di un gate contribuisce a determinare quante operazioni utili possano essere eseguite prima che il rumore domini; insieme agli errori di controllo, al crosstalk, al readout e alla connettività, è uno dei principali colli di bottiglia per algoritmi profondi come Shor sui processori attuali.

C.1.3 La Rivoluzione Algoritmica degli Anni Novanta

Negli anni successivi alla proposta di Deutsch, la comunità scientifica si interrogò sulla effettiva esistenza di problemi che un computer quantistico potesse risolvere in modo *probabilmente* più efficiente di qualunque macchina classica. La risposta arrivò in forma dirompente nel corso di pochi anni.

Nel 1994, Daniel Simon³ identificò un problema oracle per il quale un algoritmo quantistico richiede esponenzialmente meno interrogazioni di qualunque algoritmo classico probabilistico. Nello stesso anno Peter Shor pubblicò algoritmi quantistici polinomiali per fattorizzazione e logaritmo discreto; nella formulazione textbook della fattorizzazione, il costo è spesso riassunto come $O(n^3)$ [1]. Le implicazioni furono immediate: RSA dipende dalla difficoltà della fattorizzazione, mentre Diffie–Hellman ed *Elliptic Curve Digital Signature Algorithm* (ECDSA) dipendono da varianti del logaritmo discreto. Una macchina quantistica sufficientemente grande comprometterebbe quindi molte delle primitive a chiave pubblica oggi impiegate, pur senza rendere insicura ogni componente simmetrica dei protocolli.

³L'**algoritmo di Simon**, presentato nel 1994 e pubblicato in forma estesa nel 1997, dimostrò nel modello oracle una separazione esponenziale rispetto agli algoritmi classici probabilistici. Shor riconobbe che la sua tecnica si ispirava direttamente alle idee di Simon.

Due anni dopo, nel **1996**, Lov Grover presentò un algoritmo di tutt'altra natura, ma di eguale importanza [49]. Mentre Shor aveva risolto un problema specifico della teoria dei numeri, Grover affrontò una sfida universale: la **ricerca non strutturata**. Dato un insieme di N elementi senza alcuna struttura o ordinamento, trovare quello che soddisfa una certa proprietà richiede, nel caso classico, un numero di interrogazioni proporzionale a N . Grover dimostrò che un algoritmo quantistico può fare lo stesso lavoro in sole $O(\sqrt{N})$ interrogazioni – uno *speedup quadratico* – e che questo risultato è *ottimale*: nessun algoritmo quantistico può fare di meglio [51]. Sebbene meno spettacolare dello speedup esponenziale di Shor, il vantaggio di Grover è universale, applicabile a qualunque problema di ricerca, e porta con sé implicazioni profonde per la crittografia simmetrica, l'ottimizzazione combinatoria e la ricerca in grandi basi di dati.

Questi due algoritmi – Shor e Grover – rappresentano ancora oggi i pilastri dell'informatica algoritmica quantistica: il primo come dimostrazione del potere *esponenziale* del calcolo quantistico su problemi strutturati, il secondo come strumento *universale* di accelerazione quadratica applicabile al problema computazionale più fondamentale.

C.1.4 RSA e le Implicazioni Crittografiche dell'Algoritmo di Shor

Per comprendere appieno la portata dell'algoritmo di Shor è necessario esaminare la struttura matematica del sistema crittografico che esso minaccia. Il sistema **RSA**, proposto nel 1977 da Rivest, Shamir e Adleman e pubblicato nel 1978 [3], è uno degli schemi storicamente centrali della crittografia a chiave pubblica. Continua a essere impiegato in certificati e firme, ma i protocolli moderni possono adottare anche primitive diverse, in particolare su curve ellittiche.

Struttura del Protocollo RSA

La sicurezza di RSA si basa sulla **asimmetria computazionale** tra due operazioni inverse: moltiplicare due numeri primi grandi p e q per ottenere $N = p \cdot q$ è un'operazione elementare (complessità $O(n^2)$ per numeri a n bit), mentre fattorizzare N per recuperare p e q è computazionalmente intrattabile con gli algoritmi classici noti. La generazione delle chiavi segue i passi seguenti:

1. Scegliere due primi grandi p e q (tipicamente a 1024–2048 bit ciascuno) e calcolare $N = p \cdot q$;
2. Calcolare la funzione di Eulero $\phi(N) = (p-1)(q-1)$;
3. Scegliere e coprimo con $\phi(N)$ (convenzionalmente $e = 65537 = 2^{16} + 1$, primo di Fermat);
4. Calcolare $d \equiv e^{-1} \pmod{\phi(N)}$ tramite l'algoritmo di Euclide esteso.

La **chiave pubblica** è la coppia (N, e) ; la **chiave privata** è d . Cifrare un messaggio $m < N$ produce il crittogramma $c = m^e \bmod N$; decifrarlo richiede $m = c^d \bmod N$. La correttezza si fonda sul **teorema di Eulero** — la generalizzazione del piccolo teorema di Fermat a moduli composti, secondo cui $m^{\phi(N)} \equiv 1 \pmod{N}$ per m coprimo con N : essendo $ed \equiv 1 \pmod{\phi(N)}$, si ha $ed = 1 + k\phi(N)$ e quindi $(m^e)^d = m^{1+k\phi(N)} \equiv m \pmod{N}$. Tutta la sicurezza risiede nell'impossibilità di calcolare d da (N, e) senza conoscere la fattorizzazione di N .

Perché l'Algoritmo di Shor Rompe RSA

Il miglior algoritmo classico per la fattorizzazione di un intero a n bit è il *General Number Field Sieve* (GNFS) [2], con complessità sub-esponenziale:

$$T_{\text{GNFS}}(N) = O(\exp[c \cdot (\log N)^{1/3} (\log \log N)^{2/3}]) \quad (\text{C.4})$$

Per N a 2048 bit, questa espressione implica circa 2^{112} operazioni classiche — un numero astronomico, ben oltre la capacità di qualunque infrastruttura computazionale esistente o prevedibile in ambito classico. I migliori risultati pratici al 2020 hanno raggiunto la fattorizzazione di RSA-250 (829 bit) [22], impiegando circa 2700 anni-CPU di calcolo distribuito.

L'algoritmo di Shor riduce questa complessità a $O(n^3)$ — un salto da sub-esponenziale a **polinomiale** — rendendo la fattorizzazione di chiavi RSA-2048 un problema risolvibile in ore su un computer quantistico fault-tolerant con qualche migliaio di qubit logici. La struttura dell'algoritmo, descritta in dettaglio nel Capitolo 3, sfrutta la Quantum Fourier Transform per trovare il periodo di una funzione modulare, problema che si riduce efficientemente alla fattorizzazione.

Implicazioni Pratiche per la Sicurezza Informatica

Le conseguenze sono sistemiche e riguardano più in generale la crittografia a chiave pubblica vulnerabile a Shor: RSA, Diffie–Hellman e le primitive su curve ellittiche. Questi algoritmi compaiono, in combinazioni diverse, in:

- **HTTPS e TLS:** per autenticazione e scambio delle chiavi di sessione, a seconda della versione e della configurazione;
- **Certificati X.509:** nell’infrastruttura a chiave pubblica usata per autenticare servizi e soggetti;
- **SSH:** il protocollo standard per l’accesso remoto sicuro ai server;
- **Firme digitali:** documenti legali, transazioni finanziarie, aggiornamenti software firmati.

Un computer quantistico crittograficamente rilevante renderebbe vulnerabili anche comunicazioni registrate in precedenza quando la riservatezza della chiave di sessione dipende da una primitiva a chiave pubblica attaccabile da Shor. È il rischio *“harvest now, decrypt later”*: acquisire oggi dati cifrati ancora sensibili e conservarli per una futura decifratura. Il NIST sottolinea però che non esiste una data affidabile per la comparsa di una macchina di questo tipo; le stime spaziano da pochi anni a decenni. Le scadenze 2030–2035 presenti nelle roadmap istituzionali sono obiettivi di migrazione e dismissione degli algoritmi vulnerabili, non previsioni della data in cui RSA-2048 verrà violato [63, 64].

C.1.5 Crittografia Post-Quantistica: la Risposta dell’Industria

La consapevolezza della minaccia quantistica ha spinto il NIST a lanciare nel 2016 un processo di standardizzazione internazionale per la **crittografia post-quantistica** (*Post-Quantum Cryptography* (PQC)) – algoritmi crittografici resistenti agli attacchi di computer quantistici, progettati per essere eseguiti su hardware classico convenzionale e quindi immediatamente distribuibili.

Gli Standard NIST 2024

Dopo otto anni di valutazione pubblica, crittoanalisi internazionale e tre successivi turni di selezione tra 69 candidati iniziali, nell’agosto 2024 il NIST ha pubblicato i primi tre standard definitivi [65]:

- **ML-KEM** (FIPS 203, ex CRYSTALS-Kyber): meccanismo di incapsulamento di chiavi basato sul problema *Module Learning With Errors* (MLWE) su reticoli. È il sostituto designato per lo scambio di chiavi RSA e Diffie-Hellman nei protocolli TLS;
- **ML-DSA** (FIPS 204, ex CRYSTALS-Dilithium): schema di firma digitale basato sullo stesso fondamento matematico di ML-KEM. Sostituisce RSA e ECDSA per le firme digitali;
- **SLH-DSA** (FIPS 205, ex SPHINCS+): schema di firma basato esclusivamente su funzioni hash, senza assunzioni algebriche. Più conservativo per sicurezza, adottato come backup contro potenziali attacchi futuri ai reticoli.

La sicurezza di questi schemi si fonda su problemi matematici – trovare vettori corti in reticoli ad alta dimensione, o invertire funzioni hash – per i quali non sono noti algoritmi quantistici con speedup esponenziale. In particolare, l’algoritmo di Grover fornisce al più uno speedup quadratico sui problemi basati su hash, motivando l’aumento dei parametri di sicurezza (es. passaggio da 128 a 256 bit di sicurezza classica equivalente).

La Sfida della Migrazione

La transizione verso la crittografia post-quantistica è un’operazione di ingegneria su scala globale, paragonabile per complessità alla migrazione da IPv4 a IPv6, ma con urgenza maggiore. Le principali sfide includono:

- **Eterogeneità dei sistemi:** miliardi di dispositivi embedded, sensori industriali e infrastrutture legacy con cicli di vita decennali che non riceveranno aggiornamenti software;
- **Overhead di banda:** il materiale crittografico post-quantistico è sensibilmente più voluminoso. Al livello di sicurezza più basso (ML-KEM-512), la chiave di incapsulamento occupa 800 byte e il crittogramma 768 byte, contro i circa 256 byte di una chiave pubblica RSA-2048: un fattore di crescita che pesa sui protocolli a bassa latenza e sui dispositivi con banda limitata;
- **Migrazione a doppio binario:** nella fase transitoria, molte implementazioni adottano schemi *ibridi* (classico + post-quantistico in parallelo) per mantenere compatibilità con sistemi non aggiornati.

Aziende come Google (Chrome 131), Apple (iMessage PQ3) [66] e Cloudflare hanno già avviato implementazioni ibride nei loro protocolli di produzione. La coesistenza di questa transizione con la ricerca attiva su algoritmi quantistici come Shor definisce il contesto strategico in cui si inserisce il presente lavoro: comprendere i limiti reali di Shor su hardware NISQ è rilevante non solo teoricamente, ma anche per calibrare con maggiore precisione le tempistiche e le priorità della transizione crittografica globale.

C.1.6 Lo Stato dell'Arte: dall'Era NISQ ai Computer Fault-Tolerant

Nonostante la solidità teorica di questi risultati, la realizzazione pratica di computer quantistici di scala sufficiente ad eseguirli su istanze realmente significative incontra ostacoli formidabili. I sistemi quantistici fisici sono intrinsecamente fragili: qualunque interazione non controllata con l'ambiente esterno – vibrazioni termiche, campi elettromagnetici, imperfezioni nei materiali – provoca fenomeni di **decoerenza** – ossia la perdita progressiva della coerenza quantistica causata dall'entanglement indesiderato tra il qubit e i gradi di libertà ambientali, che sopprime le interferenze quantistiche riducendo il sistema a una miscela statistica classica – degradando lo stato quantistico in modo irreversibile prima che il calcolo possa completarsi. A differenza di un bit classico, che può essere protetto dalla ridondanza, un qubit non può essere copiato (teorema del no-cloning [23]) né letto senza distruggerlo: questo rende la correzione degli errori quantistici un problema radicalmente diverso e molto più complesso di quella classica.

Il panorama tecnologico attuale è dominato da un insieme variegato di piattaforme hardware in competizione, ciascuna con vantaggi e svantaggi distinti:

- **Qubit superconduttori** (IBM, Google, Rigetti): alta velocità dei gate ($\sim 50\text{--}500\text{ ns}$), scalabilità su chip in silicio, ma tempi di coerenza limitati a $\sim 100\text{--}500\text{ }\mu\text{s}$ e necessità di raffreddamento a 15 mK ;
- **Ioni intrappolati** (IonQ, Quantinuum): tempi di coerenza eccellenti ($> 1\text{ s}$), fedeltà dei gate superiore al 99.9% anche su due qubit, ma gate lenti ($\sim 10\text{--}100\text{ }\mu\text{s}$) e scalabilità più complessa;
- **Fotoni** (PsiQuantum, Xanadu): operano a temperatura ambiente, naturalmente adatti alla comunicazione quantistica, ma la generazione deterministica di fotoni entangled rimane una sfida aperta;
- **Spin in silicio** (Intel, UNSW): compatibili con i processi di fabbricazione CMOS esistenti, potenzialmente scalabili a milioni di qubit, ma con tecnologia di controllo ancora in fase di sviluppo.

Un momento simbolico di questa corsa fu raggiunto nel **2019**, quando il team di Google rivendicò il raggiungimento della cosiddetta *quantum supremacy*⁴ con il processore Sycamore a 53 qubit [67]: un calcolo specifico fu completato in 200 secondi, mentre le stime attribuivano al più potente supercomputer classico un tempo di circa 10.000 anni per lo stesso compito. Nel 2023, IBM ha presentato il processore *Condor* a 1.121 qubit e ha delineato una roadmap [48] verso sistemi con architettura fault-tolerant entro la fine del decennio, con il target *Quantum Starling* previsto per il 2029.

Tuttavia, la distanza tra i dispositivi attuali e un computer quantistico *fault-tolerant* — in grado di eseguire Shor su istanze RSA reali — è ancora enorme. I processori odierni sono comunemente descritti nel regime NISQ, termine introdotto da John Preskill nel 2018 [4]: il numero di qubit fisici, da solo, non li rende fault-tolerant, perché errori, connettività e assenza di correzione scalabile limitano la profondità affidabile. In questo scenario, la ricerca si concentra su due direzioni parallele e complementari:

- **QEC:** tecniche per codificare qubit logici in qubit fisici ridondanti, in modo da rilevare e correggere gli errori durante il calcolo, a costo di un overhead fisico significativo (centinaia o migliaia di qubit fisici per qubit logico);
- **Noise mitigation:** tecniche che, senza richiedere overhead hardware, riducono l'effetto del rumore a livello di post-processing classico, estendendo la portata dei circuiti eseguibili su hardware NISQ.

C.1.7 Approfondimento della Sezione 1.2: Filo delle Due Fasi e Risultati Guida

Origine nel corpo: dettaglio quantitativo trasferito dalla Sezione “Motivazione e contributo del presente lavoro” del Capitolo 1. I valori anticipano soltanto il filo argomentativo; protocolli, incertezza e discussione completa restano nel Capitolo 5.

La prima fase confronta tre configurazioni sugli stessi istogrammi rumorosi di Shor:

⁴Il termine *quantum supremacy*, coniato da John Preskill nel 2012, indica il punto in cui un computer quantistico esegue un calcolo specifico in un tempo che nessun supercomputer classico potrebbe eguagliare in modo pratico. La rivendicazione di Google con il processore Sycamore è rimasta controversa: IBM sostenne che il calcolo avrebbe potuto essere eseguito classicamente in 2.5 giorni anziché 10.000 anni, con un algoritmo migliorato. Questo dibattito evidenzia la difficoltà di definire e misurare il vantaggio quantistico.

- **Metodo 1** (baseline classica, TOP-1): usa come candidato la misura più frequente e richiede in media $\bar{M}_1 = 1.37$ esecuzioni nel preset di riferimento e 1.50 in quello di stress;
- **Metodo 2** (classificatore ML TOP-1 seguito da TOP-4): una SVM, selezionata su validation e valutata su test indipendente, filtra gli istogrammi prima della ricerca sui quattro candidati più frequenti; ottiene $\bar{M}_2 = 1.50$ e 1.37, senza un miglioramento significativo rispetto a M1;
- **Ablazione** M_{TOP4} : applica TOP-4 senza classificatore, ottiene $\bar{M} = 1.00$ in entrambi i preset ed è significativamente migliore della variante filtrata ($p_{\text{Holm}} = 0.0033$ e 0.0096). L’audit delle etichette mostra inoltre che TOP-16 produce 2000/2000 positivi: quel problema è a classe unica, mentre l’etichetta TOP-1 descrive soltanto la riuscita del candidato dominante (Sezione D.2).

Il confronto delimita il post-processing: TOP-4 recupera informazione ancora presente nell’istogramma, ma una regola a valle non può ricostruire struttura già cancellata o spostata fuori dall’insieme osservato. Da qui nasce il passaggio alla seconda fase, che protegge l’informazione durante l’esecuzione mediante codici a ripetizione, Steane e surface code. Il simbolo p_L viene usato nei memory experiment per il tasso di fallimento logico e, separatamente, nell’analisi di sensibilità di Shor per un parametro fenomenologico applicato ai gate compilati inclusi nel proxy; i due oggetti non vengono identificati direttamente.

Nella decodifica della sindrome, una rete che sostituisce il matching lo pareggia soltanto a parità di modello e richiede grandi volumi di dati. Un decoder ibrido, nel quale la rete apprende quando ribaltare la decisione di MWPM, riduce invece l’errore logico fino al 21% nel modello simulato con errori correlati non rappresentati dal grafo analitico; quando il rumore coincide con il modello previsto, il metodo ricade sul riferimento entro la risoluzione dell’esperimento. Questo contrasto con la prima fase fonda il criterio unificante della tesi: un modello appreso apporta valore quando osserva informazione strutturale pertinente che il metodo analitico non utilizza.

C.1.8 Approfondimento della Sezione 1.3: Struttura Estesa della Tesi

Origine nel corpo: roadmap analitica trasferita dalla Sezione “Struttura della tesi” del Capitolo 1. La versione breve nel corpo conserva la sequenza dell’argomento; quella seguente rende referenziabile il contenuto di ciascuna parte.

Capitolo 1 — Introduzione. Presenta il limite del calcolo classico, la nascita del paradigma quantistico e la motivazione crittografica. Introduce il contributo e il filo delle due fasi: verifica del post-processing basato su classificazione ML e TOP-K, quindi protezione del calcolo mediante QEC e decodifica della sindrome.

Capitolo 2 — Problema di ricerca, obiettivi e criteri di valutazione. Formula il problema, definisce il divario fra esecuzione ideale e rumorosa e fissa il contratto sperimentale. Comprende obiettivi, sei domande di ricerca, ipotesi, contributi, perimetro e sintesi delle metriche; requisiti, preset e protocolli completi sono sviluppati nella Sezione C.2.

Capitolo 3 — Fondamenti teorici e metodologia sperimentale. Riunisce i concetti indispensabili — qubit, porte, circuiti ed entanglement come risorsa fragile — e l’analisi di Shor: riduzione al *period finding*, Quantum Phase Estimation, QFT, complessità $O((\log N)^3)$ e implicazioni crittografiche. La parte sul rumore presenta fonti fisiche e canali fondamentali, incluso il proxy illustrativo di nessun evento pari a circa 3,4% per una lunghezza di input $n = 15$ bit, che non coincide con la probabilità di successo di Shor. Il confronto fra quattro macro-strategie anti-rumore motiva l’approccio adottato. Nello stesso capitolo sono definiti disegno delle campagne, pipeline a tre stadi, mitigazione, TOP-1, filtro ML, ablazione TOP-K e protocollo QEC, mantenendo distinta la metrica dei memory experiment dal proxy per gate dell’analisi di sensibilità.

Capitolo 4 — Architettura e implementazione. Traduce il disegno in una piattaforma riproducibile. Descrive Qiskit, Aer su CPU, la valutazione non riuscita del backend GPU, il noise model parametrizzato, la transpilazione e WSL2; presenta poi QFT inversa, circuito di Shor, pipeline M1/TOP-K/M2 e classificatore. Completa il quadro l’implementazione del ciclo QEC, dal codice a ripetizione a Steane e al surface code, con MWPM e decoder appreso e ibrido.

Capitolo 5 — Risultati e discussione sperimentale. Nella prima fase stabilisce la baseline e separa mediante ablazione il contributo di TOP-4 da quello del classificatore. Nella seconda verifica ripetizione e Steane, quindi misura errore logico e soglia del surface code con Stim e PyMatching; confronta sugli stessi campioni MWPM, decoder appreso e ibrido. Include inoltre l’analisi di sensibilità di Shor a un proxy Pauli uniforme per gate compilato, senza tradurlo in requisiti hardware in assenza di

compilazione fault-tolerant, e la valutazione sperimentale della QFT approssimata. La discussione finale mette in relazione risultati NISQ, QEC, sensibilità e AQFT distinguendone unità e limiti.

Capitolo 6 — Conclusioni e sviluppi futuri. Ricompatta le due fasi, verifica le ipotesi, risponde alle sei domande di ricerca, dichiara contributi e limiti e isola il criterio che unifica post-processing e decodifica della sindrome. Le attività non eseguite restano esplicitamente separate dai risultati.

Appendice A — Trattazione teorica estesa. Sviluppa i postulati della meccanica quantistica e l'insieme completo delle porte (Sezione A.1), l'anatomia di una QPU e il confronto fra tecnologie di qubit (Sezione A.2), gli approfondimenti su Shor e sullo stato dell'arte della fattorizzazione (Sezione A.3) e le derivazioni dei canali di rumore in forma di Kraus (Sezione A.4).

Appendice B — Strumenti, ambiente e codice. Motiva le scelte tecnologiche, confronta framework e metodi di simulazione Aer, documenta GPU e WSL e riporta i componenti implementati (Sezioni B.1 e B.2). Raccoglie inoltre configurazioni, parametri, seed e istruzioni di riproduzione.

Appendice C — Contesto e motivazione. Contiene gli approfondimenti richiamati nei primi due capitoli: principi fisici, storia algoritmica, RSA, PQC, hardware, roadmap estesa, motivazione crittografica, obiettivi analitici, requisiti, use case, input, output e protocollo completo delle metriche (Sezioni C.1 e C.2).

Appendice D — Fase 1 nel dettaglio. Riporta gli otto sweep su M_{TOP4} , il confronto previsione-esito per parametro e il protocollo di confronto con la Zero-Noise Extrapolation (Sezione D.1); documenta inoltre architetture di classificazione, addestramento, metriche e audit delle etichette (Sezione D.2).

Appendice E — Fase 2 nel dettaglio. Conserva i richiami e i protocolli completi dei laboratori su ripetizione e Steane (Sezione E.1), il funzionamento del surface code, la griglia sperimentale e i controlli non-Pauli (Sezione E.2), quindi architettura e addestramento del decoder con i confronti rispetto a MWPM e BP+OSD (Sezione E.3). Raccoglie inoltre i dettagli e le verifiche dell'analisi di sensibilità di Shor (Sezione E.4) e verifica il criterio conclusivo su un caso esterno al perimetro

principale: la mappatura circuitale studiata su una snapshot di calibrazione offline e riproducibile (Sezione E.5).

C.2 Dettaglio degli obiettivi e del piano sperimentale

Questa sezione raccoglie il materiale di dettaglio del Capitolo 2: motivazione crittografica, formulazione analitica degli obiettivi, contributi, requisiti, use case, input/output, metriche e test. Ogni blocco dichiara la propria origine nel corpo e completa la sintesi senza modificarne il perimetro.

C.2.1 Motivazione: la Vulnerabilità della Crittografia Attuale

La quasi totalità delle comunicazioni digitali sicure — dalle transazioni bancarie alle connessioni *HyperText Transfer Protocol Secure* (HTTPS), dai protocolli *Secure Shell* (SSH) ai certificati digitali — è protetta da sistemi crittografici la cui sicurezza si fonda su un presupposto computazionale: la difficoltà di fattorizzare il prodotto di due grandi numeri primi. Il sistema RSA, ad esempio, costruisce la propria chiave pubblica come $N = p \cdot q$, dove p e q sono primi di centinaia o migliaia di bit. Finché non esiste un algoritmo classico efficiente per recuperare p e q a partire da N , la chiave privata rimane al sicuro.

Per comprendere perché i sistemi crittografici attuali siano strutturalmente vulnerabili all'algoritmo di Shor, è necessario mettere a confronto le complessità computazionali in gioco. Il miglior algoritmo classico per la fattorizzazione, il GNFS, richiede un tempo:

$$T_{\text{classico}}(N) = O\left(\exp\left((c + o(1))(\log N)^{1/3}(\log \log N)^{2/3}\right)\right) \quad (\text{C.5})$$

che, pur crescendo più lentamente di un esponenziale puro, rimane del tutto impraticabile per i valori di N usati in crittografia. A titolo di esempio, fattorizzare un numero RSA a 2048 bit con algoritmi classici richiederebbe un tempo stimato superiore all'età dell'universo, anche impiegando l'intera potenza di calcolo disponibile sul pianeta.

Nel 1994, Peter Shor dimostrò che un computer quantistico è in grado di eseguire la stessa operazione in tempo **polinomiale** [1]:

$$T_{\text{Shor}}(N) = O((\log N)^3) \quad (\text{C.6})$$

Lo scarto tra le due espressioni non è una differenza di grado, ma una differenza di *natura*: il tempo classico cresce in modo super-polinomiale con la lunghezza della chiave, mentre il tempo quantistico cresce polinomialmente. **Aumentare la lunghezza della chiave RSA non è quindi una contromisura strutturale** contro Shor: raddoppiare n porta il costo asintotico textbook da $O(n^3)$ a $8O(n^3)$. Sul piano concreto questo fattore può richiedere molte più risorse fault-tolerant e ritardare un attacco, ma non ripristina un problema computazionalmente difficile per un avversario quantistico scalabile. La stessa vulnerabilità asintotica si estende ai sistemi basati sul logaritmo discreto (Diffie–Hellman, ECDSA).

La minaccia ha già una conseguenza operativa: nel modello *harvest now, decrypt later*, dati cifrati oggi e destinati a restare sensibili vengono conservati in vista di una futura macchina crittograficamente rilevante. Non esiste però una previsione affidabile della sua data di arrivo. Le roadmap dei produttori descrivono obiettivi ingegneristici e il NIST sottolinea che le stime spaziano da pochi anni a decenni; proprio l’incertezza, unita ai tempi lunghi di migrazione, motiva l’adozione anticipata della PQC [63, 48].

Gli ostacoli pratici sono la scala, la profondità fault-tolerant e il **rumore**. I processori attuali operano in regime NISQ: dispongono di molti qubit fisici, ma decoerenza, errori di gate, readout, connettività e overhead di correzione impediscono di sostenere il numero di operazioni logiche richiesto da Shor su chiavi crittografiche. Comprendere questi limiti e stabilire quali strategie li riducano — e in quale punto della pipeline i modelli di machine learning contribuiscano davvero — è la ragione d’essere del presente lavoro.

C.2.2 Approfondimento della Sezione 2.2: Obiettivi Specifici

Origine nel corpo: formulazione estesa degli obiettivi sintetizzati nella Sezione “Obiettivi specifici” del Capitolo 2.

1. **Fondamenti del calcolo quantistico.** Fornire una trattazione rigorosa del qubit, della sovrapposizione, dell’entanglement, delle porte logiche e dell’interferenza, base necessaria per comprendere l’algoritmo discusso nella Sezione 3.
2. **Analisi teorica dell’algoritmo di Shor.** Descrivere la riduzione della fattorizzazione al *period finding*, la QFT e la phase estimation; analizzare la complessità polinomiale e le conseguenze per RSA e la sicurezza post-quantum.

3. **Implementazione su Qiskit e analisi sperimentale.** Costruire la QFT, l'esponentiazione modulare e il circuito completo, verificarne il comportamento ideale e analizzarne le prestazioni sulle istanze dichiarate nel perimetro.
4. **Rumore e strategie di protezione.** Studiare decoerenza, errori di gate e readout, con attenzione a QFT e phase estimation; distinguere e valutare QEC, ZNE, *Probabilistic Error Cancellation* (PEC) e mitigazione del readout, senza attribuire a una tecnica risultati non prodotti dagli esperimenti.
5. **Ottimizzazione mediante modelli appresi.** Stabilire in quale punto della pipeline un modello migliori effettivamente la resa. A valle dell'esecuzione, il classificatore sull'istogramma è sottoposto ad ablazione contro TOP-K; durante la correzione, il decoder addestrato sulle sindromi è confrontato con il riferimento analitico a parità di campioni. Il criterio è quantitativo — iterazioni nella prima fase, logical error rate nella seconda — e mira a delimitare sia il regime di utilità sia quello di non utilità.
6. **Correzione degli errori e integrazione con Shor.** Costruire in ordine di complessità codice a ripetizione, Steane $[[7, 1, 3]]$ e surface code; misurare la corrispondenza sindrome-errore, la soppressione di p_L in funzione di p e d e il comportamento di soglia. Studiare poi Shor sotto un residuo Pauli fenomenologico per gate, senza identificarlo direttamente con le metriche aggregate dei benchmark QEC.

Questi obiettivi producono tre classi di output: una ricostruzione teorica verificabile, una piattaforma sperimentale riproducibile e confronti quantitativi contro baseline esplicite. Le domande DR1–DR6 e le ipotesi restano nel corpo, dove costituiscono l'indice logico dei risultati.

C.2.3 Approfondimento della Sezione 2.4: Contributi e Confini Interpretativi

Origine nel corpo: dettaglio dei quattro contributi riassunti nella Sezione 2.4.

Progettazione e verifica di TOP-K. Rispetto alle tecniche strutturali applicate prima della misura, la prima fase valuta una regola di post-processing direttamente sull'istogramma rumoroso. Il protocollo appaiato separa il contributo della ricerca multi-candidato da quello del classificatore senza confonderlo con una diversa estrazione Monte Carlo.

Quantificazione del fattore ρ . I due preset illustrativi usano lo stesso circuito $N = 15$; per ogni coppia replica–iterazione, M1, TOP-4 e M2 ricevono lo stesso istogramma. Le differenze sono analizzate con Wilcoxon appaiato–Pratt. Le istanze Beauregard più grandi forniscono controlli ideali di correttezza e costo, non osservazioni delle campagne rumorose. I risultati quantitativi provengono soltanto dagli artefatti schema 2 generati dopo la correzione del moltiplicatore $\times 7 \bmod 15$.

Caratterizzazione a livelli. Ripetizione, Steane e surface code misurano il logical error rate nei rispettivi protocolli; M8 misura separatamente la sensibilità di Shor a un proxy Pauli uniforme per gate. In assenza di una compilazione fault-tolerant, la tesi non sovrappone sulla curva M8 tassi per ciclo o per memoria e non confronta codici sotto unità non commensurabili.

Regime di utilità del machine learning. A valle di Shor, M1–TOP-4–M2 separa il classificatore dalla ricerca multi-candidato. Nel surface code, modello appreso e decoder analitico ricevono gli stessi detection events. Questa simmetria sperimentale permette di esprimere il criterio di applicabilità in termini verificabili anziché attribuire il risultato alla sola scelta dell’algoritmo di apprendimento.

C.2.4 Requisiti Funzionali

Definiti gli obiettivi e il contributo originale, è possibile formalizzare cosa il sistema sperimentale deve concretamente fare. I requisiti seguenti traducono le ipotesi di ricerca in vincoli tecnici verificabili, e costituiscono il riferimento rispetto al quale l’implementazione descritta nel Capitolo 4 viene valutata nel Capitolo 5.

Il sistema sperimentale deve soddisfare i seguenti requisiti funzionali:

1. **Esecuzione configurabile entro il perimetro validato.** La campagna rumorosa e la demo pubblica usano l’istanza canonica $N = 15$, $a = 7$ e un numero configurabile di shot entro limiti dichiarati. Le implementazioni Beauregard per $N = 21$ e $N = 35$ sono sottoposte a test ideali e audit di risorse, ma non sono esposte come campagne rumorose equivalenti.
2. **Modello di rumore parametrizzabile.** Il sistema permette di configurare separatamente i parametri Aer λ_{1q} e λ_{2q} , i tempi T_1/T_2 , il readout asimmetrico della demo e l’eventuale sovrarotazione coerente. I preset della campagna restano uniformi e illustrativi; non riproducono una calibrazione hardware nominata.

3. **Raccolta e archiviazione degli output.** Il sistema registra la distribuzione di misura del registro di controllo per ogni esecuzione e la archivia in formato strutturato. La riproducibilità usa un seed base e calendari derivati separati per replica, iterazione, simulatore e risoluzione dei pareggi.
4. **Analisi statistica del Metodo 1.** Il sistema implementa la pipeline del Metodo 1: selezione della misura più frequente dell'istogramma (strategia TOP-1, senza sogliatura preliminare), stima del periodo r tramite frazioni continue, verifica della correttezza dei fattori estratti. Misura e registra il numero di iterazioni necessarie per raggiungere il primo risultato corretto.
5. **Training, validazione e inferenza del Metodo 2.** Il sistema genera il dataset dal simulatore rumoroso, addestra il classificatore con divisione training/validation/test e ne valuta le metriche. Il modello corrente apprende l'etichetta TOP-1: decide se il candidato dominante conduce ai fattori; se accetta l'istogramma, M2 applica poi la ricerca TOP-4. L'audit TOP-16 è a classe singola e non produce un classificatore.
6. **Confronto sistematico sui use case previsti.** M1, TOP-4 e M2 vengono eseguiti sugli stessi istogrammi nei due preset $N = 15$, producendo metriche appaiate secondo la Sezione 2.6. Per $N = 21$ e $N = 35$ si riportano correttezza ideale e costo compilato; non si afferma che la probabilità rumorosa sia zero, perché la campagna rumorosa non viene eseguita.
7. **Riproducibilità.** Tutti gli esperimenti sono riproducibili: seed fisso, parametri esplicitati, codice disponibile nel repository del progetto. Questo requisito è condizione necessaria per la validazione accademica dei risultati [53].

C.2.5 Approfondimento della Sezione 2.5: Campagne e Validazioni

Origine nel corpo: specifica completa del perimetro sintetizzato nella Sezione 2.5.

Il disegno separa le campagne rumorose dalle validazioni ideali:

- **Campagne rumorose N15:** stesso $N = 15$, stessa base $a = 7$, stesso circuito e due preset, *riferimento* e *stress*. Sono scenari uniformi illustrativi, non calibrazioni di una QPU reale.

- **Validazioni ideali N21/N35:** i moltiplicatori modulari di Beauregard sono verificati senza rumore mediante truth table, ripristino dei registri ausiliari e prove QPE ideali. Il loro costo esclude una campagna Aer rumorosa equivalente; l'assenza di tale campagna non viene interpretata come probabilità rumorosa nulla.

Preset delle campagne rumorose

La Tabella C.1 riporta le sole configurazioni rumorose. I simboli ϵ_{1q} e ϵ_{2q} indicano esattamente il parametro λ passato da Aer a `depolarizing_error`; non sono probabilità Pauli aggregate né dati di calibrazione hardware. Il modello è uniforme su tutti i qubit.

Parametro	Riferimento	Stress
N	15	15
a	7	7
Periodo r atteso	4	4
n_{count} (qubit)	8	8
M_{shots}	1024	1024
$\epsilon_{1q} = \lambda_{1q}$	1×10^{-3}	5×10^{-3}
$\epsilon_{2q} = \lambda_{2q}$	1×10^{-2}	5×10^{-2}
T_1 (ns)	100 000	50 000
T_2 (ns)	80 000	30 000
p_{ro}	2×10^{-2}	5×10^{-2}

Tabella C.1: Preset uniformi illustrativi per le campagne rumorose N15. Non rappresentano la calibrazione di una QPU reale.

Razionale per la scelta delle istanze

N15 — riferimento e stress. La base $a = 7$ ha periodo $r = 4$ e consente di estrarre i fattori 3 e 5. Il moltiplicatore controllato implementa realmente $y \mapsto 7y \bmod 15$: l'operazione elementare viene ripetuta `power` volte, anziché selezionare erroneamente un ramo mediante `power % 4`. Il circuito compilato con Qiskit 2.5, `optimization_level=2`, seed di transpiler fissato e base `rz/sx/x/cx` ha profondità 412 e contiene 224 porte `cx`; questi dati e l'hash SHA-256 della netlist sono registrati nel manifest. Le rotazioni `rz` sono cambi di frame virtuali, con durata ed errore nulli; `sx` e `x` hanno durata 50 ns e `cx` 300 ns. Depolarizzazione e rilassamento termico sono composti soltanto su `sx/x` e `cx`; il readout è una matrice di confusione simmetrica.

N21 e N35 — validazione ideale. Per $N = 21$, $a = 2$, e $N = 35$, $a = 6$, si usa l'aritmetica reversibile di Beauregard. La verifica controlla l'azione sui vettori di base validi, il ritorno a zero degli ancilla e la QPE ideale. Come audit di risorse, il circuito completo $N = 21$, $n_{\text{count}} = 8$, compilato globalmente nella stessa base a livello 2, conta 21 036 `cx` e profondità 23 081. Al valore illustrativo $\lambda = 10^{-3}$, il proxy circuitale registrato è 2.6987×10^{-9} : indica accumulo circuitale, non la probabilità di successo di Shor. L'esclusione dal rumore è una scelta di validità e costo, non un risultato sulla robustezza delle due istanze.

C.2.6 Parametri di Input e Output

Parametri di Input

La Tabella C.2 elenca i parametri di configurazione del sistema. I loro valori vengono fissati per ciascun use case prima dell'esecuzione e rimangono costanti per tutta la durata dell'esperimento corrispondente.

Parametro	Unità	Descrizione
N	intero	Numero da fattorizzare
a	intero	Base della moltiplicazione modulare ($\gcd(a, N) = 1$)
n_{count}	intero	Qubit del registro di controllo
M_{shots}	intero	Shot per esecuzione del simulatore
λ_{1q}	adimensionale	Parametro Aer del canale depolarizzante 1Q
λ_{2q}	adimensionale	Parametro Aer del canale depolarizzante 2Q
T_1	ns	Tempo di rilassamento (amplitude damping)
T_2	ns	Tempo di dephasing
p_{ro}	adimensionale	Probabilità di readout error

Tabella C.2: Parametri principali del sistema sperimentale. λ segue la convenzione Aer e non coincide con la probabilità aggregata di Pauli non identità.

Output del Sistema

Per ogni use case e per ciascuno dei due metodi, il sistema produce le seguenti quantità:

- I fattori trovati p e q tali che $p \cdot q = N$, con verifica esplicita della correttezza ($p \neq 1, q \neq 1, p \cdot q = N$);
- Il numero di iterazioni necessarie per il primo risultato corretto;
- La distribuzione di frequenza completa degli output del registro di controllo su M_{shots} esecuzioni;
- Le metriche di confronto tra i due metodi definite nella Sezione 2.6.

Per il solo Metodo 2, vengono prodotte in aggiunta:

- L'accuratezza del classificatore e il punteggio F1 sul test set;
- La curva ROC con l'area sottostante (AUC).

C.2.7 Approfondimento della Sezione 2.6: Definizioni Complete delle Metriche

Origine nel corpo: definizioni operative estese della sintesi in Tabella 2.1 e della Sezione 2.6.1.

Metriche della prima fase

Per il **Metodo 1** si registrano: $\bar{M}_1$, numero medio di esecuzioni necessarie al primo risultato corretto sulle repliche riuscite, sempre accompagnato dal tasso di successo; σ_{M_1} , deviazione standard delle iterazioni; $P_{\text{success},1}(M_{\text{max}})$, probabilità di successo entro il budget preregistrato $M_{\text{max}} = 50$.

Per il **Metodo 2** si registrano analogamente $\bar{M}_2$, σ_{M_2} e $P_{\text{success},2}(M_{\text{max}})$ sulle stesse repliche. La qualità predittiva è descritta da accuratezza, F1-score della classe positiva e AUC sul test set indipendente. Tali metriche qualificano il classificatore, ma la metrica operativa del metodo resta il numero di iterazioni fino a una fattorizzazione verificata.

La metrica descrittiva principale è il fattore di riduzione $\rho = \bar{M}_1/\bar{M}_2$ dell'Equazione (2.1). $\rho > 1$ indica soltanto una media inferiore per il metodo al denominatore: la significatività viene stabilita sulle differenze appaiate e non dedotta dal rapporto fra medie.

Metriche QEC e analisi di sensibilità

- p_L : probabilità che il qubit logico subisca un errore non corretto dopo codifica, rumore, estrazione di sindrome e decodifica; è misurata in funzione dell'errore fisico p e della distanza d .
- **Pendenza di soppressione**: esponente γ nella relazione $p_L \propto p^\gamma$ a piccolo p ; per distanza $d = 3$ il comportamento atteso è $\gamma \simeq 2$.
- **Break-even**: regione $p_L < p$, nella quale il qubit logico è più affidabile del qubit fisico non protetto.
- **Soglia** p_{th} : valore intorno al quale si incrociano le curve del surface code a distanze diverse; sotto soglia, aumentare d deve ridurre p_L .
- $P_{\text{success}}(p_L)$ e $P(k)$: in M8, p_L denota la probabilità totale di un Pauli non identità dopo ogni gate compilato incluso nel proxy, non il tasso per ciclo dei benchmark QEC. La probabilità cumulativa dopo k esecuzioni indipendenti è $P(k) = 1 - (1 - P_s)^k$.

Ogni stima Monte Carlo riporta numero di campioni, seed e incertezza binomiale. La diversa definizione di p_L nei memory experiment e in M8 viene sempre dichiarata per impedire confronti numerici non commensurabili.

C.2.8 Approfondimento della Sezione 2.3: Protocollo Statistico e Test

Origine nel corpo: dettagli dei test e dei criteri richiamati dopo le ipotesi e nella sintesi delle metriche del Capitolo 2.

Appaiamento, fallimenti e pareggi

Ogni replica genera un unico calendario di istogrammi: M_1 , M_{TOP4} e M_2 analizzano gli stessi conteggi nello stesso ordine. Un fallimento entro M_{max} è codificato con la sentinella $M_{max} + 1$, così rimane distinto dal successo all’ultima iterazione e non scompare dai vettori inferenziali. Il confronto usa il test dei ranghi con segno di Wilcoxon con `zero_method='pratt'` e soglia $\alpha = 0.05$: $M_1 > TOP-4$ e $M_1 > M_2$ sono unilaterali, $TOP-4 - M_2$ è bilaterale. Quando sono riportati più confronti della stessa famiglia, i valori p vengono corretti con Holm.

In caso di conteggi uguali, l’ordine dei candidati non dipende né dall’ordine del dizionario né dal valore numerico della bitstring: è stabilito da una priorità SHA-256 dipendente dal seed della coppia replica–iterazione. Seed base, calendari distinti per simulatore e tie-breaking, configurazione e hash degli artefatti sono registrati nel manifest.

Mitigazione e regole decisionali

Il readout error è incorporato nel simulatore come matrice di confusione simmetrica parametrizzata da p_{ro} ; non viene applicata una correzione post-hoc. La sensibilità a p_{ro} è misurata nell’artefatto parametrico schema 2 corrente (Sezione D.1.6); gli artefatti precedenti alla correzione del circuito non alimentano i risultati.

M_1 seleziona la sola moda dell’istogramma, senza sogliatura preliminare, e applica direttamente `extract_factors`. $TOP-4$ estende il tentativo ai quattro esiti più frequenti. M_2 usa l’etichetta $TOP-1$ per decidere se accettare l’istogramma e, in caso positivo, applica $TOP-4$: proprio il confronto con $TOP-4$ privo di filtro isola l’effetto del classificatore.

Matrice di verifica

La matrice sintetica che collega componenti, controlli e criteri di evidenza è stata mantenuta nel corpo, nella Tabella 3.6. Le sottosezioni precedenti ne conservano qui le definizioni operative estese.

Appendice D

Fase 1: la campagna classica nel dettaglio

D.1 Campagna parametrica e confronto con la Zero-Noise Extrapolation

Questa sezione riporta la campagna parametrica schema 2 sulla strategia TOP-K, di cui il Capitolo 5 riporta la sintesi. Il protocollo usa $N = 15$, $a = 7$, $n_{\text{count}} = 8$, 30 repliche, budget massimo 50 e, salvo diversa indicazione, 1 024 shot. Il preset di riferimento è $\lambda_{1q} = 10^{-3}$, $\lambda_{2q} = 10^{-2}$, $T_1/T_2 = 100/80 \mu\text{s}$ e readout simmetrico 2%.

M1 e TOP-K condividono lo stesso istogramma per ogni coppia replica-iterazione. I fallimenti sono codificati con sentinella 51 e i pareggi con una priorità SHA-256 dipendente dal seed. I confronti usano Wilcoxon appaiato-Pratt. Poiché non è applicata una correzione globale fra le otto famiglie, i p di questa sezione hanno finalità esplorativa.

Il materiale seguente approfondisce le Sezioni 5.1.1–5.1.5 del corpo. In quei punti restano il disegno, i risultati decisivi e la loro interpretazione; qui sono raccolti il contratto riproducibile, le validazioni accessorie e tutte le diagnostiche necessarie a verificarli senza interrompere il filo della discussione.

D.1.1 Contratto sperimentale e perimetro delle validazioni

Il percorso rumoroso è limitato a $N = 15$, $a = 7$ e otto qubit di controllo. Smolin, Smith e Vargo hanno mostrato che una versione compilata di Shor costruita sfruttando la conoscenza della risposta può fattorizzare prodotti di due primi in tempo costante

e con due soli qubit coerenti; l'istanza non è quindi presentata come evidenza di scalabilità, ma come banco di prova comune per strategie di post-processing sul medesimo istogramma [68].

Il circuito è compilato nella base `rz/sx/x/cx` con `optimization_level=2` e seed del transpiler 20260819. I gate RZ sono virtuali; SX/X hanno durata illustrativa di 50 ns e CX di 300 ns. Il modello uniforme combina depolarizzazione a uno e due qubit, rilassamento/dephasing e readout: non riproduce una calibrazione né la topologia di una QPU specifica. Ogni iterazione usa 1 024 shot, il budget massimo è 50 e ogni confronto principale comprende 30 repliche. Le strategie della stessa replica ricevono il medesimo istogramma; i pareggi sono risolti mediante una priorità SHA-256 dipendente dal seed. I fallimenti al budget sono codificati con sentinella 51 e le differenze appaiate sono valutate con Wilcoxon–Pratt e correzione di Holm entro ciascuna famiglia.

Una campagna storica usava semi consecutivi che condividevano quasi tutti gli shot derivati da Aer. I relativi valori non sono riutilizzati. La campagna schema 2 separa replica e iterazione di più del budget di shot e registra nel JSON seed, manifest, revisione del rumore e revisione del post-processing.

Tabella D.1: Invarianti del circuito compilato impiegato da training, baseline, sweep e ZNE.

Grandezza	Valore
Qubit totali	12
Profondità / size	412/604
RZ / SX / CX	302/70/224
Misure	8
SHA-256 circuito	c7f4cf3bc1735dce9d5b56d8e09686e...
Qiskit / Aer	2.5.0/0.17.2

L'aritmetica Beauregard per $N = 21$ e $N = 35$ è stata verificata separatamente nel caso ideale mediante truth table esaustive dei moltiplicatori controllati, pulizia delle ancille e confronto della legge QPE con quella teorica. La distanza di variazione totale è 1.38×10^{-9} per $N = 21$, $a = 2$, $r = 6$ e 2.50×10^{-13} per $N = 35$, $a = 6$, $r = 2$. Per $N = 21$, otto controlli e compilazione congiunta nella base esecutiva producono 21 036 CX e profondità 23 081. Al valore diagnostico $\lambda_{2q} = 10^{-3}$, la probabilità che

nessuna CX applichi un Pauli non-identità è

$$\left(1 - \frac{15}{16}\lambda_{2q}\right)^{21036} = 2.70 \times 10^{-9};$$

non è una fedeltà né una probabilità di fattorizzazione.

Tabella D.2: Separazione fra validazione ideale e campagna rumorosa. Nel corpo è mantenuto il solo confronto centrale su $N = 15$ e il controllo di scala su $N = 21$; $N = 35$ resta documentato qui.

Istanza	Ideale	Rumorosa
$N = 15, a = 7$	truth table e QPE	campagna completa
$N = 21, a = 2$	truth table e QPE	esclusa per costo
$N = 35, a = 6$	truth table e QPE	esclusa per costo

D.1.2 Variazione di K

Tabella D.3: M1 e TOP-K sui medesimi 30 vettori di iterazioni. $K = 1$ coincide con il controllo M1.

K	$\bar{M}_1$	$\bar{M}_{\text{TOP}K}$	successo	ρ	p
1	1.37	1.37	100%	1.000	1.000
2	1.37	1.00	100%	1.367	< 0.001
3	1.37	1.00	100%	1.367	< 0.001
4	1.37	1.00	100%	1.367	< 0.001
6	1.37	1.00	100%	1.367	< 0.001
8	1.37	1.00	100%	1.367	< 0.001

La soglia osservata è $K = 2$, non $K = r = 4$. Questo risultato dipende dalla banda di esiti che la post-elaborazione mappa su un periodo utile e non costituisce una regola generale per altre istanze.

D.1.3 Variazione di λ_{2q}

Per il canale Aer a due qubit, $\mathcal{E}(\rho) = (1 - \lambda)\rho + \lambda I/4$, la probabilità aggregata della componente Pauli non-identità è $15\lambda/16$. Sul circuito da 224 CX il proxy di nessun evento è quindi $(1 - 15\lambda_{2q}/16)^{224}$; non è una probabilità di successo.

Tabella D.4: Il tasso di successo è 100% per tutte le 30 repliche di ciascun punto.

λ_{2q}	proxy	$\bar{M}_1$	$\bar{M}_{\text{TOP4}}$	ρ	p
10^{-3}	8.11×10^{-1}	1.93	1.00	1.933	< 0.001
5×10^{-3}	3.49×10^{-1}	1.90	1.00	1.900	< 0.001
10^{-2}	1.21×10^{-1}	1.37	1.00	1.367	< 0.001
2×10^{-2}	1.44×10^{-2}	1.33	1.00	1.333	0.001
5×10^{-2}	2.14×10^{-5}	1.33	1.00	1.333	< 0.001
10^{-1}	2.65×10^{-10}	1.57	1.07	1.469	0.001

Al punto estremo TOP-4 richiede in media 1.07 iterazioni: la robustezza non è assoluta, pur restando il successo 30/30 entro il budget.

D.1.4 Variazione degli Shot

Tabella D.5: Risultati su 30 repliche per punto.

shot	$\bar{M}_1$	$\bar{M}_{\text{TOP4}}$	ρ	p
128	1.37	1.00	1.367	0.001
256	1.57	1.00	1.567	< 0.001
512	1.57	1.00	1.567	0.001
1024	1.37	1.00	1.367	< 0.001
2048	1.70	1.00	1.700	< 0.001

TOP-4 è stabile già a 128 shot; M1 oscilla senza andamento monotono, coerentemente con una competizione campionaria fra quattro picchi.

D.1.5 Analisi Congiunta $K \times \lambda_{2q}$

Tabella D.6: Ogni cella è $\bar{M}_{\text{TOP}K}$ su 30 repliche e ha successo 100%.

K	5×10^{-3}	10^{-2}	2×10^{-2}	5×10^{-2}
2	1.00	1.00	1.00	1.00
4	1.00	1.00	1.00	1.00
6	1.00	1.00	1.00	1.00
8	1.00	1.00	1.00	1.00

D.1.6 Parametri Secondari

RZ è virtuale (0 ns), SX/X dura 50 ns e CX 300 ns. T_2 viene variato insieme a T_1 mantenendo $T_2 = 0.8T_1$.

Tabella D.7: TOP-4 ottiene $\bar{M} = 1.00$ e successo 100% in tutti i punti.

T_1/T_2 (μs)	$\bar{M}_1$	$\bar{M}_{\text{TOP4}}$	ρ	p
20/16	1.47	1.00	1.467	< 0.001
50/40	2.20	1.00	2.200	< 0.001
100/80	1.37	1.00	1.367	< 0.001
200/160	1.60	1.00	1.600	0.001
500/400	1.80	1.00	1.800	< 0.001

Tabella D.8: TOP-4 ottiene $\bar{M} = 1.00$ e successo 100% in tutti i punti.

λ_{1q}	$\bar{M}_1$	ρ	p
10^{-4}	1.60	1.600	< 0.001
5×10^{-4}	1.43	1.433	< 0.001
10^{-3}	1.37	1.367	< 0.001
5×10^{-3}	1.67	1.667	< 0.001
10^{-2}	1.57	1.567	< 0.001
2×10^{-2}	1.47	1.467	< 0.001

Tabella D.9: TOP-4 ottiene $\bar{M} = 1.00$ e successo 100% in tutti i punti.

p_{ro}	$\bar{M}_1$	ρ	p
0%	1.33	1.333	0.004
1%	1.50	1.500	< 0.001
2%	1.37	1.367	< 0.001
5%	1.47	1.467	< 0.001
10%	1.30	1.300	0.002
20%	1.47	1.467	< 0.001

Le oscillazioni non monotone di M1 sconsigliano inferenze causali sui singoli parametri con questa numerosità. Il risultato supportato è più limitato: TOP-4 non varia nei punti osservati.

D.1.7 Livello di Ottimizzazione

Tabella D.10: Conteggi della compilazione effettiva e metriche su 30 repliche.

livello	CX	profondità	$\bar{M}_1$	$\bar{M}_{\text{TOP4}}$	ρ
0	242	427	1.47	1.00	1.467
1	236	421	1.50	1.00	1.500
2	224	412	1.37	1.00	1.367
3	224	412	1.37	1.00	1.367

D.1.8 Diagnostiche della struttura QPE

Le diagnostiche decisive sono mantenute nel corpo: la Figura 5.2 confronta le repliche estreme, l'Equazione 5.1 definisce l'indice descrittivo e la Tabella 5.3 con la Figura 5.3 mostrano lo scarto dal proxy di nessun evento. Gli sweep precedenti ne conservano qui il contesto parametrico.

D.1.9 Confronto con Zero-Noise Extrapolation

La ZNE digitale scala λ_{1q} e λ_{2q} , mantenendo fissi coerenza e readout. ZNE-2 usa due livelli e ZNE-3 tre livelli condivisi; non è gate folding su hardware.

Tabella D.11: I p sono corretti con Holm nella famiglia M1 contro ZNE-2, ZNE-3 e TOP-4.

strategia	successo	$\bar{M}$	shot/iter.	shot medi totali	p_{Holm}
M1	100%	1.37	1 024	1 399	—
ZNE-2	100%	1.50	2 048	3 072	1.000
ZNE-3	100%	1.57	3 072	4 813	1.000
TOP-4	100%	1.00	1 024	1 024	0.0024

Validazione fuori dalla campagna rumorosa. I moltiplicatori Beauregard per $N = 21/35$ sono validati idealmente e non entrano negli sweep. Il contratto completo e il valore coerente con la parametrizzazione Aer del canale a due qubit sono riportati nella Sezione D.1.1; nessuno dei proxy è interpretato come probabilità di successo.

D.2 Il classificatore ML: architettura, addestramento e metriche

Questa sezione documenta il classificatore valutato nella prima fase sperimentale (Capitolo 5) e, soprattutto, separa due domande che nella prima versione della campagna erano state confuse: *il candidato più frequente conduce ai fattori?* (TOP-1) e *esiste un candidato utile fra i primi sedici?* (TOP-16). Tutti i numeri riportati provengono dagli artefatti schema 2 rigenerati con il circuito $N = 15$, $a = 7$ corretto, il medesimo manifest del circuito e seed deterministici.

D.2.1 L’Ipotesi alla Prova: il Classificatore ML

Il Metodo 2 usa un classificatore come *gate di qualità*: riceve la distribuzione normalizzata sui $2^8 = 256$ esiti e, se approva l’istogramma, tenta l’estrazione dei fattori dai quattro candidati più frequenti. L’ablazione M_{TOP4} applica la stessa ricerca senza filtro e consente quindi di distinguere l’effetto del classificatore da quello della ricerca multi-candidato.

Per ciascuno dei due scenari sono stati generati 2 000 istogrammi da 1 024 shot. I cinque parametri del modello uniforme (λ_{1q} , λ_{2q} , T_1 , T_2 e readout) sono stati campionati indipendentemente in un intervallo relativo del $\pm 50\%$ attorno al preset. Il dataset è stato separato in train, validation e test con proporzioni 60/20/20: la scelta dell’architettura usa soltanto la validation; il test viene valutato una sola volta dopo il refit su train+validation.

La regola TOP-1 produce 1 262 positivi e 738 negativi in UC1 (63.1%/36.9%), e 1 453 positivi e 547 negativi in UC2 (72.7%/27.3%). Random Forest, SVM RBF calibrata e MLP (256, 128) sono state confrontate mediante F1 sulla validation. La Tabella D.12 riporta la valutazione indipendente del modello selezionato.

Tabella D.12: Metriche TOP-1 sul test set indipendente (400 istogrammi per scenario)

Scenario	Modello selezionato	F1	Accuratezza	AUC-ROC
UC1	SVM	0.919	0.895	0.965
UC2	SVM	0.923	0.888	0.958

Sulla validation l’SVM ottiene F1 pari a 0.908 in UC1 e 0.929 in UC2, contro 0.811/0.842 della Random Forest e 0.874/0.906 dell’MLP. Queste metriche mostrano che il modello riproduce bene la regola TOP-1; non dimostrano però che il filtro

riduca il costo operativo. Quel confronto richiede gli stessi istogrammi per M1, TOP-4 e M2 ed è riportato nella Sezione 5.1.4.

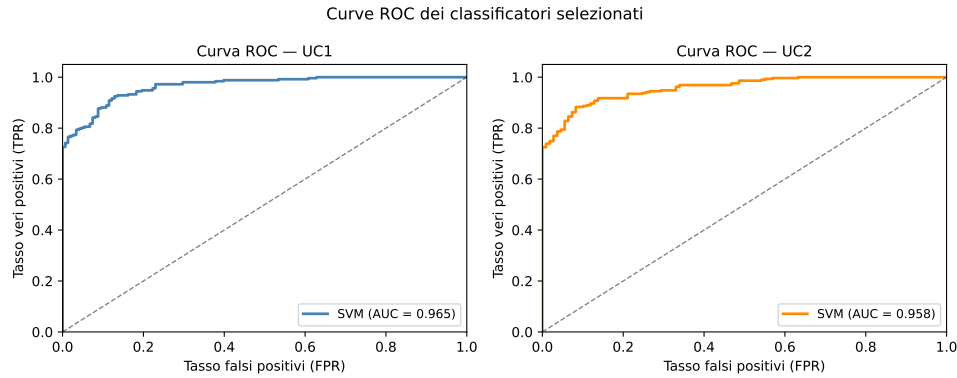


Figura D.1: Curve ROC sul test set indipendente dei due classificatori SVM selezionati mediante validation. Le aree sono 0.965 (UC1) e 0.958 (UC2); la diagonale rappresenta un classificatore casuale.

D.2.2 Audit delle Etichette: TOP-1 e TOP-16 Sono Problemi Diversi

La campagna rigenerata calcola entrambe le etichette sullo *stesso* istogramma, eliminando l'ambiguità del dataset storico. L'esito è riassunto nella Tabella D.13.

Tabella D.13: Bilanciamento delle due regole sul dataset completo. TOP-16 è a classe unica e non consente un addestramento discriminante.

Regola	UC1: positivi/negativi	UC2: positivi/negativi
TOP-1	1 262/738	1 453/547
TOP-16	2 000/0	2 000/0

Il runner non forza un modello sul dataset TOP-16: registra esplicitamente `single-class-dataset` e interrompe l'addestramento per quella regola. M2 usa perciò soltanto i modelli TOP-1 e conserva nel payload `label_top_k=1`. Questo controllo evita di attribuire capacità predittiva a un classificatore costante.

Una diagnostica separata su 60 esecuzioni nominali UC1 chiarisce la struttura della classe TOP-1. La moda è $y = 0$ in 30/60 casi e conduce ai fattori negli altri 30/60; i primi quattro candidati contengono invece almeno un esito utile in 60/60 casi. La Tabella D.14 riporta la distribuzione della moda.

Tabella D.14: Distribuzione della misura più frequente su 60 istogrammi nominali indipendenti.

Valore della moda	Occorrenze	Frazione
0	30	50.0%
64	5	8.3%
128	15	25.0%
192	10	16.7%

L'esito $y = 0$ rappresenta la fase nulla e non fornisce il periodo, ma non implica che l'istogramma sia privo di segnale. Nel campione rappresentativo della Figura D.2, la moda $y = 0$ raccoglie 176 conteggi, mentre $y = 64$, 192 e 128 ne raccolgono rispettivamente 160, 141 e 131: tutti e tre conducono ai fattori. La massa complessiva sui quattro picchi è 0.594.

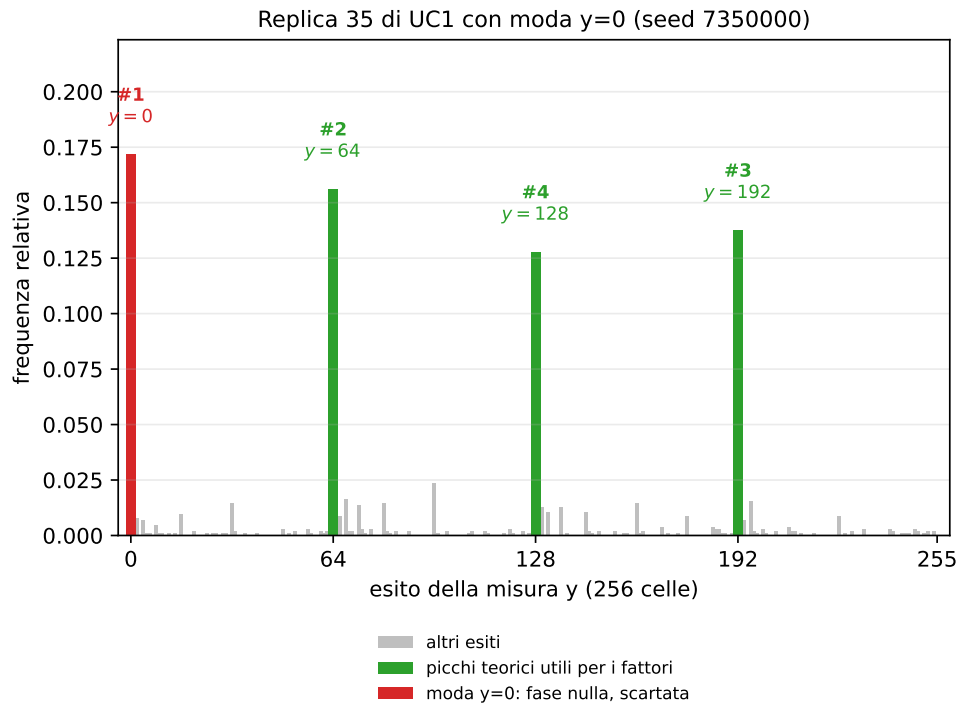


Figura D.2: Esempio nominale UC1 in cui TOP-1 fallisce perché la moda è $y = 0$, mentre i successivi tre picchi conducono ai fattori. Il classificatore TOP-1 apprende quindi una regola sul candidato dominante, non una misura generale della presenza di segnale quantistico.

Perché TOP-16 Produce una Classe Quasi Sempre Positiva

Per $N = 15$, $a = 7$ e otto bit di controllo, 63 dei 256 esiti conducono ai fattori tramite la procedura di frazioni continue adottata. Anche per un istogramma uniforme, la probabilità che nessuno dei primi K candidati sia utile è quindi

$$P_{\text{neg}}(K) = \frac{\binom{256-63}{K}}{\binom{256}{K}}. \quad (\text{D.1})$$

Per $K = 16$ essa vale soltanto 0.9275%: la regola assegna dunque un'etichetta positiva a oltre il 99% degli istogrammi uniformi per puro effetto combinatorio. Il risultato 2000/0 osservato nei due dataset è coerente con questo limite e non autorizza a concludere che il segnale sia perfetto.

Tabella D.15: Probabilità di non includere alcuno dei 63 esiti utili scegliendo K celle da 256 senza informazione sulla distribuzione.

Regola	$P_{\text{neg}}(K)$
TOP-1	75.39%
TOP-2	56.76%
TOP-4	32.06%
TOP-16	0.93%

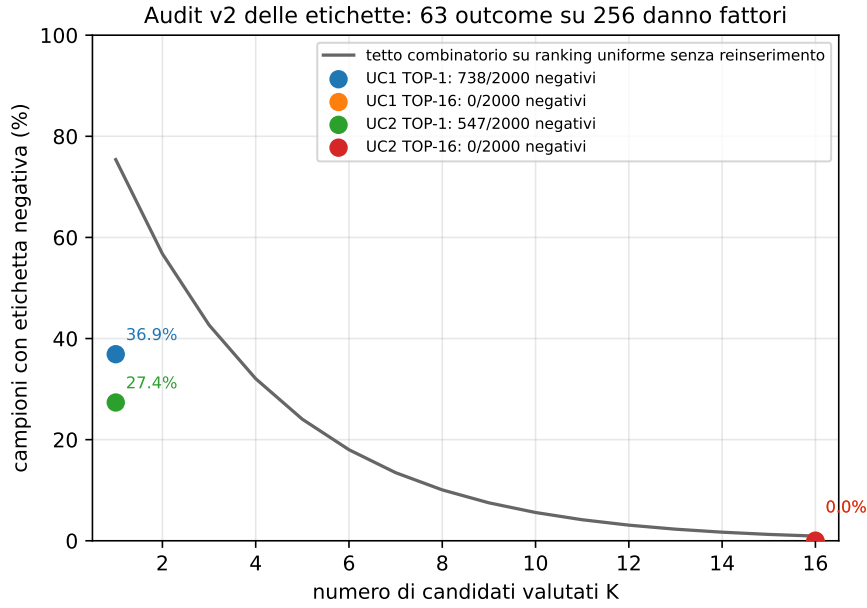


Figura D.3: Decadimento di $P_{\text{neg}}(K)$ al crescere del numero di candidati. La figura è rigenerata dai dati schema 2 e rende visibile perché TOP-16 non definisce un problema di classificazione bilanciato su questa istanza.

Il verdetto ha un perimetro preciso: non mostra che l'apprendimento automatico sia inutile in generale, ma che un filtro a valle addestrato su TOP-1 replica una decisione già superata da TOP-4, mentre la regola TOP-16 è quasi sempre positiva per costruzione. Questo motiva lo spostamento dell'apprendimento dalla distribuzione finale alla sindrome di errore nelle sezioni del Capitolo 5 dedicate alla QEC.

Appendice E

Fase 2: la campagna QEC nel dettaglio

E.1 Laboratori preliminari QEC: codici a ripetizione e di Steane

Questa sezione conserva la trattazione estesa e i protocolli dei laboratori introduttivi sintetizzati nella Sezione 5.2.4 e nella Sezione 5.2.5 del Capitolo 5. Le curve e i risultati decisivi restano nel corpo; qui sono raccolti gli strumenti concettuali, le lookup table e le assunzioni necessarie a verificarli.

E.1.1 Dalla ridondanza alla codifica per stabilizzatori

L'idea classica di trasmettere tre copie di un bit e decidere a maggioranza non si trasferisce direttamente al caso quantistico. Il teorema di no-cloning [23] impedisce di copiare uno stato ignoto e la misura diretta distruggerebbe la sovrapposizione da proteggere. La QEC codifica invece l'informazione di un qubit logico nelle correlazioni di più qubit fisici [28, 10, 29]. Operatori ausiliari rivelano quale classe di errore si è verificata senza rivelare i coefficienti dello stato logico.

Il ciclo generale comprende: (i) codifica di $|\psi_L\rangle$; (ii) azione del rumore; (iii) estrazione della sindrome mediante ancille; (iv) decodifica classica; (v) correzione fisica o aggiornamento del *Pauli frame*. Uno stabilizzatore S lascia invariati gli stati del codice, $S|\psi_L\rangle = |\psi_L\rangle$: un esito -1 segnala l'uscita dal sottospazio valido senza distinguere gli stati logici che lo compongono.

Un codice $[[n, k, d]]$ usa n qubit fisici per k qubit logici e ha distanza d , pari al peso minimo di un errore logico non rilevato. Può correggere

$$t = \left\lfloor \frac{d-1}{2} \right\rfloor \quad (\text{E.1})$$

errori arbitrari. Il teorema della soglia garantisce in linea di principio un regime in cui l'aumento della distanza riduce l'errore logico [24]; la sua osservazione quantitativa è oggetto della Sezione 5.3.4.

Codice	Parametri	Corregge	Pro / Contro	Uso nella tesi
Repetition (bit-flip)	$[[3, 1, 1]]$ su X	solo bit-flip	semplice / non corregge phase-flip	primo laboratorio
Repetition (phase-flip)	3 qubit, base H	solo phase-flip	mostra dualità X/Z / parziale	controllo duale
Shor code	$[[9, 1, 3]]$	errore arbitrario singolo	storico, intuitivo / più pesante	richiamo didattico [28]
Steane code	$[[7, 1, 3]]$	errore arbitrario singolo	CSS, compatto / distanza bassa	secondo laboratorio [10]
Five-qubit	$[[5, 1, 3]]$	errore arbitrario singolo	minimo / non CSS	confronto teorico
Surface code	d^2 dati $+O(d^2)$ ancille	errori locali su griglia 2D	scalabile / overhead elevato	campagna principale [11]

Tabella E.1: Confronto esteso e progressione metodologica: repetition per introdurre la sindrome, Steane per un errore arbitrario, surface code per soglia e scalabilità.

E.1.2 Protocollo completo del codice a ripetizione

Il codice contro errori X codifica $|0_L\rangle = |000\rangle$ e $|1_L\rangle = |111\rangle$: lo stato $\alpha|0\rangle + \beta|1\rangle$ diventa $\alpha|000\rangle + \beta|111\rangle$, uno stato entangled e non tre copie. Gli stabilizzatori Z_1Z_2 e Z_2Z_3 misurano la parità tramite due ancille. Il duale nella base di Hadamard usa X_1X_2 e X_2X_3 e protegge dagli errori Z; la concatenazione delle due protezioni conduce al codice di Shor a nove qubit.

Il primo controllo inietta deterministicamente un errore su ciascun qubit dato e verifica che la sindrome identifichi la posizione senza misurare l'informazione logica.

Errore iniettato	Sindrome (a_0, a_1)	Qubit dedotto	Esito
nessuno	$(0, 0)$	—	✓
su q_0	$(1, 0)$	q_0	✓
su q_1	$(1, 1)$	q_1	✓
su q_2	$(0, 1)$	q_2	✓

Tabella E.2: Lookup table verificata per il codice bit-flip e per il duale phase-flip. L'ancilla a_0 misura la parità di $\{0, 1\}$ e a_1 quella di $\{1, 2\}$. La coincidenza cifra per cifra a parità di seed verifica l'implementazione della dualità, non costituisce una conferma indipendente del fatto algebrico.

Il secondo controllo prepara lo stato logico, applica a ciascun qubit un errore indipendente con probabilità p , estrae la sindrome, corregge e decodifica. I 20 000 campioni per punto (seed 42) producono la curva riportata nella Figura 5.4. Un errore Z resta invisibile al codice bit-flip, e viceversa per il duale: è il limite che motiva il passaggio allo Steane.

E.1.3 Struttura e protocollo completo del codice di Steane

Lo Steane [10] è un codice CSS derivato dal codice di Hamming [7, 4, 3]: codifica un qubit in sette, ha distanza 3 e corregge un errore X, Z o Y su un singolo qubit.

Parametro	Valore	Significato
n	7	qubit fisici
k	1	qubit logici
d	3	distanza
t	1	errori arbitrari correggibili (Eq. E.1)
Tipo	CSS	sindromi X e Z separate

Tabella E.3: Parametri del codice di Steane $[[7, 1, 3]]$.

Nella convenzione adottata i generatori sono

$$\begin{aligned}
\text{tipo X: } & IIIXXXX, \quad IXXIIXX, \quad XIXIXIX, \\
\text{tipo Z: } & IIIZZZZ, \quad IZZIIZZ, \quad ZIZIZIZ.
\end{aligned} \tag{E.2}$$

Gli stabilizzatori Z rilevano gli errori X, quelli X rilevano gli errori Z e un errore Y accende entrambe le famiglie. I tre bit di sindrome, letti come numero binario, indicano la posizione del qubit colpito.

Il protocollo comprende: preparazione di $|0_L\rangle$ o $|+_L\rangle$; iniezione manuale di X, Z e Y su ciascuno dei sette qubit; misura della sindrome; costruzione della lookup table; stima della correzione al variare di p ; distinzione fra errori sui dati e sulle ancille. La preparazione di $|0_L\rangle$ usa tre qubit seme in sovrapposizione e una rete CNOT; i sei stabilizzatori restituiscono 000000 su tutti i 2 000 campioni, verificando l'appartenenza al code space.

Qubit	Sindrome	Valore	Correzione
q_0	001	1	applica X (o Z) su q_0
q_1	010	2	applica su q_1
q_2	011	3	applica su q_2
q_3	100	4	applica su q_3
q_4	101	5	applica su q_4
q_5	110	6	applica su q_5
q_6	111	7	applica su q_6

Tabella E.4: Tabella sindrome–qubit–correzione verificata per tutti i 21 casi: X, Z e Y su ciascuno dei sette qubit.

La simulazione Monte Carlo usa 200 000 campioni per punto (seed 42), applica la correzione a peso minimo e verifica il residuo logico. La versione implementata assume errori sui qubit dati e sindrome ideale: l'estrazione non è fault-tolerant. Un guasto sulle ancille può corrompere la sindrome e indurre una correzione errata; i cicli rumorosi e ripetuti del surface code affrontano precisamente questo limite.

E.2 Surface code: protocollo e controlli sul modello

Questa sezione approfondisce le Sezioni 5.3.1–5.3.5 del Capitolo 5. Nel corpo restano curve, soglie, legge di scala e conclusioni; qui sono conservati il funzionamento del codice, la griglia sperimentale, le verifiche di riproducibilità e il controllo sui canali non-Pauli.

E.2.1 Surface code e decodifica MWPM

Il surface code [30, 11] dispone su una griglia bidimensionale qubit dati e qubit di misura. Gli stabilizzatori sono locali e un patch planare ruotato di distanza d usa d^2 dati, oltre a un numero dello stesso ordine di ancille. Le misure sono ripetute per più

round perché anche la sindrome è rumorosa; una variazione fra due cicli costituisce un *detection event*.

Nel bulk, una catena d'errore produce tipicamente detection events in coppie; al bordo uno dei due estremi può terminare sul confine. Il MWPM associa gli eventi minimizzando un peso collegato alla probabilità delle catene [14]. Se errore reale e correzione differiscono per un ciclo che attraversa il patch, si verifica un errore logico silenzioso: la sua frequenza è p_L .

La stima richiede milioni di circuiti con centinaia di qubit, ma i memory experiment sono circuiti stabilizer. Stim [13] sfrutta il teorema di Gottesman–Knill per campionare detection events; PyMatching costruisce il grafo dal DEM e decodifica. Il Listato B.14 nella Sezione B.2 documenta l'implementazione. Shor non è un circuito stabilizer: per questo i tassi del memory experiment non sono trasferiti direttamente alla successiva analisi di sensibilità.

E.2.2 Griglia sperimentale e riproducibilità

Variabile	Valori	Osservazione attesa
distanza d	3, 5, 7, 9	p_L diminuisce con d sotto soglia
rounds	d e $2d$	cicli di memoria quantistica
p fisico	$10^{-4} \dots 2 \times 10^{-2}$	curva p - p_L
shot	$\geq 10^4$ per punto	intervalli di confidenza stabili
base	memory X e memory Z	controllo delle due protezioni

Tabella E.5: Griglia prescritta. La campagna eseguita copre le quattro distanze, entrambe le basi, rounds = d e $p \in [2 \times 10^{-3}, 10^{-2}]$; lo scaling a $2d$ resta un'estensione.

Il modello è circuit-level: depolarizzazione dopo ogni Clifford e flip di reset e misura con probabilità p . Il campione è dimensionato punto per punto per raccogliere circa duecento eventi logici, da 2×10^5 a 4×10^7 shot. Una prima passata da 2×10^5 stima l'ordine di grandezza di p_L ; la seconda fissa la numerosità. A $d = 9$ e $p = 2 \times 10^{-3}$ una numerosità fissa darebbe due soli eventi e il 70% di incertezza relativa, mentre il campionamento adattivo la porta al 3.5%. Seed e numerosità sono registrati per ogni punto. Una ripetizione indipendente ha riprodotto tutte le stime entro l'errore statistico e le stesse soglie.

E.2.3 Errori coerenti, leakage e controllo full-state

Stim rappresenta gate Clifford ed errori Pauli casuali. L'hardware può però produrre sovrarotazioni coerenti, leakage verso $|2\rangle$ e correlazioni temporali; una simulazione solo Clifford–Pauli può quindi essere ottimistica [17]. In una sovrarotazione $U = e^{-i\varepsilon H}$ le ampiezze si sommano coerentemente, mentre un errore stocastico accumula probabilità.

Il divario è illustrato da un qubit soggetto a N operazioni con $p = 10^{-2}$, modellate come X stocastico oppure come $R_X(\theta)$ con $\sin^2(\theta/2) = p$. Le probabilità esatte sono $[1 - (1 - 2p)^N]/2$ e $\sin^2(N\theta/2)$; nel tratto iniziale crescono come Np e N^2p . A $N = 8$ il secondo errore è circa sette volte il primo per i parametri scelti.

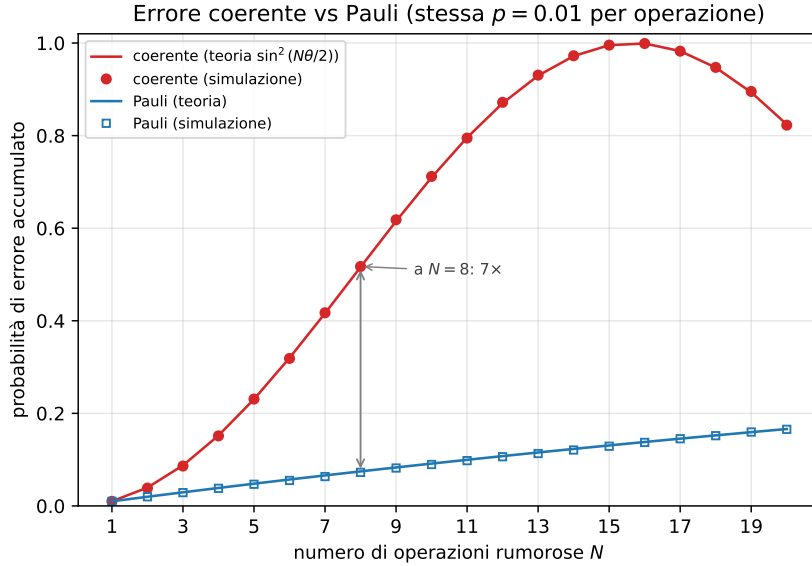


Figura E.1: Accumulo su un qubit a parità di probabilità marginale per operazione. La simulazione full-state (marcatori) coincide con le espressioni teoriche; il fattore osservato non è universale.

Lo studio di riferimento riporta, in altri regimi, errori logici 1.7% contro 23.5% in presenza di leakage e 18.1% contro 38.7% con errori coerenti; per un surface code a distanza 9 il divario può raggiungere un fattore 25–56 e spostare la soglia di circa il 20% [17]. Questi valori non sono trasferiti alla campagna della tesi: motivano un approccio channel-first, nel quale il canale fisico è definito una volta e approssimato solo dopo aver stabilito quale informazione preservare.

Il controllo interno usa un codice a distanza 3 simulato full-state per quattro cicli, confrontando X di Pauli e R_X coerente a pari probabilità marginale. A

$p = 3 \times 10^{-2}$ il canale coerente produce meno flip grezzi (0.237 contro 0.262) ma un errore MWPM leggermente maggiore (0.0789 contro 0.0767, +2.9%). Le misure ripetute interrompono parte dell’accumulo coerente, senza equivalere a un twirling Pauli completo.

Il decoder ibrido ottiene guadagni 1.019, 1.036, 1.041 sotto Pauli e 1.029, 1.048, 1.052 sotto sovrarotazione, per $p = 1, 2, 3 \times 10^{-2}$. Lo scarto è circa una deviazione standard e non è significativo. Il risultato esclude quindi la regola semplicistica “non-Pauli implica vantaggio appreso”: ciò che il matching non rappresenta è la non decomponibilità in archi, come nel crosstalk che accende più di due detector. Le soglie del corpo restano baseline entro il modello Pauli; leakage e canali più ricchi sono sviluppi futuri.

E.3 La decodifica appresa: la campagna completa

Questa sezione completa le evidenze E1–E4 presentate nella Sezione 5.3.6 con i controlli di scala, trasferibilità, calibrazione e scelta della baseline. Le analisi seguenti rendono verificabili i limiti del decoder ibrido senza interrompere il filo dei risultati principali.

Il limite: il vantaggio non sopravvive alla distanza 7. Il dato più importante della Tabella 5.9 è però l’ultimo blocco, e va enunciato senza attenuazioni. A distanza 7 l’ibrido **non ribalta alcuno shot** in nessuna delle cinque configurazioni: la soglia scelta in validazione è ogni volta quella corrispondente a “non ribaltare mai”, e il decoder ibrido coincide con il MWPM cifra per cifra. Il guadagno svanisce, e la progressione lungo le tre distanze è inequivocabile: 18–21% a $d = 3$, 1–3% a $d = 5$, esattamente zero a $d = 7$.

Due meccanismi possono spiegarlo, e vanno separati perché conducono a rimedi diversi. Il primo è la **scarsità di esempi**: l’evento da riconoscere è il fallimento del matching, la cui frequenza è p_L , e p_L decresce esponenzialmente con d . In assenza di crosstalk, a $d = 7$ si hanno $p_L = 5.5 \times 10^{-4}$ e 8×10^5 shot, ossia 264 esempi positivi nella partizione di addestramento, per un ingresso a 336 dimensioni. Il secondo è la **capacità rappresentativa**: la rete densa impiegata tratta i detector come dimensioni indipendenti, e ciò che distingue le due classi di errore su un reticolo grande è una correlazione a lungo raggio fra detector distanti.

Un primo indizio viene dal semplice conteggio. La spiegazione per scarsità è esatta nella configurazione senza crosstalk, dove i fallimenti del matching sono effettivamente poche centinaia; ma nelle altre quattro configurazioni a $d = 7$ la rete dispone da 1.8×10^4 a 2.2×10^5 esempi positivi — fino a duecentomila — e continua a non ribaltare alcuno shot. La spiegazione copre dunque una configurazione su cinque.

L’indizio non basta però a stabilire la causa, perché fra $d = 5$ (che funziona) e $d = 7$ (che non funziona) cambiano *due* cose insieme: la distanza del codice e il numero di detector. L’esperimento E9 le separa, sfruttando il fatto che il numero di detector è il prodotto fra gli stabilizzatori per ciclo e il numero di cicli, e può quindi essere variato a distanza fissa. Ne risulta un incrocio 2×2 le cui celle diagonali distinguono le due ipotesi, riportato nella Tabella E.6.

d	cicli	detector	$p_{ct} = 0.005$		$p_{ct} = 0.02$	
			es. pos.	residuo	es. pos.	residuo
7	3	144	5 933	si astiene	61 900	interviene
7	7	336	17 665	si astiene	139 275	si astiene
5	14	336	48 657	si astiene	188 582	si astiene
5	5	120	17 235	interviene	96 167	interviene

Tabella E.6: Esperimento E9: incrocio fra distanza del codice e numero di detector, ottenuto variando il numero di cicli di estrazione della sindrome. “Es. pos.” è il numero di fallimenti del MWPM disponibili all’addestramento su 4.8×10^5 shot; “residuo” indica se il criterio di validazione seleziona una soglia che ribalta almeno uno shot su diecimila. Le righe centrali sono le celle diagonali, quelle che distinguono le ipotesi: a distanza 5 con 336 detector e 1.9×10^5 esempi il residuo si astiene, a distanza 7 con 144 detector e tre volte meno esempi interviene. Il comportamento segue la colonna, non la riga.

L’esito è netto su entrambe le diagonali e va letto per colonne. A 336 detector il residuo si astiene sempre, a qualunque distanza e con qualunque numerosità — fino a 1.9×10^5 esempi positivi; a 120 e 144 detector interviene, anche a distanza 7 e anche con tre volte meno esempi. **L’ipotesi della distanza è confutata, quella della larghezza dell’ingresso è confermata.**

Una sfumatura completa l’enunciato e lo rende più preciso. A 144 detector con soli 5 933 esempi positivi il residuo si astiene: la numerosità è dunque un vincolo reale, al margine inferiore. Non è però il vincolo che opera nell’esperimento E4 a distanza 7, dove gli esempi erano 1.4×10^5 e la rete si asteneva comunque. La formulazione corretta è che disporre di abbastanza esempi è *necessario ma non sufficiente*, e che a parità di numerosità sufficiente è la larghezza dell’ingresso a decidere.

Che cosa significhi esattamente “si astiene” (E12). Il verbo va però disinnescato, perché astenersi è l’esito di una *selezione*: fra le soglie candidate figura quella corrispondente a “non ribaltare mai”, e che vinca in validazione significa che agire non conviene, non che non vi sia nulla da vedere. La distinzione non è oziosa, perché le due letture conducono a rimedi opposti — un’architettura diversa nella prima, una regola di decisione diversa nella seconda. L’esperimento E12 la risolve misurando l’AUC del residuo, che quantifica quanto la rete *ordini* i fallimenti del matching indipendentemente da qualunque soglia, e affiancando alla soglia scelta onestamente in validazione una soglia *oracolo*, la migliore sul test set stesso: inutilizzabile in pratica, ma limite superiore assoluto di ciò che qualunque regola di decisione potrebbe ottenere.

detector	<i>d</i>	cicli	AUC	guadagno onesto	guadagno oracolo
24	3	3	0.912	1.2215	1.2233
120	5	5	0.769	1.0206	1.0215
144	7	3	0.788	1.0066	1.0066
336	7	7	0.621	1.0000	1.0000
336	5	14	0.573	1.0000	1.0003

Tabella E.7: Esperimento E12 a $p_{ct} = 2 \times 10^{-2}$. L’AUC è calcolata sul test set con bersaglio “il MWPM ha sbagliato”. Il guadagno oracolo usa la soglia migliore sul test set stesso e non è quindi realizzabile: serve unicamente a stabilire quanto del divario sia imputabile alla calibrazione. Ai livelli di crosstalk più bassi il quadro è il medesimo, con AUC sistematicamente più alte (0.958 a 24 detector, 0.771 a 336).

Il risultato smentisce entrambe le letture immediate. **La rete non è cieca:** a 336 detector l’AUC vale 0.57–0.77, ben sopra il caso, e la rete ordina dunque i fallimenti del matching assai meglio del lancio di una moneta. **Ma non si tratta neppure di calibrazione:** il guadagno oracolo è 1.0000, sicché nemmeno scegliendo la soglia sui dati di valutazione si otterrebbe alcunché.

La spiegazione sta nella forma stessa della correzione residuale, ed è quantitativa. Il ribaltamento è *simmetrico*: ribaltare uno shot che il matching aveva risolto costa esattamente quanto ribaltarne uno che aveva sbagliato rende. Perché l’operazione sia in attivo occorre dunque che fra gli shot ribaltati la **precisione superi il cinquanta per cento** — una soglia netta, non un ottimo graduale. L’AUC necessaria a raggiungerla dipende dal tasso di base: a 24 detector e crosstalk 5×10^{-3} il tasso di base è del 3.1% e un’AUC di 0.958 concentra i fallimenti abbastanza da superare la soglia, con un guadagno del 19%; a 336 detector, a parità di tasso di base, un’AUC di 0.771 richiederebbe un fattore di concentrazione quattordici volte superiore a

quello che produce. Nessuna soglia porta la precisione sopra il cinquanta per cento, e il guadagno è allora *esattamente* 1.000 — non approssimativamente: è questa discontinuità a spiegare il salto brusco osservato lungo la Tabella E.6.

Ne discendono due conseguenze. La prima è che **l’astensione del criterio di validazione è corretta e non conservativa**: la soglia oracolo conferma che intervenire non converrebbe, e il metodo si comporta esattamente come dichiarato al momento della sua costruzione. La seconda riguarda il rimedio, e ne precisa il criterio di successo: un’architettura convoluzionale o a grafo è utile se e soltanto se innalza l’AUC quanto basta a riportare la precisione sopra la soglia del cinquanta per cento, che è una condizione verificabile in anticipo su una rete addestrata — assai più informativa dell’osservazione che il modello “non regge”.

Questa è la delimitazione del risultato di M10, e conviene formularla in positivo. Il guadagno del decoder ibrido è supportato da un valore nominale estremamente piccolo del test di McNemar — a $d = 3$, $p < 10^{-320}$ — ma vive nel regime in cui la sindrome è stretta abbastanza da lasciare alla rete densa una discriminazione sufficiente a superare la soglia di precisione. Il valore p quantifica l’incompatibilità con l’ipotesi nulla nel modello del test, non la dimensione pratica dell’effetto, riportata separatamente tramite la riduzione di p_L . Poiché è proprio la crescita della distanza a costituire la strategia della correzione d’errore scalabile, il metodo così com’è non accompagna il codice nel regime che conta. Il rimedio indicato dalla diagnosi non è raccogliere più campioni — ne bastavano già — ma innalzare la discriminazione a parità di larghezza dell’ingresso, il che significa cambiare architettura: una rete convoluzionale o a grafo, che condivida i pesi fra regioni equivalenti del reticolo anziché trattare i 336 detector come dimensioni indipendenti, sfrutta la simmetria traslazionale del problema e rappresenta con pochi parametri le correlazioni locali che la rete densa deve stimare una per una. È l’estensione naturale, e la si indica fra gli sviluppi futuri (Capitolo 6).

E.3.1 E6 — e se il modello del decoder fosse semplicemente sbagliato?

Gli esperimenti fin qui condotti concedono al MWPM un vantaggio che l’hardware non offre: il detector error model che ne pesa gli archi è generato da Stim *dal circuito stesso*, ed è quindi una descrizione esatta del rumore. Un decoder in produzione lavora invece su un modello ricavato dalla calibrazione del dispositivo — misurata a intervalli, soggetta a deriva, inevitabilmente diversa dal rumore del momento. Se il

vantaggio del modello appreso nascesse da questa discrepanza, sarebbe un vantaggio molto più generale di quello osservato con il crosstalk, perché la mis-calibrazione è la condizione ordinaria di ogni QPU.

L’esperimento la isola: il circuito viene campionato con il rumore reale, mentre il matching è costruito su un modello in cui l’errore di *misura* è sbagliato di un fattore $(1 + \delta)$ e quello di gate resta corretto.

Perché la perturbazione dev’essere sbilanciata. Un primo tentativo scalava tutte le probabilità di errore dello stesso fattore, e non produceva alcun effetto misurabile: il costo rilevato era 1.000, 1.004 e 1.000 per δ pari a 0, +100% e −50%. La ragione è istruttiva e merita di essere esplicitata, perché delimita ciò che una calibrazione errata può e non può fare. Il MWPM minimizza il peso totale del matching, e il peso di un arco vale $\log \frac{1-p_e}{p_e} \approx -\log p_e$ per p_e piccolo: moltiplicare tutte le p_e per una costante trasla *tutti* i pesi della stessa quantità, e un matching di peso minimo è invariante rispetto a una traslazione uniforme. Ciò che rende il matching vulnerabile non è dunque sbagliare la *scala* del rumore, ma il suo *bilanciamento*: il rapporto fra archi temporali, che rappresentano errori di misura, e archi spaziali, che rappresentano errori sui dati. È questo rapporto che l’esperimento perturba.

d	δ	MWPM esatto	MWPM sbagliato	costo	rete	rete/MWPM
3	−80%	0.00407	0.00415	1.018	0.00525	0.79
	−50%	0.00435	0.00435	1.000	0.00493	0.88
	0	0.00427	0.00427	1.000	0.00558	0.77
	+100%	0.00387	0.00382	0.987	0.00443	0.86
	+300%	0.00388	0.00362	0.931	0.00429	0.84
	+900%	0.00385	0.00392	1.017	0.00463	0.85
5	−80%	0.00147	0.00153	1.034	0.02982	0.05
	−50%	0.00172	0.00173	1.010	0.02855	0.06
	0	0.00163	0.00163	1.000	0.02784	0.06
	+100%	0.00174	0.00175	1.005	0.02774	0.06
	+300%	0.00169	0.00167	0.985	0.02888	0.06
	+900%	0.00159	0.00191	1.199	0.03052	0.06

Tabella E.8: Errore logico del MWPM quando il modello che ne pesa gli archi attribuisce all’errore di misura il valore $p(1 + \delta)$ anziché quello vero. La colonna “costo” è il rapporto fra l’errore logico con modello sbagliato e quello con modello esatto; “rete/MWPM” confronta il decoder appreso con il matching mis-calibrato — valori inferiori a 1 indicano che la rete è peggiore. Errore fisico reale $p = 3 \times 10^{-3}$, 4×10^5 campioni per configurazione.

L’esito è netto in entrambe le direzioni. Il MWPM si rivela **notevolmente robusto**: sbagliando l’errore di misura di un fattore cinque in difetto o dieci in

eccesso, l'errore logico cambia tipicamente dell'1–3%, e in due configurazioni *migliora* leggermente. L'unico caso in cui la mis-calibrazione morde davvero è $d = 5$ con $\delta = +900\%$, dove il costo sale al 20%: sovrastimare di un ordine di grandezza l'errore di misura induce il matching a preferire spiegazioni temporali degli eventi rilevati anche quando la spiegazione corretta è spaziale. Ma occorre appunto un errore di calibrazione di un ordine di grandezza perché l'effetto sia visibile.

La rete, dal canto suo, **non batte il matching in nessuna delle dodici configurazioni**, neppure quelle in cui esso lavora sul modello sbagliato. A $d = 3$ resta indietro del 15–25%; a $d = 5$ è peggiore di un fattore venti, in coerenza con il crollo osservato nell'esperimento precedente.

Che cosa questo precisa del criterio. Il risultato è negativo, ma corregge un'imprecisione nel modo in cui il criterio della tesi era stato finora formulato. Si era detto che il modello appreso produce valore dove il metodo analitico è “cieco” rispetto a una parte dell'informazione; questo esperimento mostra che la cecità rilevante non è quella dei *parametri* ma quella della *struttura*. Un modello con i parametri sbagliati continua a rappresentare le stesse classi di errore, e il matching che ne deriva resta quasi ottimale perché la geometria del problema — quali eventi rilevati siano plausibilmente connessi — non cambia. Un modello che non contempla affatto una classe di errore, come il crosstalk fra qubit di dato adiacenti dell'esperimento E2, sbaglia invece la geometria stessa, e lì il margine esiste. La formulazione corretta del criterio è dunque: *l'apprendimento paga quando la classe di errore non è rappresentabile nella struttura del decoder, non quando è rappresentata con parametri imprecisi*.

Ne discende anche un'indicazione pratica di segno opposto a quella che ci si potrebbe attendere: investire in una calibrazione più accurata del modello di rumore, entro un ordine di grandezza, non produce guadagni apprezzabili sulla decodifica. È un'osservazione che vale la pena tenere presente perché la calibrazione periodica è un costo operativo reale nei dispositivi attuali.

E.3.2 E7 — e se si scegliesse semplicemente un decoder analitico migliore?

Resta l'obiezione più severa che si possa muovere a tutto l'impianto della sezione. Il MWPM è lo standard di fatto, ma non è il miglior decoder analitico disponibile: esso richiede che ogni meccanismo di errore sia decomponibile in archi di un grafo, e

il crosstalk fra qubit di dato adiacenti viola precisamente questa ipotesi. Un decoder che non la richieda — **BP+OSD**, ossia *belief propagation* seguita da *ordered statistics decoding*, che opera direttamente sulla matrice di parità del detector error model [69, 70] — potrebbe catturare da solo il margine attribuito all'apprendimento. Se così fosse, il risultato di E4 non misurerebbe il valore di un modello appreso ma il costo di una scelta subottimale del decoder di riferimento.

Il confronto è condotto sugli stessi circuiti e con la stessa misinformazione: entrambi i decoder analitici ricevono il DEM *nominale*, privo di crosstalk, che è quanto un decoder reale avrebbe a disposizione. La significatività è valutata con il test di McNemar, appaiato sugli stessi shot.

d	p_{ct}	MWPM	BP+OSD	BP+OSD/MWPM	ibrido/MWPM	McNemar
3	0	0.00428	0.00374	1.146	0.993	1×10^{-9}
	5×10^{-3}	0.03193	0.03286	0.972	1.185	3×10^{-7}
	1×10^{-2}	0.05912	0.06034	0.980	1.202	5×10^{-7}
	2×10^{-2}	0.11176	0.11340	0.986	1.213	2×10^{-6}
	4×10^{-2}	0.20811	0.20826	0.999	1.189	$p = 0.77$
5	0	0.00192	0.00143	1.339	1.000	2×10^{-7}
	5×10^{-3}	0.03565	0.02807	1.270	1.011	2×10^{-113}
	1×10^{-2}	0.08638	0.07348	1.176	1.033	3×10^{-140}
	2×10^{-2}	0.19998	0.18288	1.094	1.029	6×10^{-108}
	4×10^{-2}	0.37561	0.36788	1.021	1.005	2×10^{-12}

Tabella E.9: Confronto fra il MWPM, il decoder BP+OSD e il decoder ibrido appreso, tutti sul medesimo modello di rumore nominale. Le due colonne di guadagno sono rapporti rispetto al MWPM: valori superiori a 1 indicano un errore logico minore. In grassetto, per ciascuna riga, il migliore fra i due approcci alternativi. Il valore di significatività si riferisce al confronto BP+OSD contro MWPM. Errore fisico $p = 3 \times 10^{-3}$, 2×10^5 campioni per configurazione.

Il primo dato da registrare è che **BP+OSD è effettivamente un decoder migliore**: sul rumore nominale, quello per cui il modello è corretto, riduce l'errore logico del 14.6% a distanza 3 e del 33.9% a distanza 5, con significatività schiacciante. Il vantaggio cresce con la distanza, come atteso: più il codice è grande, più numerose sono le configurazioni di errore degeneri che il matching, vincolato alla decomposizione in archi, non sa distinguere.

Il secondo dato è però quello che conta, ed è la struttura delle due colonne di guadagno letta lungo la colonna p_{ct} .

*Il guadagno di BP+OSD **decresce** al crescere del crosstalk; quello del decoder ibrido **cresce**.*

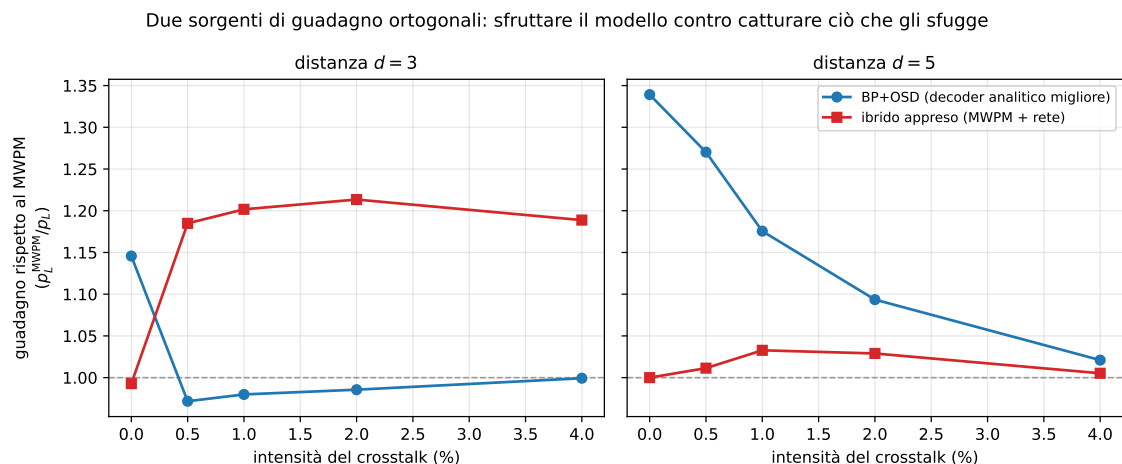


Figura E.2: Guadagno rispetto al MWPM dei due approcci alternativi, in funzione dell'intensità del crosstalk. La linea tratteggiata è il MWPM stesso. BP+OSD (blu) parte dal guadagno massimo, quando il modello di rumore che riceve è corretto, e lo perde man mano che il crosstalk lo rende incompleto; il decoder ibrido (rosso) parte da zero, perché a modello corretto non ha nulla da aggiungere, e guadagna in misura crescente. A distanza 3 le due curve si incrociano immediatamente; a distanza 5 il vantaggio iniziale di BP+OSD è così ampio da mantenerlo davanti anche eroso. Figura rigenerabile da `figure_src/gen_confronto_decoder.py`.

A distanza 5 il vantaggio di BP+OSD scende monotonamente da 1.339 a 1.021 mentre il crosstalk aumenta; a distanza 3 esso si annulla immediatamente, e il decoder diventa lievemente *peggiore* del matching. Specularmente, il guadagno dell'ibrido è nullo in assenza di crosstalk e sale fino a 1.213. I due metodi non competono sullo stesso asse: **BP+OSD sfrutta meglio il modello che riceve, il decoder appreso cattura ciò che al modello sfugge**. Il primo rende al massimo quando il modello è corretto e si degrada man mano che diventa incompleto — perché fidarsi più a fondo di una descrizione sbagliata è un danno, non un vantaggio; il secondo rende zero quando il modello è corretto, perché non c'è nulla da aggiungere, e cresce esattamente nella misura in cui il modello si allontana dal dispositivo.

Ne discende una conclusione più articolata di quella che l'esperimento si proponeva di verificare, e va enunciata riconoscendo entrambi i suoi lati. Da un lato l'obiezione è **in parte fondata**: a distanza 5, dove il vantaggio di partenza di BP+OSD è ampio, esso resta superiore all'ibrido in tutte le configurazioni, e adottare un decoder analitico migliore rende più che addestrare una rete. Chi disponga di un budget di ingegneria limitato dovrebbe spenderlo lì per primo, ed è una raccomandazione pratica che questa tesi non avrebbe potuto formulare senza l'esperimento. Dall'altro lato l'obiezione **non elimina il risultato**: a distanza 3 l'ibrido guadagna 18–21%

proprio dove BP+OSD perde.

Resta da chiarire che cosa significhi “agire su assi ortogonali”. L’inferenza immediata è che i due guadagni siano *sommabili*, e che costruire il residuo appreso sopra BP+OSD anziché sopra il matching debba produrre il prodotto dei due — ossia, a distanza 5 e crosstalk 5×10^{-3} , un guadagno di $1.289 \times 1.010 = 1.302$. È una previsione quantitativa, e l’esperimento successivo la mette alla prova anziché darla per buona.

E.3.3 E8 — le due sorgenti si sommano davvero? E10 — il pavimento comune

La previsione e il suo esito. L’esperimento E8 sostituisce il decoder di base della (5.4): il residuo appreso viene costruito sopra BP+OSD anziché sopra il matching, e i quattro bracci — MWPM, BP+OSD, ciascuno con e senza residuo — sono misurati *sui medesimi campioni*, il che rende ogni confronto appaiato ed elimina il disallineamento fra le campagne precedenti. Le due ipotesi in gioco erano dichiarate prima dell’esecuzione: quella *moltiplicativa*, per cui il combinato approssima il prodotto; e quella di *saturazione*, per cui il residuo trova sopra BP+OSD assai meno di quanto trovasse sopra il matching, perché una parte di ciò che recuperava era semplicemente il costo di un decoder analitico subottimale. La seconda è l’ipotesi sfavorevole al risultato di questa sezione, ed è quella che i dati sostengono.

d	p_{ct}	BP+OSD	MWPM+res	combinato	prodotto previsto
3	0	1.181	1.079	1.190	1.274
3	0.005	0.985	1.192	1.183	1.173
3	0.01	0.982	1.205	1.203	1.183
3	0.02	0.985	1.211	1.200	1.193
3	0.04	0.997	1.184	1.177	1.180
5	0	1.180	1.000	1.180	1.180
5	0.005	1.289	1.010	1.289	1.302
5	0.01	1.196	1.026	1.200	1.228
5	0.02	1.093	1.028	1.104	1.123
5	0.04	1.024	1.007	1.024	1.032

Tabella E.10: Esperimento E8. Tutti i guadagni sono rapporti $p_L^{\text{MWPM}}/p_L^{\text{metodo}}$ misurati sui medesimi 8×10^5 campioni per configurazione, 2×10^5 dei quali di test. “Combinato” è il residuo appreso costruito sopra BP+OSD; “prodotto previsto” è il valore che l’ipotesi moltiplicativa richiede. Il combinato non lo raggiunge mai quando entrambi i componenti hanno un guadagno reale, e coincide invece con il migliore dei due singoli.

Il confronto decisivo è nelle righe in cui *entrambi* i componenti guadagnano. A distanza 3 ve n’è una sola, quella senza crosstalk: il prodotto richiede 1.274, il combinato si ferma a 1.190, ossia esattamente il guadagno di BP+OSD da solo — e il residuo, sopra BP+OSD e senza crosstalk, ribalta lo 0.04% delle predizioni con $p = 0.58$, cioè non trova nulla. Nelle altre righe di quel blocco il “prodotto” è soddisfatto soltanto perché degenerare: con BP+OSD a 0.98 esso coincide per costruzione con il guadagno del residuo. A distanza 5, dove BP+OSD conserva un vantaggio ampio anche sotto crosstalk, tutte e cinque le righe sono informative e tutte e cinque smentiscono la previsione: lo scarto dal prodotto è negativo ovunque, da -0.007 a -0.027 , e il residuo migliora BP+OSD in modo statisticamente significativo in *una sola* configurazione su cinque, e ivi dell’uno per cento.

Il decoder combinato eguaglia sempre il migliore dei due singoli, mai il loro prodotto. Le due sorgenti di guadagno non sono sommabili: sono alternative.

Perché: esiste un pavimento comune. L’esperimento E10 individua la ragione. Sei bracci sono misurati sui medesimi campioni a distanza 3, in ordine crescente di informazione utilizzata: il decoder *banale* che predice sempre l’assenza di flip logico, la rete usata *da sola* senza alcun modello analitico, il MWPM, BP+OSD, e i due residui costruiti sui due decoder analitici.

p_{ct}	banale	MWPM	BP+OSD	rete sola	MWPM+res	BP+OSD+res	disp.
0	0.04863	0.00432	0.00367	0.00431	0.00405	0.00354	19.4%
0.005	0.08369	0.03106	0.03195	0.02704	0.02676	0.02677	1.0%
0.01	0.11611	0.05928	0.06035	0.04965	0.04885	0.04982	2.0%
0.02	0.17464	0.11169	0.11292	0.09222	0.09108	0.09156	1.2%
0.04	0.26492	0.20753	0.20819	0.17565	0.17413	0.17536	0.9%

Tabella E.11: Esperimento E10: errore logico dei sei bracci a distanza 3, tutti sugli stessi 8×10^5 campioni. La colonna “disp.” è la dispersione relativa fra i tre bracci di fondo — rete sola, MWPM+residuo, BP+OSD+residuo. In presenza di crosstalk essi convergono entro il 2% pur partendo da metodi radicalmente diversi; in sua assenza divergono del 19% e si ordinano per qualità intrinseca del decoder.

Il quadro si legge in due regimi, ed è la struttura stessa del risultato.

A *modello corretto* (prima riga) il pavimento non esiste: i metodi si ordinano per qualità, BP+OSD davanti a tutti con 3.54×10^{-3} e la rete sola ultima con 4.31×10^{-3} , e la dispersione fra i bracci di fondo è del 19%. Qui l’unica cosa che conta è quanto bene si sfrutti il modello, e il residuo appreso non ha nulla da aggiungere.

A *modello strutturalmente incompleto* (le altre quattro righe) i tre bracci di fondo collassano entro lo 0.9–2.0% l’uno dall’altro, mentre i due decoder analitici da cui provengono restano il 16–18% sopra. Vi arriva anche la rete usata da sola, che non dispone di alcun decoder analitico: il pavimento non è dunque una proprietà del metodo di decodifica, ma **dell’informazione contenuta nella sindrome**. Sotto crosstalk ciascuno di questi metodi la estrae per intero, e incontra lo stesso limite.

L’onestà richiede una precisazione sulla parola “coincidono”. Il test di McNemar fra la rete sola e il MWPM+residuo dà $p = 7.8 \times 10^{-4}$, 8.2×10^{-4} e 2.6×10^{-3} ai tre livelli superiori di crosstalk: con 2×10^5 campioni di test si risolve anche uno scarto dell’uno per cento, e quelle differenze residue sono *reali*. Ciò che si afferma è più debole e più preciso: i tre metodi atterrano entro l’1–2% l’uno dall’altro, e tale scarto è un ordine di grandezza inferiore al divario che separa tutti e tre dal decoder analitico da cui partono.

Il fallimento di E8 discende allora da E10 senza bisogno di ulteriori ipotesi: impilare i due metodi non porta sotto il pavimento, perché il pavimento non appartiene a nessuno dei due.

Il pavimento è del problema o del modello? (E11) Alla lettura appena data si può muovere un’obiezione che va presa sul serio, perché è la stessa che questa sezione muove agli altri risultati. I tre bracci che convergono contengono *tutti e tre la medesima rete*, un percettrone (128, 64) addestrato sugli stessi 4.8×10^5 campioni:

che convergano potrebbe significare soltanto “questo è quanto estrae questa rete”, e il pavimento sarebbe una proprietà del modello, non del problema. Per distinguere le due letture occorrono metodi più potenti di quel percettrone. L’esperimento E11 ne mette due alla prova, a distanza 3 e sui medesimi campioni.

Il primo è una rete con la stessa ricetta ma architettura (512, 256, 128), ossia circa quaranta volte i parametri. Il secondo è il **decoder empirico a massima verosimiglianza**: per ogni sindrome osservata in addestramento si predice l’osservabile di maggioranza, con ricaduta sul MWPM per le sindromi mai viste. Quest’ultimo non stima parametri — memorizza — e non ha dunque alcun limite di capacità.

p_{ct}	pavimento	rete 40×	residuo 40×	tabella empirica	copertura
0.005	0.02688	0.02687	0.02617	0.02765	98.1%
0.01	0.04935	0.04920	0.04850	0.05032	97.4%
0.02	0.09251	0.09222	0.09169	0.09499	95.6%
0.04	0.17602	0.17463	0.17377	0.18324	91.3%

Tabella E.12: Esperimento E11, distanza 3. “Pavimento” è il valore di E10 ricalcolato sugli stessi campioni. “Copertura” è la frazione di shot di test la cui sindrome era già stata osservata in addestramento, e misura quanto la tabella empirica sia effettivamente una tabella anziché un MWPM travestito.

L’obiezione è **respinta**: la rete con quaranta volte i parametri atterra sul pavimento entro lo 0.8%, e la tabella empirica — che di capacità ne ha infinita — risulta addirittura dal 2 al 4% *peggiore*. Il pavimento non è dunque il tetto dell’architettura scelta.

Sarebbe però scorretto concluderne che esso coincida con il limite informativo della sindrome, e la ragione sta nella tabella stessa. La sua copertura è alta (91–98%) ma la sua *profondità* non lo è: a crosstalk 4×10^{-2} le sindromi distinte osservate sono 63 180 su 4.8×10^5 campioni di addestramento, ossia circa sette osservazioni per sindrome, e un voto di maggioranza su sette campioni è in larga parte rumore. La tabella non fallisce perché l’informazione manchi, ma perché a questa numerosità non riesce a stimarla; essa mette dunque alla prova l’ipotesi della capacità — e la scarta — senza stabilire alcun limite superiore informativo. Nella stessa direzione va notato che il residuo costruito sulla rete grande scende dell’1–3% sotto il pavimento in tutti e quattro i punti, in modo piccolo ma sistematico: il pavimento non è perfettamente duro.

L’enunciato che i dati sostengono è pertanto il seguente, e va tenuto entro questi limiti: *il pavimento è robusto a un aumento di quaranta volte della capacità del modello*

e a un'alternativa non parametrica, e non è quindi un artefatto dell'architettura; se esso coincida con il limite informativo della sindrome resta una questione aperta, che solo una decodifica esatta a massima verosimiglianza potrebbe chiudere.

E.3.4 E5 — il decoder è trasferibile fra dispositivi?

Gli esperimenti precedenti addestrano e valutano il decoder sul *medesimo* profilo di rumore. È una scelta che lascia aperta l'obiezione più concreta che si possa muovere a un decoder appreso: se il modello funzionasse soltanto sul profilo su cui è stato addestrato, andrebbe riaddestrato per ogni esemplare di QPU e a ogni deriva di calibrazione, e il costo pratico ne annullerebbe il vantaggio. La domanda decisiva è dunque se la rete abbia appreso una *regola* o le idiosincrasie di un profilo.

La verifica consiste nell'addestrare su un'intensità di rumore correlato e valutare su un'altra, senza alcun riaddestramento. Il confronto ha due termini di riferimento: il MWPM, che non si addestra e resta quindi invariante, e la rete addestrata sullo stesso profilo di test, che rappresenta il limite superiore ottenibile conoscendo perfettamente il dominio.

Addestrata su	Valutata su p_{ct}			
	5×10^{-3}	10^{-2}	2×10^{-2}	4×10^{-2}
5×10^{-3}	<i>0.02804</i>	0.05000	0.09684	0.18749
10^{-2}	0.02800	<i>0.04912</i>	0.09492	0.18140
2×10^{-2}	0.02814	0.04915	<i>0.09286</i>	0.17800
4×10^{-2}	0.02841	0.04878	0.09312	<i>0.17643</i>
MWPM (nessun addestramento)	0.03232	0.05742	0.11172	0.20853

Tabella E.13: Errore logico dell'ibrido al variare del profilo di addestramento e di quello di valutazione. In corsivo la diagonale (addestramento e test sullo stesso profilo), in grassetto i due casi in cui una rete *trasferita* supera quella addestrata in casa. Tutte e dodici le configurazioni fuori diagonale battono comunque il MWPM.

L'esito è netto: **tutte e dodici** le reti trasferite mantengono il vantaggio sul decoder analitico, e il costo del trasferimento — il peggioramento medio rispetto alla rete addestrata sul profilo di test — è dell'1.6%. In due casi la rete trasferita risulta addirittura migliore di quella addestrata in casa, differenza che rientra nella variabilità statistica ma che conferma l'assenza di una dipendenza sistematica dal profilo di addestramento.

La regola appresa non è dunque il profilo di rumore, ma qualcosa di più generale: verosimilmente la struttura delle configurazioni di sindrome che il matching interpreta

male, la quale dipende dalla geometria del codice e dalla natura correlata dell'errore più che dalla sua intensità. Ne discende che il modello non richiede riaddestramento a ogni ricalibrazione del dispositivo — il che rimuove l'ostacolo pratico principale all'impiego di un decoder appreso, e distingue nettamente questo risultato da quello della prima fase, dove il classificatore richiedeva una fase di addestramento dedicata proprio per essere poi giudicato inutile.

E.3.5 Bilancio

I sette esperimenti principali e i controlli successivi E8–E12 convergono su una conclusione più informativa di un semplice confronto fra decoder. **Sostituire** il decoder analitico con una rete non conviene: a parità di modello di rumore la rete pareggia a distanza 3 — al prezzo di 10^5 – 10^6 campioni — e perde al crescere della distanza. **Affiancarlo** sì: un decoder ibrido in cui la rete impara soltanto *quando ribaltare* la decisione del matching riduce l'errore logico del 18–21% a distanza 3 e dell'1–3% a distanza 5 in presenza di rumore correlato, e ricade sul matching quando il rumore è quello previsto.

Il guadagno compare dunque a due condizioni congiunte, e gli esperimenti E6 ed E7 permettono di enunciarle con precisione. La prima è che il rumore *esca dalla struttura* del modello del decoder — non che il modello sia mal calibrato: perturbandone i parametri fino a un ordine di grandezza l'errore logico varia dell'1–3%, e la rete non vince in nessuna delle dodici configurazioni. La seconda è che il reticolo resti *rappresentabile* dal modello appreso: a distanza 7, su 336 detector trattati come dimensioni indipendenti, l'ibrido si astiene dall'intervenire in tutte e cinque le configurazioni — anche là dove dispone di oltre duecentomila esempi positivi, sicché il limite è di architettura e non di numerosità (Tabella E.6).

A queste due condizioni se ne aggiunge una terza, emersa dal confronto con BP+OSD, che riguarda la scelta del termine di paragone. Il guadagno dell'ibrido va misurato contro il *miglior* decoder analitico disponibile, non contro quello standard: a distanza 5 BP+OSD recupera assai più margine dell'ibrido, e chi disponga di un budget di ingegneria limitato dovrebbe spenderlo lì per primo. Le due sorgenti di guadagno restano ortogonali — quella analitica decresce al crescere del rumore correlato, quella appresa cresce — ma non per questo sommabili: E8 ed E10 mostrano che entrambe conducono al medesimo pavimento, e che impilarle non porta al di sotto di esso. La costruzione corretta è dunque *sceglierne una in base al regime*, non cumularle: il miglior decoder analitico dove il modello è corretto, il residuo appreso

dove non lo è — e in quest’ultimo caso quale sia il decoder analitico sottostante cessa di contare.

Il perimetro va dichiarato. L’architettura impiegata è una rete densa di capacità modesta, scelta per continuità con la prima fase; i decoder allo stato dell’arte adottano architetture ricorrenti e convoluzionali che sfruttano località e struttura temporale [19]. Le distanze arrivano a $d = 7$, dove il vantaggio si annulla, e il rumore correlato principale resta di tipo Pauli. Errori coerenti e leakage sono un asse ulteriore, ma il controllo della Sezione 5.3.5 non mostra un vantaggio appreso statisticamente risolto specifico per la sovrarotazione: la proprietà decisiva osservata è la non decomponibilità del meccanismo in archi, non il generico allontanamento dal modello Pauli.

Resta il fatto metodologico, ripreso nell’analisi di sensibilità di Shor del Capitolo 5, che collega questa sezione alla prima fase sperimentale: in entrambe le fasi sperimentali di questo lavoro il **complemento a basso costo ha battuto la sostituzione ambiziosa**. Nella prima, la ricerca multi-candidato TOP-4 ha battuto il classificatore che avrebbe dovuto filtrare l’output; nella seconda, una rete che si limita a segnalare quando il decoder sbaglia batte la rete che pretende di sostituirlo. L’apprendimento automatico non è né inutile né risolutivo: produce valore esattamente dove dispone di informazione che nessun metodo più semplice è in grado di sfruttare.

E.4 Analisi di sensibilità di Shor: dettagli e verifiche

Questa sezione documenta per esteso l’analisi M8/M13 sintetizzata nella Sezione 5.4.4 del Capitolo 5. Conserva la separazione fra livelli, la griglia completa, l’analisi delle ripetizioni e il contesto fault-tolerant; nel corpo rimangono definizione del proxy, curva principale e conclusione.

E.4.1 Separazione dei quattro livelli

Livello	Oggetto	Strumento	Domanda
1	Shor ideale	Qiskit Aer	limite senza rumore
2	Shor rumoroso	Aer + noise model	sensibilità agli errori fisici
3	QEC isolata	Qiskit; Stim/PyMatching	soppressione dell'errore
4	proxy per gate	Aer + canale Pauli	sensibilità a un residuo uniforme

Tabella E.14: I quattro livelli isolano domande diverse. Il quarto non è una simulazione fault-tolerant e non riceve automaticamente i tassi aggregati del terzo.

Una simulazione esplicita incontra una barriera dimensionale — sostituire ognuno dei dodici qubit di $N = 15$ con un blocco codificato conduce a centinaia di qubit — e una strutturale: Stim è efficiente sui circuiti stabilizer, mentre aritmetica modulare e QFT rendono Shor non-Clifford. Aer simula il circuito generale piccolo e Stim/PyMatching caratterizza la QEC isolata; nessuno dei due equivale a una compilazione fault-tolerant.

Il manifest M8 registra 224 CX, 302 RZ, 70 SX, 8 misure e lo SHA-256 con prefisso `c7f4cf3b` e suffisso `a50f4ba2c`. Ogni punto aggrega 20 repliche da 4096 shot, cioè 81 920 misure. Il successo per misura vale uno quando frazioni continue e controlli MCD restituiscono fattori non banali.

E.4.2 Contesto delle stime fault-tolerant

Le stime crittografiche comprendono routing, cicli di sindrome, fabbriche di stati magici e schedulazione; non derivano dall'estrapolazione di $N = 15$. Gidney ed Ekerå stimavano circa venti milioni di qubit e otto ore per RSA-2048 [6]; lavori successivi riducono il costo grazie ad aritmetica, codifica e preparazione delle risorse, ma sotto assunzioni architettureali differenti.

Lavoro	Anno	Bersaglio	Qubit fisici e tempo
Gidney ed Ekerå [6]	2019	RSA-2048	~20 milioni, 8 ore
Gidney [7]	2025	RSA-2048	<1 milione, <1 settimana
Pinnacle, qLDPC [71]	2026	RSA-2048	<100 000 (preprint)
Cain <i>et al.</i> [72]	2026	P-256; RSA-2048	~10 000 atomici; P-256 in giorni con 26 000 (preprint)

Tabella E.15: Le righe non sono direttamente confrontabili: cambiano codice, modello d'errore, ciclo e criterio di successo. Le due voci 2026 sono preprint e nessun valore discende dalle curve di questa tesi.

La discesa delle stime è trainata soprattutto da correzione e compilazione, ma resta una valutazione prospettica. La curva M8 è una sensibilità locale e non una previsione di risorse.

E.4.3 Griglia M8 completa e probabilità cumulativa

La griglia completa con intervalli Wilson, l'Equazione 5.6 e la curva delle ripetizioni sono mantenute nel corpo, rispettivamente nella Tabella 5.10 e nella Figura 5.12. Qui resta il contesto che impedisce di tradurre automaticamente quei punti in requisiti fault-tolerant.

E.4.4 Perché le metriche QEC non entrano direttamente in M8

Grandezza	Unità/contesto	Inseribile direttamente in M8?
p_L di M8	Pauli non-identità dopo ogni SX/-X/CX	sì, per definizione
errore Steane	fallimento del protocollo protetto	no
errore surface code	fallimento logico per memoria/ciclo e distanza	no
guadagno decoder	differenza nel benchmark correlato	no

Tabella E.16: La conversione richiede circuito logico, cicli, operazioni fault-tolerant e correlazioni residue.

Una verifica recente su 114 atomi mostra che una variante pre-compilata di Shor su qubit logici dimezza fino a un fattore due la distanza in variazione totale rispetto all'esecuzione fisica; ladder CX logiche mostrano errori due-quattro volte inferiori e un codice $[[16, 4, 4]]$ un fattore otto [73]. Il risultato riguarda il vantaggio del livello logico, non una fattorizzazione scalabile e resta soggetto alla critica di Smolin [68]. La cifra di 48 qubit logici su 280 atomi appartiene invece al distinto esperimento di Bluvstein *et al.* [74], non a Shor. Questa evidenza conferma solo la direzione qualitativa del proxy; non fornisce la conversione mancante.

E.5 Compilazione consapevole della struttura: un'esplorazione

Questa sezione esamina due verifiche esterne al confronto principale M1–M2: la sensibilità alla mappatura su una topologia eterogenea (M11/M11b) e il comportamento della ricerca TOP-4 nel proxy per gate (M13). Entrambe sono analisi esplorative. Il loro scopo è verificare se una metrica disponibile prima dell'esecuzione o una regola disponibile dopo la misura conservino potere informativo; non dimostrare prestazioni su una QPU reale. Il Capitolo 5 ne riporta la sintesi.

E.5.1 Previsione e selezione non sono la stessa domanda

M11 usa la snapshot offline **FakeSherbrooke** distribuita con Qiskit IBM Runtime. Le proprietà `target.error` sono interpretate come *average gate infidelity* (AGI) complessiva e convertite in un canale depolarizzante con la stessa AGI. Il readout, esposto dalla snapshot come un solo errore medio, viene modellato simmetricamente. Le direzioni native ECR sono conservate e ogni ECR compilato deve avere una calibrazione esplicita; `rz` è virtuale.

Non viene aggiunto un secondo canale T_1/T_2 . L'AGI del gate è già una misura complessiva della sua non idealità e sommarvi il rilassamento ricavato dalla stessa snapshot produrrebbe un doppio conteggio non quantificato. Di conseguenza M11 non separa il contributo della decoerenza: studia l'eterogeneità della snapshot attraverso AGI e readout.

Il punteggio di calibrazione di un circuito compilato è

$$s = 1 - \prod_{g \in C} (1 - e_g), \quad (\text{E.3})$$

dove e_g è l'errore target dell'istruzione fisica o della misura. Un punteggio minore indica una fedeltà nominale maggiore. La correlazione fra s e successo risponde alla domanda *predittiva*; scegliere il layout con s minimo e valutarlo su dati non usati per la scelta risponde alla domanda *decisionale*. Le due non vanno confuse.

E.5.2 M11: pilota sui layout

Si campionano 60 sottografi connessi di 12 qubit mediante crescita casuale. Il campionamento non è uniforme sullo spazio dei sottografi: l'esperimento resta quindi

esplorativo e le incertezze non descrivono l'intera topologia Sherbrooke. Su ciascun sottografo il circuito $N = 15$, $a = 7$ viene compilato in `sx/rz/x/ecr` con direzioni ECR native, livello di ottimizzazione 3 e seed 42.

Gli 8192 shot per layout sono divisi in otto batch con seed comuni. Quattro batch costituiscono la partizione di selezione e quattro l'holdout. Si confrontano due regole: layout con punteggio s minimo e layout con successo massimo sulla partizione di selezione; entrambe sono valutate soltanto sull'holdout. Questo evita di chiamare “migliore” il massimo osservato sugli stessi dati usati per stimarne la prestazione.

Voce	Contratto M11
backend	snapshot offline FakeSherbrooke, 127 qubit
layout richiesti	60 sottografi connessi, random-growth non uniforme
circuito	$N = 15$, $a = 7$, 8 qubit di conteggio
base / ottimizzazione	<code>sx/rz/x/ecr</code> , livello 3, seed 42
campionamento	8192 shot, 8 batch, split 4/4
inferenza	Spearman + bootstrap 2000 sui layout

Tabella E.17: Contratto preregistrato di M11. La snapshot è riproducibile tramite hash di calibrazione; non è una misura live di un dispositivo al momento dell'esecuzione.

Stima	Interpretazione ammessa
Spearman s - P_{holdout}	associazione di rango sul campione random-growth
P_{holdout} , minimo s	resa della regola basata solo sulla calibrazione
P_{holdout} , massimo train	resa di una selezione empirica su dati separati
differenza holdout	confronto esplorativo tra le due regole, non oracolo globale

Tabella E.18: La correlazione valuta capacità predittiva; il confronto holdout valuta una decisione. Nessuna riga autorizza un claim causale sull'hardware.

Dei 60 layout richiesti, 40 producono un modello di rumore valido. I 20 fallimenti non sono simulazioni con successo nullo: la snapshot assegna ad almeno un gate usato un'AGI pari a 1, valore fuori dal dominio della conversione depolarizzante adottata, e quei layout vengono pertanto esclusi. Fra i layout validi, P_{holdout} varia da 0,297 a 0,516, con mediana 0,431.

La correlazione di rango tra punteggio di calibrazione e successo holdout è $\rho_s = -0,184$, con IC bootstrap al 95% $[-0,456; 0,124]$ (2000 ricampionamenti). L'intervallo include ampiamente lo zero: sul campione random-growth il punteggio

non mostra una capacità predittiva conclusiva. Il layout selezionato dal minimo punteggio (ID 35) ottiene $P_{\text{holdout}} = 0,451$; quello selezionato dal massimo successo sulla partizione train (ID 48) ottiene 0,516 sull'holdout. La differenza descrittiva è quindi $+0,065$, ma non è accompagnata da un intervallo dedicato e non va generalizzata oltre questo pilota.

La snapshot `fake_sherbrooke` è identificata nel JSON dal proprio hash SHA-256, il cui prefisso è `04b6a48f`. Il risultato completo è nel file `results_M11_pilota_v2_20260826_222137.json`.

Le Figure E.3 e E.4 riportano rispettivamente la dispersione fra partizione di selezione e holdout sui 40 layout validi e la collocazione dei due layout selezionati sulla coupling map della snapshot.

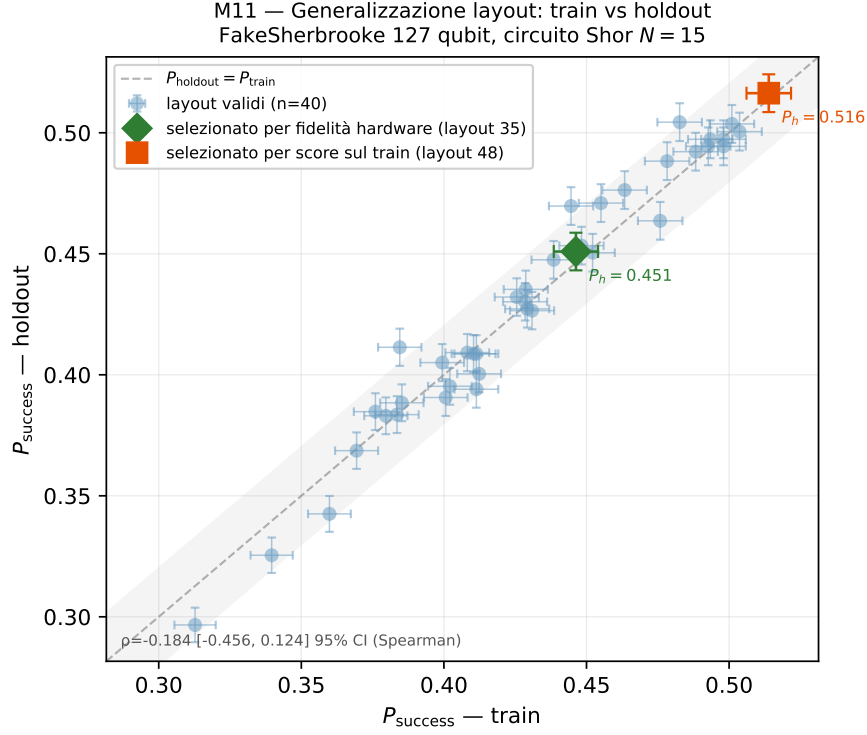


Figura E.3: I 40 layout validi nel piano P_{success} sulla partizione di selezione contro P_{success} sull'holdout, con barre d'errore statistiche e diagonale $y = x$ come riferimento. Sono evidenziati il layout scelto dal punteggio di calibrazione minimo (ID 35) e quello scelto dal successo massimo in selezione (ID 48). La dispersione attorno alla diagonale è lo scarto fra prestazione osservata sui dati usati per scegliere e prestazione su dati indipendenti: è la ragione dello split 4/4. Gli assi **non** rappresentano la correlazione ρ_s discussa nel testo, che lega il punteggio di calibrazione al successo holdout ed è una grandezza diversa. Esito osservazionale sul campione random-growth non uniforme: nessun claim causale e nessuna generalizzazione all'intera topologia. Figura rigenerabile da `figure_src/gen_m11_scatter.py` a partire da `results_M11_pilota_v2_20260826_222137.json`.

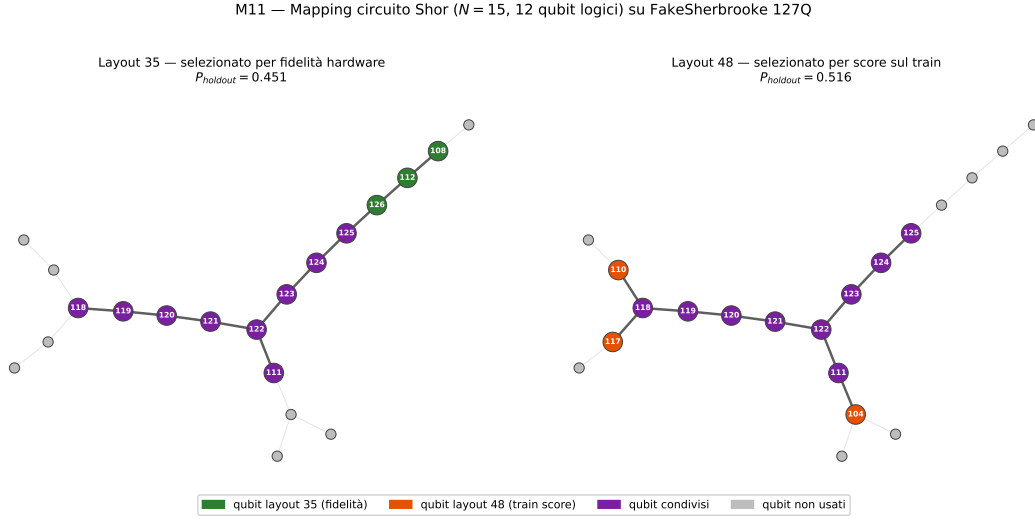


Figura E.4: Sottografo di connettività della snapshot **FakeSherbrooke** attorno ai qubit impiegati: sono disegnati i qubit dei due layout selezionati e i loro primi vicini, non l'intero registro da 127 qubit. Pannello sinistro layout 35, pannello destro layout 48; in viola i qubit comuni ai due layout, in grigio quelli non impiegati. La disposizione dei nodi sul foglio è un'**incorporazione grafica** di tipo *force-directed* e non riproduce la geometria *heavy-hex* del processore: la figura mostra quali qubit e quali accoppiamenti i due layout occupano, non dove essi si trovino fisicamente sul chip. Serve a rendere concreta l'eterogeneità della coupling map su cui il punteggio di calibrazione è costruito; non aggiunge evidenza al confronto fra le due regole di selezione. Figura rigenerabile da `figure_src/gen_m11_hardware_map.py`.

Perché la correlazione va misurata e non assunta. Gli strumenti di selezione del layout costruiscono un punteggio aggregando le grandezze di calibrazione disponibili prima dell'esecuzione. I-QMapper, per esempio, assegna a ogni layout un *Layout-Quality Score* che riassume in un solo valore gli errori di readout e di gate a due qubit del layout [75], con la stessa logica del punteggio impiegato qui. La grandezza che tali punteggi intendono approssimare non è però il punteggio stesso, ma la prestazione osservata: quanto strettamente le due siano legate è una domanda empirica, e su questo campione la risposta misurata è debole ($\rho_s = -0,184$, con intervallo che include lo zero). Il contributo di M11 va letto in questa direzione: non una critica a una regola di selezione particolare — il pilota non ne implementa nessuna di letteratura — ma la misura, su un campione dichiaratamente non uniforme, di un legame che la pratica corrente tende a dare per acquisito. Un confronto utile è con i metodi che ordinano i layout apprendendo da esecuzioni passate: Hartnett e collaboratori addestrano un modello d'errore su misure di hardware IBM e riportano una riduzione dell'errore di selezione di $3,2\times$ rispetto alla scelta casuale su un dispositivo

a 16 qubit [76]. Il contrasto è istruttivo e non contraddittorio: un punteggio *appreso da esecuzioni* mostra un guadagno misurato, mentre qui è un punteggio *derivato dalla sola calibrazione* a essere valutato, su una snapshot e su un campione di layout non uniforme.

E.5.3 M11b: readout effettivo dopo il routing

Una seconda verifica genera variazione entro 12 sottografi. Per ciascuno vengono proposti cinque *initial layout*: registri di conteggio inizialmente assegnati ai qubit con readout nominale basso o alto, layout scelto dal transpiler e due permutazioni casuali. Queste etichette non sono trattamenti. Il routing può spostare i qubit virtuali, perciò l'esposizione analizzata è l'errore medio di readout dei qubit **effettivamente misurati nel circuito finale**.

Voce	Contratto M11b
sottografi	12, ciascuno con cinque strategie descrittive
base / ottimizzazione	sx/rz/x/ecr , livello 2, seed 42
shot	8192 per strategia, otto batch, holdout 50%
esposizione	readout medio sui qubit misurati dopo routing
controlli	effetti fissi di sottografo, numero ECR, profondità
stima primaria	Spearman parziale readout–successo
incertezza	cluster bootstrap sui sottografi analizzabili, 2000 ri-campionamenti

Tabella E.19: L'unità indipendente del bootstrap è il sottografo, non la singola strategia annidata.

Etichetta iniziale	Uso nell'analisi
init-readout-basso	genera una permutazione iniziale; non garantisce il readout finale
init-readout-alto	genera variazione nella direzione opposta
transpiler	lascia la scelta iniziale al transpiler
casuale-1/2	due permutazioni riproducibili di confronto

Tabella E.20: Le etichette sono strumenti per produrre variazione entro sottografo. Il predittore usato nel modello è sempre la proprietà osservata dopo il routing.

Dei 12 sottografi richiesti, nove risultano analizzabili per tutte le cinque strategie, per un totale di 45 osservazioni holdout. Nei tre sottografi esclusi la conversione del

modello di calibrazione incontra un'AGI pari a 1, fuori dal dominio della parametrizzazione depolarizzante: anche in questo caso le righe escluse non rappresentano esecuzioni con successo nullo.

Dopo la rimozione degli effetti fissi di sottografo e il controllo per numero di ECR e profondità, la correlazione parziale di Spearman fra errore medio di readout effettivo e successo è $\rho_s = -0,605$. L'IC al 95% ottenuto con 2000 ricampionamenti cluster a livello di sottografo è $[-0,826; -0,190]$. Nel campione osservato, a maggiore errore di lettura corrisponde quindi un successo inferiore anche entro sottografo; l'intervallo non include lo zero. Il risultato resta associativo, non causale, sia per il numero ridotto di cluster sia per la possibile presenza di proprietà topologiche non misurate correlate con readout, conteggio ECR e profondità.

Il risultato canonico è `results_M11b_ruoli_v2_20260827_115109.json`; l'analisi indipendente è `analysis_M11b_v2_20260827_115119.json` e vincola il sorgente all'hash SHA-256 `1c4b5b4fad1e2289...f41c196b42152215`.

E.5.4 M13: TOP-4 sotto il proxy per gate

M13 usa lo stesso circuito, lo stesso manifest e la stessa definizione di p_L di M8. Per ciascun punto esegue 200 repliche da 1024 shot e riusa ogni istogramma per tre endpoint: massa di successo per singola misura, TOP-1 e TOP-4. Il successo TOP-K è binario per replica: almeno uno dei candidati selezionati conduce ai fattori.

Un pareggio al confine del quarto posto rende la selezione non identificata. Il runner salva quindi: (i) una scelta pseudo-casuale riproducibile; (ii) un limite inferiore, vero solo quando ogni scelta ammissibile contiene un esito utile; (iii) un limite superiore, vero quando almeno una scelta ammissibile può contenerlo. Il contrasto primario fra $p_L = 0$ e il massimo preregistrato usa un intervallo Newcombe; l'equivalenza richiede che l'intero intervallo cada entro $\pm 0,10$. Una seconda verifica usa un involuppo Wilson simultaneo su tutte le risoluzioni dei pareggi.

p_L	per misura	TOP-1	TOP-4 seeded	identificato
0	0,7512	0,805	1,000	[1,000; 1,000]
10^{-4}	0,7459	0,730	1,000	[1,000; 1,000]
5×10^{-4}	0,7377	0,690	1,000	[1,000; 1,000]
10^{-3}	0,7229	0,620	1,000	[1,000; 1,000]
$1,7 \times 10^{-3}$	0,7080	0,620	1,000	[1,000; 1,000]
2×10^{-3}	0,7003	0,650	1,000	[1,000; 1,000]
5×10^{-3}	0,6396	0,505	1,000	[1,000; 1,000]
10^{-2}	0,5652	0,645	1,000	[1,000; 1,000]
2×10^{-2}	0,4528	0,695	1,000	[1,000; 1,000]
5×10^{-2}	0,2979	0,760	1,000	[1,000; 1,000]
10^{-1}	0,2506	0,535	0,915	[0,855; 0,940]
2×10^{-1}	0,2469	0,245	0,670	[0,535; 0,860]

Tabella E.21: Endpoint estratti dall’artefatto M13 schema 2 del 26 agosto 2026. La colonna “identificato” riporta i limiti inferiore e superiore generati dai pareggi al confine del TOP-4.

Il contrasto primario preregistrato confronta il TOP-4 seeded a $p_L = 0$ con quello al massimo valore della griglia, $p_L = 0,2$. La differenza stimata è 0,330 e l’IC Newcombe 95% è [0, 266; 0, 398]. La regola di equivalenza richiedeva l’intero intervallo entro $\pm 0,10$; la regola di discriminazione richiedeva invece che il limite inferiore della diminuzione superasse 0,10. L’esito preregistrato è una diminuzione discriminante: il TOP-4 non è equivalente sul dominio completo della griglia.

Le analisi di sensibilità confermano che la gestione dei pareggi è materiale. Se tutti i pareggi sono risolti nel senso più sfavorevole, la diminuzione stimata è 0,465 con IC [0, 395; 0, 534]; nel senso più favorevole è 0,140 con IC [0, 095; 0, 195], non sufficiente per superare il margine con la stessa regola di discriminazione. L’inviluppo simultaneo Bonferroni-Wilson su tutte le risoluzioni ammissibili è [0, 059; 0, 553] e non supporta equivalenza per tutte le risoluzioni dei pareggi. Il risultato va quindi letto così: TOP-4 è estremamente efficace fino a $p_L = 0,05$ su $N = 15$, ma degrada in modo statisticamente visibile agli errori più alti della griglia.

E.5.5 Conclusione e limiti

M11 e M11b non sono un benchmark su hardware. Il simulatore e il punteggio derivano dalla stessa snapshot; questa coerenza rende il confronto controllato, ma può favorire la metrica nominale e non include deriva, crosstalk, leakage o errori coerenti. Il campionamento random-growth non è uniforme. M11b aumenta il controllo confrontando strategie entro lo stesso sottografo, ma resta osservazionale: readout, numero ECR e profondità possono essere correlati con variabili topologiche non misurate.

M13 risponde a una domanda diversa. Una strategia TOP-4 può restare efficace anche quando la massa utile per misura è fortemente degradata, soprattutto nell'istanza $N = 15$ con pochi picchi strutturali. Perciò TOP-4 deve essere riportato insieme alla metrica per misura e all'incertezza dei pareggi; da solo non misura la fedeltà dello stato quantistico e non può essere estrapolato a moduli maggiori.

Bibliografia

- [1] Peter W. Shor. “Algorithms for Quantum Computation: Discrete Logarithms and Factoring”. In: *Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 1994, pp. 124–134. DOI: 10.1109/SFCS.1994.365700.
- [2] Arjen K. Lenstra e Hendrik W. Lenstra Jr. “The Development of the Number Field Sieve”. In: *Lecture Notes in Mathematics*. Vol. 1554. Springer-Verlag, 1993. DOI: 10.1007/BFb0091537.
- [3] Ronald L. Rivest, Adi Shamir e Leonard Adleman. “A Method for Obtaining Digital Signatures and Public-Key Cryptosystems”. In: *Communications of the ACM* 21.2 (1978), pp. 120–126. DOI: 10.1145/359340.359342.
- [4] John Preskill. “Quantum Computing in the NISQ Era and Beyond”. In: *Quantum* 2 (2018), p. 79. DOI: 10.22331/q-2018-08-06-79.
- [5] Unathi Skosana e Mark Tame. “Demonstration of Shor’s Factoring Algorithm for $N = 21$ on IBM Quantum Processors”. In: *Scientific Reports* 11 (2021), p. 16599. DOI: 10.1038/s41598-021-95973-w.
- [6] Craig Gidney e Martin Ekerå. “How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits”. In: *Quantum* 5 (2021), p. 433. DOI: 10.22331/q-2021-04-15-433.
- [7] Craig Gidney. *How to factor 2048 bit RSA integers with less than a million noisy qubits*. arXiv:2505.15917. 2025. arXiv: 2505.15917.
- [8] Michael A. Nielsen e Isaac L. Chuang. *Quantum Computation and Quantum Information*. 10th Anniversary Edition. Cambridge, UK: Cambridge University Press, 2000. ISBN: 978-1107002173.

- [9] Kristan Temme, Sergey Bravyi e Jay M. Gambetta. “Error Mitigation for Short-Depth Quantum Circuits”. In: *Physical Review Letters* 119.18 (2017). Articolo fondativo della Zero-Noise Extrapolation (ZNE): propone l’amplificazione controllata del rumore seguita da estrapolazione a zero rumore per stimare l’output ideale di un circuito NISQ, p. 180509. DOI: 10.1103/PhysRevLett.119.180509.
- [10] Andrew M. Steane. “Error Correcting Codes in Quantum Theory”. In: *Physical Review Letters* 77.5 (1996), pp. 793–797. DOI: 10.1103/PhysRevLett.77.793.
- [11] Austin G. Fowler et al. “Surface Codes: Towards Practical Large-Scale Quantum Computation”. In: *Physical Review A* 86.3 (2012), p. 032324. DOI: 10.1103/PhysRevA.86.032324.
- [12] Qiskit contributors. *Qiskit: An Open-source Framework for Quantum Computing*. <https://qiskit.org>. 2023. DOI: 10.5281/zenodo.2573505.
- [13] Craig Gidney. “Stim: a fast stabilizer circuit simulator”. In: *Quantum* 5 (2021), p. 497. DOI: 10.22331/q-2021-07-06-497.
- [14] Oscar Higgott. *PyMatching: A Python package for decoding quantum codes with minimum-weight perfect matching*. arXiv:2105.13082. 2021. arXiv: 2105.13082.
- [15] Fabian Pedregosa et al. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.
- [16] Stéphane Beauregard. “Circuit for Shor’s Algorithm Using $2n + 3$ Qubits”. In: *Quantum Information and Computation* 3.2 (2003). arXiv:quant-ph/0205095. Ottimizzazione del circuito originale di Shor: riduce il numero di qubit da $O(n)$ a $2n + 3$ tramite addizione modulare in-place, pp. 175–185.
- [17] Raul Conchello Vendrell et al. *Plaquette: A hardware-aware design platform for fault-tolerant quantum computers*. arXiv:2607.08767. 2026. arXiv: 2607.08767.
- [18] Cl  mence Chevnard, Pierre-Alain Fouque e Andr   Schrottenloher. “Reducing the Number of Qubits in Quantum Factoring”. In: *Advances in Cryptology — CRYPTO 2025*. Cryptology ePrint Archive, Paper 2024/222. Springer, 2025.
- [19] Johannes Bausch et al. “Learning High-Accuracy Error Decoding for Quantum Processors”. In: *Nature* 635 (2024). AlphaQubit: decoder neurale sviluppato da Google DeepMind e Google Quantum AI. Supera i decoder analitici classici su hardware Sycamore a 241 qubit, pp. 834–840. DOI: 10.1038/s41586-024-08148-8.

- [20] Richard P. Feynman. “Simulating Physics with Computers”. In: *International Journal of Theoretical Physics* 21.6–7 (1982), pp. 467–488. DOI: 10.1007/BF02650179.
- [21] David Deutsch. “Quantum Theory, the Church-Turing Principle and the Universal Quantum Computer”. In: *Proceedings of the Royal Society of London. Series A* 400.1818 (1985), pp. 97–117. DOI: 10.1098/rspa.1985.0070.
- [22] Fabrice Boudot et al. “Factoring RSA-250”. In: *Cryptology ePrint Archive* (2020). Report 2020/697, <https://eprint.iacr.org/2020/697>.
- [23] William K. Wootters e Wojciech H. Zurek. “A Single Quantum Cannot be Cloned”. In: *Nature* 299 (1982), pp. 802–803. DOI: 10.1038/299802a0.
- [24] Dorit Aharonov e Michael Ben-Or. “Fault-Tolerant Quantum Computation with Constant Error Rate”. In: *SIAM Journal on Computing* 38.4 (2008). Versione preliminare presentata a STOC 1997; preprint arXiv:quant-ph/9906129, pp. 1207–1282. DOI: 10.1137/S0097539799359385.
- [25] Giacomo Torlai e Roger G. Melko. “Neural Decoder for Topological Codes”. In: *Physical Review Letters* 119.3 (2017), p. 030501. DOI: 10.1103/PhysRevLett.119.030501.
- [26] Adriano Barenco et al. “Elementary gates for quantum computation”. In: *Physical Review A* 52.5 (1995). Dimostra che qualsiasi operatore unitario su k qubit può essere decomposto in $O(4^k)$ porte elementari; fondamento teorico per la complessità della decomposizione UnitaryGate, pp. 3457–3467. DOI: 10.1103/PhysRevA.52.3457.
- [27] Qingxin Yang e Stefano Markidis. *When Noisy Quantum Order Finding Remains Recoverable for Shor’s Algorithm*. 2026. arXiv: 2605.16074.
- [28] Peter W. Shor. “Scheme for Reducing Decoherence in Quantum Computer Memory”. In: *Physical Review A* 52.4 (1995), R2493–R2496. DOI: 10.1103/PhysRevA.52.R2493.
- [29] Joschka Roffe. “Quantum error correction: an introductory guide”. In: *Contemporary Physics* 60.3 (2019), pp. 226–245. DOI: 10.1080/00107514.2019.1667078.
- [30] A. Yu. Kitaev. “Fault-tolerant quantum computation by anyons”. In: *Annals of Physics* 303.1 (2003), pp. 2–30. DOI: 10.1016/S0003-4916(02)00018-0.

- [31] Andrew W. Senior et al. *A Scalable and Real-Time Neural Decoder for Topological Quantum Codes*. Google DeepMind e Google Quantum AI; sottomesso l'8 dicembre 2025, revisione 11 marzo 2026. 2026. arXiv: 2512.07737.
- [32] Johannes Bausch, Andrew W. Senior et al. *Learning to Decode the Surface Code with a Recurrent, Transformer-Based Neural Network*. Decoder transformer per surface code; supera i decoder state-of-the-art su hardware Google Sycamore per distanze 3–11. 2023. arXiv: 2310.05900.
- [33] Ge Yan, Shanchuan Li e Yuxuan Du. *Rethink the Role of Neural Decoders in Quantum Error Correction*. Accettato a ICML 2026. 2026. arXiv: 2605.12046.
- [34] B. Akoramurthy e B. Surendiran. *Phase-Fidelity-Aware Truncated Quantum Fourier Transform for Scalable Phase Estimation on NISQ Hardware*. 2026. arXiv: 2604.05456.
- [35] Alok Shukla e Prakash Vedula. “A Modular, Adaptive, and Scalable Quantum Factoring Algorithm”. In: *European Physical Journal Plus* 141 (2026). arXiv:2509.05010, p. 474.
- [36] Alok Shukla e Prakash Vedula. “Towards Practical Quantum Phase Estimation: A Modular, Scalable, and Adaptive Approach”. In: *Advanced Quantum Technologies* 9.3 (2026). Preprint arXiv:2507.22460, e00683. DOI: 10.1002/qute.202500683.
- [37] Prakash Murali et al. “Noise-Adaptive Compiler Mappings for Noisy Intermediate-Scale Quantum Computers”. In: *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2019, pp. 1015–1029. DOI: 10.1145/3297858.3304075.
- [38] Swamit S. Tannu e Moinuddin K. Qureshi. “Not All Qubits Are Created Equal: A Case for Variability-Aware Policies for NISQ-Era Quantum Computers”. In: *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2019, pp. 987–999. DOI: 10.1145/3297858.3304007.
- [39] Alberto Peruzzo et al. “A variational eigenvalue solver on a photonic quantum processor”. In: *Nature Communications* 5 (2014), p. 4213. DOI: 10.1038/ncomms5213.
- [40] Edward Farhi, Jeffrey Goldstone e Sam Gutmann. *A Quantum Approximate Optimization Algorithm*. 2014. arXiv: 1411.4028.

- [41] Wojciech Hubert Zurek. “Decoherence, einselection, and the quantum origins of the classical”. In: *Reviews of Modern Physics* 75.3 (2003), pp. 715–775. DOI: 10.1103/RevModPhys.75.715.
- [42] Mohan Sarovar et al. “Detecting crosstalk errors in quantum information processors”. In: *Quantum* 4 (2020), p. 321. DOI: 10.22331/q-2020-09-11-321.
- [43] Lieven M. K. Vandersypen et al. “Experimental Realization of Shor’s Quantum Factoring Algorithm Using Nuclear Magnetic Resonance”. In: *Nature* 414 (2001), pp. 883–887. DOI: 10.1038/414883a.
- [44] Thomas Monz et al. “Realization of a Scalable Shor Algorithm”. In: *Science* 351.6277 (2016), pp. 1068–1070. DOI: 10.1126/science.aad9480.
- [45] David Beckman et al. “Efficient Networks for Quantum Factoring”. In: *Physical Review A* 54.2 (1996). arXiv:quant-ph/9602016. Stima di $5K + 1$ qubit e circa $72K^3$ gate elementari per fattorizzare un numero di K bit, pp. 1034–1063. DOI: 10.1103/PhysRevA.54.1034.
- [46] Paul Bagourd et al. *Practical Challenges in Executing Shor’s Algorithm on Existing Quantum Platforms*. 2025. arXiv: 2512.15330.
- [47] Oded Regev. *An Efficient Quantum Factoring Algorithm*. 2023. arXiv: 2308.06572.
- [48] IBM Quantum. *IBM Quantum Roadmap*. <https://www.ibm.com/roadmaps/quantum/>. Roadmap aggiornata a marzo 2026; obiettivi dichiarati soggetti a modifica. 2026.
- [49] Lov K. Grover. “A Fast Quantum Mechanical Algorithm for Database Search”. In: *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*. ACM, 1996, pp. 212–219. DOI: 10.1145/237814.237866.
- [50] Charles H. Bennett et al. “Strengths and Weaknesses of Quantum Computing”. In: *SIAM Journal on Computing* 26.5 (1997), pp. 1510–1523. DOI: 10.1137/S0097539796300933.
- [51] Christof Zalka. “Grover’s Quantum Searching Algorithm is Optimal”. In: *Physical Review A* 60.4 (1999), pp. 2746–2751. DOI: 10.1103/PhysRevA.60.2746.

- [52] A. R. Calderbank e Peter W. Shor. “Good Quantum Error-Correcting Codes Exist”. In: *Physical Review A* 54.2 (1996), pp. 1098–1105. DOI: 10.1103/PhysRevA.54.1098.
- [53] Dewang Sun, Naifeng Zhang e Franz Franchetti. “Optimization and Performance Analysis of Shor’s Algorithm in Qiskit”. In: *IEEE High Performance Extreme Computing Conference (HPEC)*. Available at https://users.ece.cmu.edu/~franzf/papers/hpec_2023_Quantum_final.pdf. IEEE, 2023.
- [54] IBM Quantum. *Shor’s Algorithm – IBM Quantum Documentation*. <https://quantum.cloud.ibm.com/docs/en/tutorials/shors-algorithm>. Tutorial ufficiale IBM per l’implementazione dell’algoritmo di Shor con Qiskit Runtime su hardware reale (*ibm_marrakesh*). Include Dynamical Decoupling e Gate Twirling come tecniche di soppressione del rumore. 2024.
- [55] Google Quantum AI. *Cirq: A Python Framework for Writing, Manipulating, and Running Quantum Circuits*. <https://quantumai.google/cirq>. Framework open-source Google per processori superconduttori Sycamore. 2024.
- [56] Ville Bergholm et al. *PennyLane: Automatic Differentiation of Hybrid Quantum-Classical Computations*. arXiv:1811.04968. Framework Xanadu orientato a circuiti variazionali e quantum machine learning. 2022.
- [57] Graychii. *Shor Algorithm Implementation*. <https://github.com/Graychii/Shor-Algorithm-Implementation>. Implementazione Python dell’algoritmo di Shor su Qiskit. 2023.
- [58] Haoran Liao et al. *Machine Learning for Practical Quantum Error Mitigation*. 2023. arXiv: 2309.17368.
- [59] John S. Bell. “On the Einstein Podolsky Rosen Paradox”. In: *Physics* 1.3 (1964), pp. 195–200. DOI: 10.1103/PhysicsPhysiqueFizika.1.195.
- [60] Alain Aspect, Philippe Grangier e Gérard Roger. “Experimental Realization of Einstein-Podolsky-Rosen-Bohm Gedankenexperiment: A New Violation of Bell’s Inequalities”. In: *Physical Review Letters* 49.2 (1982), pp. 91–94. DOI: 10.1103/PhysRevLett.49.91.
- [61] Brian D. Josephson. “Possible New Effects in Superconductive Tunnelling”. In: *Physics Letters* 1.7 (1962), pp. 251–253. DOI: 10.1016/0031-9163(62)91369-0.

- [62] Jens Koch et al. “Charge-insensitive Qubit Design Derived from the Cooper Pair Box”. In: *Physical Review A* 76.4 (2007), p. 042319. DOI: 10.1103/PhysRevA.76.042319.
- [63] National Institute of Standards and Technology. *What Is Post-Quantum Cryptography?* <https://www.nist.gov/cybersecurity-and-privacy/what-post-quantum-cryptography>. Aggiornato il 27 febbraio 2026; consultato il 26 agosto 2026. 2026.
- [64] National Security Agency. *Commercial National Security Algorithm Suite 2.0 (CNSA 2.0)*. https://media.defense.gov/2022/Sep/07/2003071834/-1/-1/0/CSA_CNSA_2.0_ALGORITHMS_.PDF. Cybersecurity Advisory, settembre 2022. Fissa le scadenze di migrazione verso algoritmi resistenti ai computer quantistici entro il 2030–2035. 2022.
- [65] National Institute of Standards and Technology. *Post-Quantum Cryptography Standards: FIPS 203, 204, 205*. <https://csrc.nist.gov/pubs/fips/203/final>. FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA). Pubblicazione formale agosto 2024. 2024.
- [66] Apple Security Engineering and Architecture. *iMessage with PQ3: The new state of the art in quantum-secure messaging*. <https://security.apple.com/blog/imessage-pq3/>. Febbraio 2024. Primo protocollo di messaggistica con crittografia post-quantistica di livello 3, basato su ML-KEM per lo scambio di chiavi ibride. 2024.
- [67] Frank Arute et al. “Quantum Supremacy Using a Programmable Superconducting Processor”. In: *Nature* 574 (2019), pp. 505–510. DOI: 10.1038/s41586-019-1666-5.
- [68] John A. Smolin, Graeme Smith e Alexander Vargo. “Oversimplifying Quantum Factoring”. In: *Nature* 499.7457 (2013). Preprint arXiv:1301.7007, dal titolo *Pretending to Factor Large Numbers on a Quantum Computer*, pp. 163–165. DOI: 10.1038/nature12290.
- [69] Pavel Panteleev e Gleb Kalachev. “Degenerate Quantum LDPC Codes With Good Finite Length Performance”. In: *Quantum* 5 (2021), p. 585. DOI: 10.22331/q-2021-11-22-585.
- [70] Joschka Roffe et al. “Decoding Across the Quantum Low-Density Parity-Check Code Landscape”. In: *Physical Review Research* 2.4 (2020), p. 043423. DOI: 10.1103/PhysRevResearch.2.043423.

- [71] Paul Webster et al. *The Pinnacle Architecture: Reducing the Cost of Breaking RSA-2048 to 100 000 Physical Qubits Using Quantum LDPC Codes*. Preprint non sottoposto a revisione paritaria alla data di consultazione. 2026. arXiv: 2602.11457.
- [72] Madelyn Cain et al. *Shor’s Algorithm Is Possible with as Few as 10,000 Reconfigurable Atomic Qubits*. Preprint non sottoposto a revisione paritaria alla data di consultazione. 2026. arXiv: 2603.28627.
- [73] Rich Rines et al. *Demonstration of a Logical Architecture Uniting Motion and In-Place Entanglement*. Collaborazione Inflection; revisione 2 aprile 2026. 2025. arXiv: 2509.13247.
- [74] Dolev Bluvstein et al. “Logical Quantum Processor Based on Reconfigurable Atom Arrays”. In: *Nature* 626 (2024). Preprint arXiv:2312.03982, pp. 58–65. DOI: 10.1038/s41586-023-06927-3.
- [75] Milana Bazayeva e Kenneth Merz. *I-QMapper: Error-Aware Layout Optimization and Device Diagnostics for NISQ Hardware*. 2026. arXiv: 2606.27508.
- [76] Gavin S. Hartnett et al. “Learning to Rank Quantum Circuits for Hardware-Optimized Performance Enhancement”. In: *Quantum* 8 (2024), p. 1542. DOI: 10.22331/q-2024-11-27-1542.

Ringraziamenti

Desidero ringraziare in primo luogo il mio relatore, **Ing. Floriano Caprio**, per la disponibilità, la pazienza e la guida preziosa fornita durante l'intero percorso di sviluppo di questo lavoro. I suoi consigli e la sua competenza sono stati fondamentali per orientarmi in un campo così vasto e in continua evoluzione.

Ringrazio l'**Università Campus Bio-Medico di Roma** per il percorso formativo offerto, che mi ha fornito gli strumenti teorici e pratici per affrontare questa tesi.

Un ringraziamento speciale va alla mia **famiglia**, che ha rappresentato un punto fermo e una fonte inesauribile di sostegno morale durante gli anni di studio.

Ringrazio infine tutti i **colleghi e amici** che hanno condiviso con me questo percorso, rendendo più leggere le giornate di studio e ricerca.

Claudio Dragotta

Roma, 2026